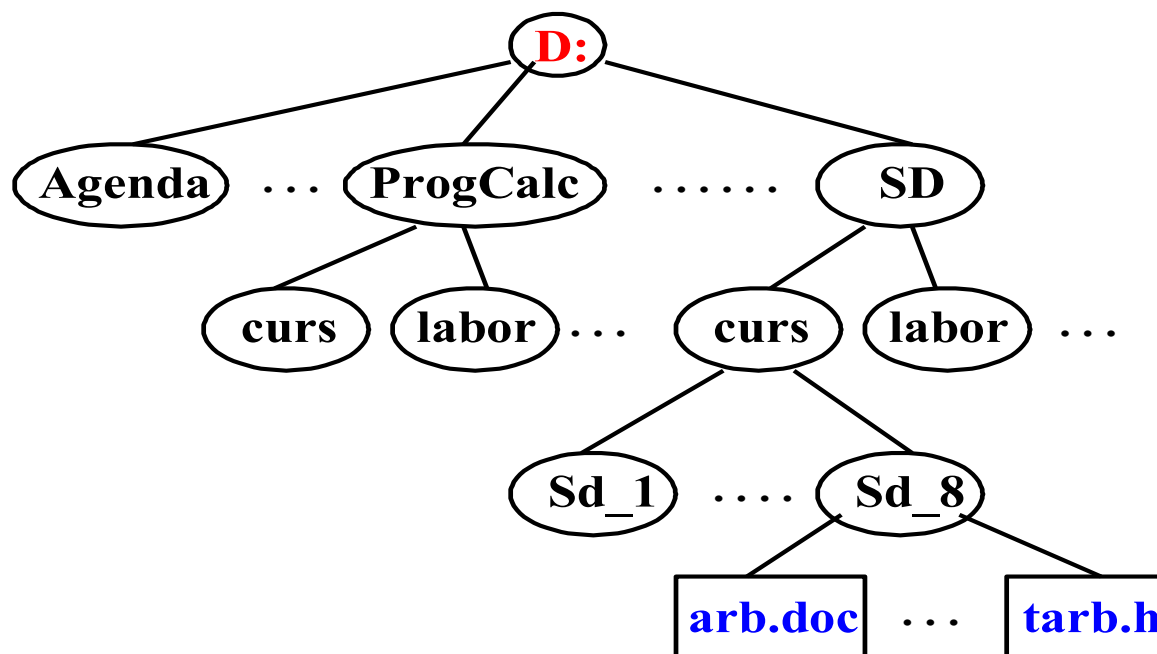


Structuri ierarhice: Arbori

O entitate oarecare poate fi **indivizibilă** sau **structurată**, în marea majoritate a cazurilor, ca o **ierarhie** de componente.

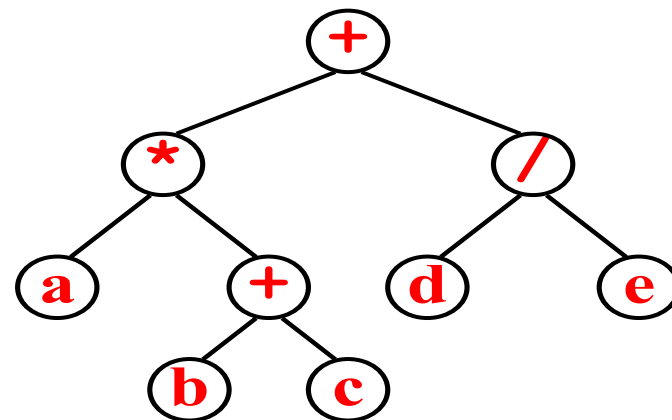
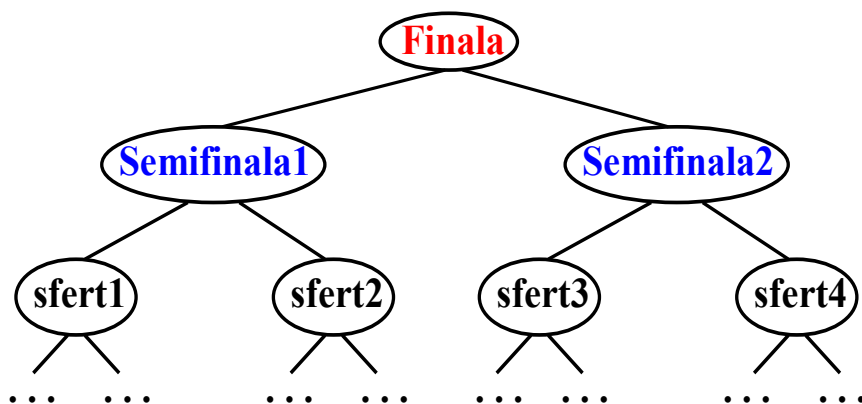
Exemple:

- informația înregistrată pe suport magnetic



Structuri ierarhice: Arbori

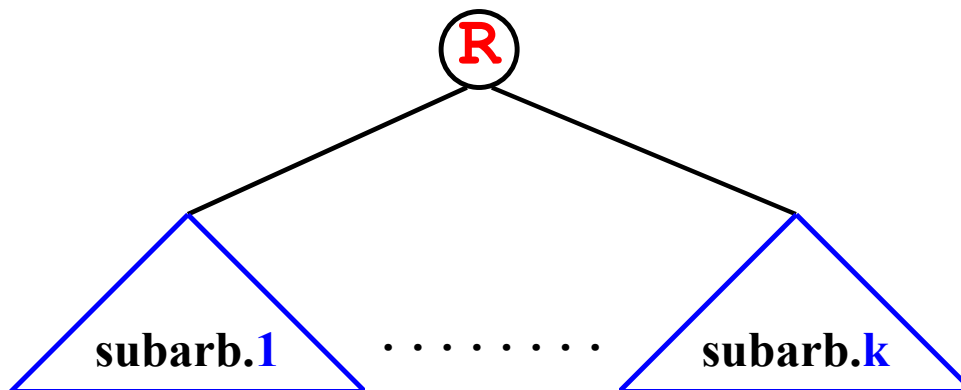
- cuprinsul unei cărți
- organizarea administrativă a societăților comerciale, ministerelor, universităților etc.
- programul meciurilor dintr-un turneu eliminatoriu
- arborele unei expresii aritmetice



Arbori

Un **arbore** este format dintr-un nod **rădăcină**, căruia îi este atașat un **număr finit de arbori**. Pentru a evidenția relația ierarhică, aceștia sunt numiți **subarbori**.

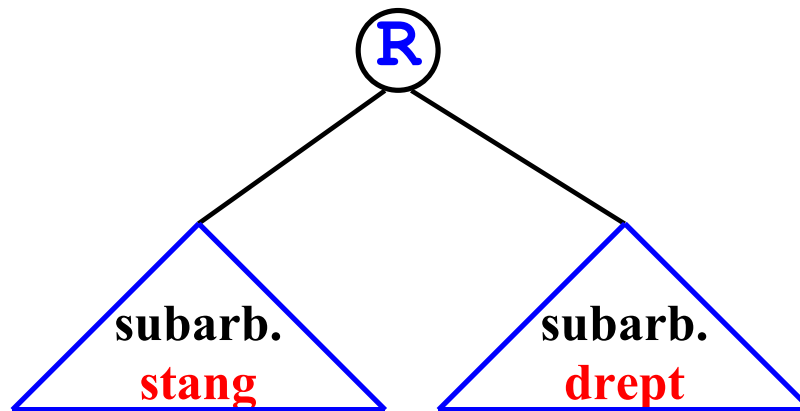
Orice nod dintr-un arbore poate fi privit ca rădăcină a unui (sub)arbore.



Arbori

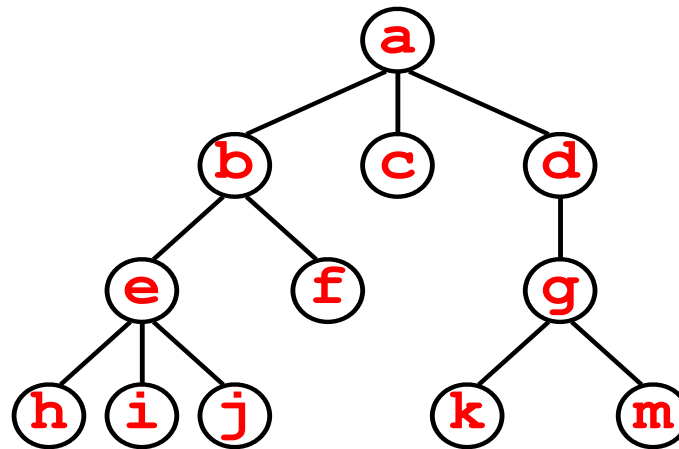
Termeni uzuali:

- **arbore vid** - nu conține nici un nod
- **arbore binar** - nodurile au **cel mult 2** subarbori
 - subarbore **stâng**
 - subarbore **drept**



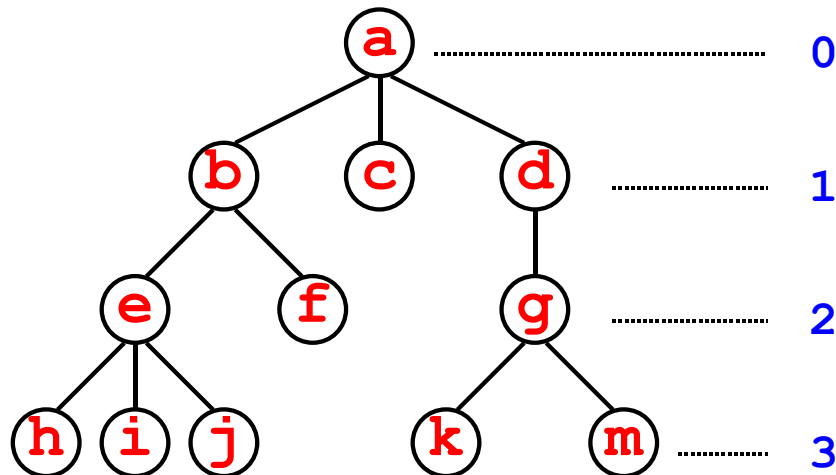
Arbori

- **arbore multicăi** – arbore în care nodurile pot avea mai mult de 2 subarbori



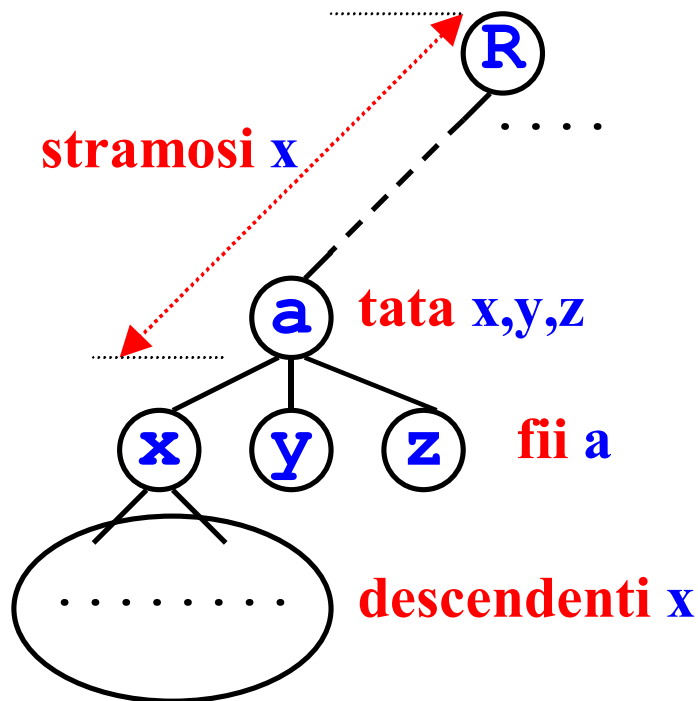
Arbori

- **ordinul** (gradul) unui nod - **numărul de subarbori atașați** nodului
- **noduri interne** - nodurile care au subarbori
- **noduri externe** (noduri terminale sau **frunze**) – nodurile fără subarbori (cu ordin 0)
- **nivelul** unui nod – distanța la care se află față de radacină (radacina are nivel 0)
- **înălțimea** unui arbore – nivelul maxim din arbore

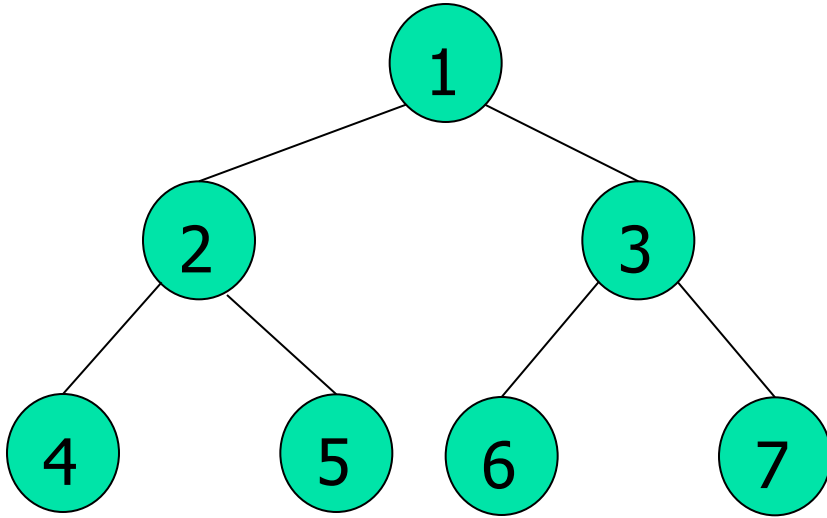


Arbori

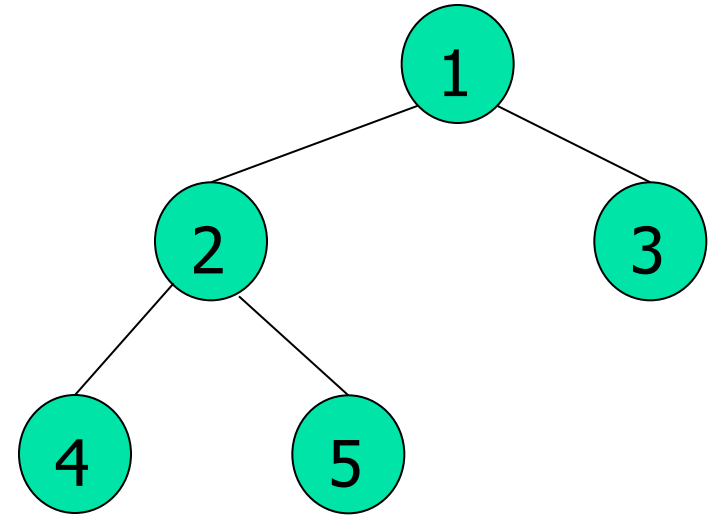
- **tată / fiu** – noduri aflate la distanța 1 (numite și **strămoș / descendent direct**)
- **frați** (siblings) – fiii aceluiași nod



Arbori

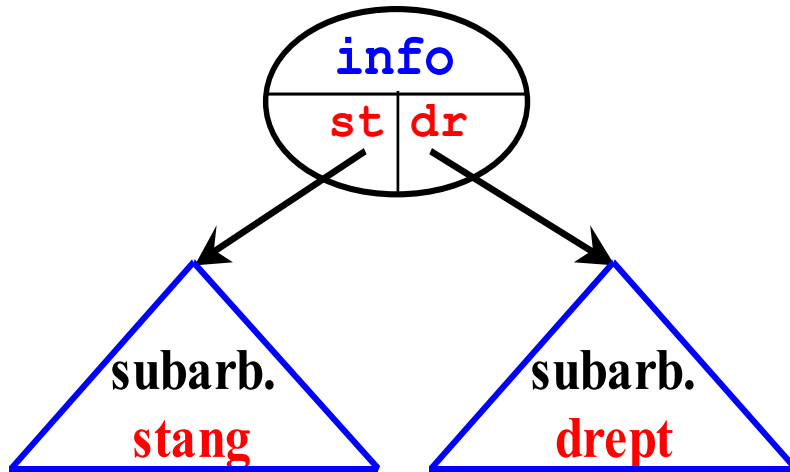


arbore plin



arbore complet

Tipul arbore binar



```
typedef struct nod
{
    int info;
    struct nod *st, *dr;
} TNod, *TArb, **AArb;
```

Adresare :

informatia: `a->info`

subarborii: `a->st`

`a->dr`

Teste arbore binar

Frunză

```
((a)->st == NULL && (a)->dr == NULL)
```

Nod interior

```
((a)->st != NULL) || ((a)->dr != NULL)
```

Teste arbore binar

Ordin nod

$((a) \rightarrow \text{st} \neq \text{NULL}) + ((a) \rightarrow \text{dr} \neq \text{NULL})$

frunza: $0 + 0 \Rightarrow 0$

numai subarbore stang: $1 + 0 \Rightarrow 1$

numai subarbore drept: $0 + 1 \Rightarrow 1$

doi subarbori: $1 + 1 \Rightarrow 2$

Numar noduri

```
int NrNoduri(TArb r)
```

```
/* numarul de noduri din arborele r -*/
```

```
{
```

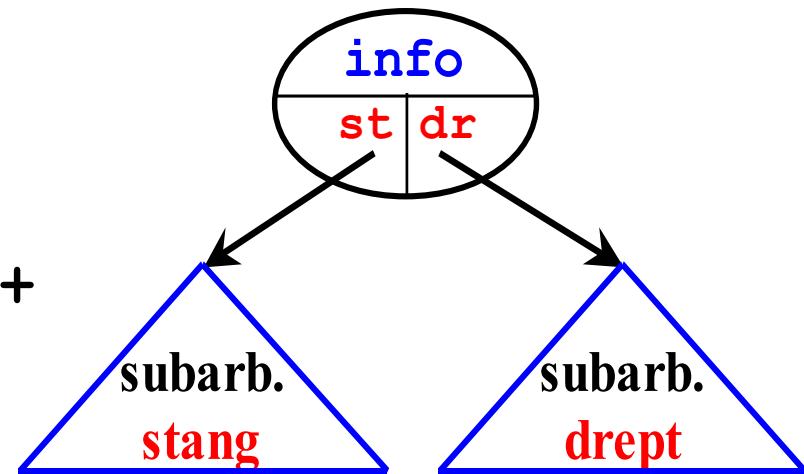
```
    if (!r) return 0;
```

```
    return 1 +
```

```
        NrNoduri(r->st) +
```

```
        NrNoduri(r->dr) ;
```

```
}
```



Numar niveluri

```
int NrNiveluri(TArb r)
```

```
/* numar niveluri (0 pentru arbore vid) */
```

```
{
```

```
    int ns, nd;
```

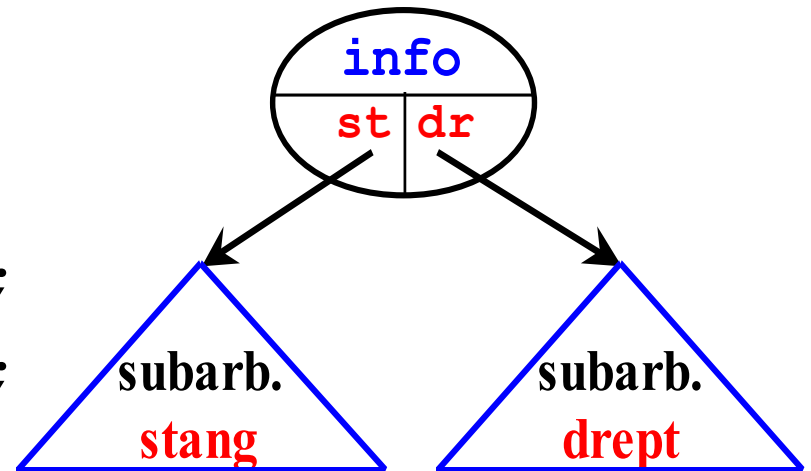
```
    if (!r) return 0;
```

```
    ns = NrNiveluri(r->st);
```

```
    nd = NrNiveluri(r->dr);
```

```
    return 1 + (ns >= nd ? ns : nd);
```

```
}
```



Operatii elementare

```
TArb ConstrNod(TInfo x, TArb s, TArb d);  
    /* -> adresa nod (cu info x,  
                                   fiu stanga s, fiu dreapta d)  
    sau NULL daca nu exista spatiu */  
  
TArb ConstrFr(TInfo x);  
    /* -> adresa frunza cu informatia x,  
    sau NULL daca nu exista spatiu */
```

Creare arbore – constructie frunza

TArb ConstrFr(TInfo x)

**/* -> adresa frunza cu informatia x, sau
NULL daca nu exista spatiu */**

{

TArb aux = (TArb)malloc (sizeof(TNod)) ;

if (!aux) return NULL;

aux->info = x;

aux->st = aux->dr = NULL;

return aux;

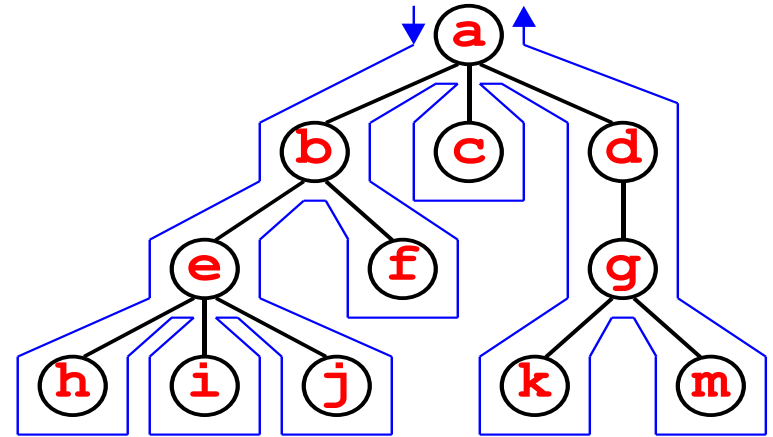
}

Creare arbore – constructie nod

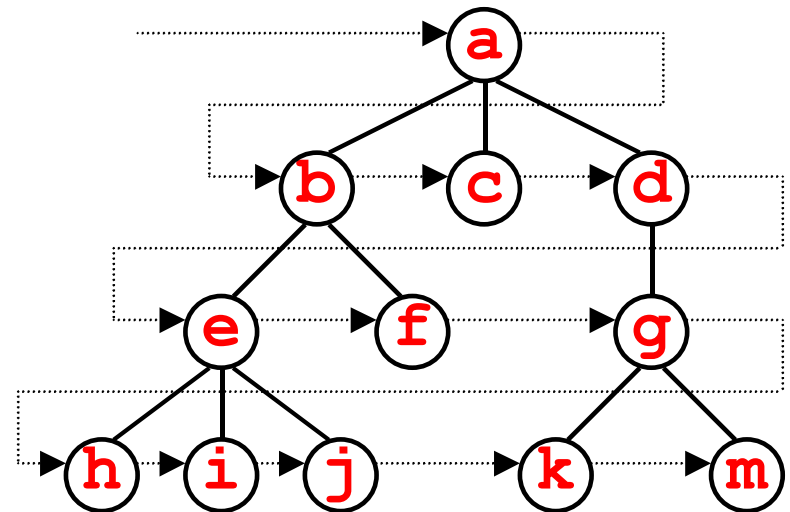
```
TArb ConstrNod(TInfo x, TArb s, TArb d)  
/* -> adresa nod (cu informatia x, fiu stanga s,  
fiu dreapta d) sau NULL in caz de eroare */  
{  
    /* incercare alocare spatiu */  
    TArb aux = (TArb)malloc (sizeof(TNod)) ;  
    if (!aux) return NULL; /* spatiu insuficient */  
  
    /* actualizeaza campurile nodului */  
    aux->info = x; aux->st = s; aux->dr = d;  
    return aux; /* intoarce adresa nodului */  
}
```

Parcurgerea unui arbore

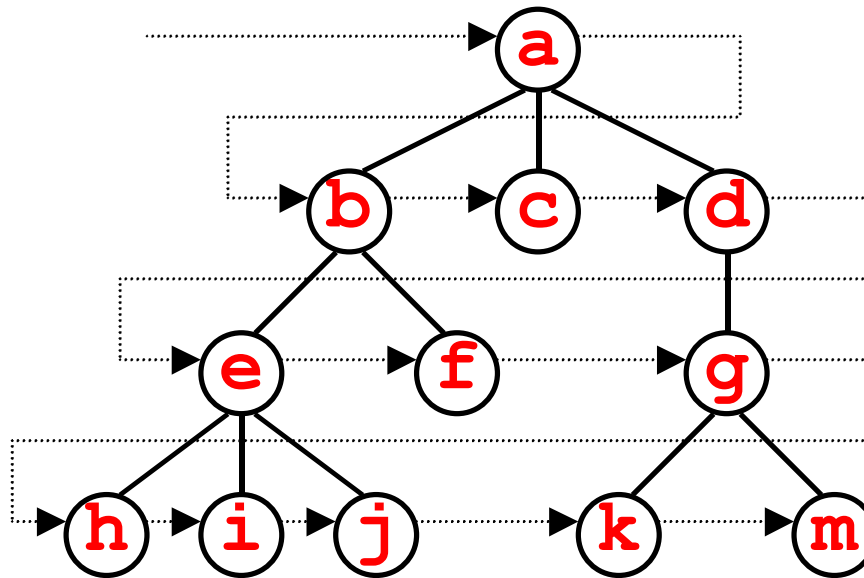
- Adancime



- Latime



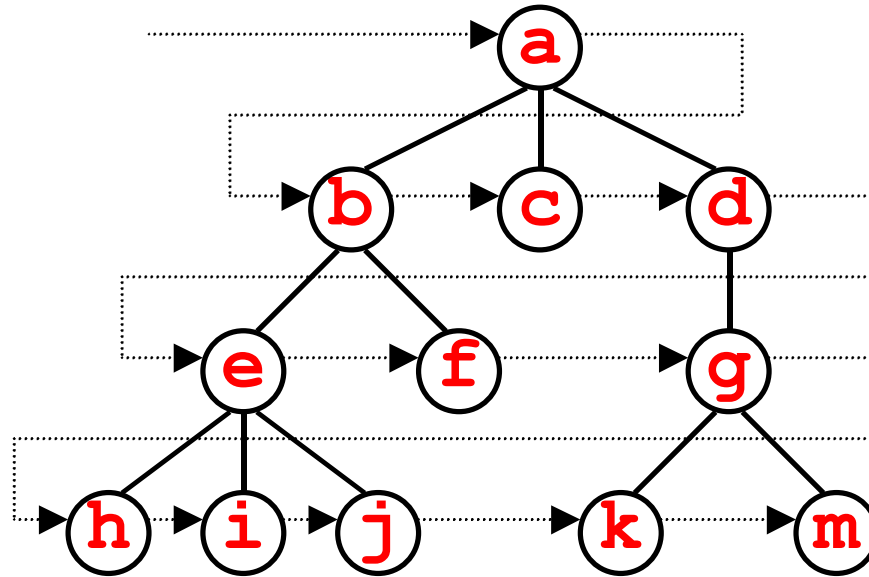
Parcurgerea unui arbore în lăţime



Parcurgere pe nivel:

a, b, c, d, e, f, g, h, i, j, k, m

Parcurgerea unui arbore în lăţime



Coadă: a

Parcurgere: a, coada: -

Inserare în coada copii(a), coada: b, c, d

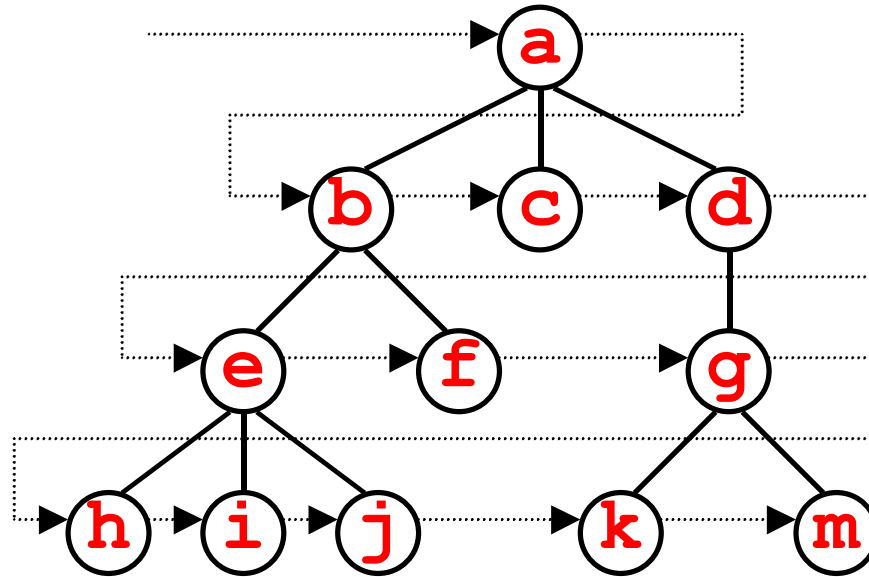
Parcurgere: a, b, coada: c, d

Inserare în coada copii(b), coada: c, d, e, f

Parcurgere: a, b, c, coada: d, e, f

Inserare în coada copii(c), coada: d, e, f

Parcurgerea unui arbore în lăţime



Parcurgere: a, b, c, coada: d, e, f

Inserare in coada copii(c), coada: d, e, f

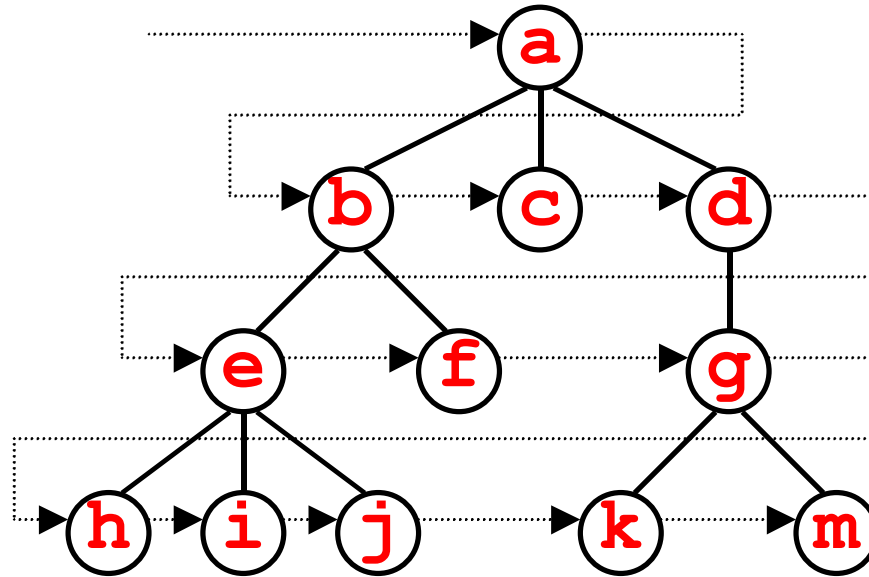
Parcurgere: a, b, c, d, coada: e, f

Inserare in coada copii(d), coada: e, f, g

Parcurgere: a, b, c, d, e coada: f, g

Inserare in coada copii(e), coada: f, g, h, i, j

Parcurgerea unui arbore în lăţime



Parcurgere: a, b, c, d, e coada: f, g

Inserare in coada copii(e), coada: f, g, h, i, j

Parcurgere: a, b, c, d, e, f coada: g, h, i, j

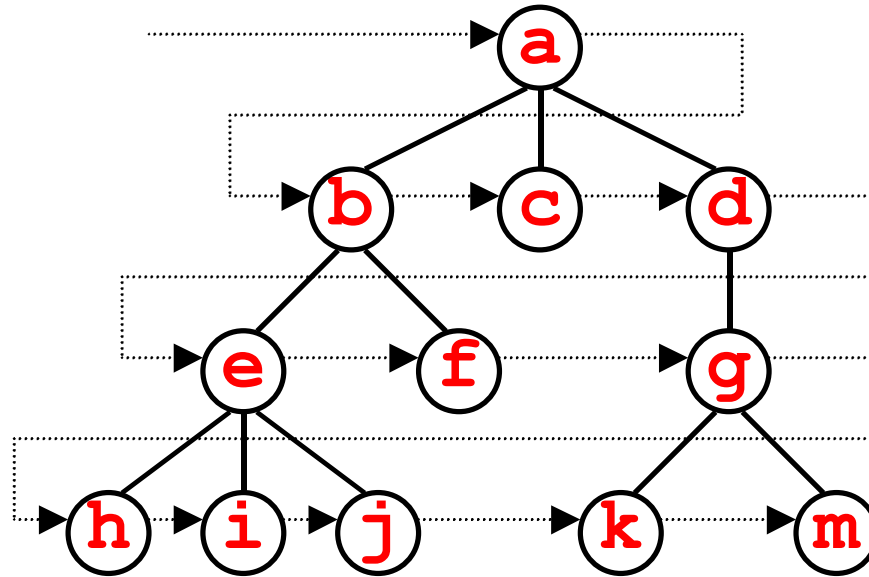
Inserare in coada copii(g), coada: h, i, j, k, m

Parcurgere: a, b, c, d, e, f, g coada: h, i, j, k, m

Inserare in coada copii(h), coada: i, j, k, m

Parcurgere: a, b, c, d, e, f, g, h, coada: i, j, k, m

Parcurgerea unui arbore în lăţime



Parcurgere: a, b, c, d, e, f, g, h, coada: i, j, k, m

Inserare in coada copii(i), coada: j, k, m

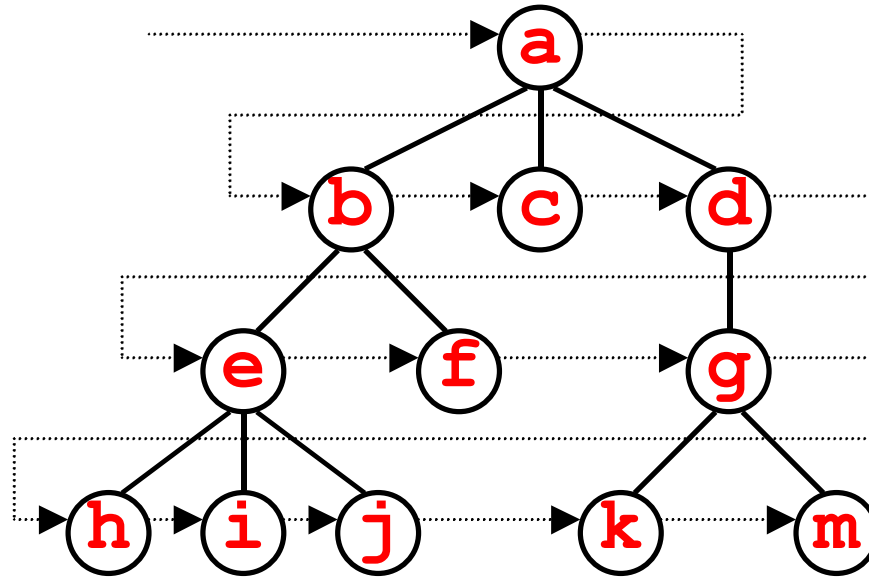
Parcurgere: a, b, c, d, e, f, g, h, i, coada: j, k, m

Inserare in coada copii(j), coada: k, m

Parcurgere: a, b, c, d, e, f, g, h, i, coada: j, k, m

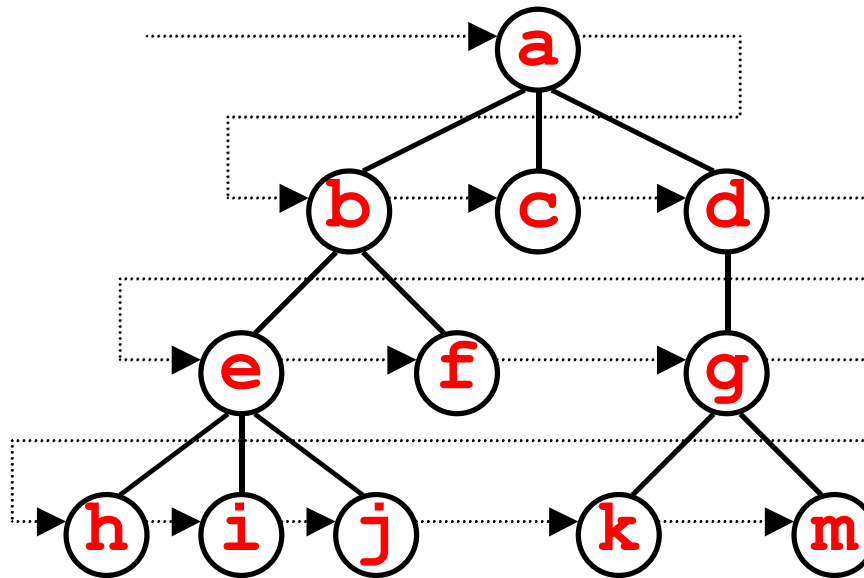
Inserare in coada copii(i), coada: j, k, m

Parcurgerea unui arbore în lăţime



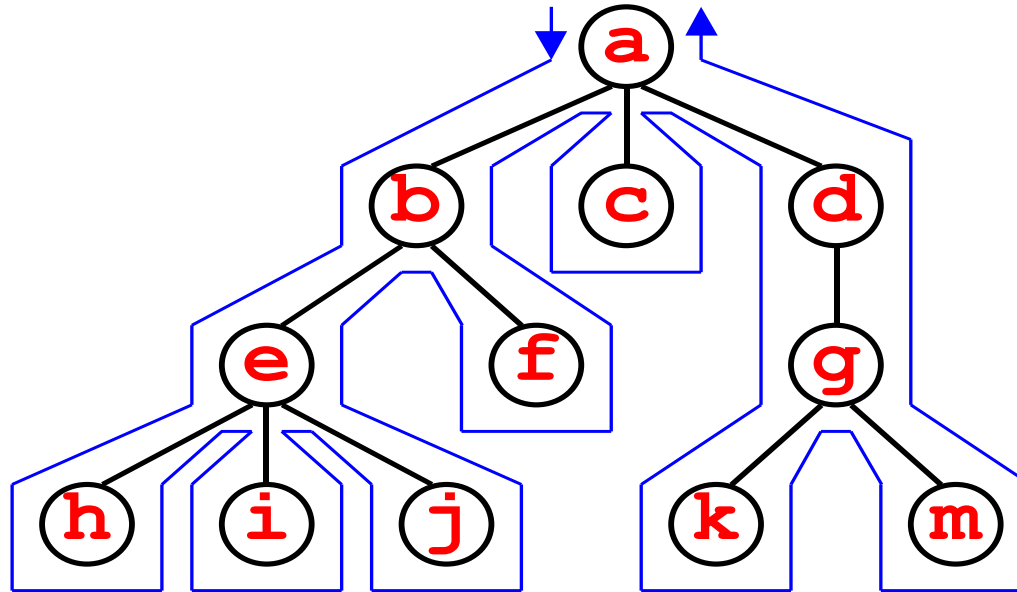
Parcurgere: a, b, c, d, e, f, g, h, i, coada: j, k, m
Inserare in coada copii(i), coada: j, k, m
Parcurgere: a, b, c, d, e, f, g, h, i, j, coada: k, m
Inserare in coada copii(j), coada: k, m
Parcurgere: a, b, c, d, e, f, g, h, i, j, k coada: m
Inserare in coada copii(k), coada: m
Parcurgere: a, b, c, d, e, f, g, h, i, j, k, m coada:-
Inserare in coada copii(m), coada: -

Parcurgerea unui arbore în lățime



```
inițializează coadă;  
inserează adresa rădăcinii în coadă;  
cât timp coadă nevidă  
{ extrage adresă arbore din coadă;  
  prelucrează informația din rădăcină;  
  pentru fiecare subarbore  
    inserează adresa în coadă;  
}
```

Parcurgerea unui arbore în adâncime

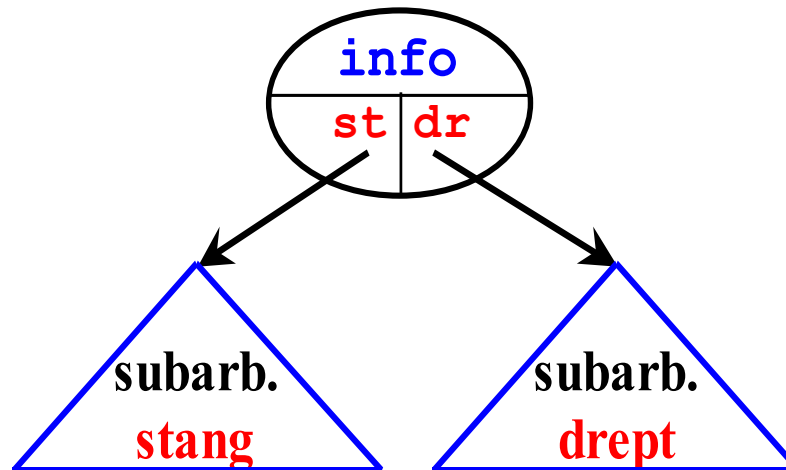


```
Parcurge_in_Adancime (arbore) :  
{ daca arbore vid atunci return;  
  altfel  
  { prelucreaza informatia din radacina;  
    pentru fiecare subarbore  
      Parcurge_in_Adancime (subarbore) ;  
  }  
}
```

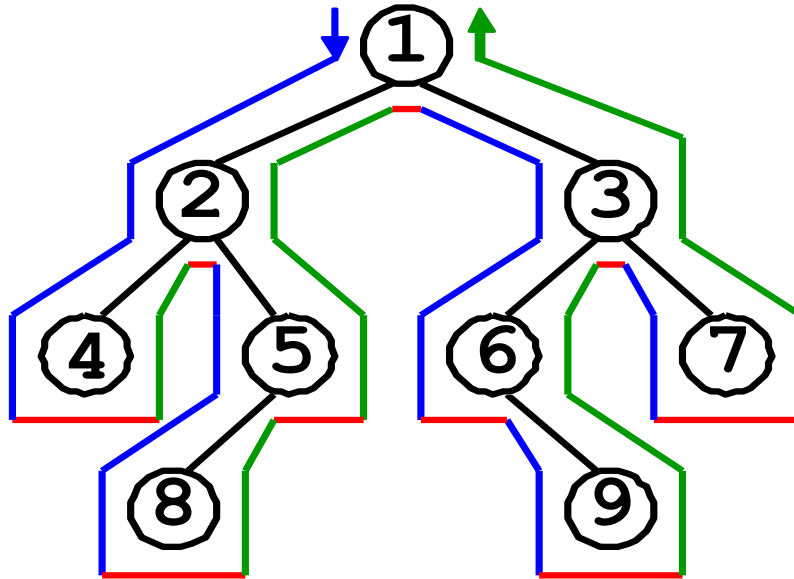
Parcurgerea unui arbore în adâncime

3 tipuri principale de parcurgeri

- **Preordine:** R, S, D
- **Postordine:** S, D, R
- **Inordine:** S, R, D

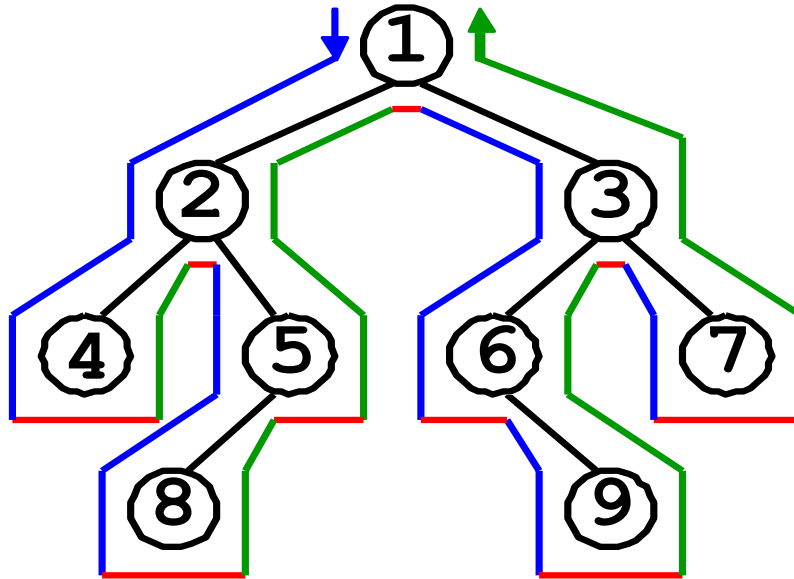


Parcurgeri în adâncime pentru arbore binar



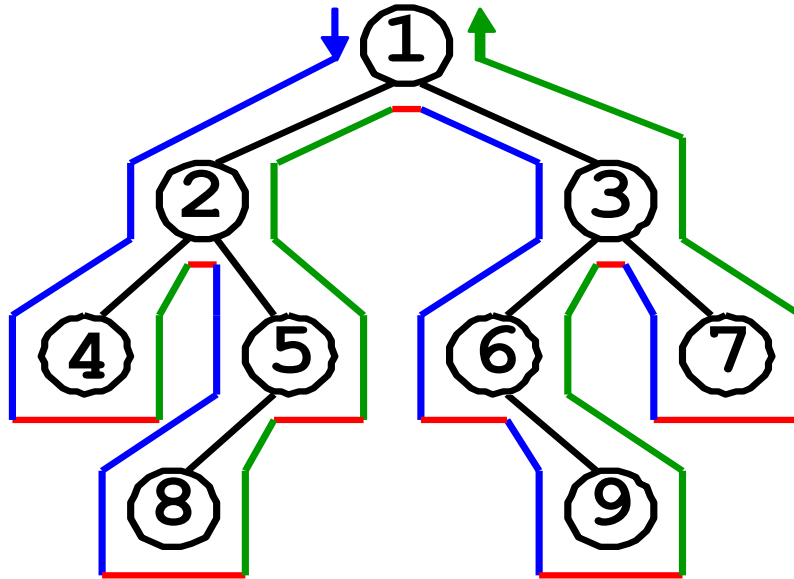
Radacina(Stanga,Dreapta) - in preordine
1, 2, 4, 5, 8, 3, 6, 9, 7

Parcurgeri în adâncime pentru arbore binar



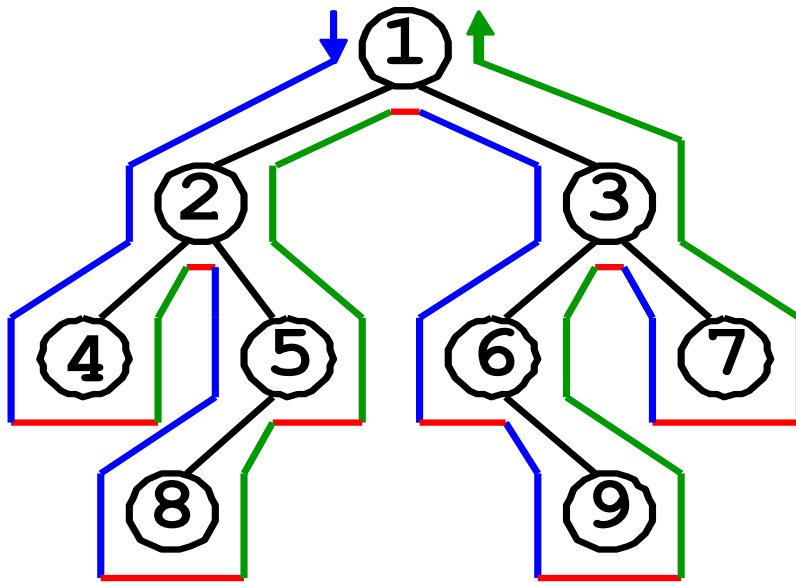
Stanga Radacina Dreapta- in inordine
4, 2, 8, 5, 1, 6, 9, 3, 7

Parcurgeri în adâncime pentru arbore binar



Stanga, Dreapta, Radacina - in postordine
4, 8, 5, 2, 9, 6, 7, 3, 1

Parcurgeri în adâncime pentru arbore binar



Parcurgere în preordine
Radacina (Stanga, Dreapta)
1, 2, 4, 5, 8, 3, 6, 9, 7

```
void RSD(Tarb a)
{
    if(!a) return;
    printf("%d ", a->info;
    RSD(a->st);
    RSD(a->dr);
}
```

Parcurgeri în adâncime pentru arbore binar

Numarare noduri din arbore, in parcurgere RSD

```
void RSD(TArb a)
```

```
{  
    if(!a) return;  
    printf("%d ", a->info;  
    RSD(a->st);  
    RSD(a->dr);  
}
```

```
int Numara_RSD(TArb a)
```

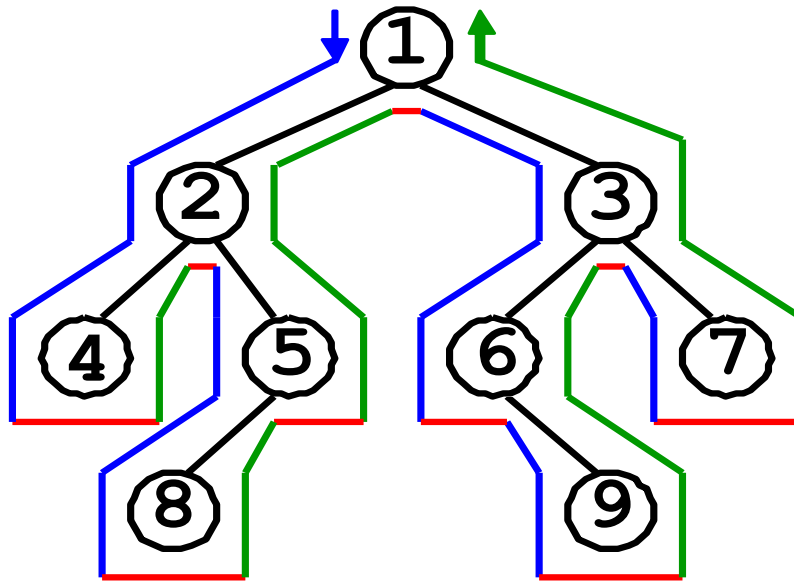
```
{  
    if(!a) return 0;  
    return 1 + RSD(a->st) +  
            RSD(a->dr);  
}
```

Parcurgeri în adâncime pentru arbore binar

Numarare noduri din arbore, in parcurgere SRD

```
int Numara_SRD (TArb a)
{
    int n;
    if(!a) return 0;
    n = Numara_SRD(a->st);
    n++; /* numara radacina */
    n += Numara_SRD(a->dr);
    return n;
}
```

Parcurgeri în adâncime pentru arbore binar



Radacina(Stanga, Dreapta) - in **preordine**

1 (2 (4, 5 (8, -)), 3 (-, 9), 7))

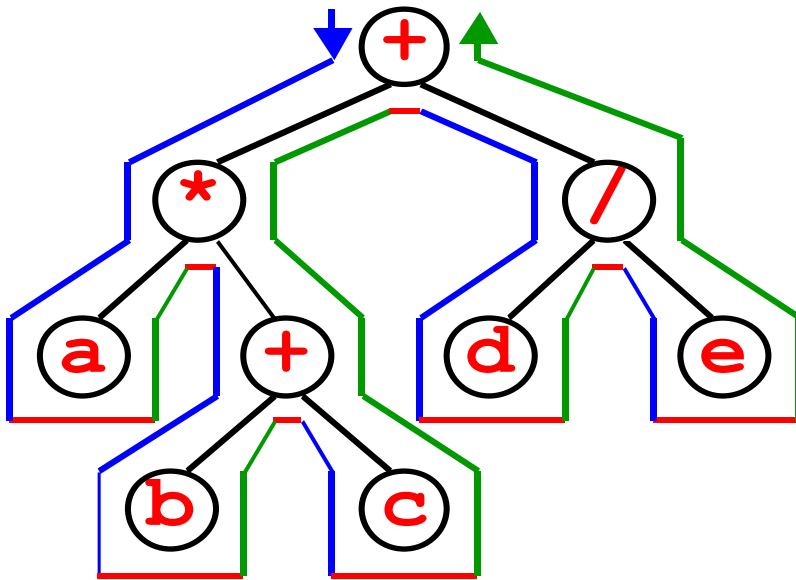
(Stanga **Radacina** Dreapta) - in **inordine**

((4 **2** (8 **5** -)) **1** ((- **6** 9) **3** 7))

(Stanga, Dreapta)**Radacina** - in **postordine**

((4, (8, -) **5**) **2**, ((-, 9) **6**, 7) **3**) **1**

Parcurgeri în adâncime pentru arbore binar



Rădăcină(Stânga, Dreapta) - în **preordine**

+ (* (a, + (b, c)), / (d, e))

(Stânga *Rădăcină* Dreapta) - în **inordine**

((a * (b + c)) + (d / e))

(Stânga, Dreapta) *Rădăcină* - în **postordine**

((a, (b, c) +) *, (d, e) /) +

Parcuregere RSD

```
void RSD(TArb a)
{
    if(!a) {printf("-"); return;}
    if(!a->st && !a->dr)
        {printf(" %d ", a->info); return;}
    printf(" %d ", a->info);
    printf(" (");
    RSD(a->st);
    printf(",");
    RSD(a->dr);
    printf(")");
}
```

Distrugere arbore

```
void distruge(TArb r)
```

```
/* functie auxiliara - distruge toate nodurile */
```

```
{
```

```
    if (!r) return;
```

```
    /* distruge subarborele stang */
```

```
    distruge (r->st);
```

```
    /* distruge subarborele drept */
```

```
    distruge (r->dr);
```

```
    free (r);           /* distruge nodul radacina */
```

```
}
```


Creare arbore

```
void DistrArb(TArb *a)
```

```
/* distruge toate nodurile arborelui de la  
adresa a */
```

```
{
```

```
    if (!(*a)) return;
```

```
    /* distruge toate nodurile */
```

```
    distruge (*a);
```

```
    *a = NULL;
```

```
}
```

Cutare informatie

```
int Cauta(TArb a, int x)
{
    if (!a) return;

    if(a->info == x) return 1;

    int r = Cauta(a->st, x);
    if(r) return 1;

    return Cauta(a->dr, x);
}
```

Cautare informatie + nivelul pe care se afla

```
int Cauta(TArb a, int x)
```

```
{  
    if (!a) return -1;  
    n = aux(a, x, 0);  
    return n;  
}
```

```
int aux(TArb a, int x, int niv)
```

```
{  
    if(!a) return -1;  
    if(a->info == x) return niv;  
    int r = aux(a->st, x, niv+1);  
    if(r >= 0) return r;  
    return aux(a->dr, x, niv+1);  
}
```

Cautare informatie maxima

```
int Max(TArb a) //a - arbore nevid
```

```
{ int m, t;
```

```
  m = a->info;
```

```
  if(a->st) {
```

```
    t = Max(a->st);
```

```
    if(t > m) m = t;
```

```
  }
```

```
  if(a->dr) {
```

```
    t = Max(a->dr);
```

```
    if(t > m) m = t;
```

```
  }
```

```
  return m;
```

```
}
```

Cautare informatie maxima + nivel

```
int Max(TArb a, int* max)
{
    int nivm;
    if(!a) return -1;

    *max = a->info;
    nivm = 0;
    aux(a, 0, max, &nivm);
    return nivm;
}
```

Cautare informatie + nivel

```
void aux(TArb a, int nivCrt, int* max, int* nivM)
{
    if(!a) return;
    if(a->info >= *max) {
        *max = a->info;
        *nivM = nivCrt;
    }
    aux(a->st, nivCrt+1, max, nivM);
    aux(a->dr, nivCrt+1, max, nivM);
}
```

Media informatiilor din arbore

```
float Media(TArb a, int* n)
{
    int suma = 0;
    *n = 0;

    if(!a) return 0;
    aux(a, &suma, n);
    if(n) return (float)suma/(*n);
}
```

Media informatiilor din arbore

```
void aux(TArb a, int* suma, int* n)
{
    if(!a) return;

    (*suma) += a->info;
    (*n)++;

    aux(a->st, suma, n);
    aux(a->dr, suma, n);
}
```


Media informatiilor de pe nivelurile > niv

```
float Media(TArb a, int* n, int niv)
{
    int suma = 0;
    *n = 0;

    if(!a) return 0;
    aux(a, &suma, n, 0, niv);
    if(*n) return (float) suma / (*n);
}
```

Media informatiilor de pe nivelurile $> n$

```
void aux(TArb a, int* suma, int* n,
        int nivCrt, int niv)
{
    if(!a) return;

    if(nivCrt > niv)
    {
        (*suma) += a->info;
        (*n)++;
    }

    aux(a->st, suma, n, nivCrt+1, niv);
    aux(a->dr, suma, n, nivCrt+1, niv);
}
```

Cautare informatie + nivel

```
int Cauta(TArb a, int x)
```

```
/* intoarce nivelul pe care se afla x sau -1 */
{ int r;
  if(!a) return -1;
  if(a->info == x) return 0;
  r = Cauta(a->st, x);
  if(r < 0)
  {
    r = Cauta(a->dr, x);
    if(r < 0) return -1;
  }
  return r+1;
}
```

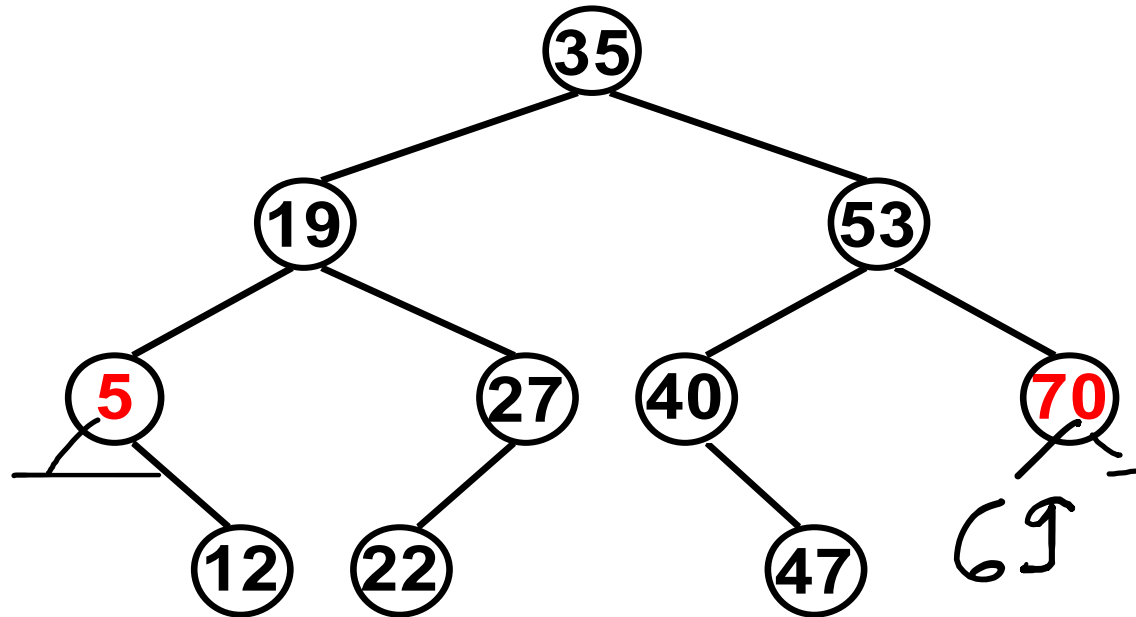
Arbori binari de cautare

Asigura cautarea rapida a nodului cu o anumita cheie.

Un arbore binar nevid este **arbore de cautare** daca indeplineste urmatoarele conditii:

- **cei doi subarbori** (daca exista) sunt, la randul lor, **arbori de cautare**
- **cheia nodului radacina** este **succesoarea** tuturor cheilor din subarborele stang si **predecesoarea** tuturor cheilor din subarborele drept
- $\text{orice info}(S) < \text{info}(R) < \text{orice info}(D)$
- S, D – arbori binari de cautare

Arbore de cautare cu chei intregi

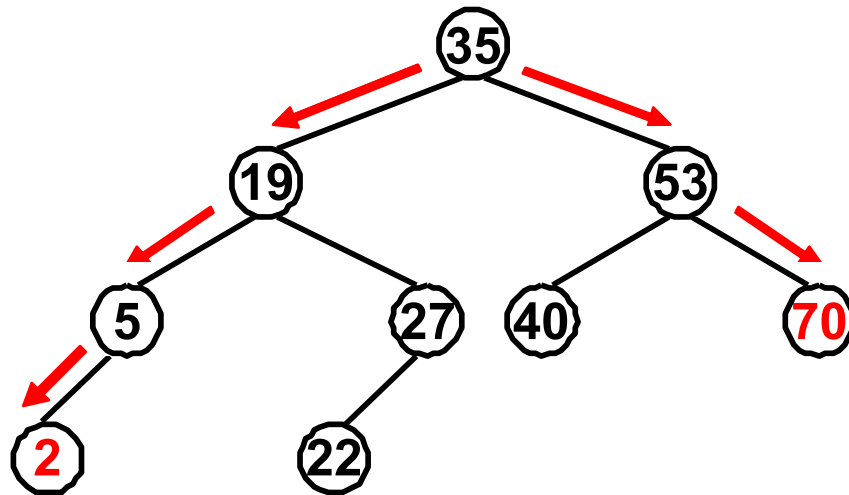


Parcurgerea **S****R****D** $\Rightarrow$ ordine crescatoare

5, 12, 19, 22, 27, 35, 40, 47, 53, 70

Arbore de cautare cu chei intregi

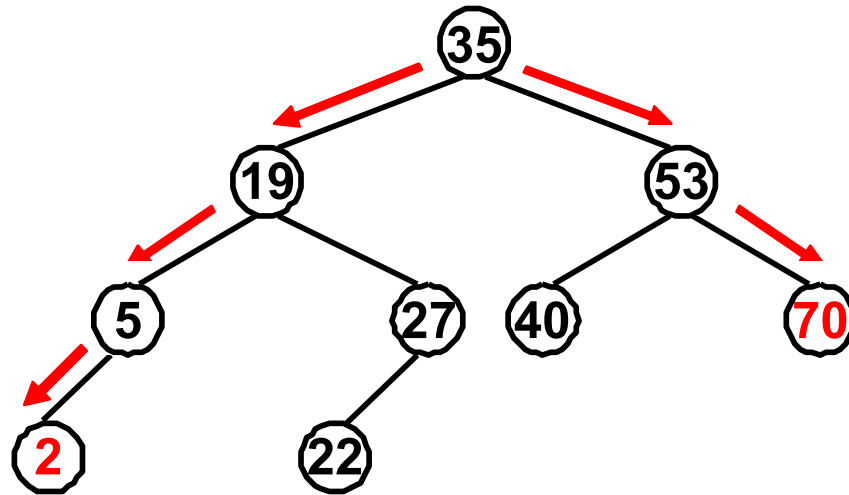
Valorile **minima** dintr-un arbore binar de cautare



```
TArb a;  
while (a->st != NULL)  
    a = a->st;  
//a - nodul cu informatia minima
```

Arbore de cautare cu chei intregi

Valorile **maxima** dintr-un arbore binar de cautare

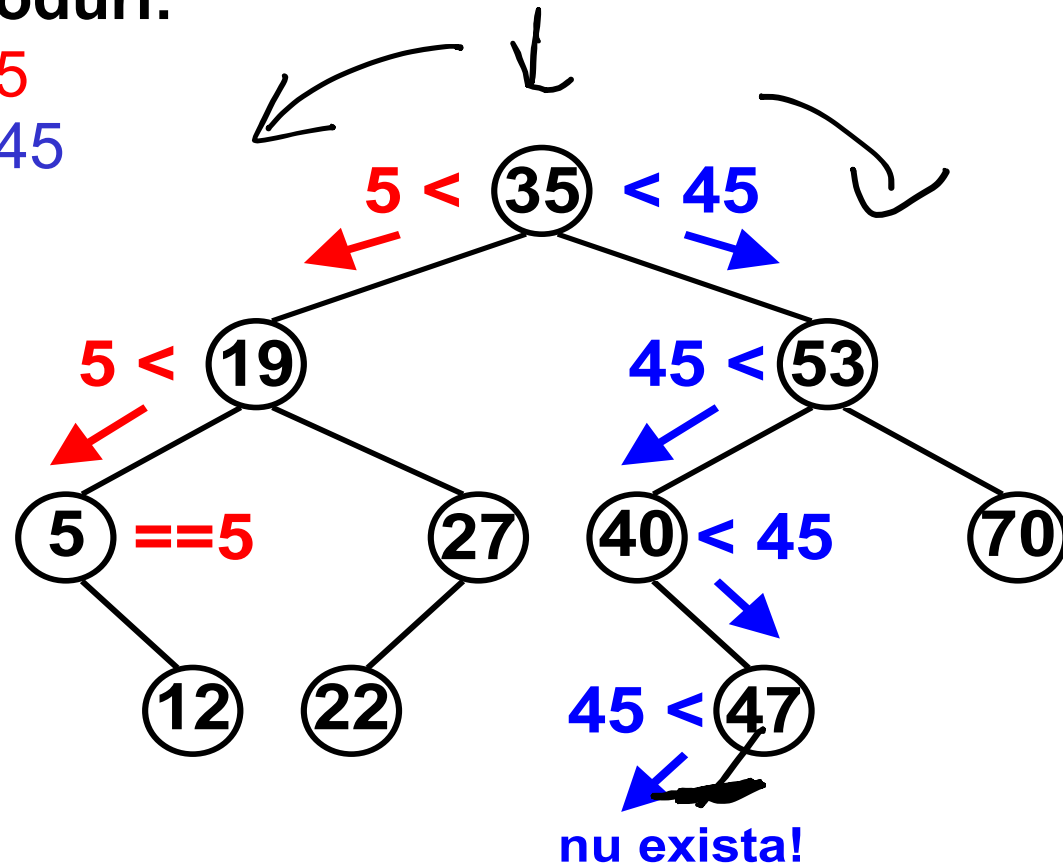


```
TArb a;  
while (a->dr != NULL)  
    a = a->dr;  
//a - nodul cu informatia maxima
```

Arbore de cautare cu chei intregi

Cautare noduri:

- cu cheia **5**
- cu cheia **45**



Numar maxim comparatii = numar niveluri

Arbore de cautare cu chei intregi

Cautarea nodului cu cheie data

```
TArb Cauta(TArb a, int cheie)
//NULL daca nu exista cheie in a
//nodul gasit
{ while (a) /* cat timp exista nod */
  {
    if (a->info == cheie) /* gasit */
      return a;
    if (cheie < a->info)
      a = a->st; /* deplasare stanga */
    else
      a = a->dr; /* deplasare dreapta */
  }
  return NULL;
}
```

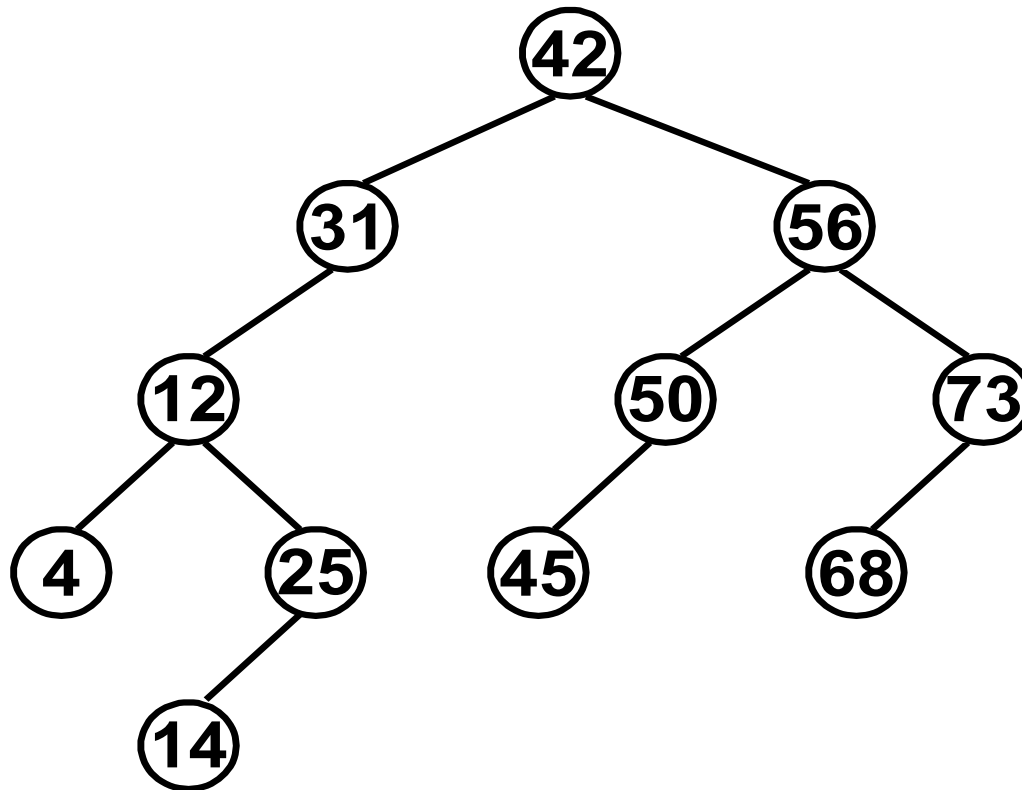
Arbore de cautare cu chei intregi

Cautarea nodului cu cheie data – cautare recursiva

```
TArb Cauta(TArb a, int cheie)  
{  
    if(!a) return 0;  
  
    if(a->info == x) return 1;  
  
    if(x < a->info) return Cauta(a->st, x);  
  
    return Cauta(a->dr, x);  
}
```

Construire arbore binar de cautare

Secventa chei: 42, 31, 56, 73, 12, 25, 50, 4, 45, 14, 68



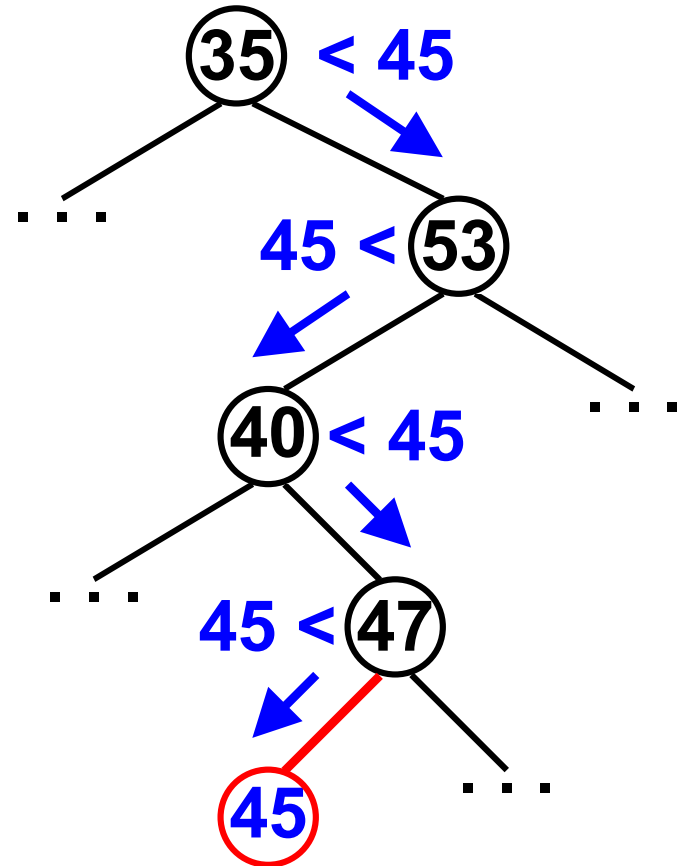
Inserare in arbore binar de cautare

Etape:

- cautarea locului de inserare
- inserarea propriu-zisa

Cazuri speciale:

- cheia de inserat
exista deja $\Rightarrow -1$
- nu exista spatiu
pentru un nou nod $\Rightarrow 0$



Inserare in arbore binar de cautare

```
int Insert_ABC(TArb* a, int x)
{
    TArb aux, n = *a, p = NULL;
    //n - parcurge arb, p - parintele nodului de ins
    int s; //fiu in care se face ins

    while (n)
    {
        if ( x == n->info) return -1;
        if ( x < n->info)
        {
            p = n; s = 0; n = n->st;
        }
        else
        {
            p = n; s = 1; n = n->dr;
        }
    }
}
```

Inserare in arbore binar de cautare

```
/* inserare */  
    aux = ConstrFrunza(x);  
    if(!aux) return 0; /* intoarce 0 */  
  
    if(!p) *a = aux;  
    else  
    {  
        if(s) p->dr = aux;  
        else p->st = aux;  
    }  
  
    return 1; /* intoarce 1 */  
}
```

Inserare recursiva in arbore binar

```
TArb Insert_ABC_R(TArb a, int x)
{
    if (a == NULL) return ConstrFrunza(x);

    if (x < a->info)
        a->st = Insert_ABC_R(a->st, x);
    else
        a->dr = Insert_ABC_R(a->dr, x);

    return a;
}
```

Inserare in arbore binar de cautare

```
int Insert_ABC(TArb *aa, int cheie)
{ /* cautare */
    while (*aa)
    { if ( cheie == (*aa)->info) return -1;
      if ( cheie < (*aa)->info)
          aa = &((*aa)->st);
      else
          aa = &((*aa)->dr);
    }

    /* inserare */
    *aa = ConstrFrunza(cheie);
    if(!*aa) return 0; /* intoarce 0 */
    return 1; /* intoarce 1 */
}
```


Constructie frunza

```
TArb ConstrFrunza(int x)
```

```
{
```

```
    TArb aux = (TArb)malloc(sizeof(TNod));
```

```
    if (!aux) return NULL;
```

```
    aux->info = x;
```

```
    aux->st = NULL;
```

```
    aux->dr = NULL;
```

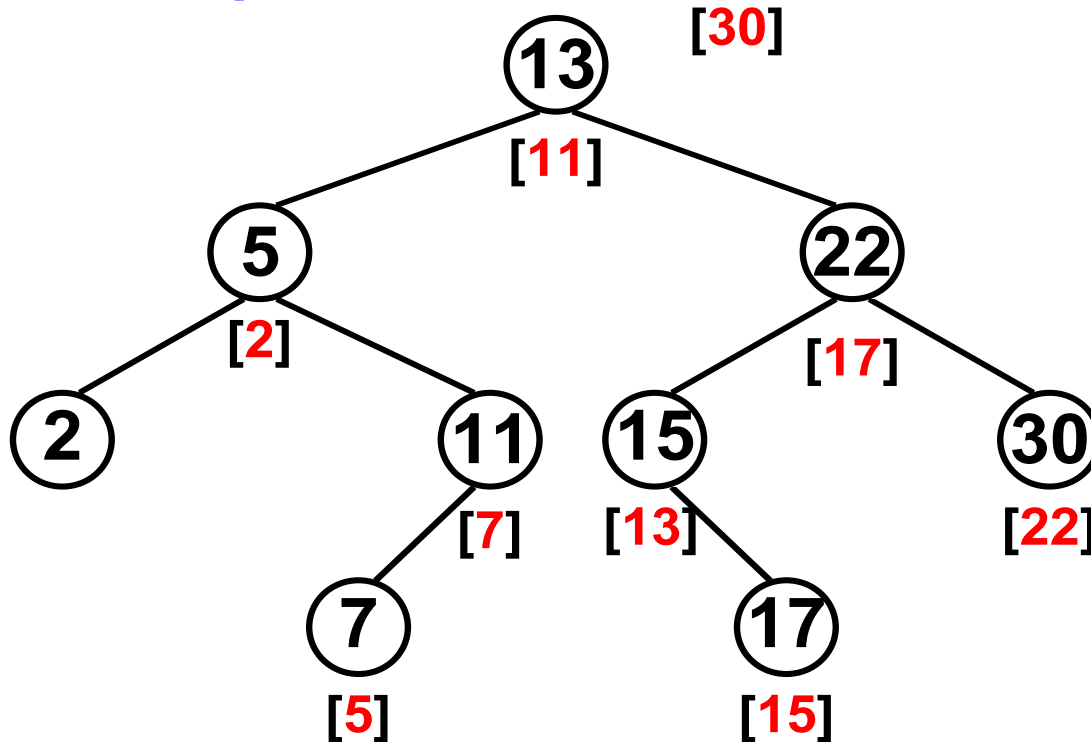
```
    return aux;
```

```
}
```

Test arbore binar de cautare

Bazata pe parcurgerea SRD:

- informatia din radacina (sub)arborelui analizat trebuie sa fie **> precedenta informatie analizata.**



Test arbore binar de cautare

Functia de test, apelata de utilizator cu un **singur parametru - arborele analizat**, initializeaza variabila care va memora ultima informatie analizata si apeleaza o functie auxiliara recursiva, cu doi parametri:

- **adresa (sub)arborelui analizat**
- **adresa variabilei de referinta**

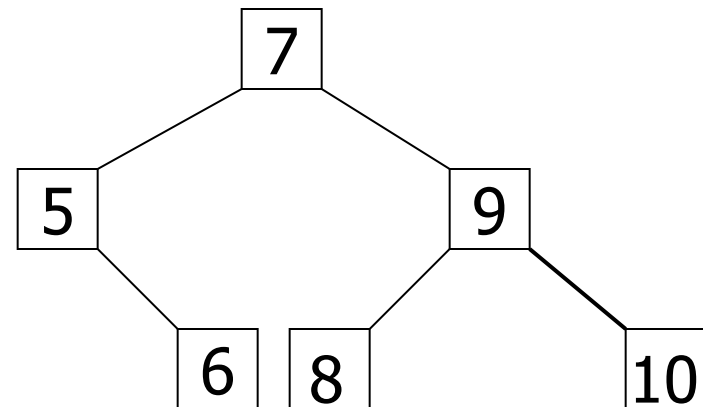
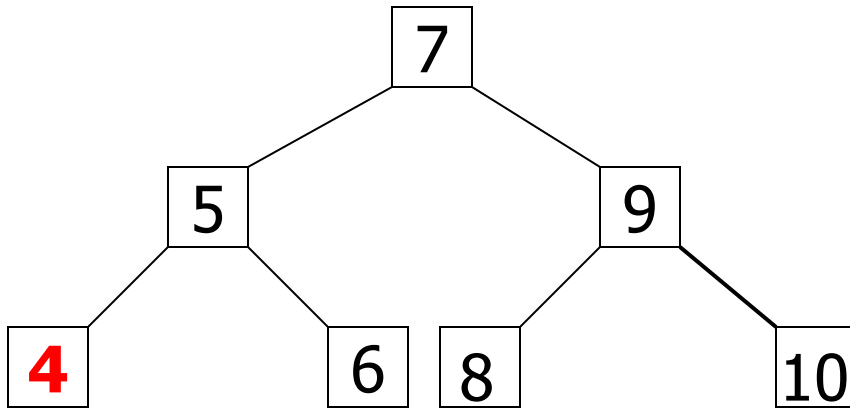
```
int TestAC(TArb a)
{ int ref;
  if (!a) return 1;
  ref = MinAC(a) - 1;
  return aux(a, &ref);
}
```

Test arbore binar de cautare

```
int aux(TArb a, int *p)  
{ /* test subarbore stang */  
    if (a->st)  
        if (!aux(a->st, p)) return 0;  
  
    /* test relatie de ordine */  
    if (a->info <= *p) return 0;  
  
    *p = a->info; /* actualizare referinta */  
  
    if (a->dr) /* test subarbore drept */  
        return aux(a->dr, p);  
    else  
        return 1;  
}
```

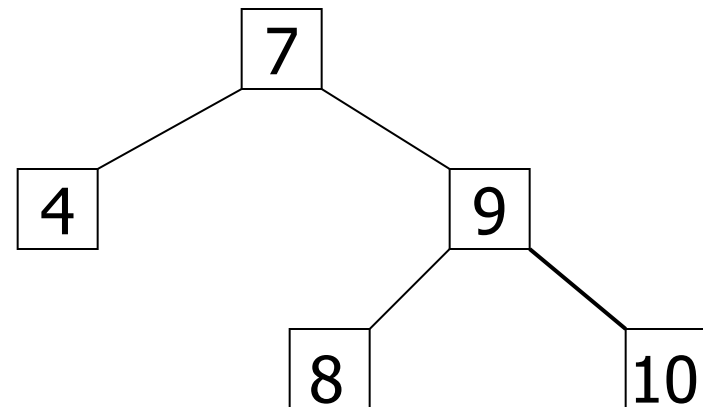
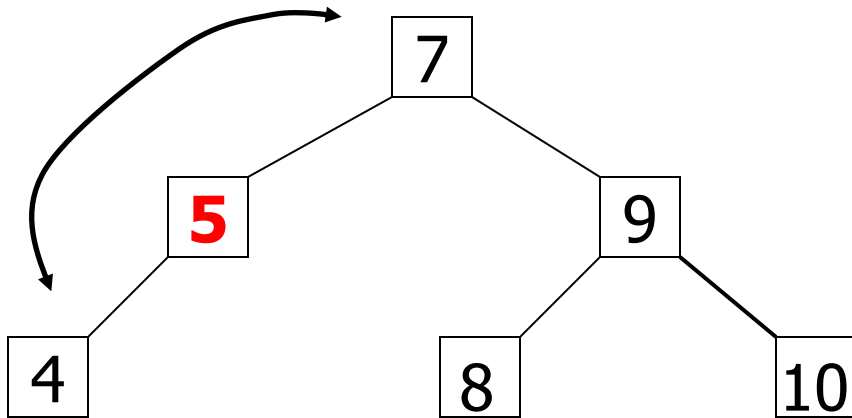
Eliminare nod

Eliminare frunza – eliminare nod cu info **4**



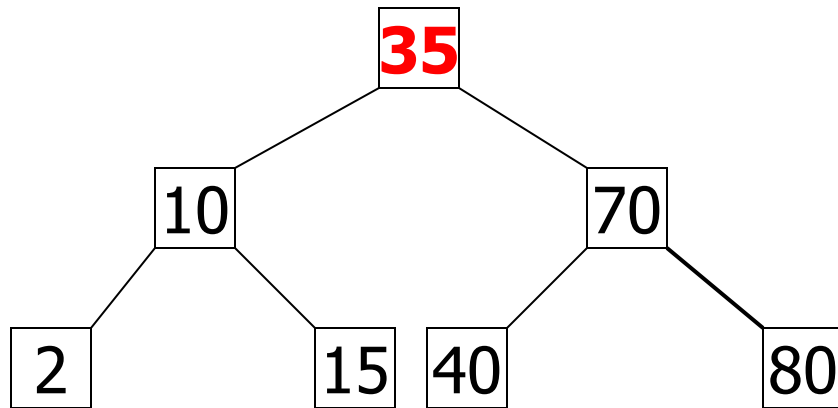
Eliminare nod

Eliminare nod de ordin 1 – eliminare nod cu info **5**

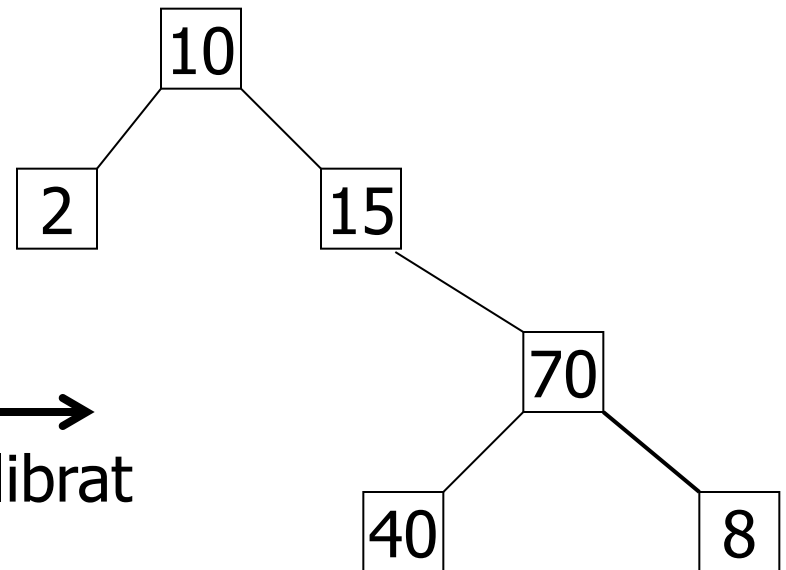


Eliminare nod

Eliminare nod de ordin 2: nodul cu info **35**

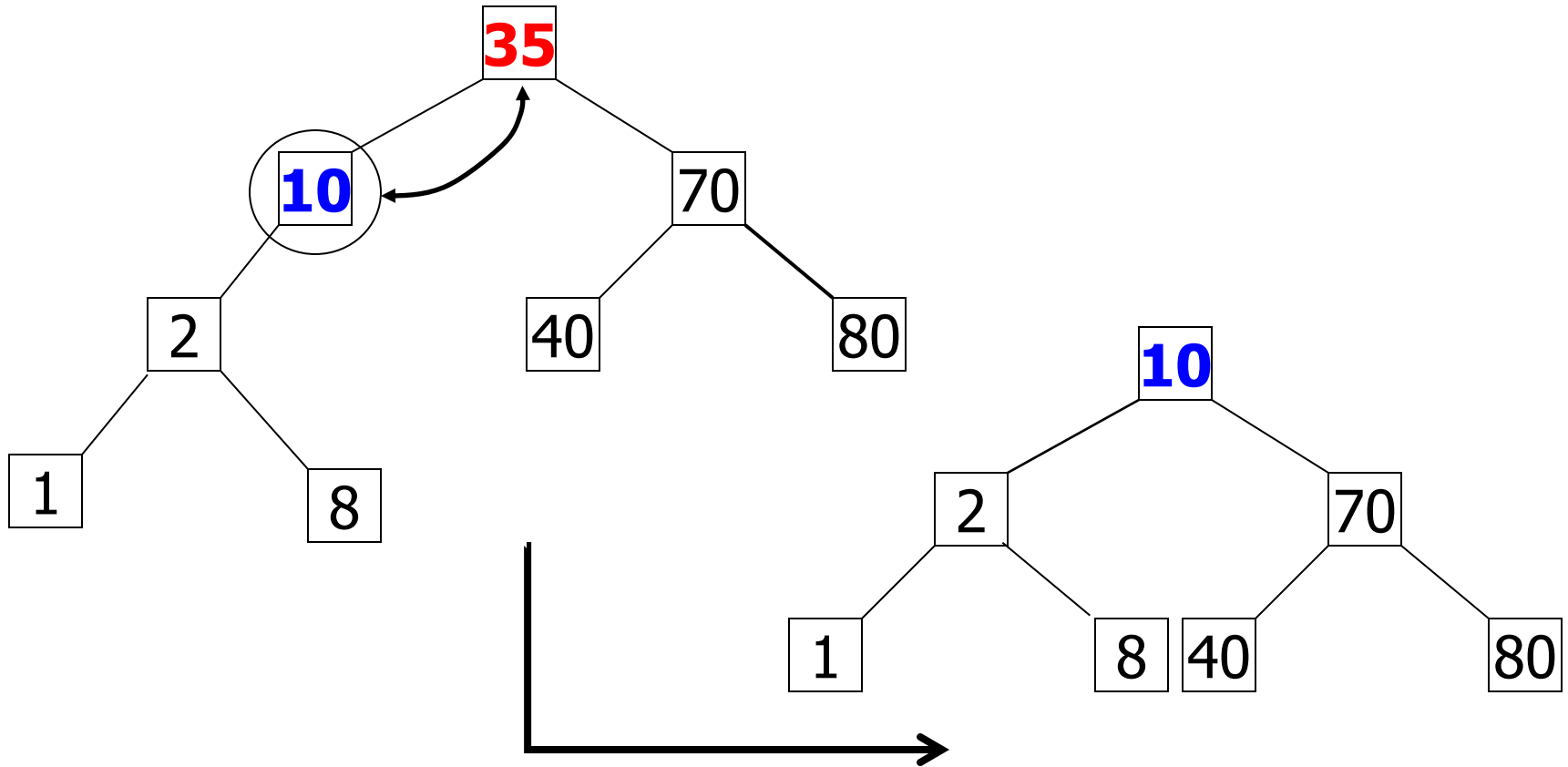


Arbore dezechilibrat



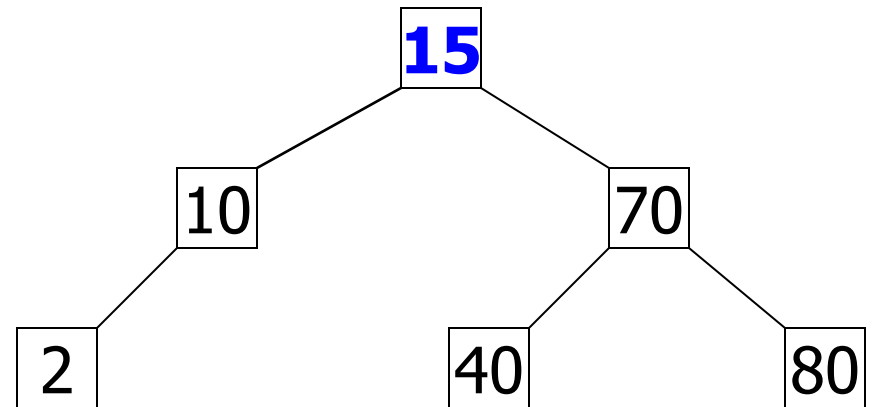
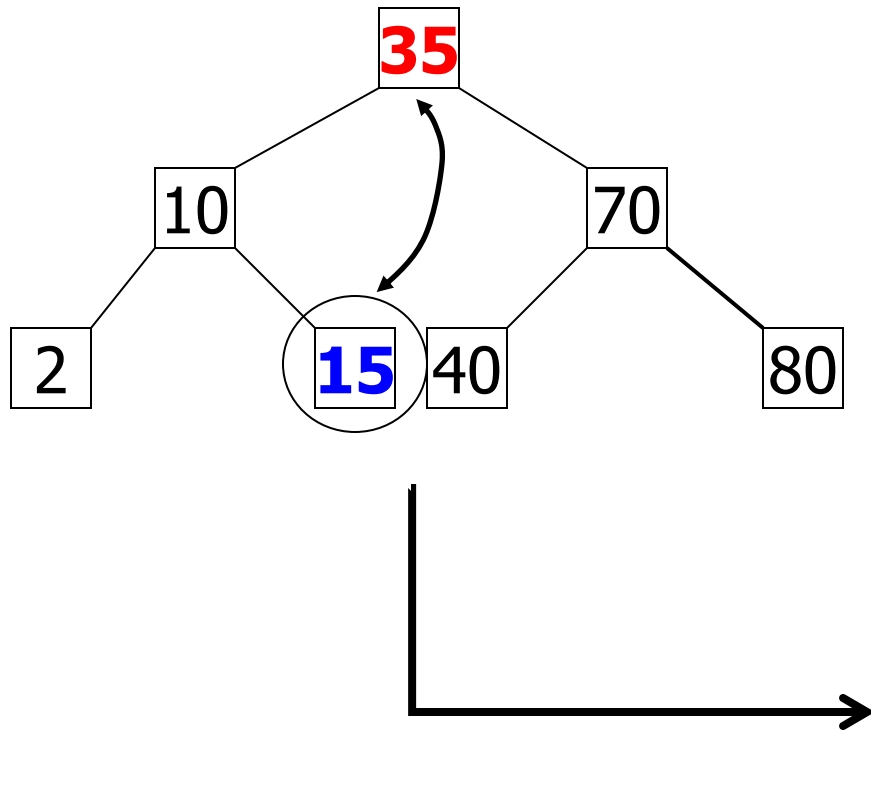
Eliminare nod

Eliminare nod de ordin 2: nodul cu info **35**



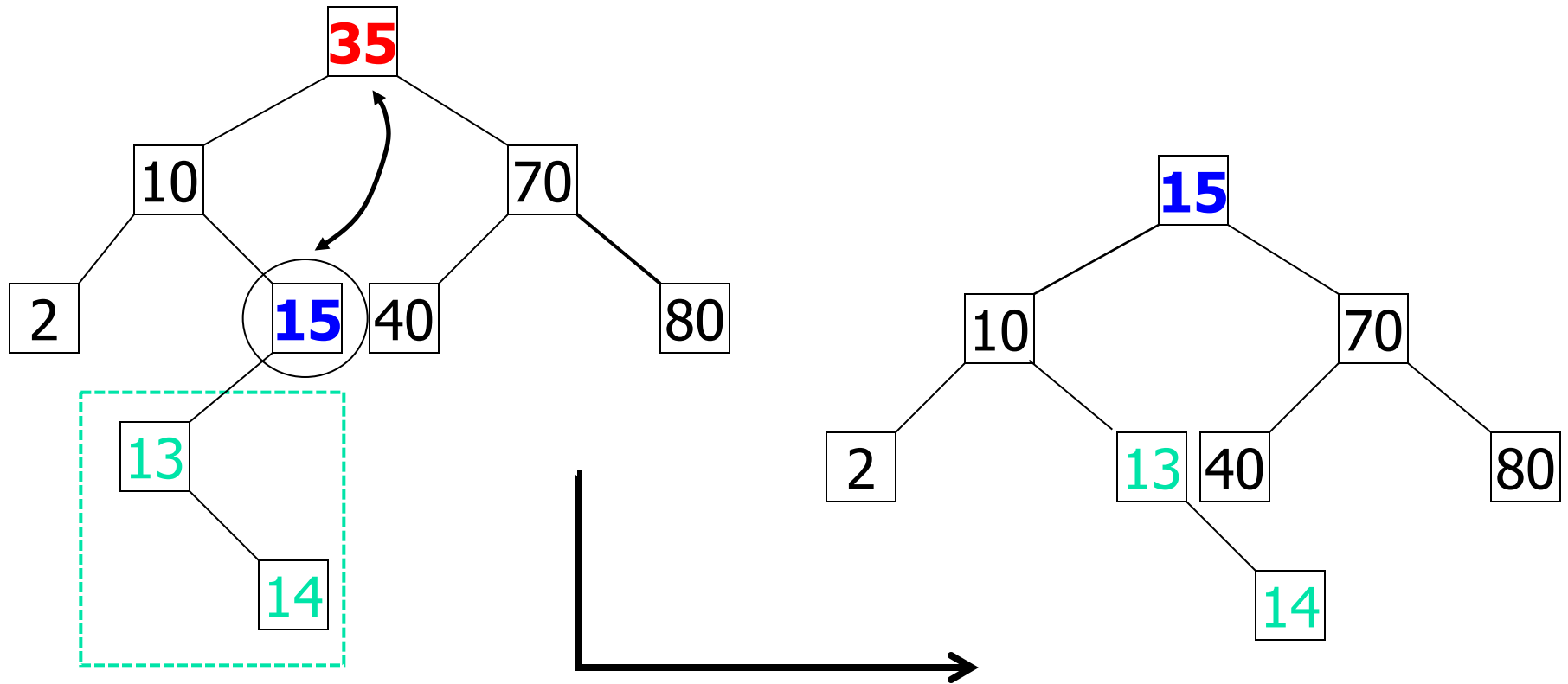
Eliminare nod

Eliminare nod de ordin 2 – eliminare nod cu info **35**



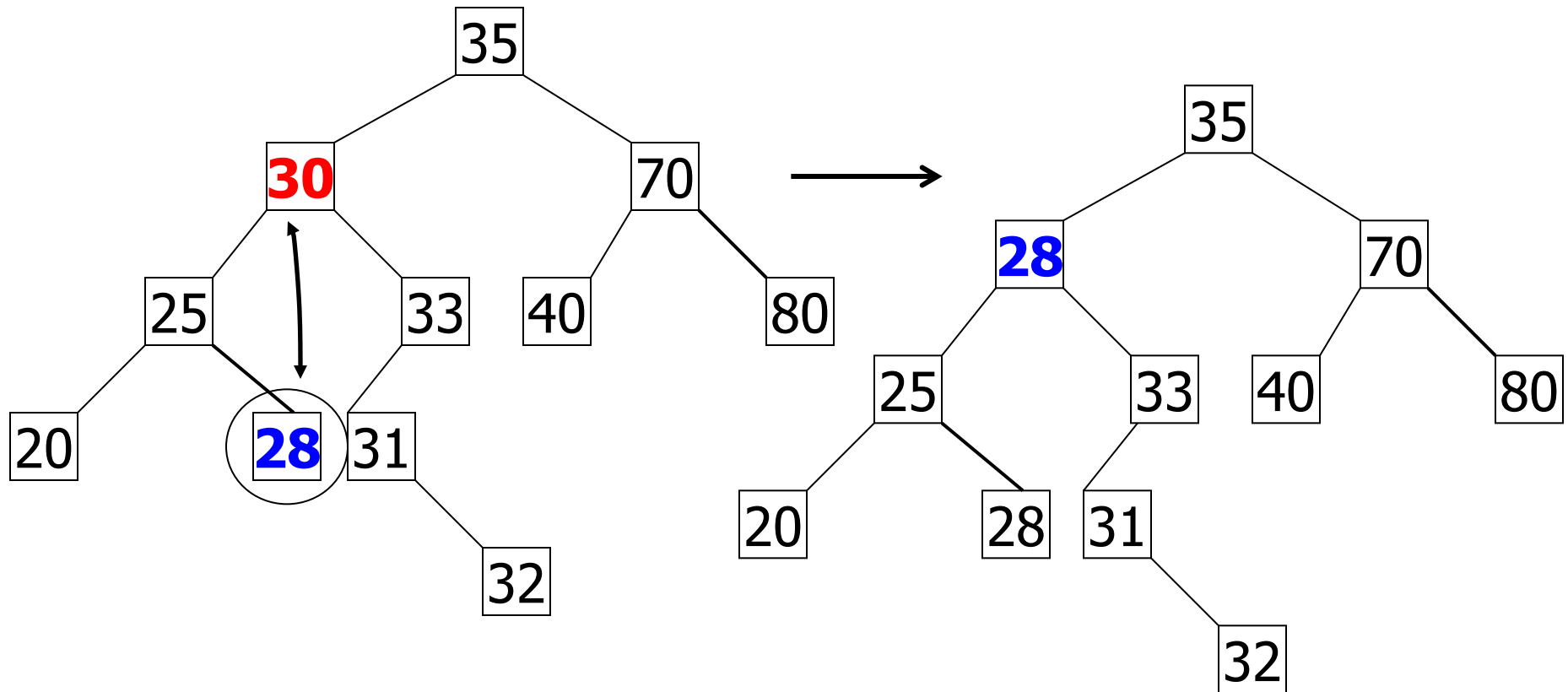
Eliminare nod

Eliminare nod de ordin 2 – eliminare nod cu info **35**



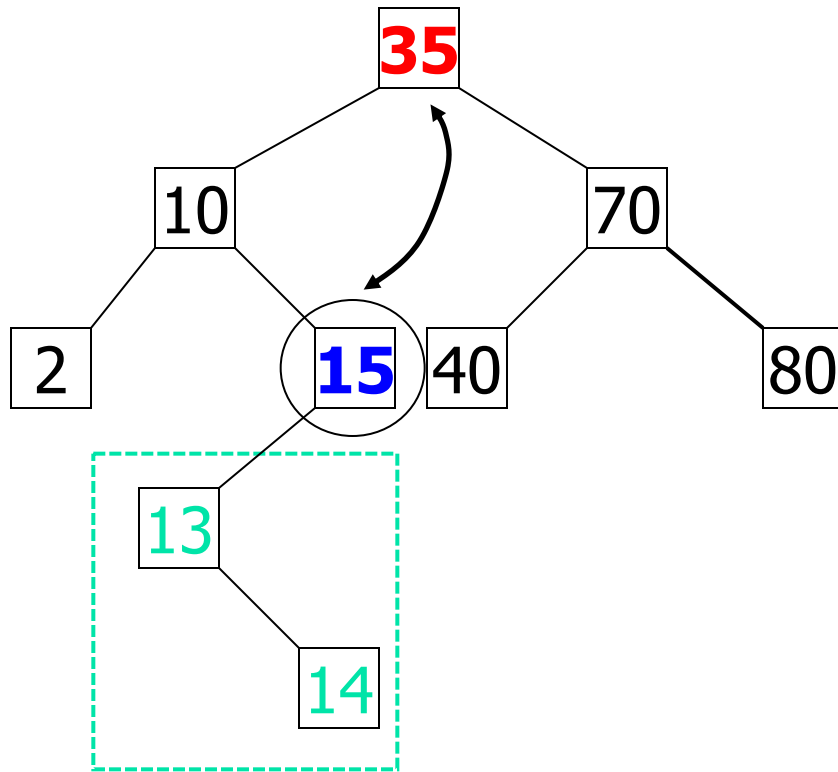
Eliminare nod

Eliminare nod de ordin 2 – eliminare nod cu info **30**



Eliminare nod

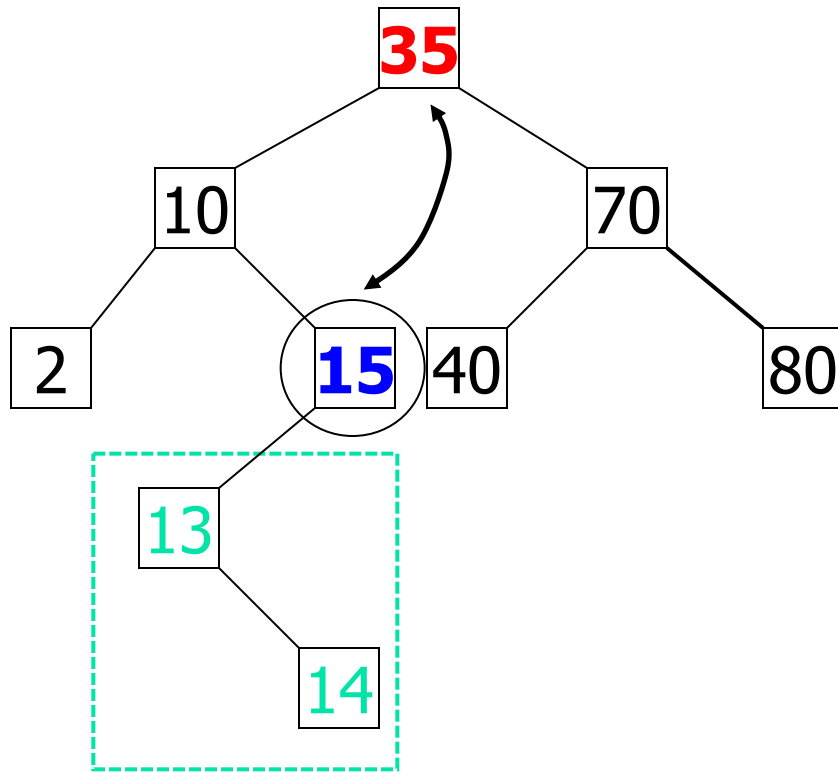
Eliminare nod de ordin 2 – eliminare nod cu info **35**



1. Localizare nod de elim (n) + parinte (p)
2. Daca $\text{ordin}(n) = 0$ sau 1
 1. Actualizeaza fiul parintelui p

Eliminare nod

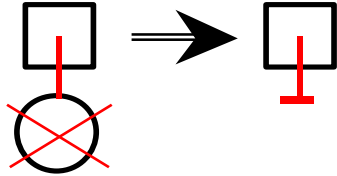
Eliminare nod de ordin 2 – eliminare nod cu info **35**



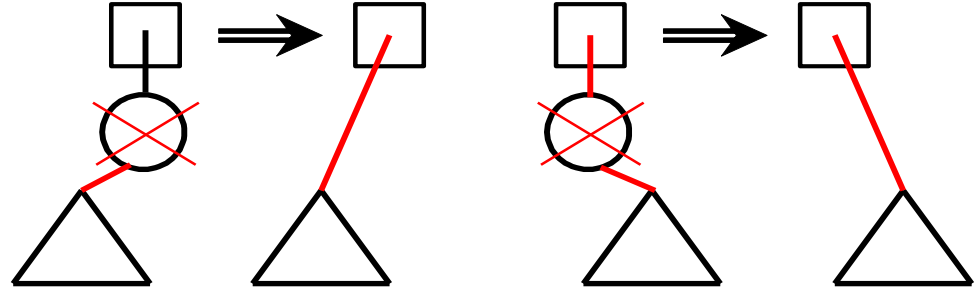
1. Localizare nod de elim (n) + parinte (p)
2. Daca $\text{ordin}(n) = 0$ sau 1
 1. Actualizeaza fiul parintelui p
3. Daca $\text{ordin}(n) = 2$
 1. Pivot = max(subarb stg pt n) + parinte pivot
 2. Plaseaza in n info din pivot
 3. Actualizeaza fiu parinte pivot

Eliminare nod

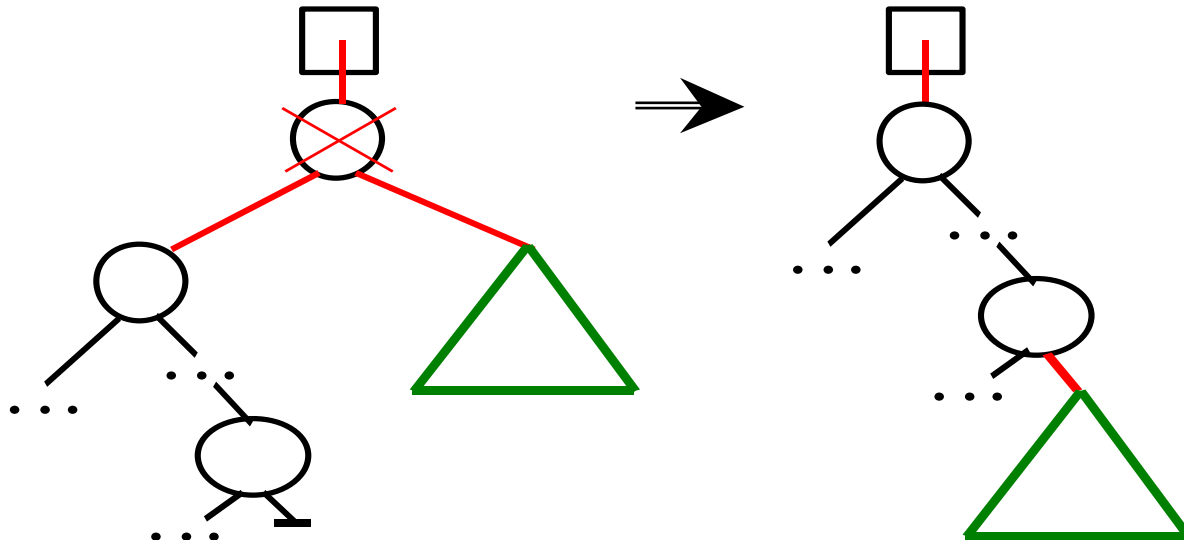
- frunza



- de ordin 1

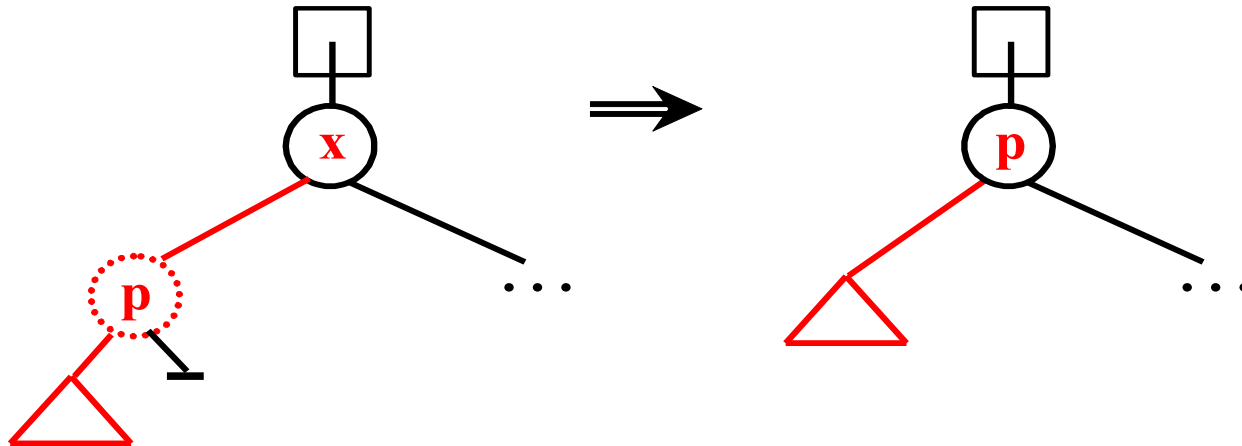


- de ordin 2 – eliminare cu posibila dezechilibrare



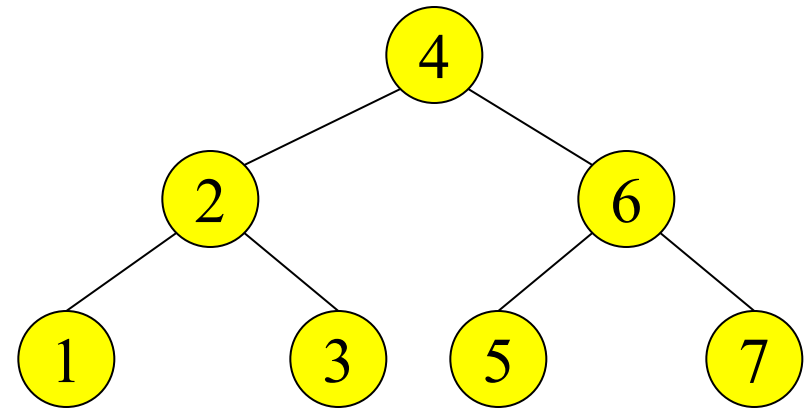
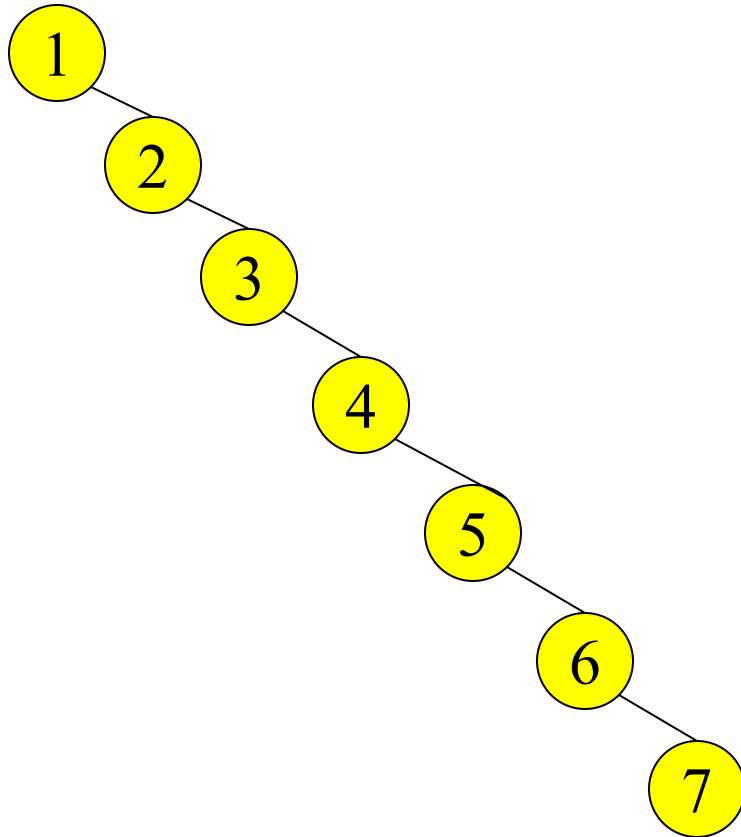
Eliminare nod

- de ordin 2 – eliminare cu pivot stanga, caz particular



Arbori binari de cautare

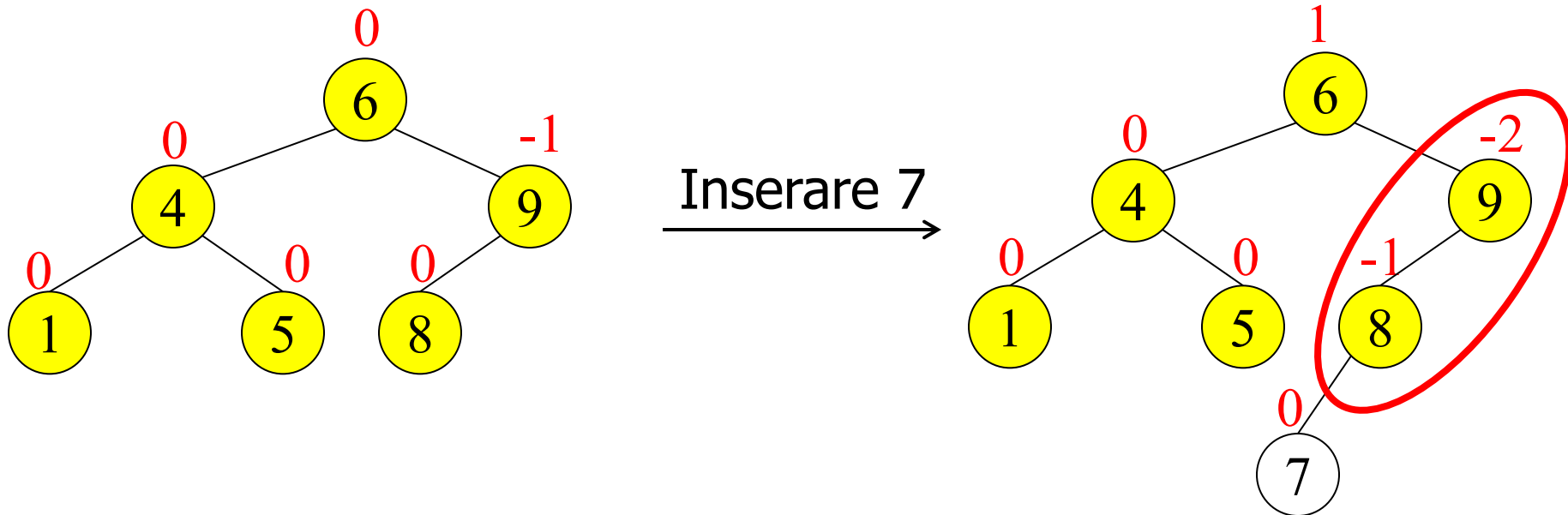
Constructie arbore binar de cautare: 1, 2, 3, 4, 5, 6, 7



Arbore echilibrat

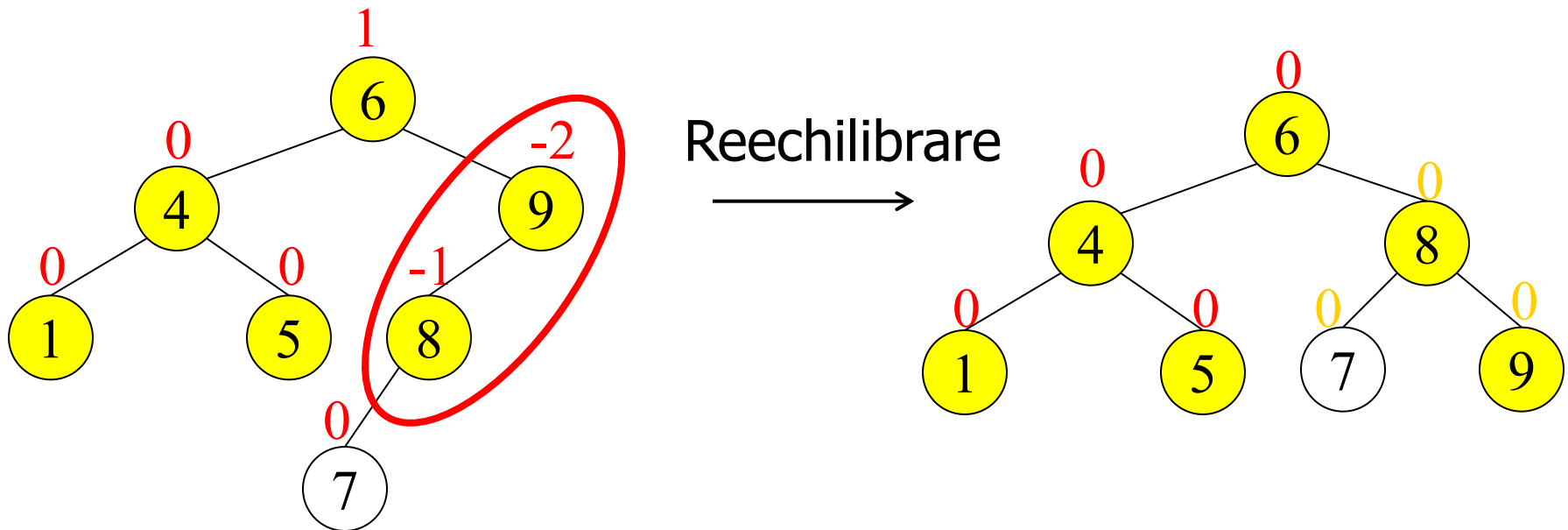
Arbori binari de cautare AVL

- Pentru orice nod **diferenta** intre **inaltimile** celor 2 subarbori este < 2
- Inserarea si eliminarea pot necesita **reechilibrarea arborelui**; in acest scop se utilizeaza **factorul de echilibru** = $H(dr) - H(st)$



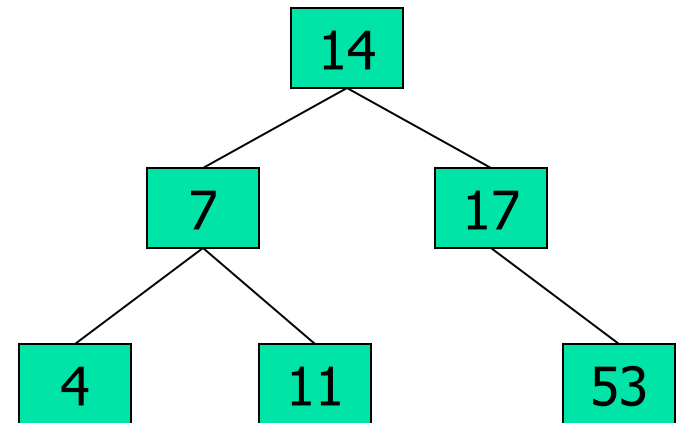
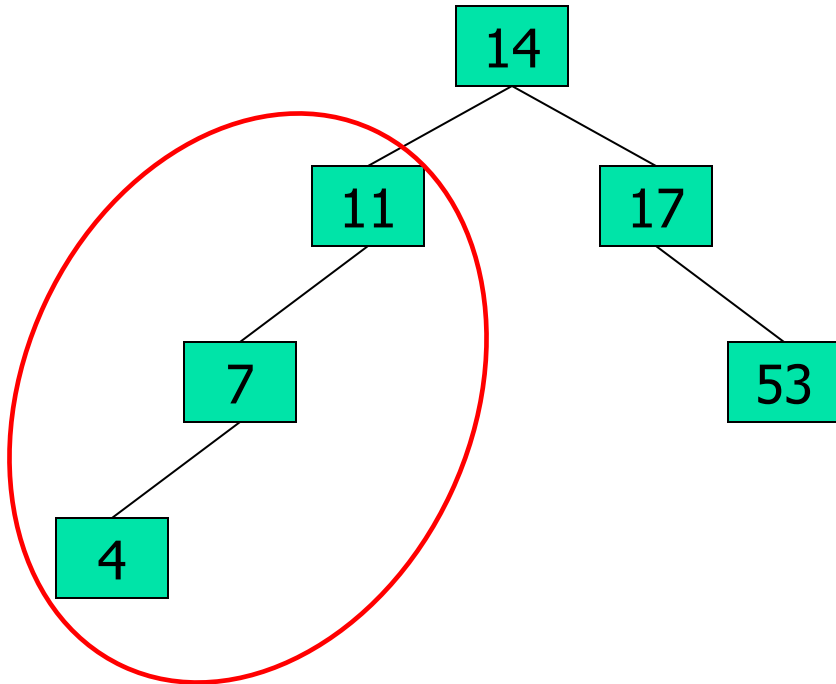
Arbori binari de cautare echilibrati - AVL

- Pentru orice nod **diferenta** intre **inaltimile** celor 2 subarbori este < 2
- Inserarea si eliminarea pot necesita **reechilibrarea** arborelui; in acest scop se utilizeaza **factorul de echilibru** = $H(dr) - H(st)$

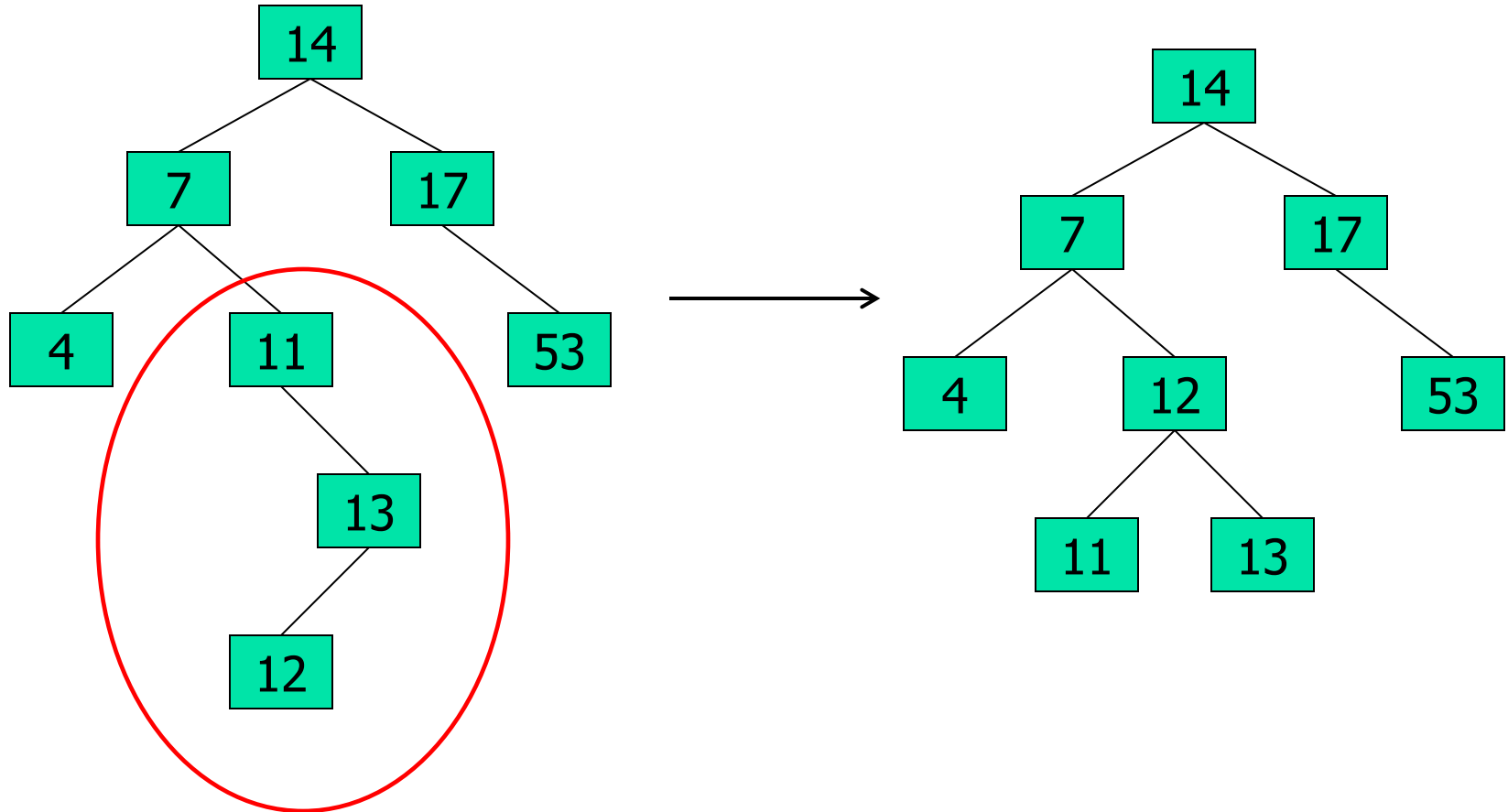


Arbori binari de cautare echilibrati - AVL

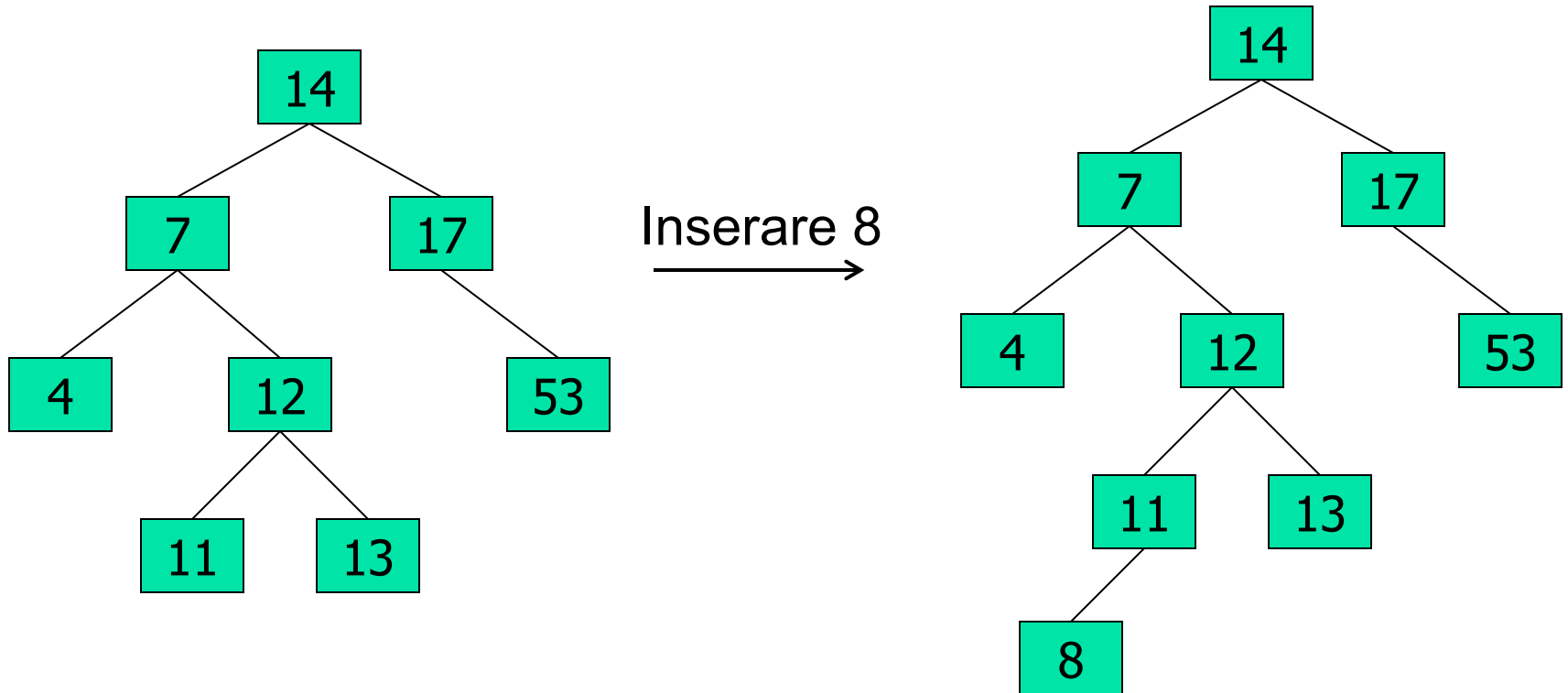
Inserare: 14, 17, 11, 7, 53, 4



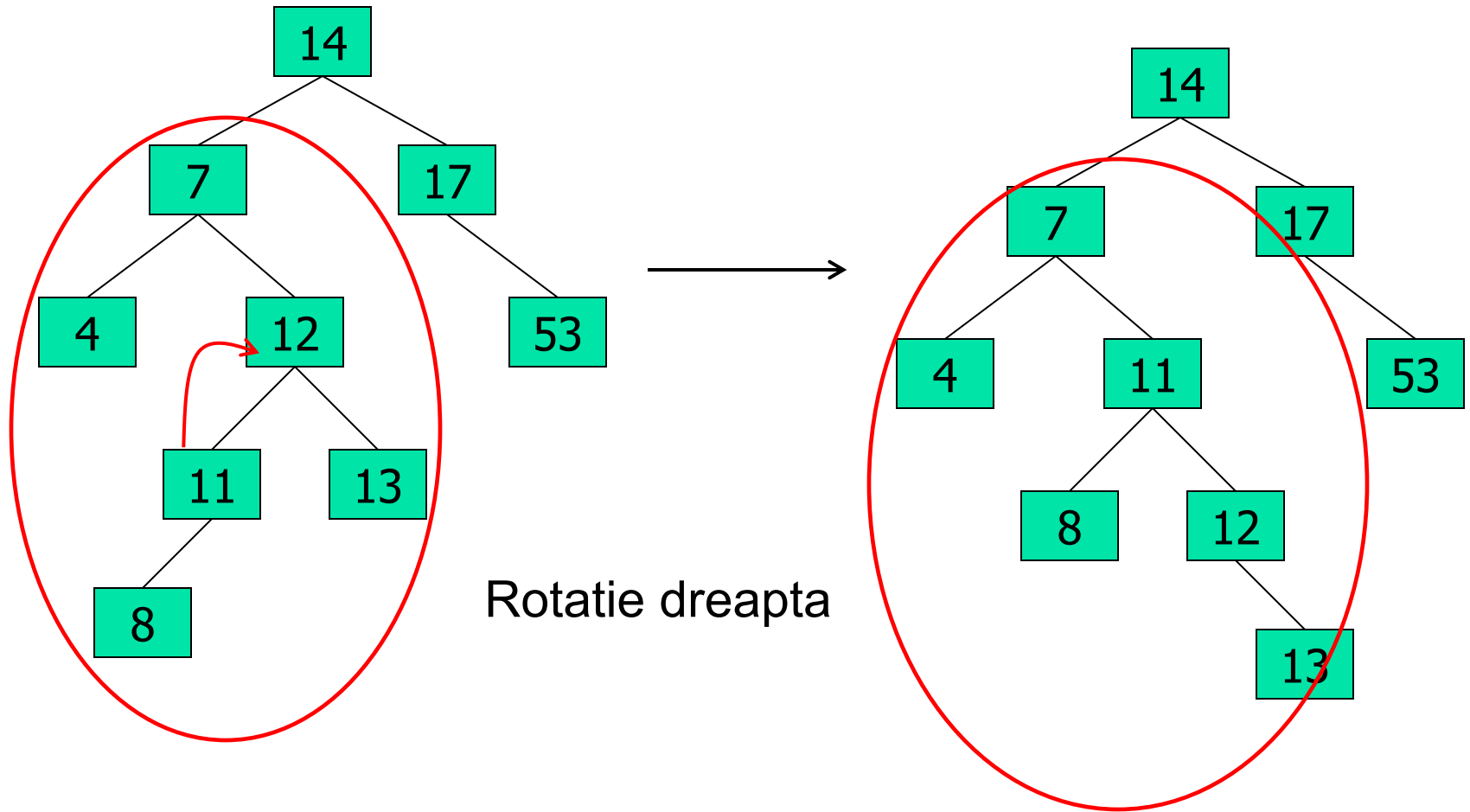
Arbori binari de cautare echilibrati - AVL



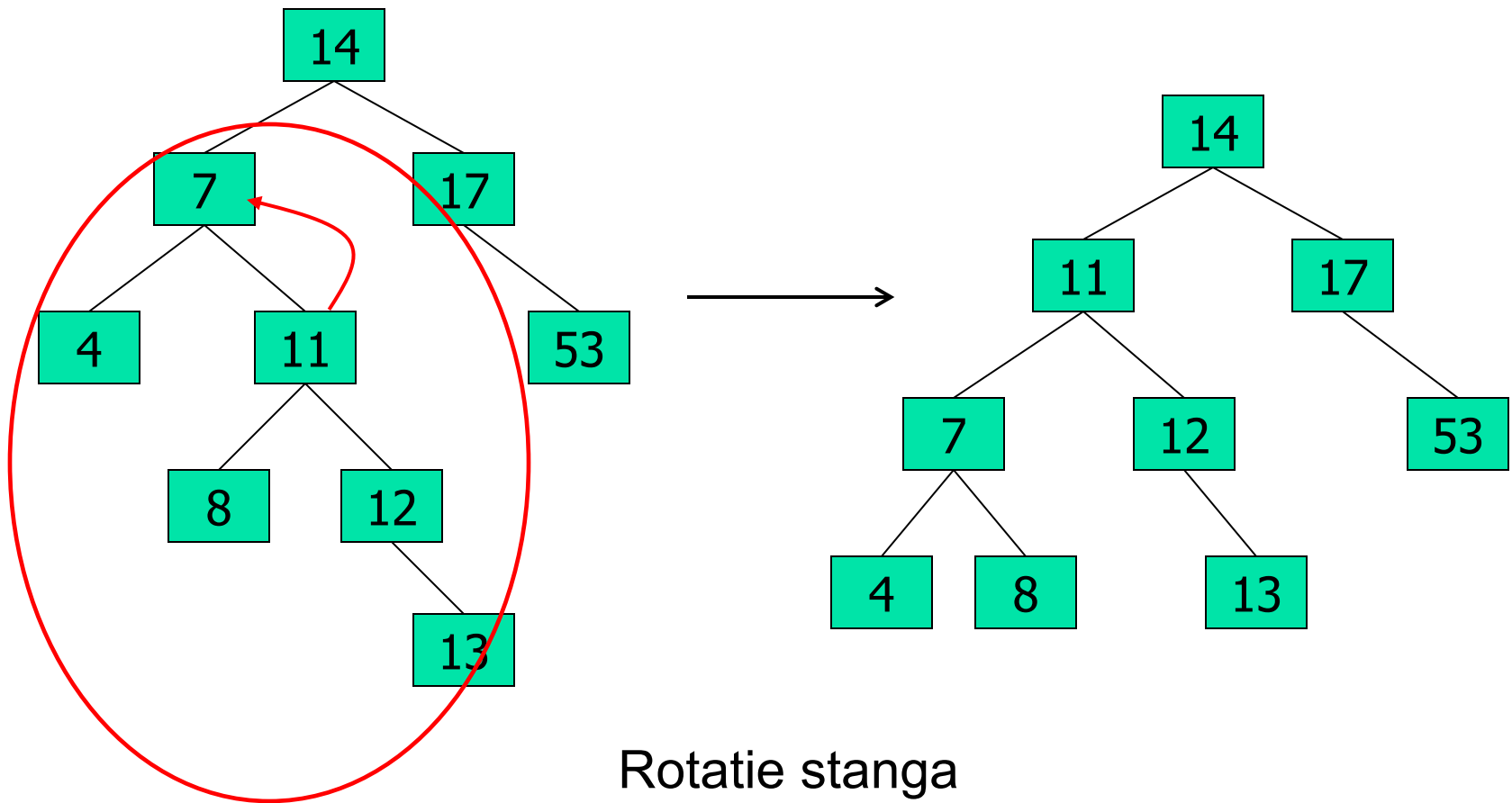
Arbori binari de cautare echilibrati - AVL



Arbori binari de cautare echilibrati - AVL

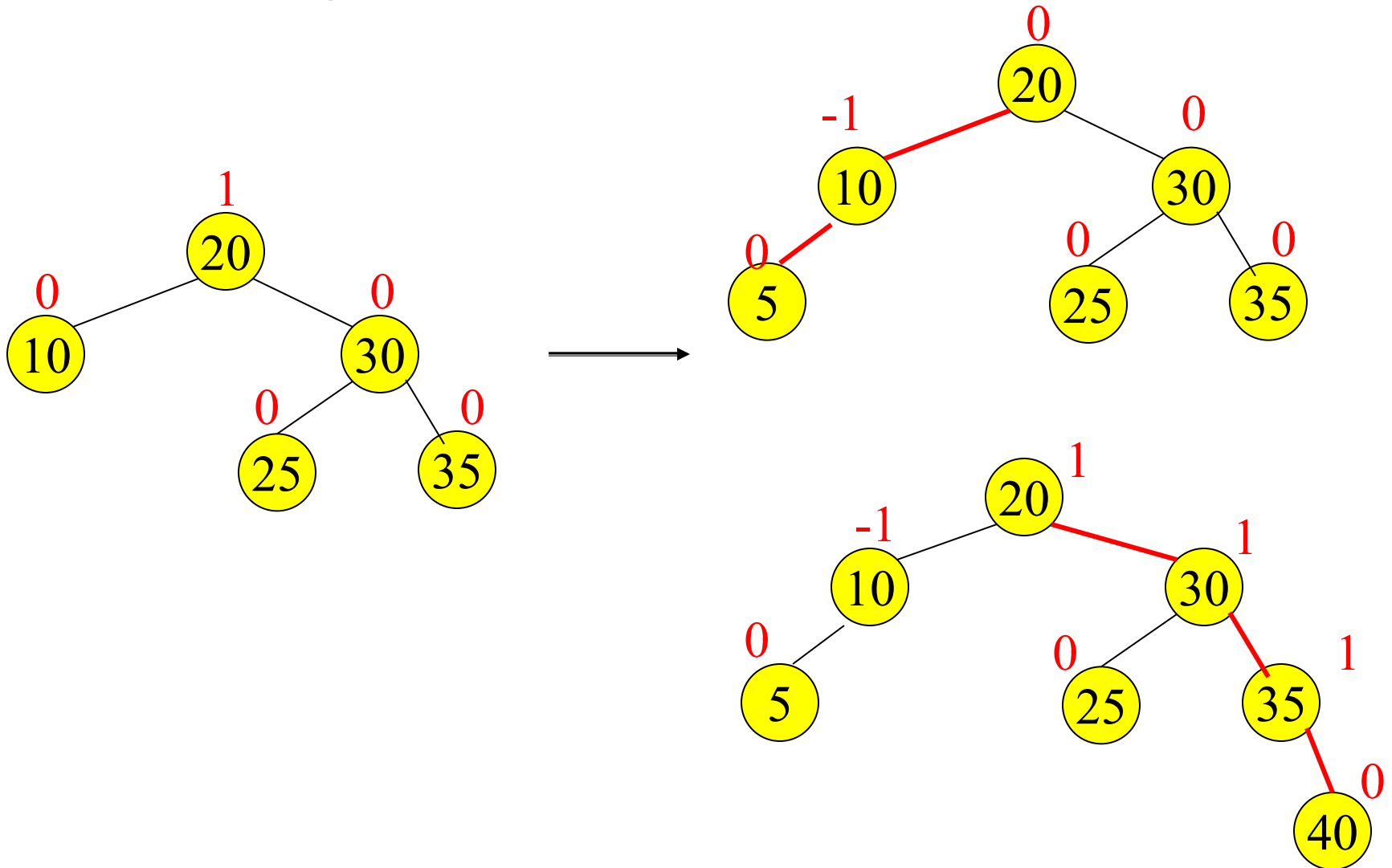


Arbori binari de cautare echilibrati - AVL



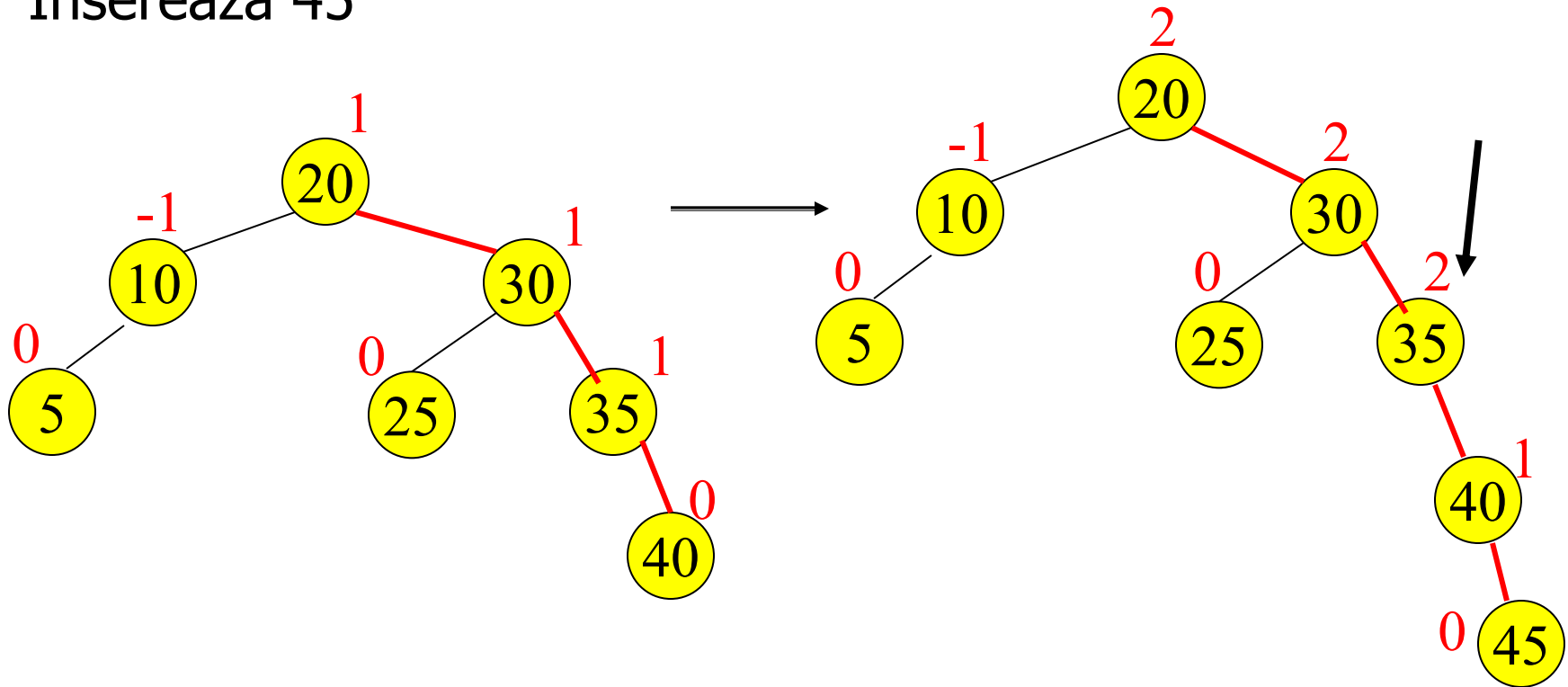
Exemplu: Arbori AVL

Insereaza 5, 40



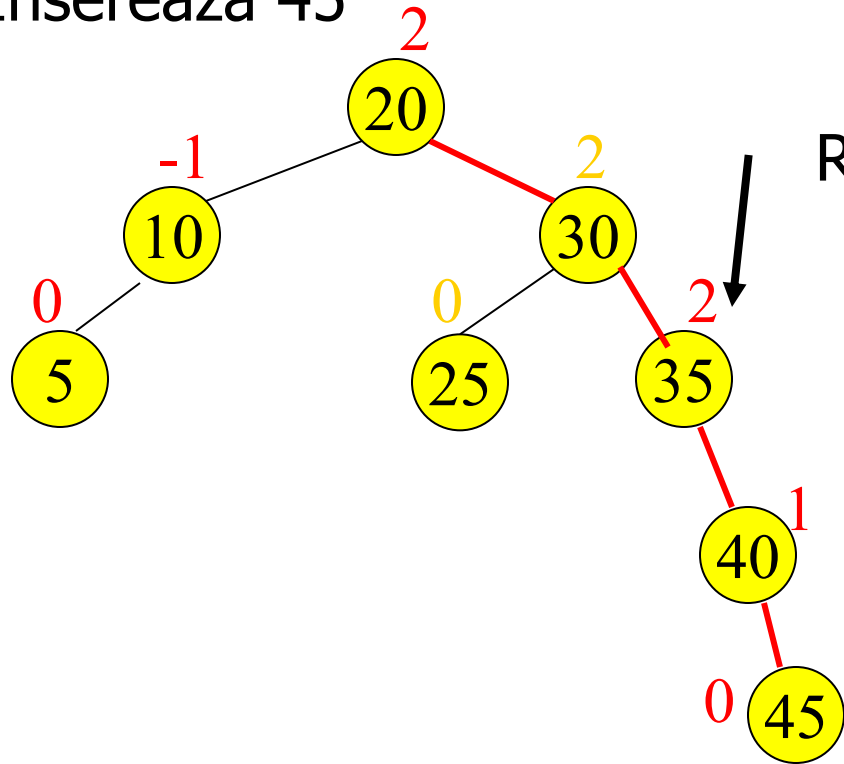
Exemplu: Arbori AVL

Insereaza 45

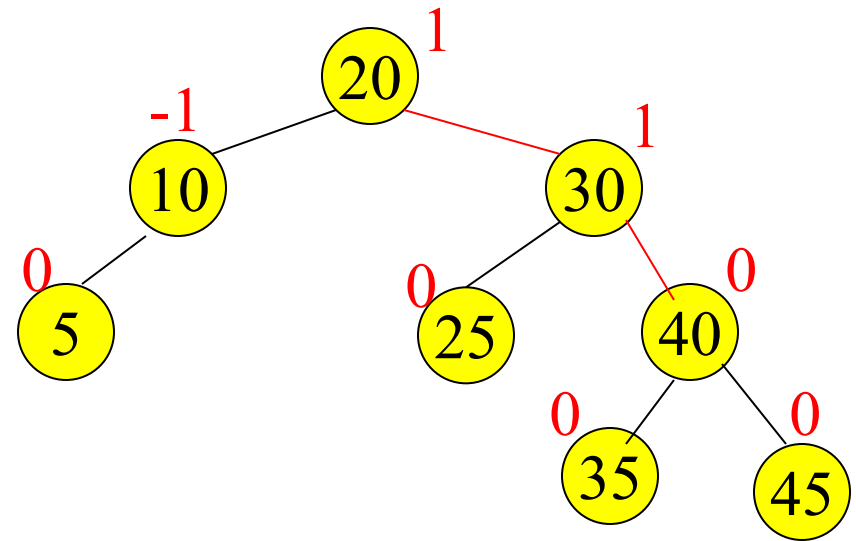


Exemplu: Arbori AVL

Insereaza 45

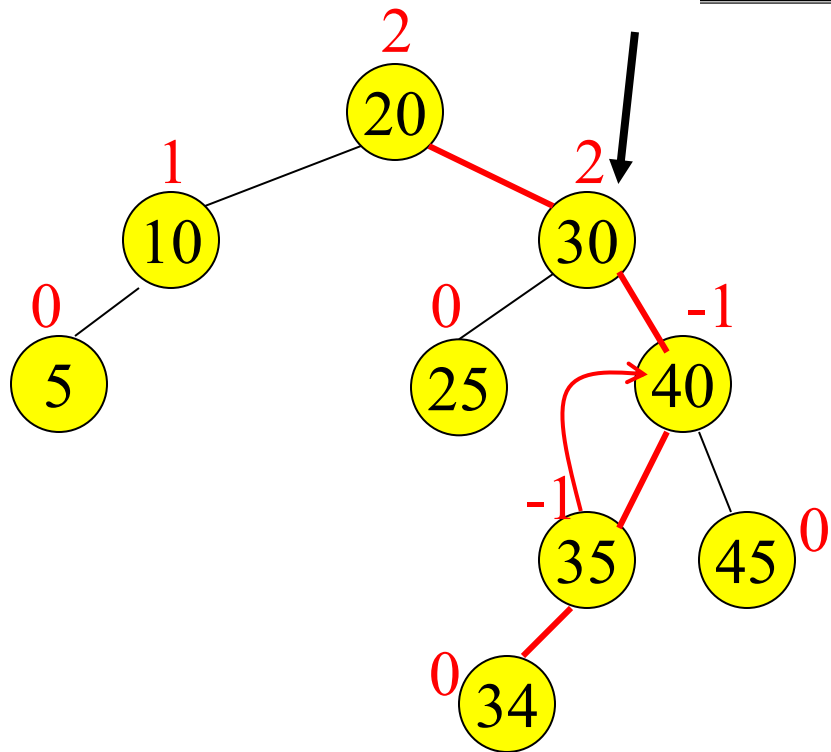


Rotatie stanga

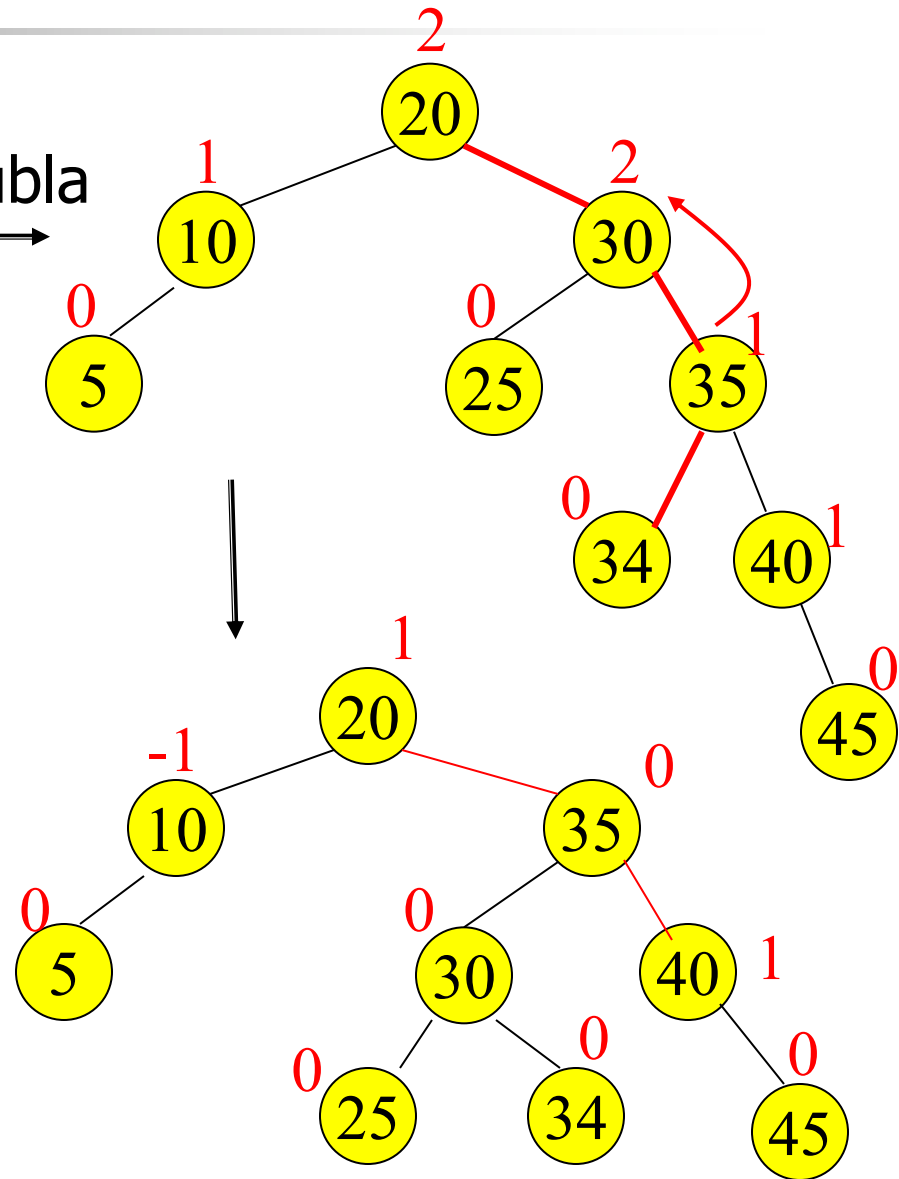


Exemplu: Arbori AVL

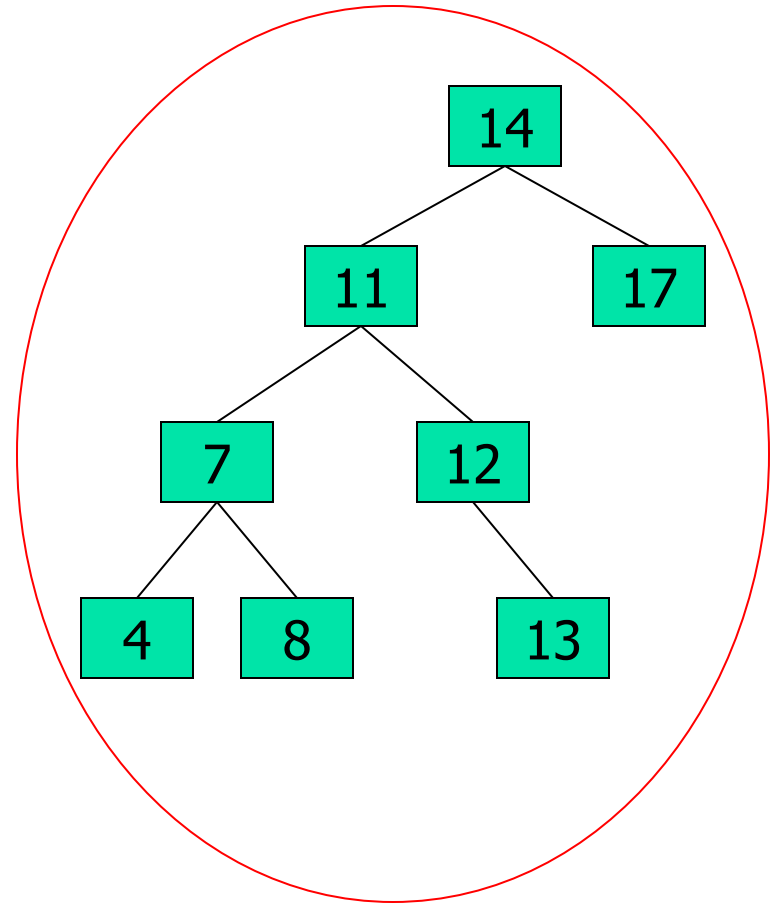
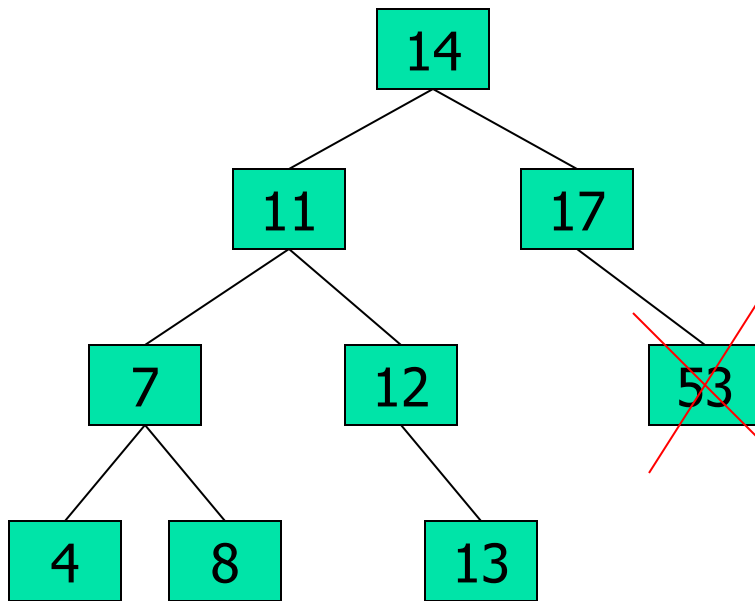
Inserare 34



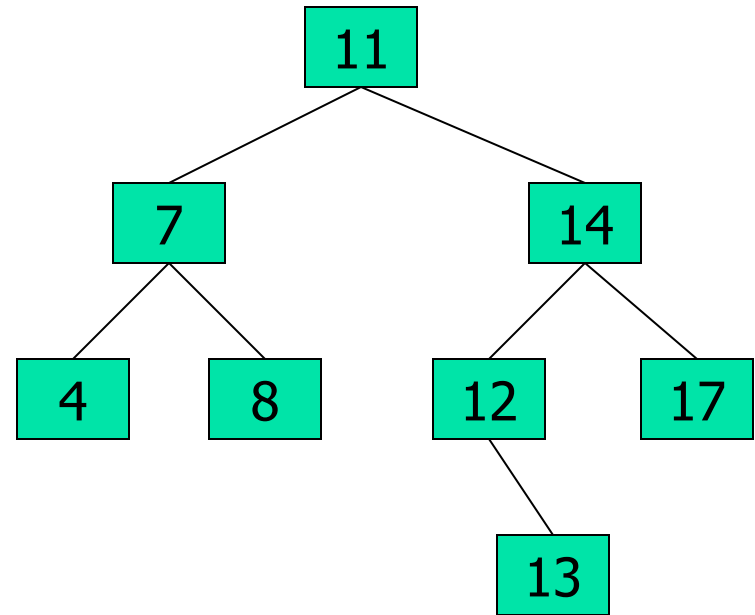
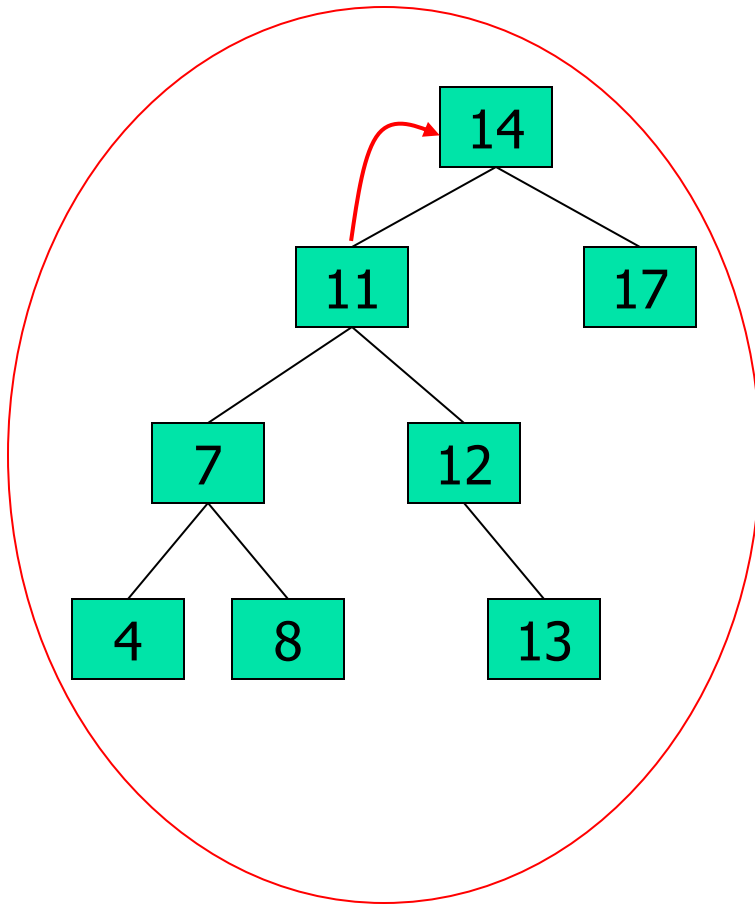
Rotatie dubla



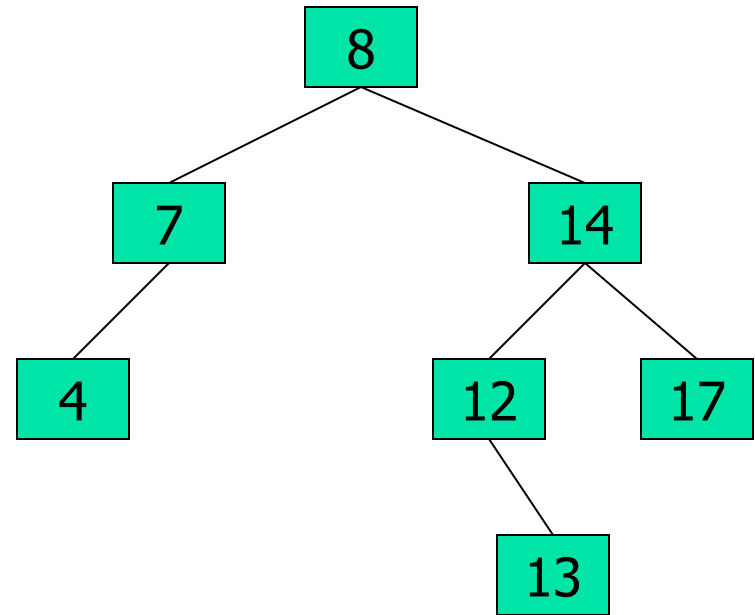
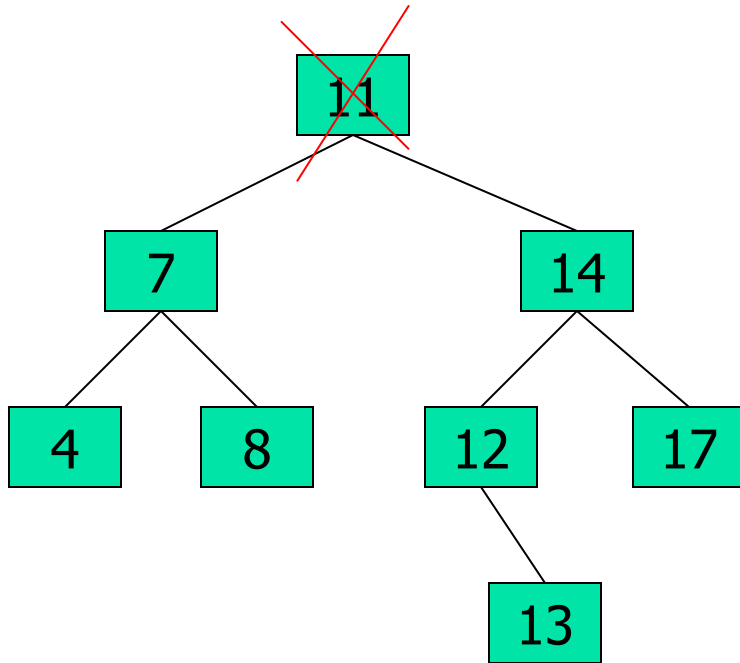
Eliminare din arbori AVL



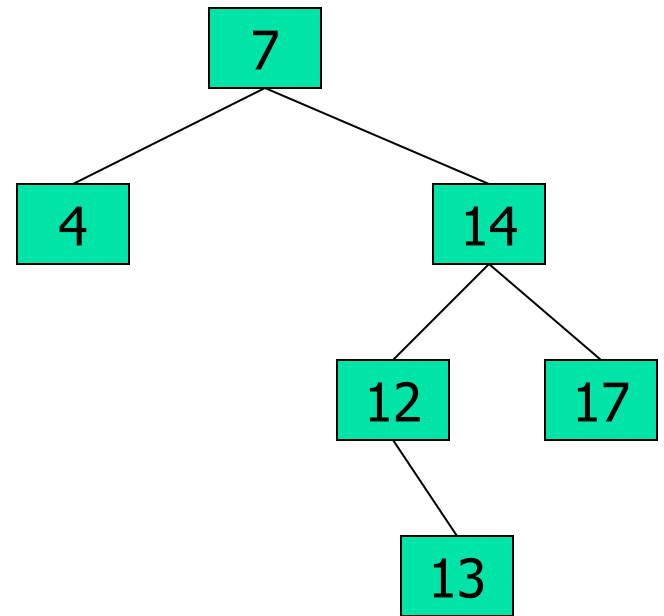
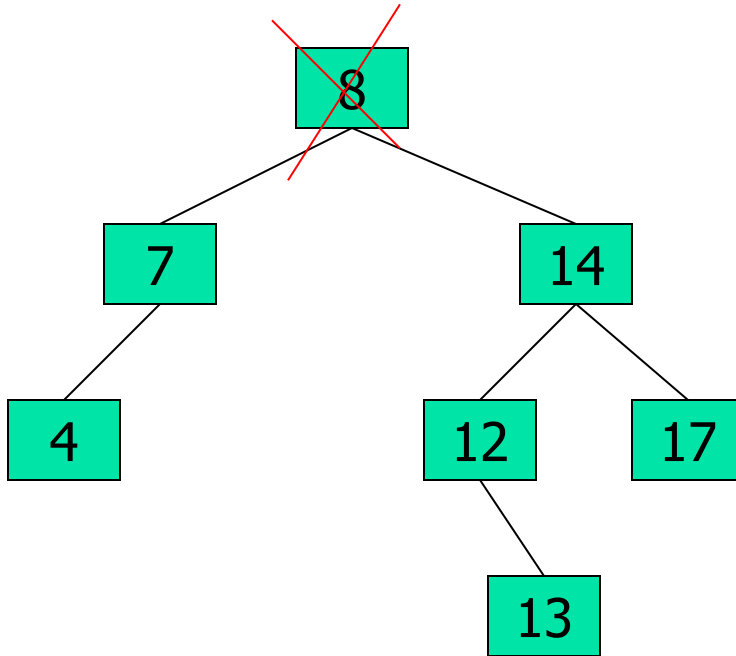
Eliminare din arbori AVL



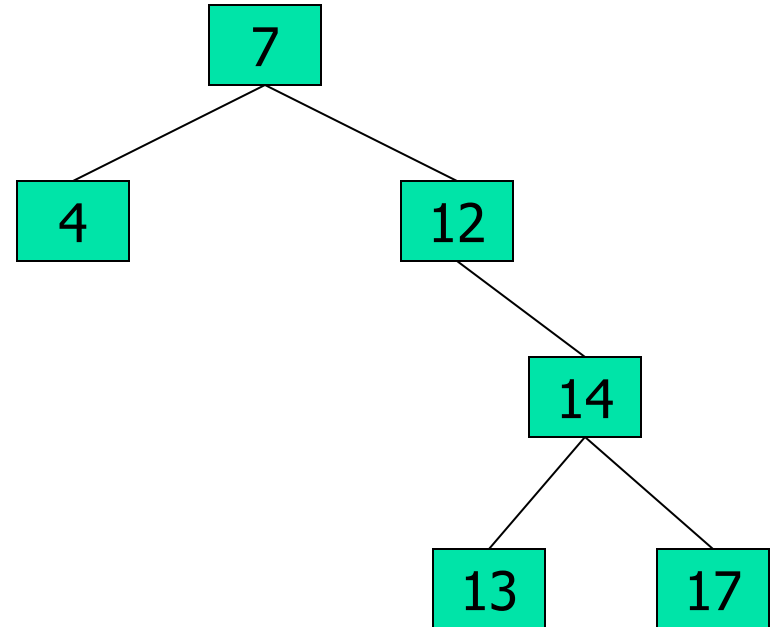
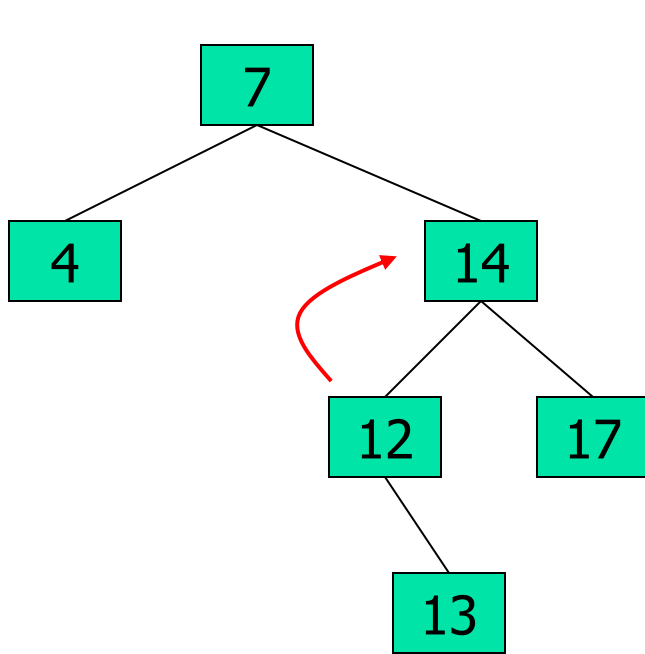
Eliminare din arbori AVL



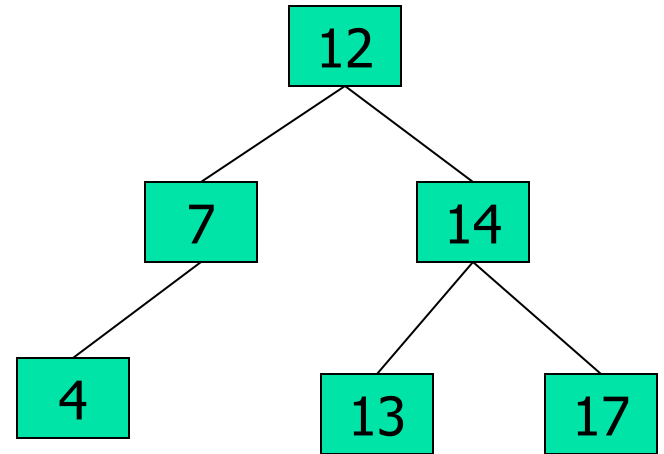
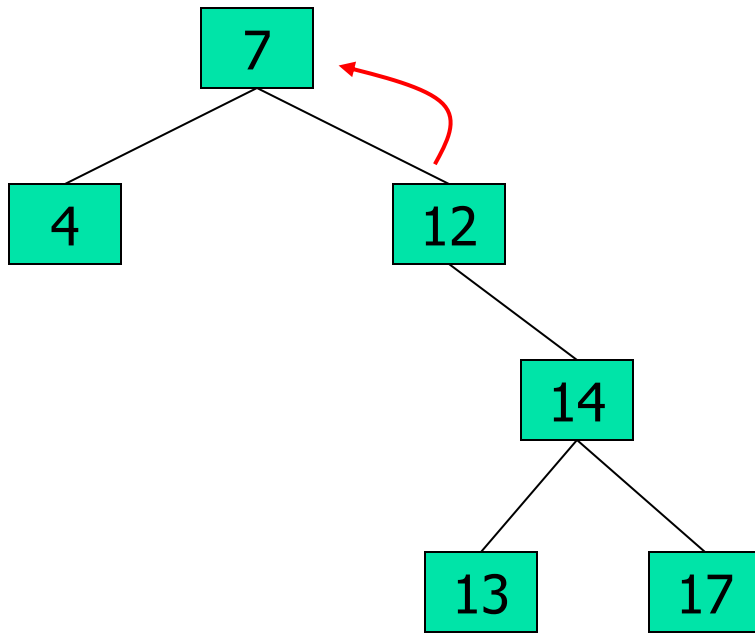
Eliminare din arbori AVL



Eliminare din arbori AVL



Eliminare din arbori AVL



Inserare in arbori AVL

- Insereaza informatia ca o frunza (analog ca in arborii binari de cautare)
 - Nodurile - de la frunza adaugata la radacina arborelui - isi pot modifica factorul de echilibru
 - Se actualizeaza factorii de echilibru pe calea de la frunza inserata la radacina
 - Daca se obtin factori de echilibru 2 sau -2, atunci se reechilibreaza arborele folosind rotatii stanga / dreapta sau rotatii duble stanga / dreapta

Colectii cu disciplina de prelucrare dictata de ordinea inserarii elementelor

Stiva: ultimul venit – primul servit
(**L**ast **I**n **F**irst **O**ut – **LIFO**)



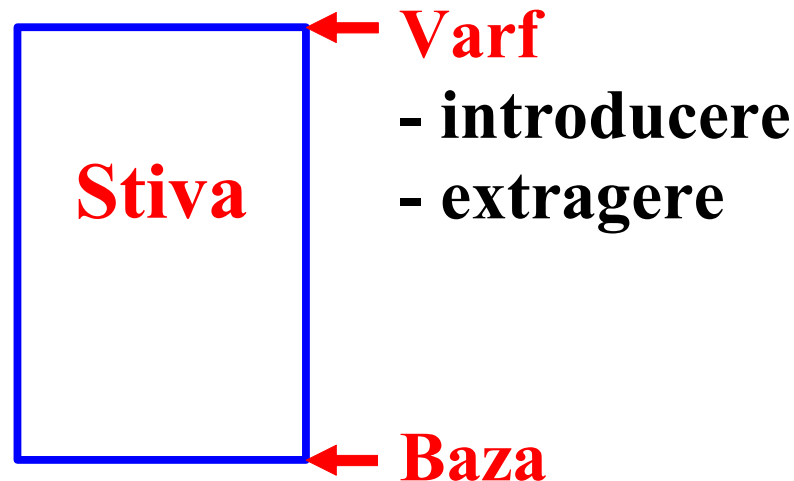
Coadă: primul venit – primul servit
(**F**irst **I**n **F**irst **O**ut – **FIFO**)



Colectii cu disciplina de prelucrare dictata de ordinea inserarii elementelor

Stiva: ultimul venit – primul servit
(**Last In First Out – LIFO**)

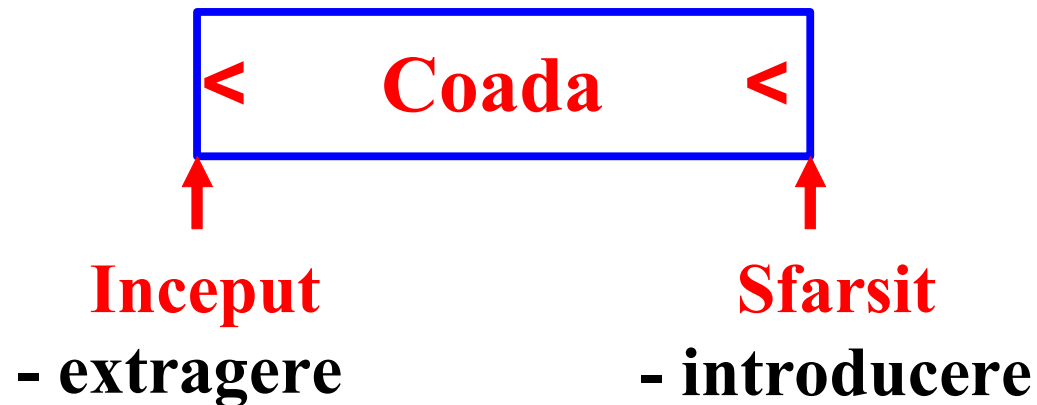
Analogii: tub de medicamente, dosare puse
teanc in ordinea primirii si parcurse la finalul
zilei in ordinea in care sunt asezate etc.



Colectii cu disciplina de prelucrare dictata de ordinea inserarii elementelor

Cooda: primul venit – primul servit
(**F**irst **I**n **F**irst **O**ut – **FIFO**)

Analogii: cereri rezolvate in ordinea inregistrarii



Operatii de baza

Stiva: (Last In First Out – LIFO):

- Initializare stiva (InitS)
- Adauga element in stiva (Push)
- Extrage element din stiva (Pop)
- Obtine elementul din varful stivei – fara a modifica continutul acesteia (Top)
- Elibereaza spatiul ocupat de intreaga structura (DistrS)

Operatii de baza

Cooda: (Last In First Out – LIFO)

- Initializare coada (InitQ)
- Adauga element in coada (IntrQ)
- Extrage element din coada (ExtrQ)
- Obtine primul element din coada – fara a modifica continutul acesteia (PrimQ)
- Elibereaza spatiul ocupat de intreaga structura (DistrQ)

Alte operatii

- Test structura vida
- Numar elemente
- Dimensiune element din structura
- Transforma structura in structura vida
- Muta primul element din structura sursa in structura destinatie
- Transfera continutul structurii sursa in structura destinatie

Mutarea unui element

Coda: `int MutaQ(void* ad, void* as);`

Destinatie	Sursa
< 1, 2, 3 <	< 20 , 13, 8, 5 <
⇓	
< 1, 2, 3, 20 <	< 13, 8, 5 <

Stiva: `int MutaS(void* ad, void* as);`

Destinatie	Sursa
1, 2, 3 :	20, 13, 8, 5 :
⇓	
1, 2, 3, 5 :	20, 13, 8 :

Echivalenta cu **extragere** urmata de **introducere**.

Transfer continuut Coada / Stiva

Concatenare cozi

```
int ConcatQ(void* ad, void* as);
```

Destinatie

Sursa

< 1, 2, 3 <

< 13, 8, 5 <



< 1, 2, 3, 13, 8, 5 <

<<

Rastoarna stiva **sursa** in stiva destinatie

```
int Rastoarna(void* ad, void* as);
```

Destinatie

Sursa

| 11, 22 :

| 21, 22, 23 :



| 11, 22, 23, 22, 21 :

|:

Pot fi realizate prin **mutari repetate**.

Suprapunere Stive

```
int Suprapune (void* ad, void* as) ;
```

Destinatie	Sursa
11, 22 :	21, 22, 23 :
⇓	
11, 22, 21, 22, 23 :	:

Poate fi realizata prin **mutari repetate**, folosind o **stiva auxiliara**:

```
{ initializeaza stiva aux;  
  cat timp sursa nevida  
  { muta element din sursa in aux; }  
  cat timp aux nevida  
  { muta element din aux in destinatie; }  
  distruge stiva aux;  
}
```

Inversare Stiva

```
int InvS(void* a);
```

| 1, 2, 3, 4, 5 :

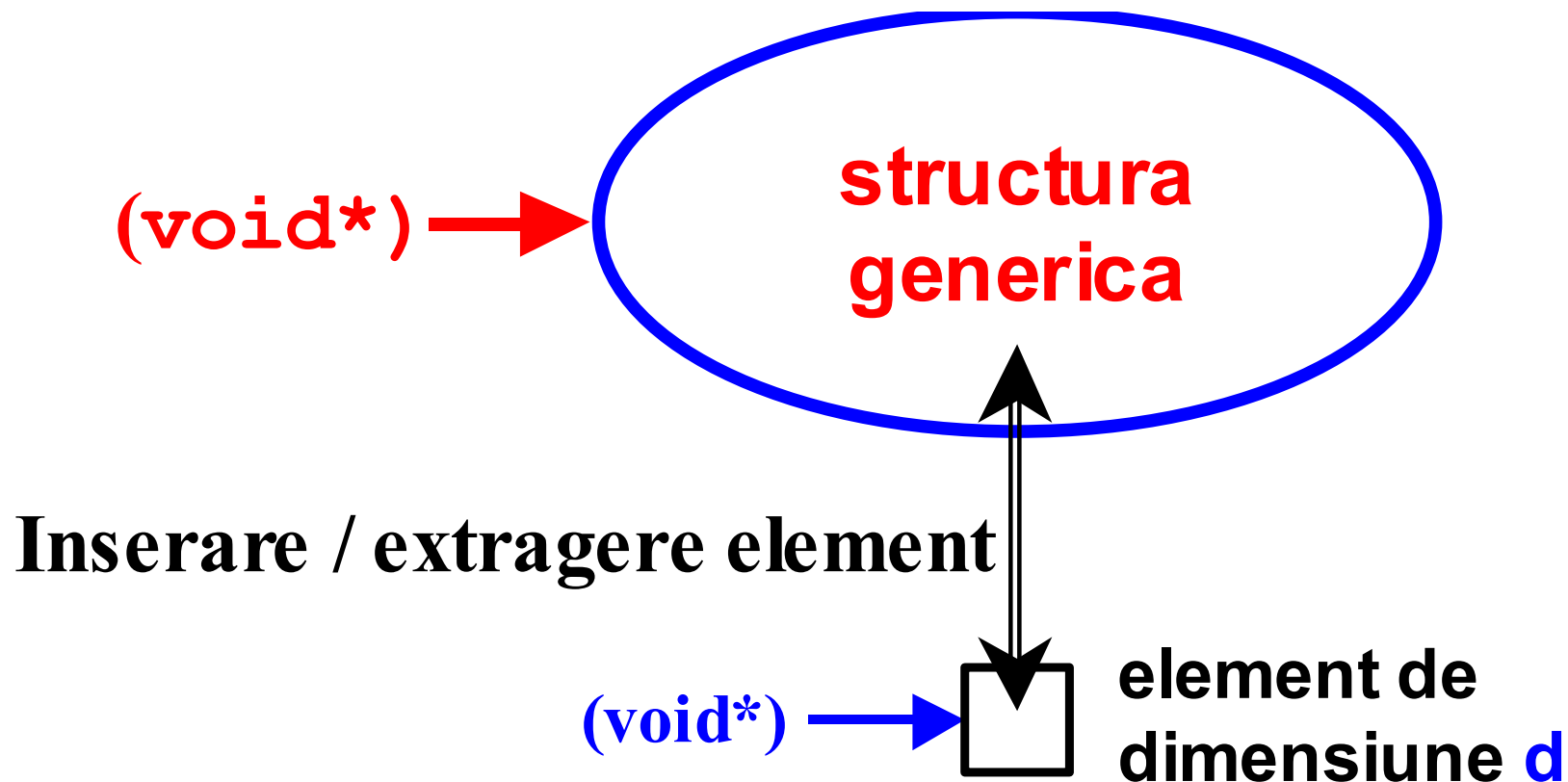
⇒

| 5, 4, 3, 2, 1 :

Poate fi realizata prin **mutari repetate**, folosind
o **coada auxiliara**:

```
{ initializeaza coada;  
  cat timp stiva nevida  
  { extrage element din stiva;  
    introduce elementul in coada;  
  }  
  cat timp coada nevida  
  { extrage element din coada;  
    introduce elementul in stiva;  
  }  
  distruge coada;  
}
```

Coada si Stiva generice



Coada si Stiva generice - operatii standard

Initializare structura cu elemente de dimensiune **d**:

```
void* InitQ(size_t d, ...);
```

```
void* InitS(size_t d, ...);
```

Rezultat: NULL sau adresa structura

Inserare:

```
int IntrQ(void* a, void* ae);
```

```
int Push(void* a, void* ae);
```

Rezultat: 1 – succes / 0 - esec

Coada si Stiva generice - operatii standard

Extragere

```
int ExtrQ(void* a, void* ae);  
int Pop(void* a, void* ae);
```

Rezultat: 1 – succes / 0 - esec

Eliberarea intregului spatiu ocupat de structura:

```
void DistrQ(void** aa);  
void DistrS(void** aa);
```

Test structura vida:

```
int VidaQ(void* a);  
int VidaS(void* a);
```


Coada si Stiva generice - operatii standard

Copiere element accesibil:

```
int PrimQ(void* a, void* ae);  
int Top(void* a, void* ae);
```

Numar elemente:

```
size_t NrEQ(void* a);  
size_t NrES(void* a);
```

Dimensiune elemente:

```
size_t DimEQ(void* a);  
size_t DimES(void* a);
```

Cooda si Stiva generice – alte operatii

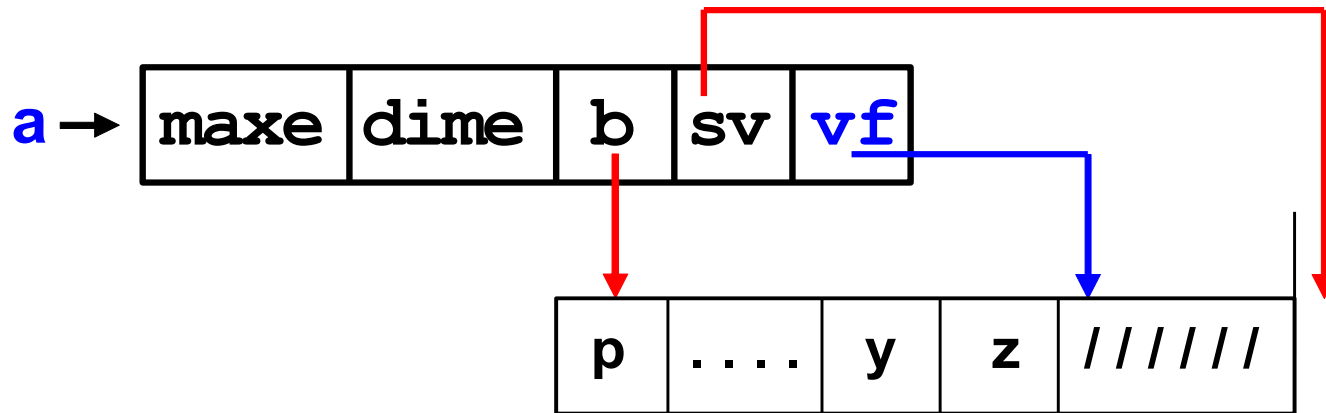
Transformare in structura vida:

```
void ResetQ(void* a);
```

```
void ResetS(void* a);
```

Implementare **stiva** generica – A [1]

A. Folosind un **vector** de elemente de dimensiune **d**

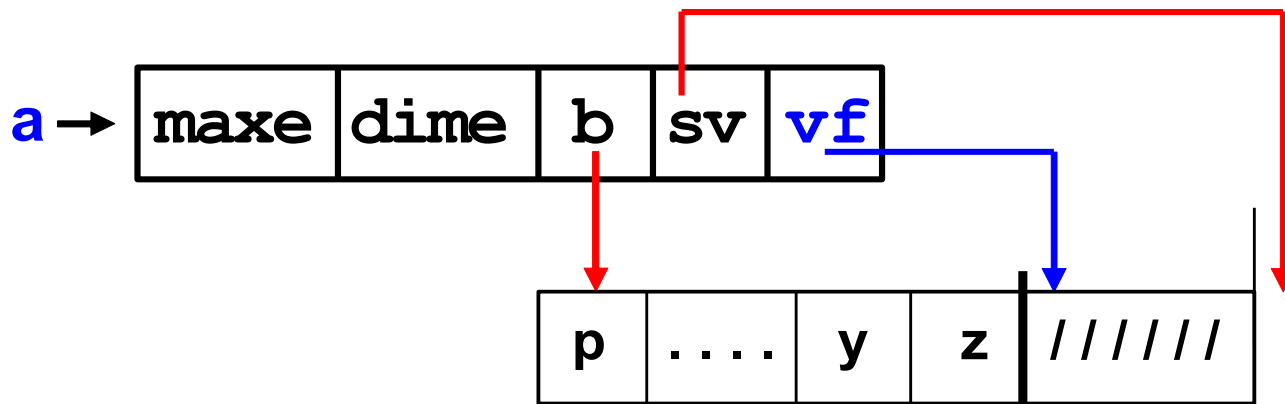


```
typedef struct
{ size_t maxe, dime; /* nr.maxim elemente,
                      dimensiune element */
  char *b, *sv, *vf; /* baza, sfarsit vector,
                      varf stiva */
} TStiva, *ASt;
```

Stiva vida: **`a->vf == a->b`**

Implementare **stiva** generica – A [2]

```
/*- macrodefinitii utile -*/  
/*- acces la campuri -*/  
#define DIME(a) ((Ast)(a))->dime)  
#define BS(a) ((Ast)(a))->b)  
#define SV(a) ((Ast)(a))->sv)  
#define VF(a) ((Ast)(a))->vf)  
/*- teste -*/  
#define DIMDIF(s,d) (DIME(s) != DIME(d))  
#define VIDA(a) (VF(a) == BS(a))  
#define PLINA(a) (VF(a) == SV(a))
```



Implementare **stiva** generica – A [3]

Initializare

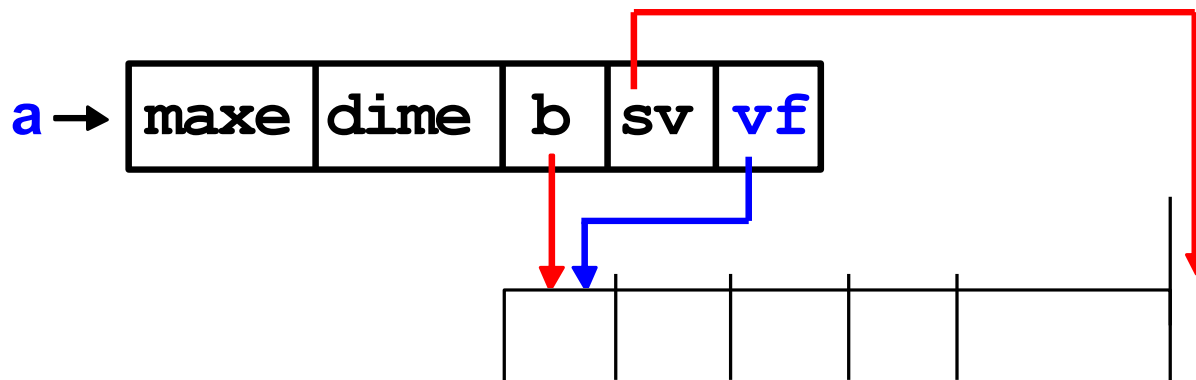
```
a = InitS(sizeof(tip_element), maxim);
```

In cazul unei initializari reusite

```
a->maxe == maxim
```

```
a->dime == sizeof(tip_element)
```

```
a->sv == a->b + a->maxe * a->dime
```



Implementare **stiva** generica – A [4]

```
void* InitS(size_t d,...)
```

```
{
```

```
    ASt a;
```

```
    va_list ap;
```

```
    a = (ASt)malloc(sizeof (TStiva));
```

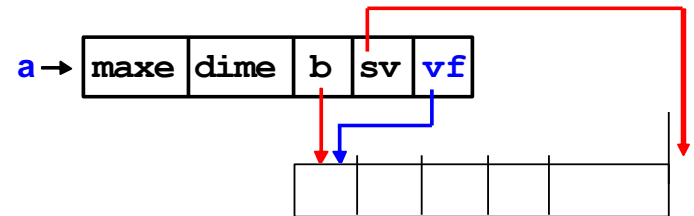
```
    if (!a) return NULL;
```

```
    va_start(ap,d);
```

```
    a->maxe = va_arg(ap,size_t);
```

```
    va_end(ap);
```

```
    a->dime = d;
```



Implementare **stiva** generica – A [5]

```
/* continuare InitS(size_t d,...)*/
```

```
/* aloca spatiu pentru vector si test esec */
```

```
a->b = (char*)calloc(a->maxe, d);
```

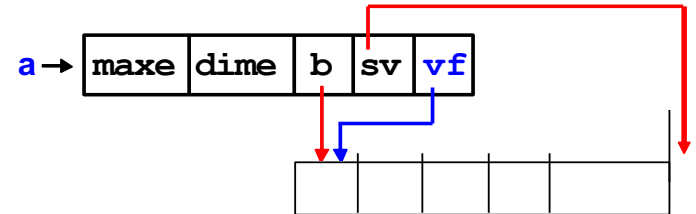
```
if (!a->b)
```

```
{
```

```
    free(a);
```

```
    return NULL;
```

```
}
```



```
/* actualizeaza adrese din descriptor */
```

```
a->vf = a->b;
```

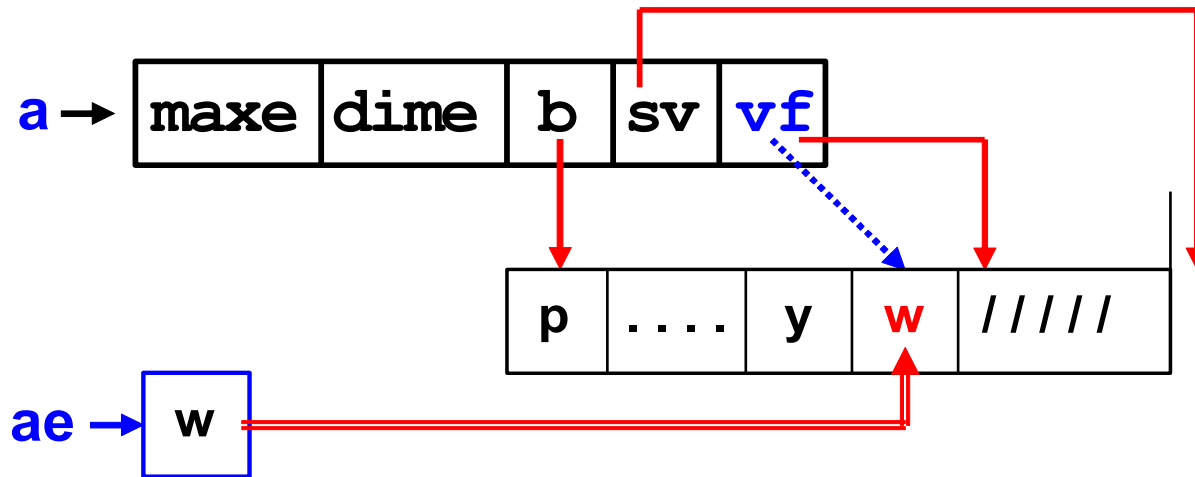
```
a->sv = a->b + d * a->maxe;
```

```
return (void *)a;
```

```
}
```

Implementare **stiva** generica – A [6]

Inserare



```
int Push(void* a, void* ae)
```

```
{
```

```
    if (PLINA(a)) return 0;
```

```
    memcpy (VF(a), ae, DIME(a));
```

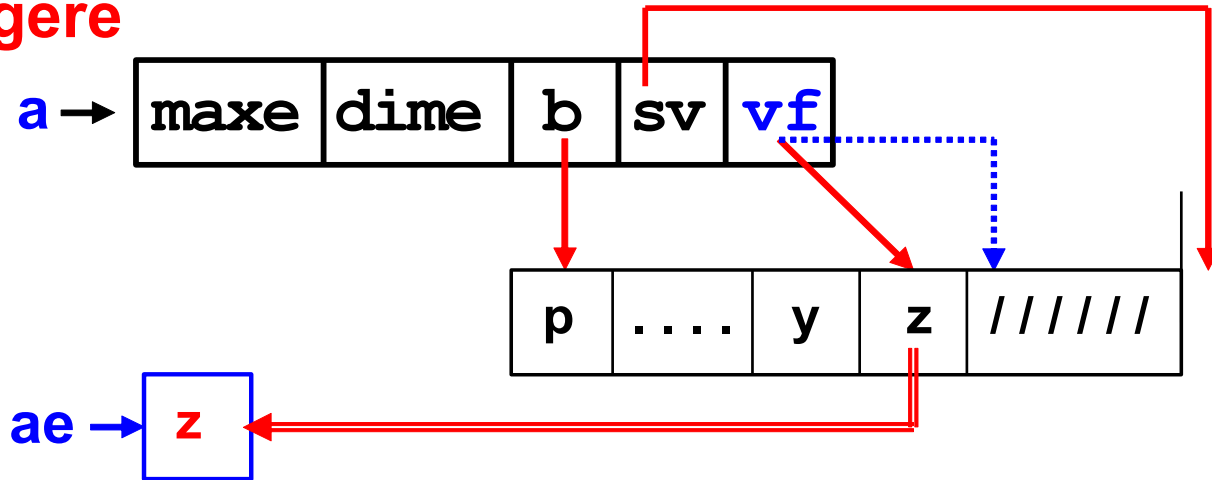
```
    VF(a) += DIME(a);
```

```
    return 1;
```

```
}
```


Implementare **stiva** generica – A [7]

Extragere



```
int Pop(void* a, void* ae)
```

```
{
```

```
    if (VIDA(a)) return 0;
```

```
    VF(a) -= DIME(a);
```

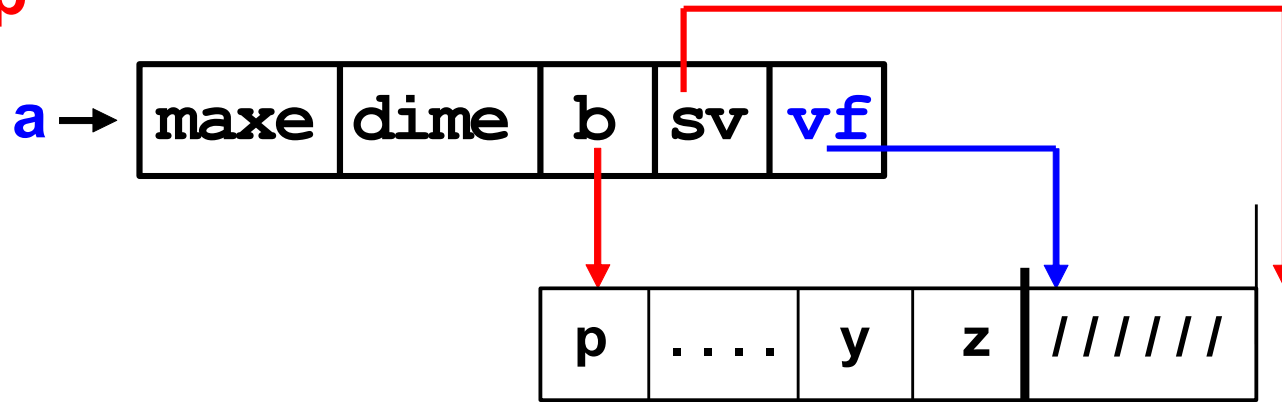
```
    memcpy(ae, VF(a), DIME(a));
```

```
    return 1;
```

```
}
```

Implementare **stiva** generica – A [8]

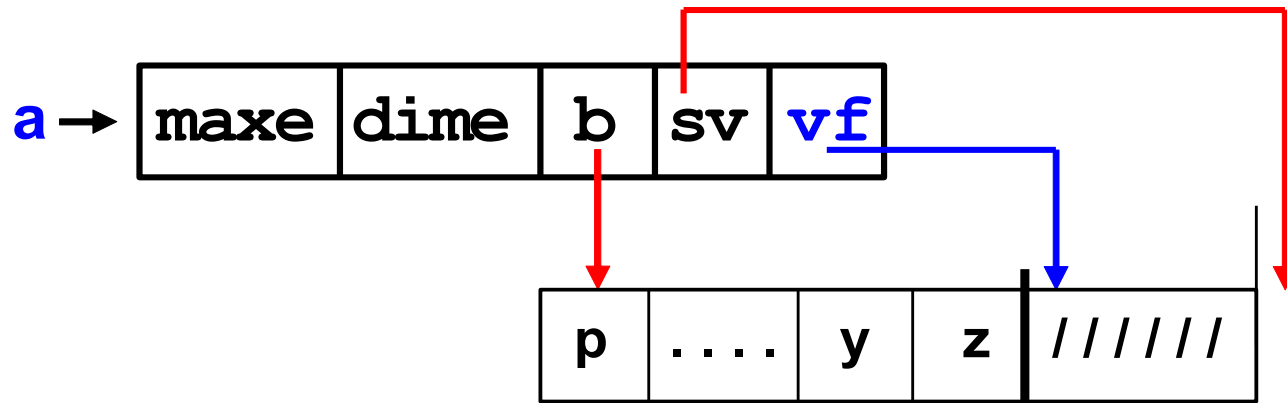
Top



```
int Top(void* a, void* ae)
{
    if (VIDA(a)) return 0;
    memcpy (ae, VF(a) - DIME(a), DIME(a));
    return 1;
}
```

Implementare **stiva** generica – A [9]

Numar elemente



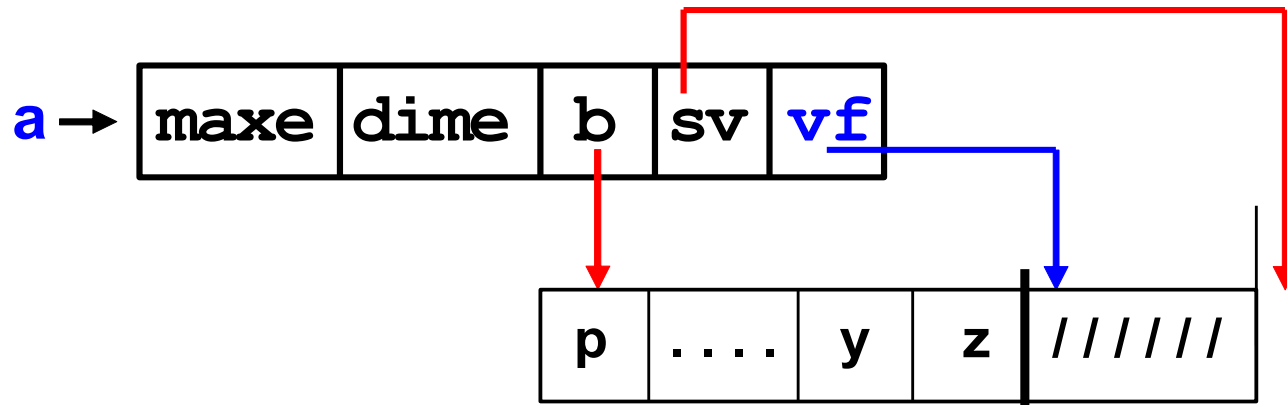
```
int NrES(void* a)
```

intoarce numarul de elemente din stiva

$((VF(a) - BS(a)) / DIME(a))$

Implementare **stiva** generica – A [10]

Reset



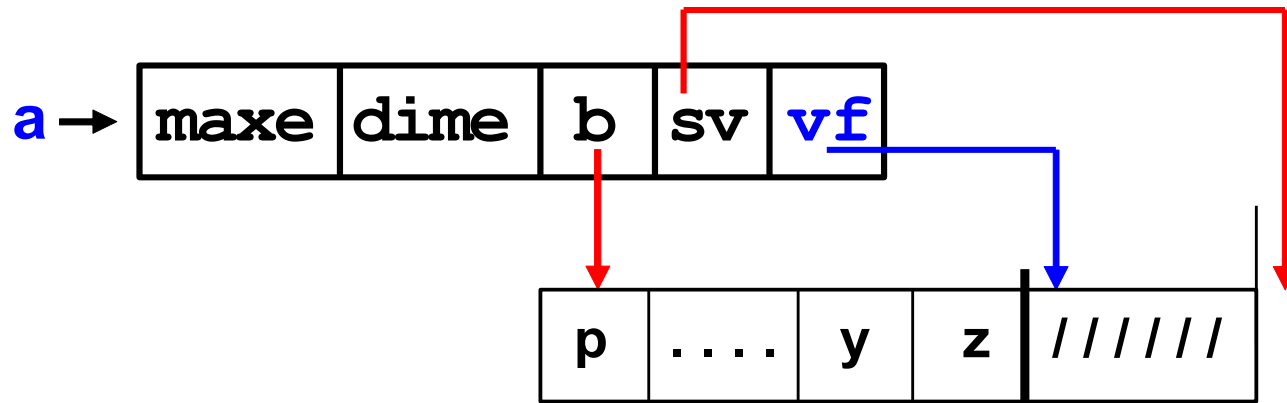
```
void ResetS(void* a)
```

```
/* transforma stiva in stiva vida*/
```

```
seteaza varful stivei la baza acesteia
```

Implementare **stiva** generica – A [11]

Distrugere stiva



```
void DistrS(void** a)
```

```
/* elibereaza intregul spatiu ocupat de stiva*/
```

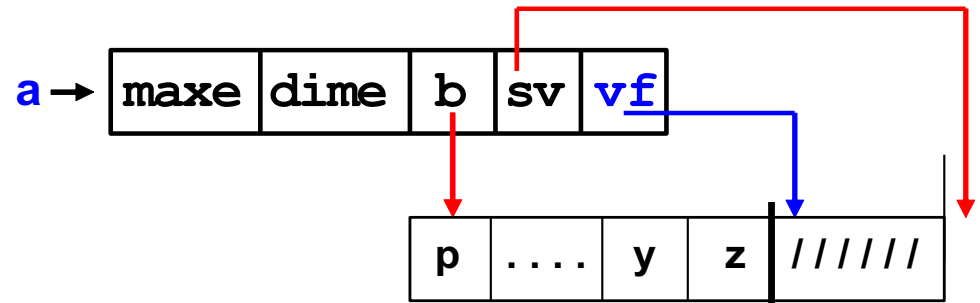
```
    elibereaza spatial ocupat de vector
```

```
    elibereaza spatial ocupat de descriptorul structura
```

```
    marcheaza stiva ca vida
```

Implementare **stiva** generica – A [12]

```
void DistrS(void **aa)
{
    free(BS(*aa));
    free(*aa);
    *aa = NULL;
}
```



Implementare **stiva** generica – A [13]

```
int MutaS(void *ad, void *as)
{
    if(DIMDIF(as,ad)) return -1;
    if(VIDA(as)) return 0;
    if(PLINA(ad)) return -2;

    /* mutare element */
    VF(as) -= DIME(as);
    memcpy(VF(ad), VF(as), DIME(as));
    VF(ad) += DIME(ad);
    return 1;
}
```

Implementare **stiva** generica – A [14]

```
int Suprapune(void *ad, void *as)
{
    size_t n = VF(as) - BS(as);
    if(DIMDIF(as,ad)) return -1;
    if(VIDA(as)) return 0;
    if(VF(ad) + n > SV(ad)) return -2;

    /* transfer elemente */
    memcpy(VF(ad), BS(as), n);
    VF(ad) += n;
    VF(as) = BS(as);
    return 1;
}
```

Destinatie

Sursa

| 11, 22 :

| **21, 22, 23 :**



| 11, 22, **21, 22, 23 :**

|:

Implementare **stiva** generica – A [15]

```
int Rastoarna (void *ad, void *as)
```

```
{
```

```
    size_t n = VF(as) - BS(as);
```

```
    if(DIMDIF(as,ad)) return -1;
```

```
    if(VIDA(as)) return 0;
```

```
    if(VF(ad) + n > SV(ad)) return -2;
```

```
    /* muta din varf sursa in varf destinatie */
```

```
    while(VF(as) > BS(as))
```

```
    {
```

```
        VF(as) -= DIME(as);
```

```
        memcpy(VF(ad), VF(as), DIME(as));
```

```
        VF(ad) += DIME(as);
```

```
    }
```

```
    return 1;
```

```
}
```

Destinatie

Sursa

| 11, 22 :

| 21, 22, 23 :



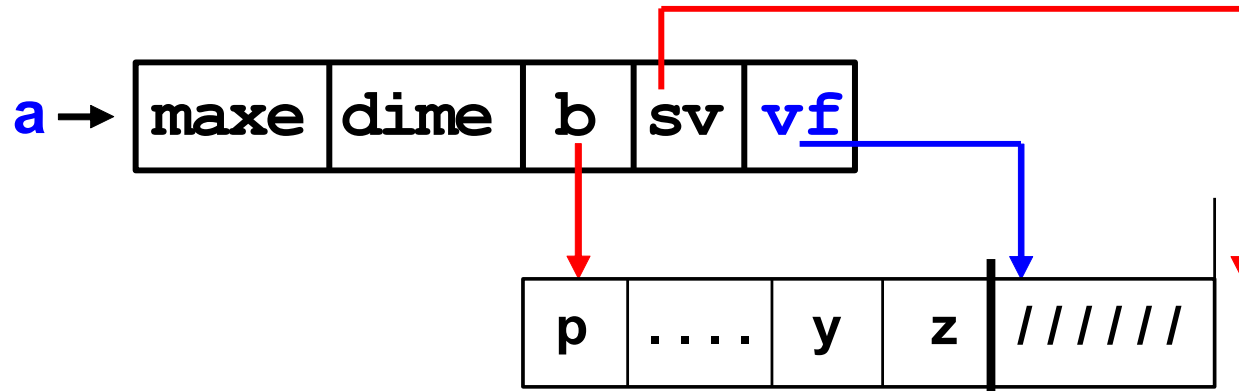
| 11, 22, 23, 22, 21 :

|:

Implementare **stiva** generica – A [16]

```
int InvS(void *a)
{
    char *temp = VF(a), *p, *q;
    if(temp >= SV(a)) temp = (char *)malloc(DIME(a));
    if(!temp) return 0;
    /* inverseaza perechi de elemente; p, q */
    for(p = BS(a), q = VF(a)-DIME(a); p < q;
        p += DIME(a), q -= DIME(a))
    {
        memcpy(temp, p, DIME(a));
        memcpy(p, q, DIME(a));
        memcpy(q, temp, DIME(a));
    }

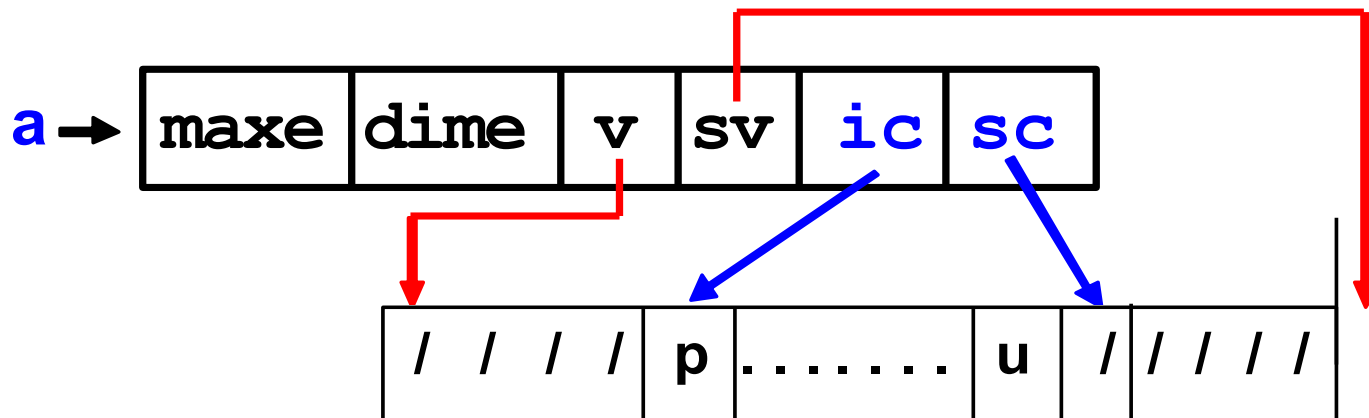
    if(temp != VF(a)) free(temp);
    return 1;
}
```



Implementare **coada** generica – A [1]

A. Folosind un **vector** prelucrat circular, cu santinela

```
typedef struct
{
    size_t maxe, dime; /* nr.maxim elemente,
                        dimensiune element */
    char *v, *sv, /* inceput, sfarsit vector */
        *ic, *sc; /* inceput, sfarsit coada */
} TCoadaV, *AQV;
```

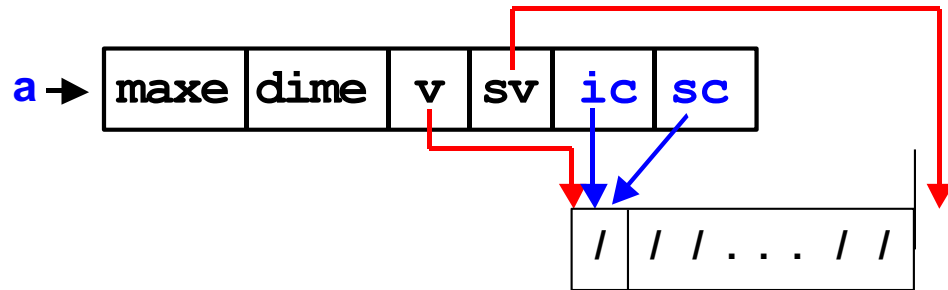


Implementare coada generica – A [1]

Coadă vida

maxe = 6

v, ic, sc



Inserare 2, 3, 4

v, ic



sc



3 eliminari

v



ic, sc



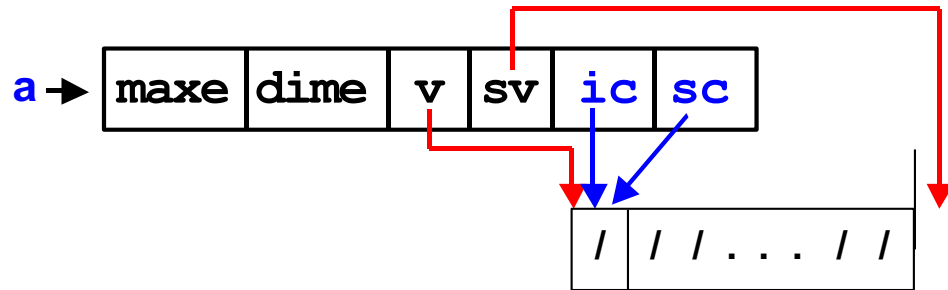
Coadă vida

Implementare coada generica – A [1]

Coadă vida

maxe = 6

v, ic, sc



Inserare 2, 3, 4

v, ic



sc



Inserare 5, 6

v, ic



sc

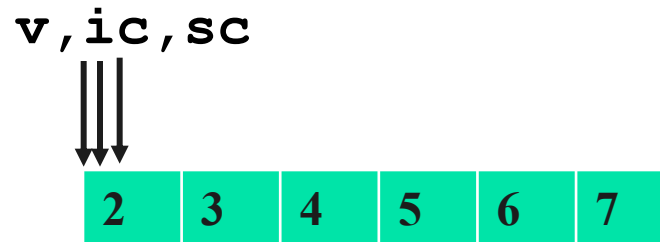


Implementare **coada** generica – A [1]

`maxe = 6`



Inserare 7



Avans circular `ic, sc` → coada circulara

Implementare **coada** generica – A [1]

maxe = 6

v, ic, sc



Coadă plină

Extragere doua elemente

v, sc

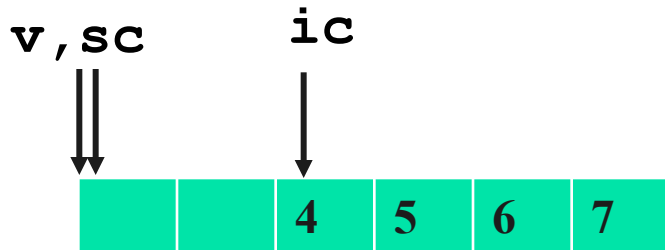


ic

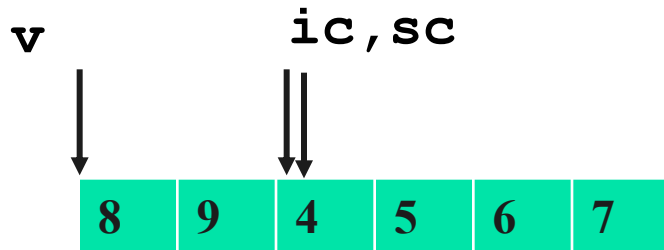


Implementare **coada** generica – A [1]

maxe = 6



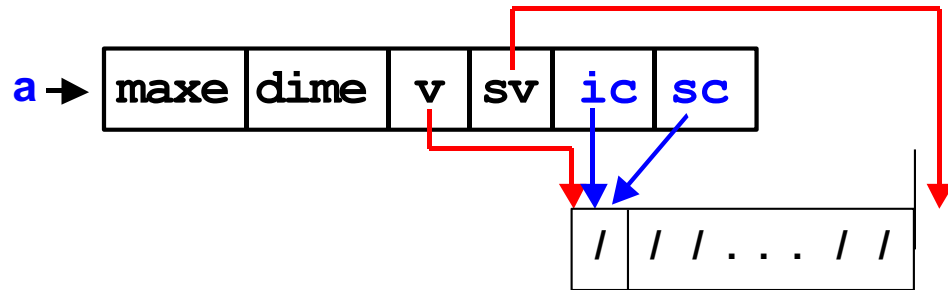
Inserare: 8, 9



Coadă plină

Implementare coada generica – A [1]

Coada cu maxe
elemente plus un
element suplimentar
= santinela



Coada vida: `maxe = 6` (vector alocat cu `maxe+1` elemente)

`v, ic, sc`

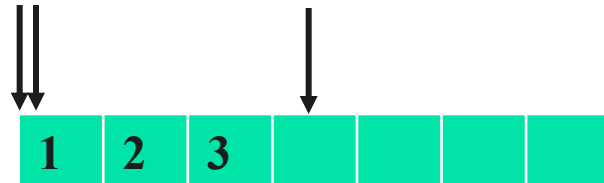


`ic == sc`

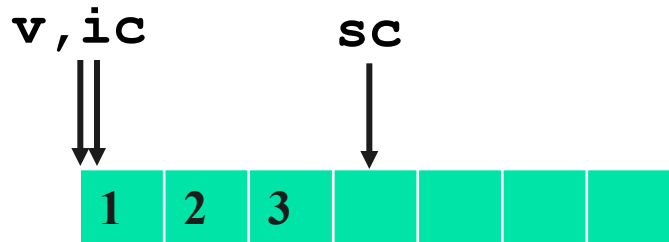
Inserare: 1, 2, 3

`v, ic`

`sc`



Implementare **coada** generica – A [1]

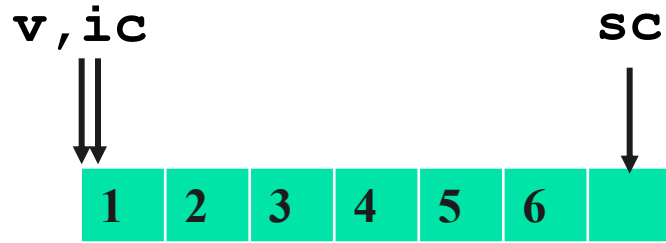


Inserare: 4, 5, 6

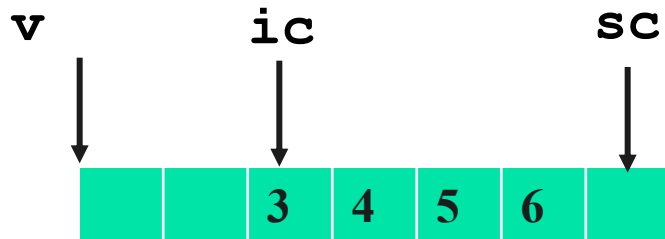


Cooda plina

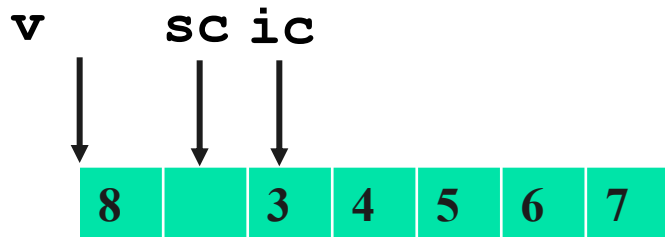
Implementare **coada** generica – A [1]



2 eliminari

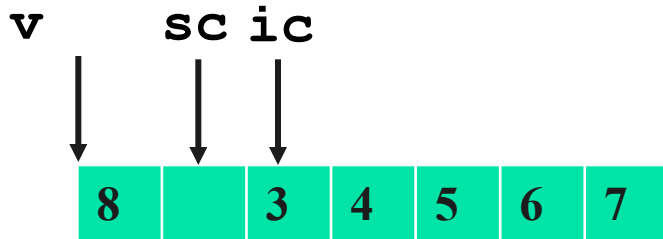


Inserare: 7, 8

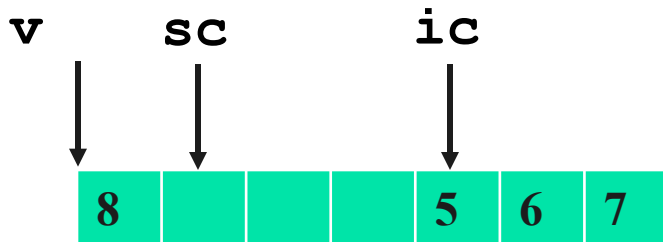


Coadă plină

Implementare **coada** generica – A [1]



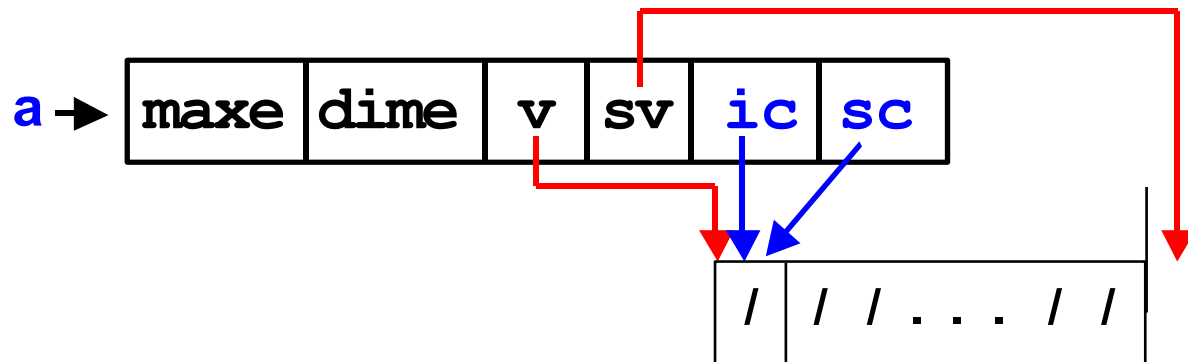
2 eliminari



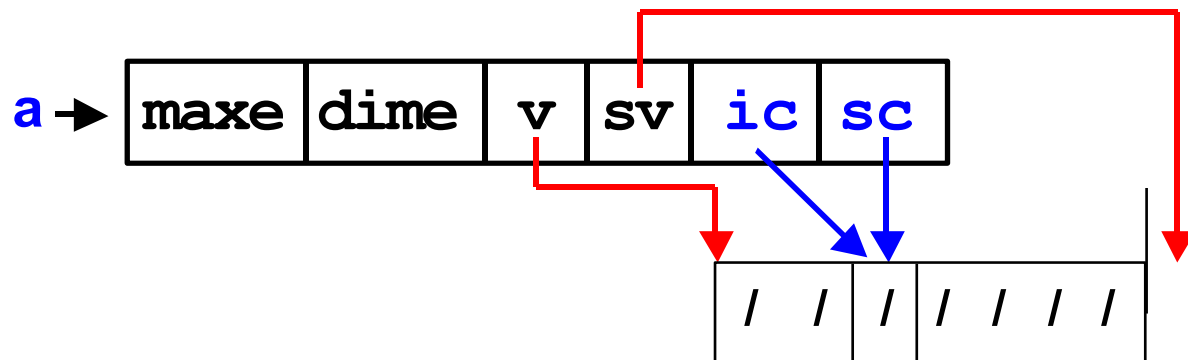
Implementare coada generica – A [2]

Cooda vida: $a \rightarrow ic == a \rightarrow sc$

- dupa initializare

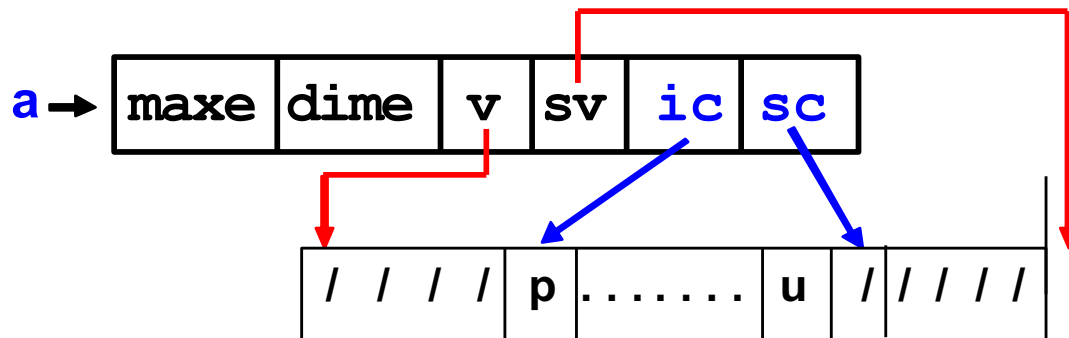


- dupa numar egal de inserari si eliminari



Implementare coada generica – A [3]

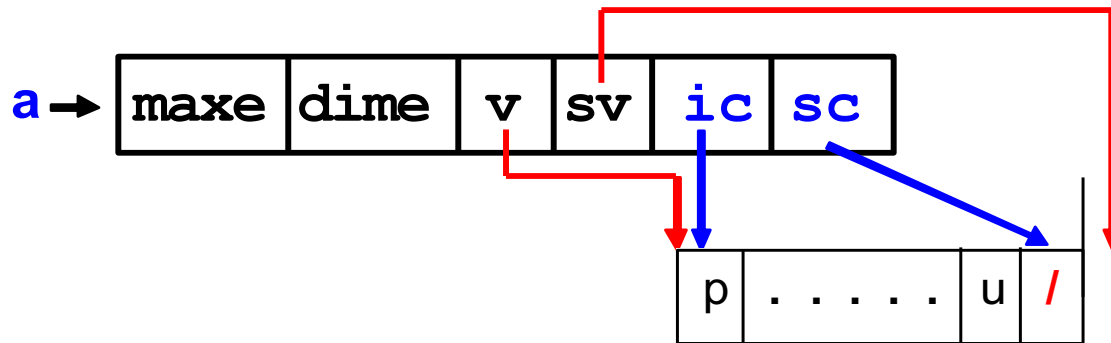
```
/*- macrodefinitii utile -*/  
/*- acces la campuri -*/  
#define V(a) (((AQV) (a)) ->v)  
#define SV(a) (((AQV) (a)) ->sv)  
#define IC(a) (((AQV) (a)) ->ic)  
#define SC(a) (((AQV) (a)) ->sc)  
#define DIME(a) (((AQV) (a)) ->dime)  
/*- teste si operatii simple -*/  
#define DIMDIF(s,d) (DIME(s) != DIME(d))  
#define VIDA(a) (IC(a) == SC(a))
```



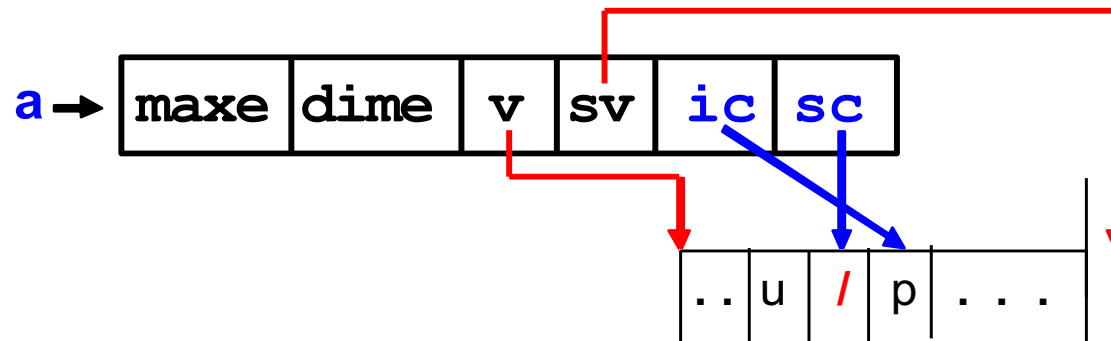
Implementare coada generica – A [6]

Coadă plină:

- cazul $IC(a) == V(a) : SC(a) + DIME(a) == SV(a)$



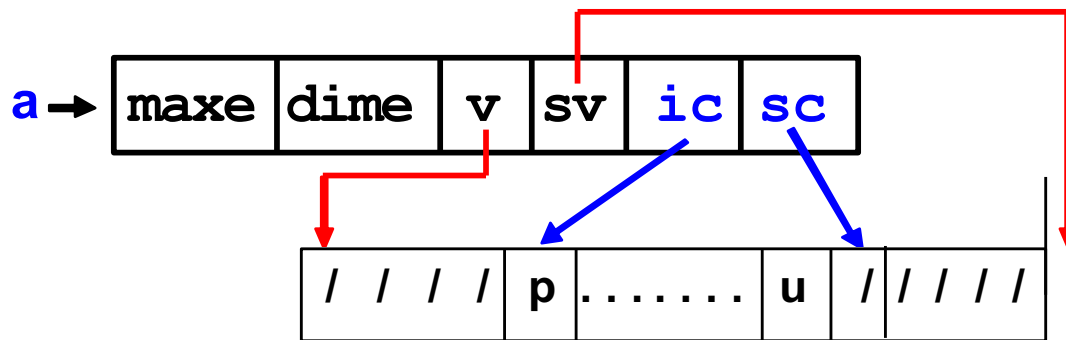
- cazul $IC(a) > V(a) : SC(a) + DIME(a) == IC(a)$



Implementare coada generica – A [4]

Avans (a,p)

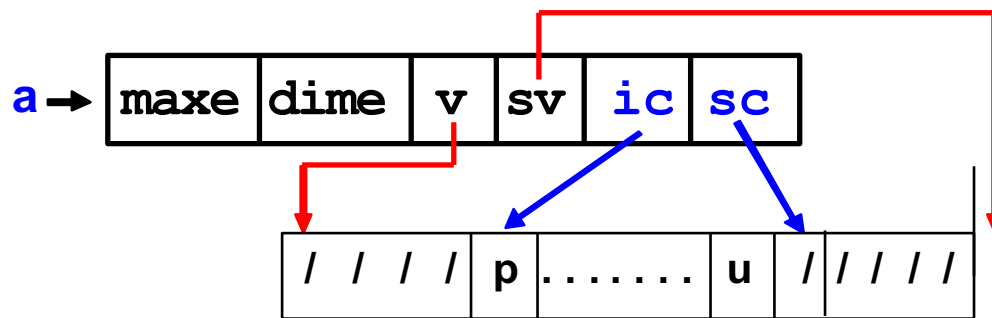
*daca $p + DIME(a) < SV(a)$ atunci $p += DIME(a)$
altfel $p = V(a)$*



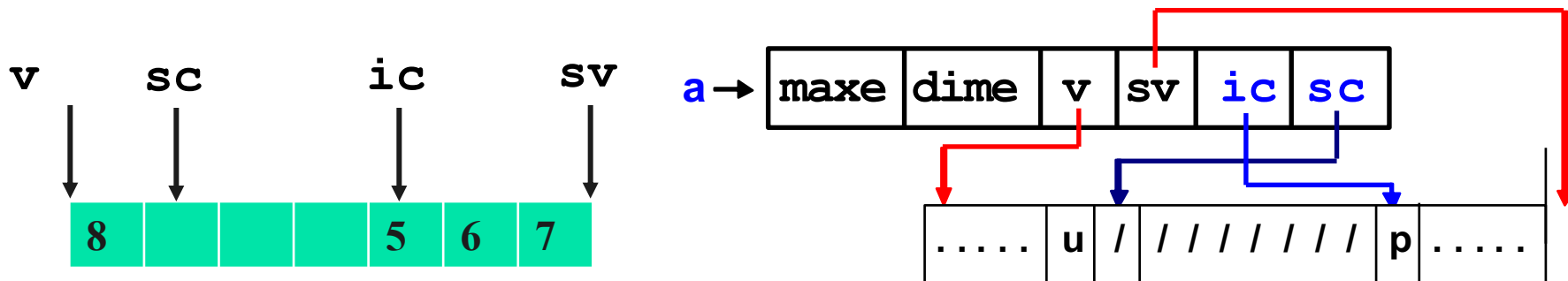
Implementare coada generica – A [5]

Spatiu ocupat, exprimat in octeti: Ocupat(a)

- cazul $IC(a) \leq SC(a)$: $SC(a) - IC(a)$

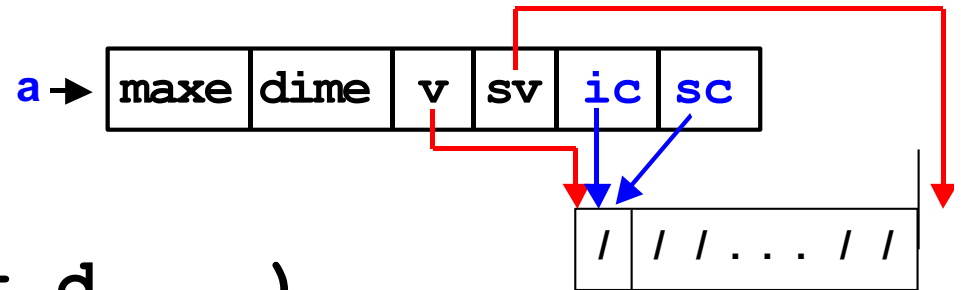


- cazul $IC(a) > SC(a)$: $SC(a) - V(a) + SV(a) - IC(a)$



Implementare coada generica – A [7]

Initializare



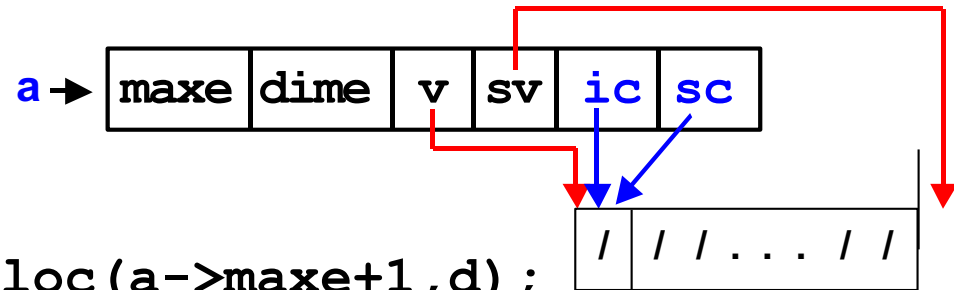
```
void* InitQ(size_t d,...)
{
    A_QV a = (A_QV)malloc(sizeof(TCodaV));
    va_list ap;

    if (!a) return NULL;
    /* preia param.2 = nr.max.elemente */
    va_start(ap,d);
    a->maxe = va_arg(ap,size_t);
    va_end(ap);

    a->dime = d;
```

Implementare coada generica – A [8]

Initializare



```
a->v = (char*)calloc(a->maxe+1,d) ;
```

```
if(!a->v)
```

```
{
```

```
    free(a) ;
```

```
    return NULL;
```

```
}
```

```
/* actualizeaza adrese din descriptor */
```

```
a->sv = a->v + (a->maxe+1) * d;
```

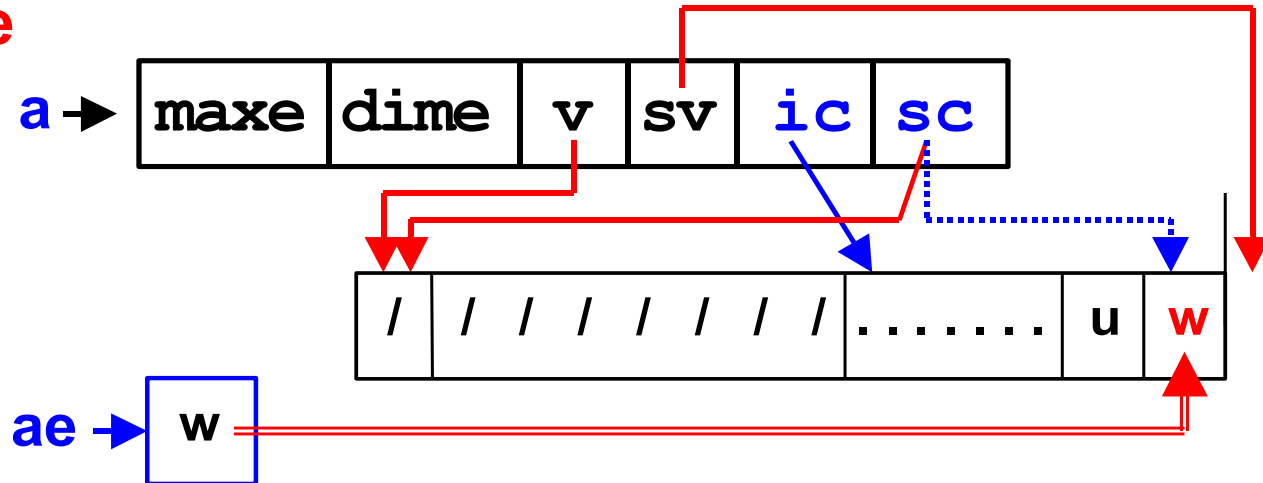
```
a->ic = a->sc = a->v;
```

```
return (void*)a;
```

```
}
```

Implementare **coada** generica – A [9]

Inserare



```
int IntrQ(void* a, void* ae)
```

```
{
```

```
    if (PLINA(a)) return 0;
```

```
    memcpy(SC(a), ae, DIME(a));
```

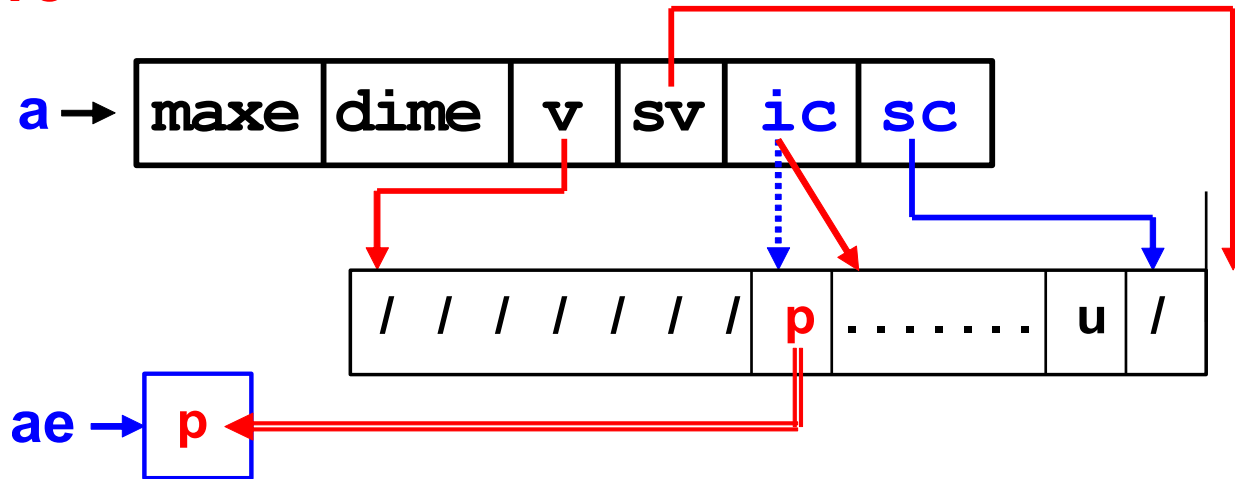
```
    Avans(a, SC(a));
```

```
    return 1;
```

```
}
```

Implementare **coada** generica – A [10]

Extragere

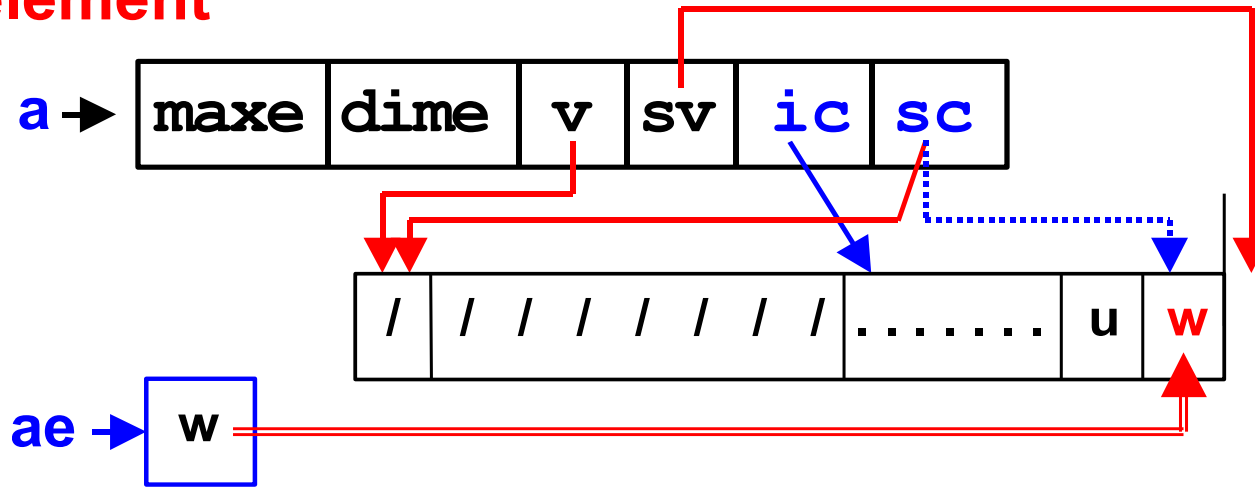


```
int ExtrQ(void* a, void* ae)
```

```
{  
    if(VIDA(a)) return 0;  
    memcpy(ae, IC(a), DIME(a));  
    Avans(a, IC(a));  
    return 1;  
}
```

Implementare coada generica – A [11]

Primul element



```
int PrimQ(void *a, void *ae)
```

```
{
```

```
    if(VIDA(a)) return 0;
```

```
    memcpy(ae, IC(a), DIME(a));
```

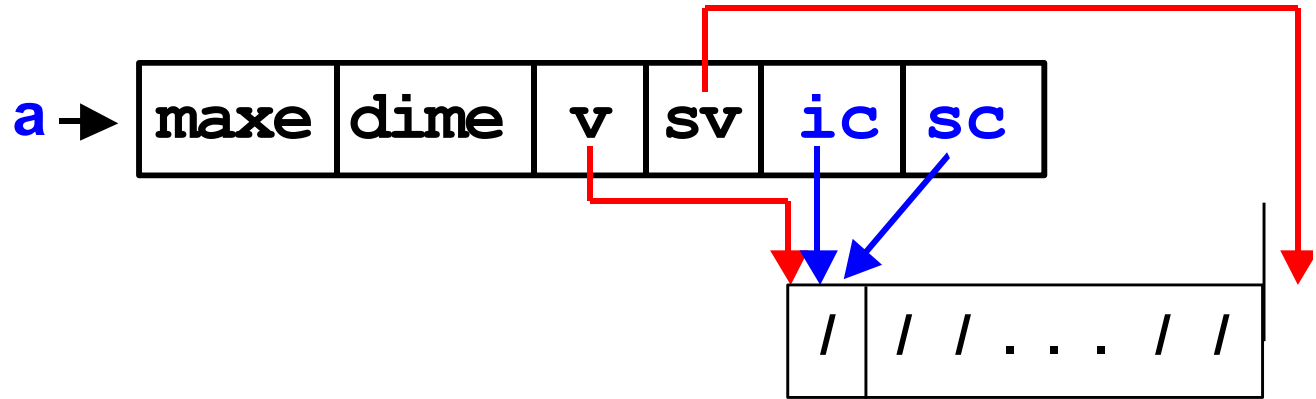
```
    Avans(a, IC(a));
```

```
    return 1;
```

```
}
```

Implementare coada generica – A [12]

Numar elemente



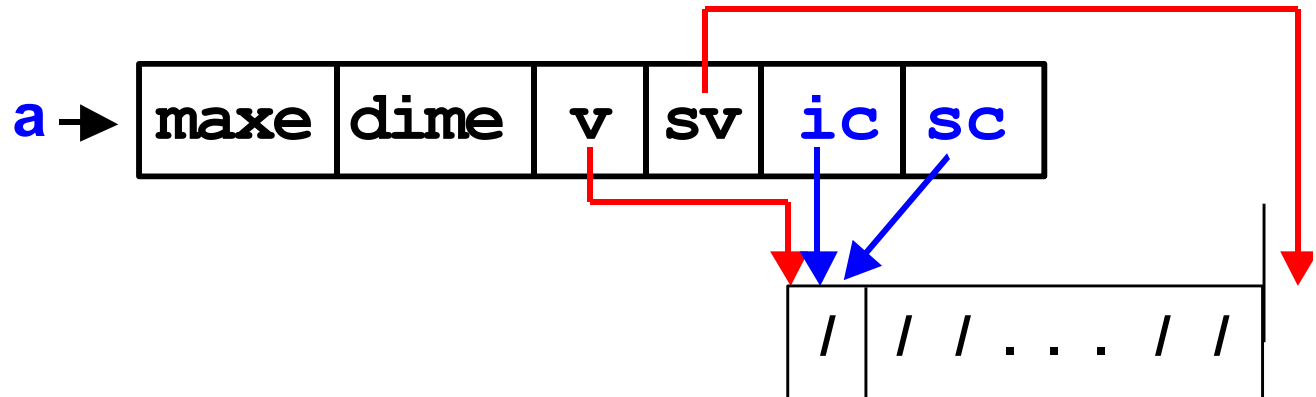
```
size_t NrEQ(void *a)
```

```
/* numarul de elemente din coada */
```

```
    intoarce spatiul ocupat / dimensiune element
```


Implementare coada generica – A [13]

Reset



```
void ResetQ(void *a)
```

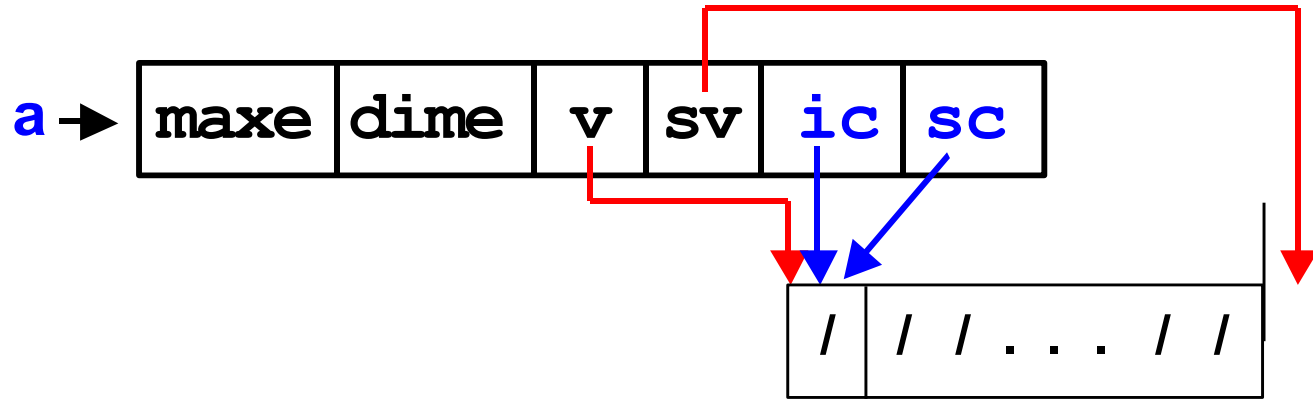
```
/* transforma coada in coada vida */
```

```
seteaza inceputul si sfarsitul cozii la  
inceputul vectorului
```

```
SC(a) = IC(a) = V(a);
```

Implementare coada generica – A [14]

Distruge coada



```
void DistrQ(void **a)
{
    free(V(*aa));
    free(*aa);
    *aa = NULL;
}
```

Implementare coada generica – A [15]

```
int MutaQ (void *ad, void *as)
{
    if(DIMDIF(as,ad)) return -1;
    if(VIDA(as)) return 0;
    if(PLINA(ad)) return -2;

    memcpy(SC(ad), IC(as), DIME(as));
    Avans(ad,SC(ad));
    Avans (as,IC(as));
    return 1;
}
```

Implementare coada generica – A [16]

```
int ConcatQ(void *ad, void *as)
{
    if(DIMDIF(as,ad)) return -1;
    if(VIDA(as)) return 0;
    if(Liber(ad) < Ocupat(as)) return -2;

    for(; IC(as) != SC(as);
        Avans(ad,SC(ad)),Avans(as,IC(as)))
        memcpy(SC(ad), IC(as), DIME(as));
    return 1;
}
```

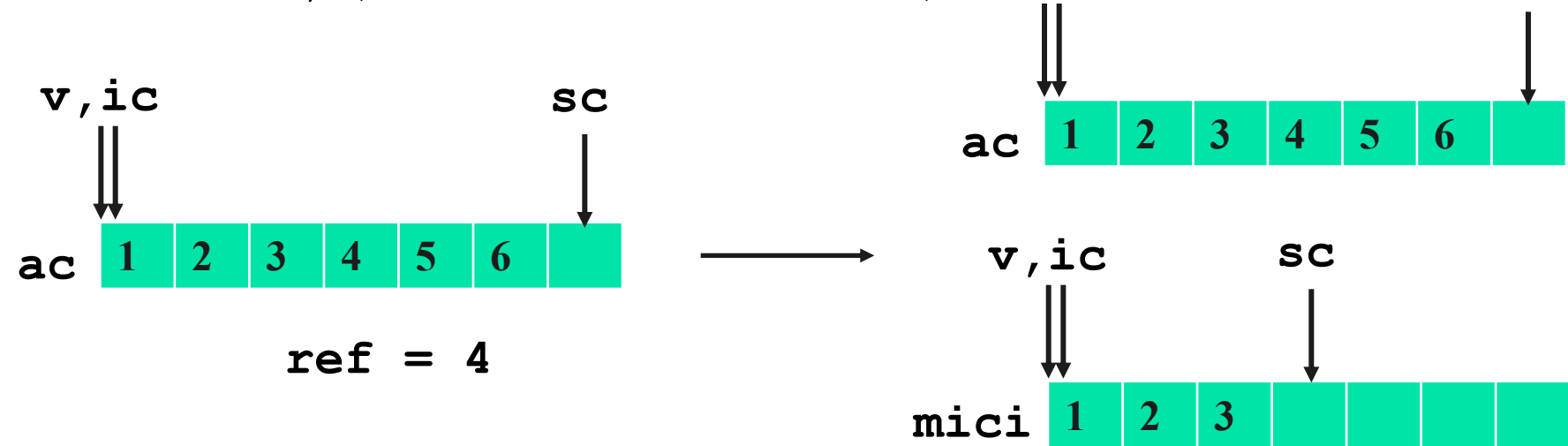
Exemple prelucrari

```
void* CopieMaiMici(void* ac, int ref)
```

- Fie o coada implementata generic, care contine elemente int
- Sa se construiasca o noua coada in care sunt copiate elementele < ref
- Elementele din cele doua cozi vor pastra ordinea relativa din coada initiala

Exemple prelucrari

```
void* CopieMaiMici(void* ac, int ref)
{ void *mici, *caux; /* adresa coada rezultat, aux */
  int x; /* valoare element */ v,ic          sc
```



```
/*- initializare cozi de lucru -*/
```

```
mici = InitQ(DimES(ac), MAX);
```

```
if (!mici) return NULL;
```

```
caux = InitQ(DimES(ac), MAX);
```

```
if (!caux){ DistrQ(&mici); return NULL; }
```

```
}
```

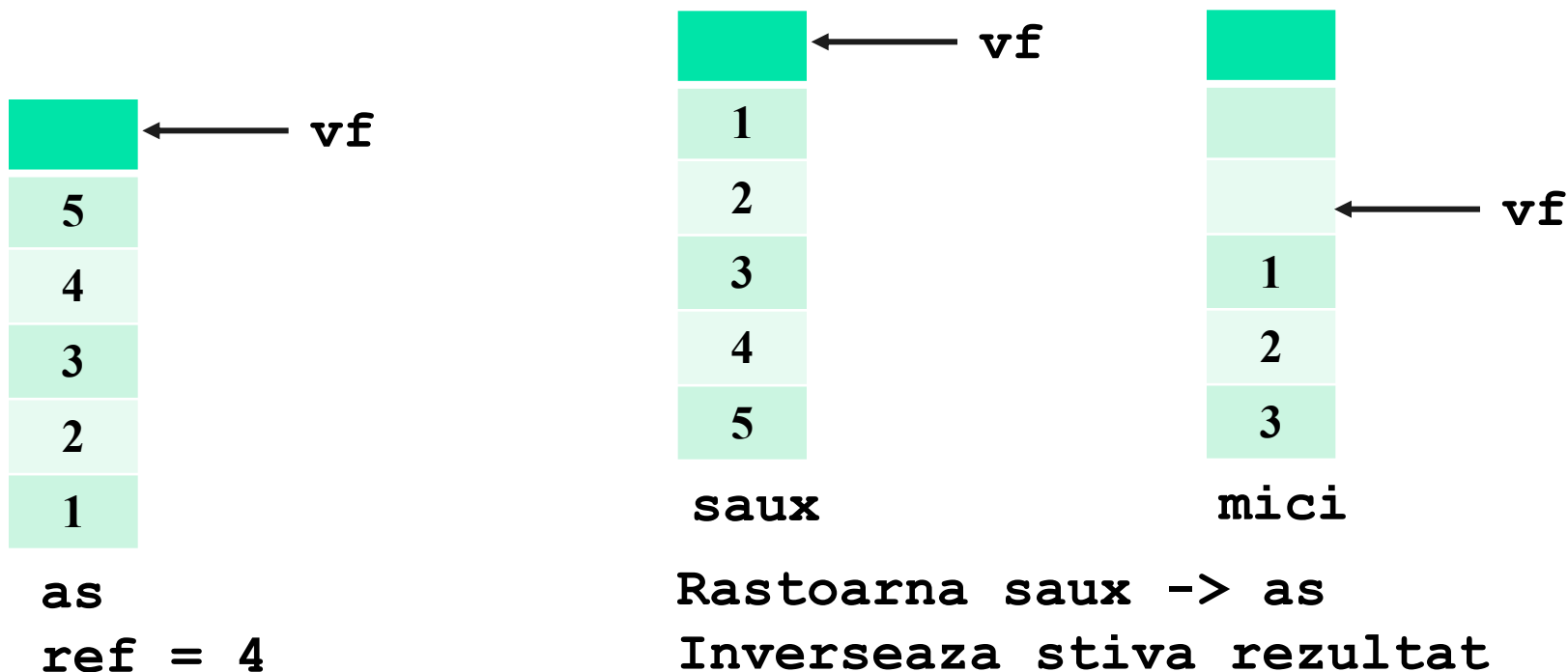
Exemple prelucrari

```
/*- transfer din coada sursa in cozile de lucru -*/  
while (ExtrQ(ac, &x))  
{ if(x < ref)  
    IntrQ(mici, &x);  
  IntrQ(caux, &x);  
}  
ac = caux; //?? NU SE POATE  
ConcatQ(ac, caux); /* muta elementele in coada initiala*,  
DistrQ(&caux); /* nu mai este necesara */  
  
/* intoarce rezultat */  
return mici;  
}
```

Exemple prelucrari

```
void* CopieMaiMici(void* as, int ref, )
```

- Fie o stiva implementata generic, care contine elemente int
- Sa se construiasca o noua stiva in care sunt transferate elementele $< \text{ref}$
- Elementele din cele doua stive vor pastra ordinea relativa din stiva initiala



Exemple prelucrari

```
void* TransferMaiMici(int ref, void* as)
{ void * mici, *saux; /* adrese stive de lucru */
  int x; /* valoare element */
  /*- initializare stive de lucru -*/

  mici = InitS(DimES(as), MAX);
  if (!mici) return NULL;

  saux = InitS(DimES(as), MAX);
  if (!mari){ DistrS(&mici); return NULL; }
}
```

Exemple prelucrari

```
/*- transfer din stiva sursa in stivele de lucru -*/
```

```
while (Pop(as, &x))  
{ if(x < ref)  
    Push(mici, &x);  
    Push(saux, &x);  
}
```

```
/* elementele mari raman in stiva sursa */
```

```
Rastoarna(as, saux);  
DistrS(&saux); /* este vida si nu mai este necesara */
```

```
/* stiva elementelor mici -> rezultat */
```

```
InvS(mici);  
return mici;  
}
```

Exemple prelucrari

```
void* TransferMaiMici(void* ac, int ref)
/* coada */
```

```
while (ExtrQ(ac, &x))
{ if(x < ref)
    IntrQ(mici, &x);
  IntrQ(caux, &x);
}
```

Devine

```
while (ExtrQ(ac, &x))
{ if(x < ref)
    IntrQ(mici, &x);
  else
    IntrQ(caux, &x);
}
```

Exemple prelucrari

```
void* TransferMaiMici(void* ac, int ref)
/* nu se cunoaste tipul elementelor
   din coada /structura */
int x;    → //void* x;

/* alocare x */
x = (int*)malloc(sizeof(int));
//x = malloc(DimEQ(ac));
if(!x) { /*elibereaza structure alocate*/
    return NULL;}

...
while (ExtrQ(ac, x))
{ if(x < ref)
    IntrQ(mici, x);
  else
    IntrQ(caux, x);
}

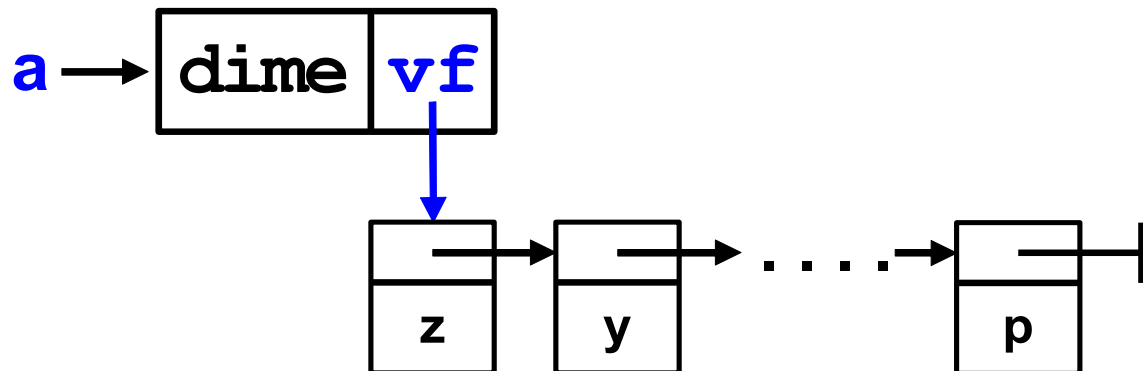
...
free(x);
```

Implementare **stiva** generica – B [1]

B. Folosind o **lista**

```
typedef struct celst
{ struct celst *urm; /* adr.urmatoarei celule */
  void* info;        /* adresa informatie */
} TCelSt, *ACelSt;

typedef struct stiva
{ size_t dime;        /* dim.element */
  ACelSt vf;          /* adresa celulei din varf */
} TStiva, *ASt;
```

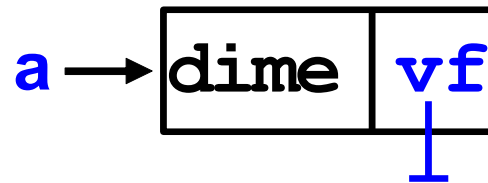


Implementare **stiva** generica – B [2]

```
#define VF(a) ((ASt)(a))->vf  
#define DIME(a) ((ASt)(a))->dime)
```

Initializare

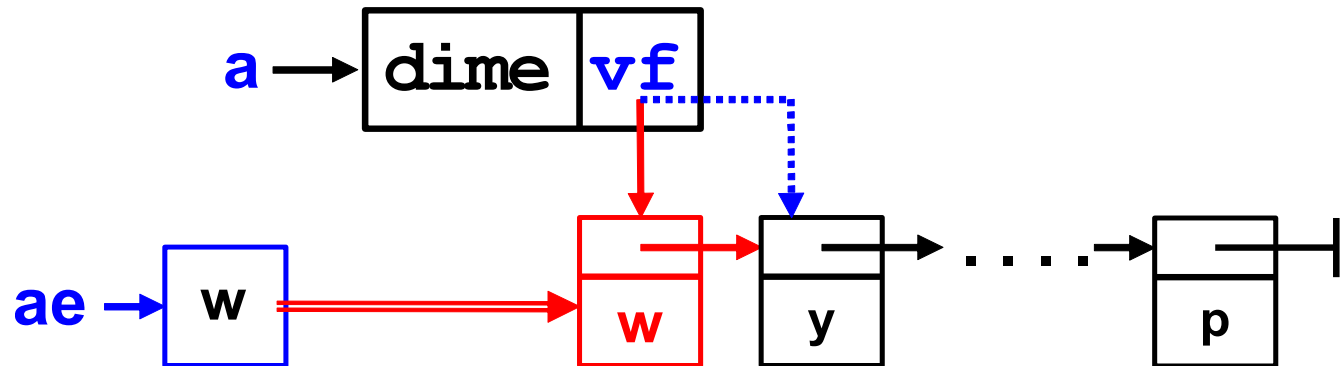
```
void* a = InitS(sizeof(tip_element));  
void* a = InitS(sizeof(int));
```



Stiva vida: **VF(a) == NULL**

Implementare **stiva** generica – B [3]

Inserare



```
ACelSt aux = (ACelSt)malloc(sizeof(TCelSt));  
if(!aux) return 0;
```

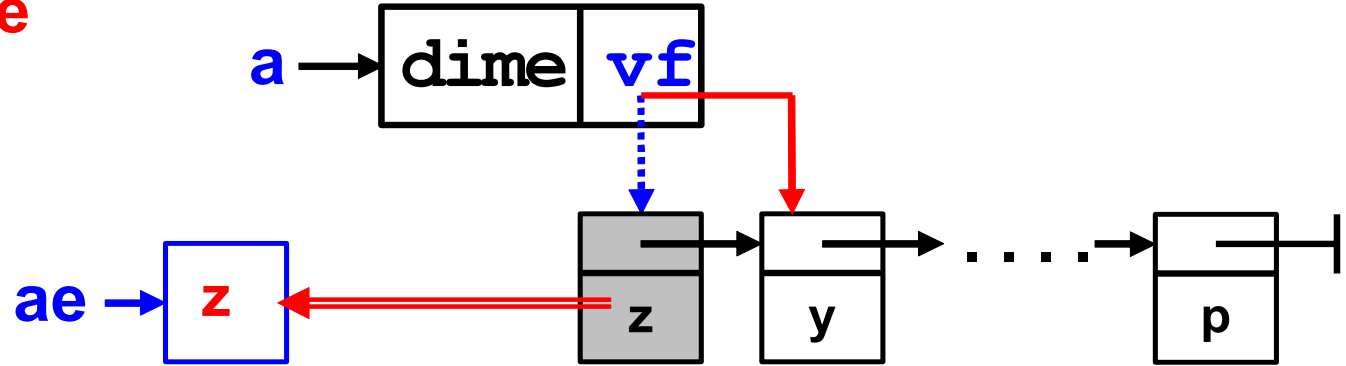
Aloca si testeaza aux->info de dimensiune `((TStiva*)a)->dime`

Copiază în aux->info valoarea de la adresa ae
`memcpy(aux->info, ae, ((TStiva*)a)->dime);`

```
aux->urm = ((TStiva*)a)->vf;  
((TStiva*)a)->vf = aux;
```

Implementare **stiva** generica – B [3]

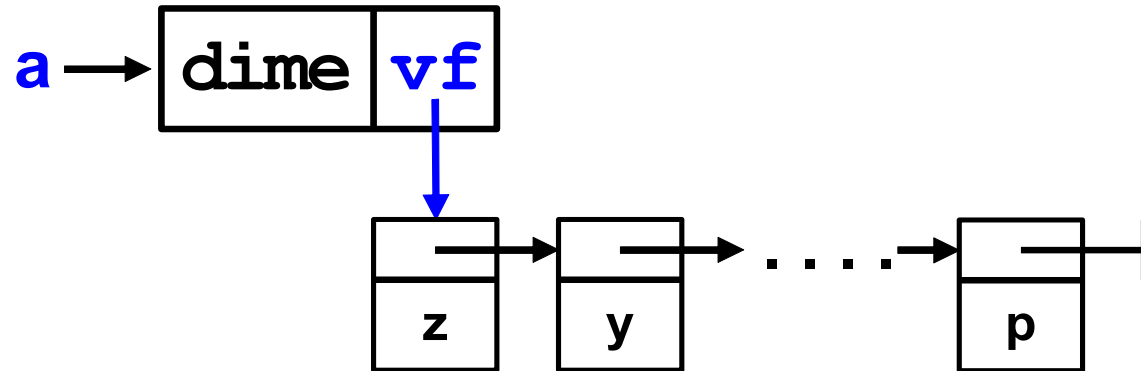
Extragere



```
int Pop(void* s, void* ae)
{ if stiva vida (((TStiva*)s)->vf == NULL) return 0;
  memcpy(ae, (((TStiva*)s)->vf->info,
            (((TStiva*)s)->dime);
  ACelSt aux = (((TStiva*)s)->vf;
  (((TStiva*)s)->vf = aux->urm;
  free(aux->info);
  free(aux);
  return 1;
}
```


Implementare **stiva** generica – B [3]

ResetS



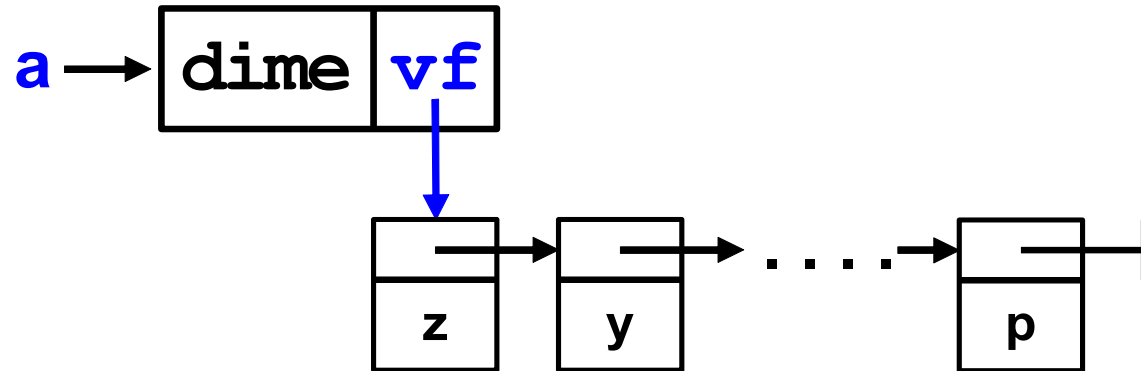
```
l = a->vf (cu cast)
```

```
Distruge lista l (l lista generica)
```

```
a->vf = NULL (cu cast)
```

Implementare **stiva** generica – B [3]

DistrS



```
l = a->vf (cu cast)
```

```
Distruge lista l (l lista generica)
```

```
a->vf = NULL (cu cast)
```

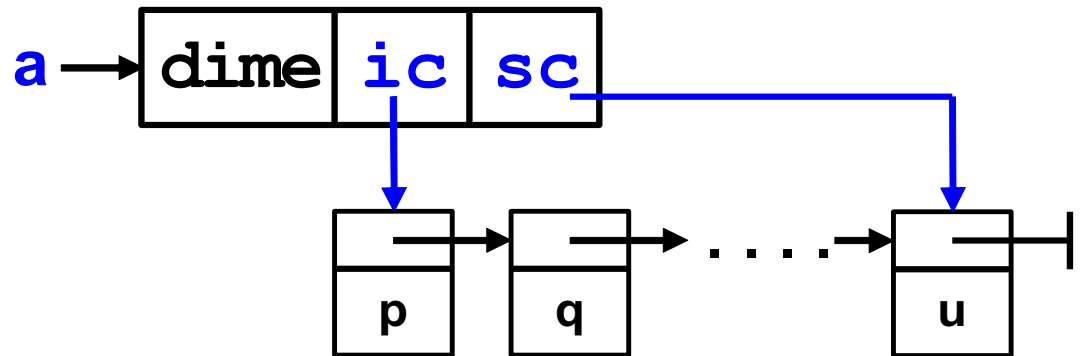
```
free(a)
```

Implementare coada generica – B [4]

B. Folosind o **lista** deschisa, fara santinela

```
typedef struct cel    /* dimensiune variabila */
{ struct cel * urm;
  void* info;
} TCell, *ACell;
```

```
typedef struct coada
{ size_t dime;    /* dim.element */
  ACell ic, sc;   /* adr.prima, ultima celula */
} TCoada, *AQ;
```

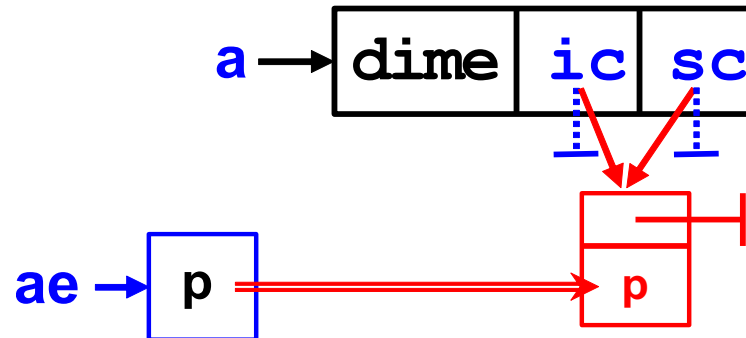


Implementare coada generica – B [5]

Coada vida: $a \rightarrow ic == NULL \ \&\& \ a \rightarrow sc == NULL$

Inserare

- in coada vida



Alocare celula + test alocare

$aux \rightarrow info = ae$ //ae trebuie sa fie alocat

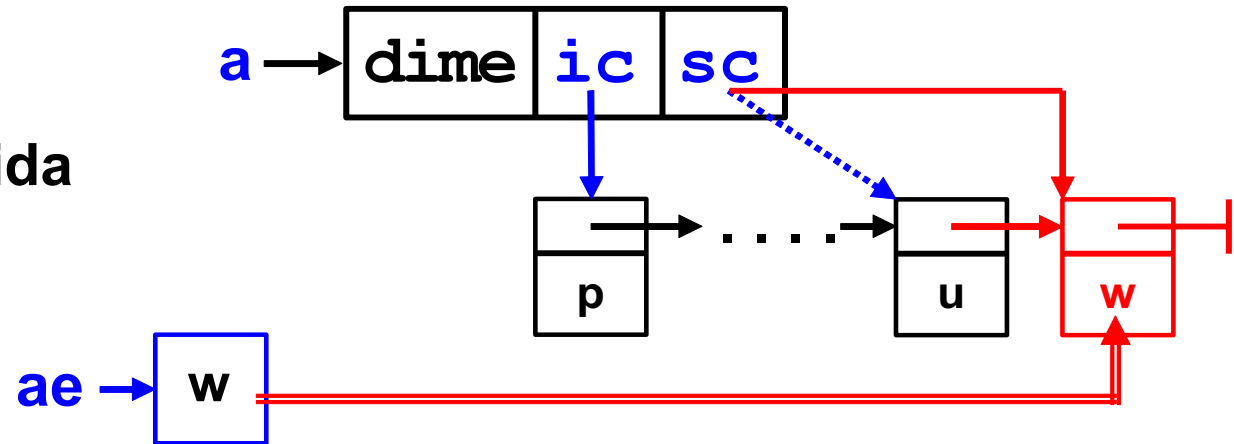
$Aux \rightarrow urm = NULL$

$ic = sc = aux$

Implementare coada generica – B [5]

Inserare

- in coada nevada



Alocare celula + test alocare

`aux->info = ae //ae trebuie sa fie alocat`

`aux->urm = NULL`

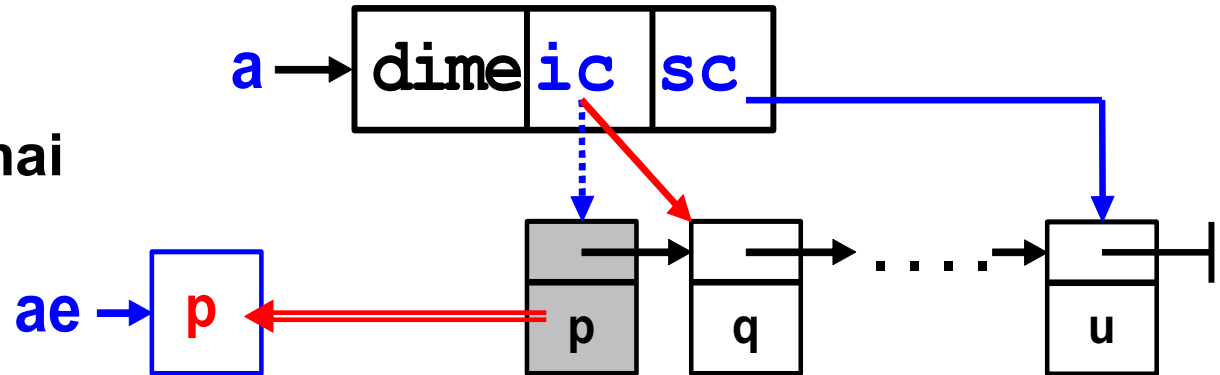
`a->sc->urm = aux`

`a->sc = aux`

Implementare coada generica – B [6]

Extragere(void* a, void* ae)

- din coada cu mai multe elemente

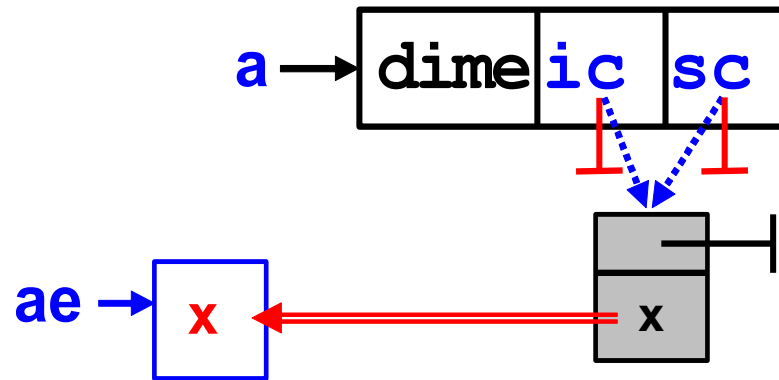


```
aux = a->ic  
a->ic = aux->urm
```

```
memcpy(ae, aux->info, a->dime)  
free(aux->info)  
free(aux)
```

Implementare **coada** generica – B [6]

Extragere - din coada cu un singur element



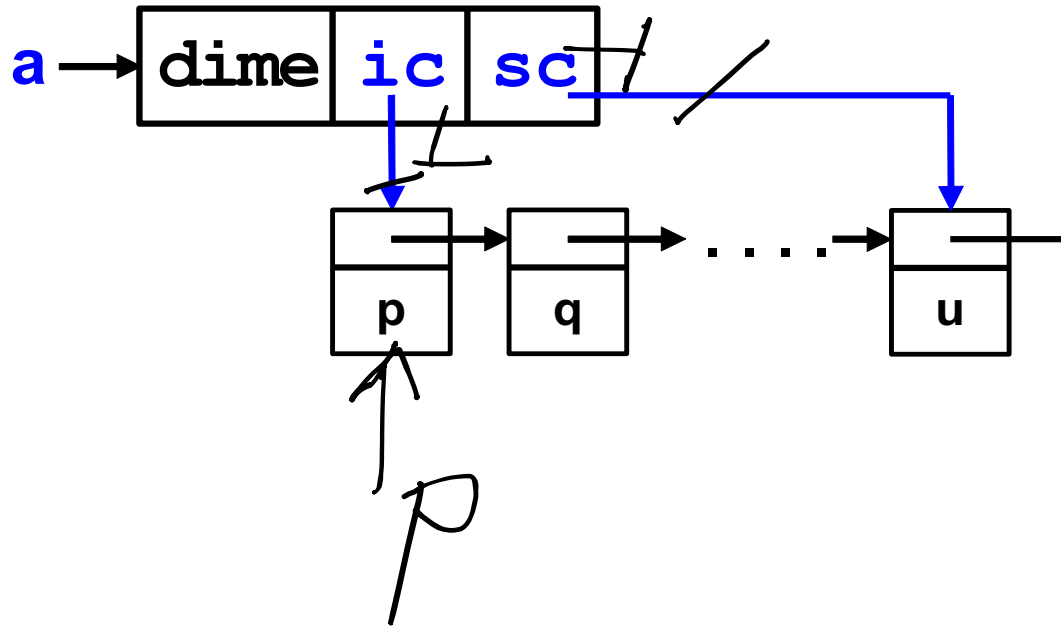
```
aux = a->ic  
a->ic = aux->urm
```

```
memcpy(ae, aux->info, a->dime)  
free(aux->info)  
free(aux)
```

```
if a->ic == NULL a->sc = NULL
```

Implementare **coada** generica – B [6]

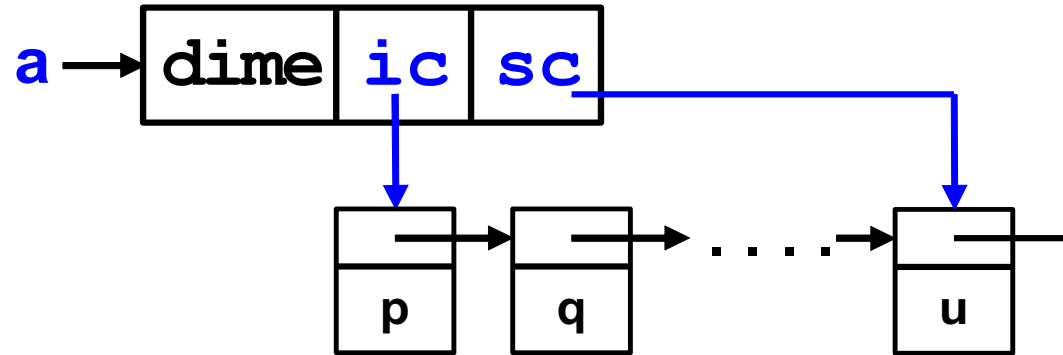
ResetQ



```
p = a->ic  
a->ic = NULL  
a->sc = NULL  
Distruge lista p
```


Implementare **coada** generica – B [6]

DistrQ



```
ResetQ(a)  
free(a)
```

Grafuri

Graful = modelul tipic pentru reprezentarea relațiilor între obiecte de același tip:

- orașe înfrățite,
- colaboratori,

Relațiile dintre obiecte pot fi reflexive sau nu

⇒ **graf neorientat** sau **orientat**.

Obiectele: **nodurile (vârfurile)** grafului

Relațiile: **muchiile (arcele)** care unesc nodurile;

Nodurile, respectiv arcele pot fi etichetate; arcelor le sunt asociate frecvent costuri.

Grafuri

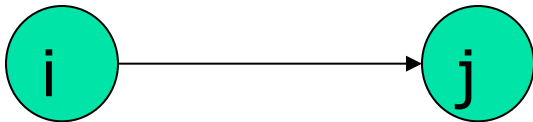
Graf orientat (en.digraph):

$G(V,A)$

unde V – mulțimea vârfurilor (vertices)

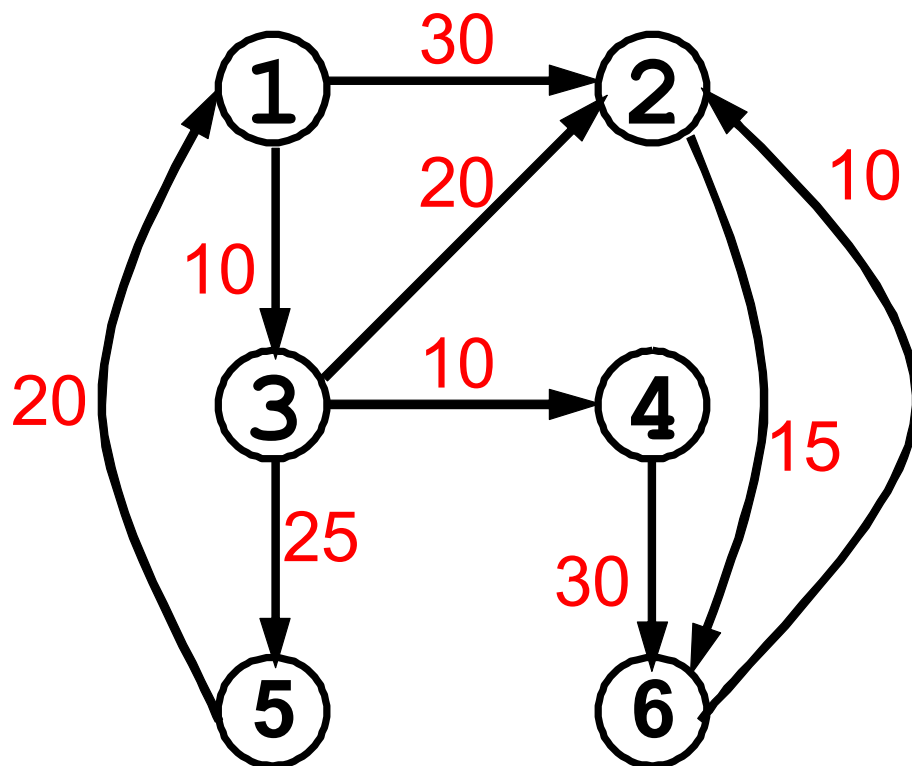
A – mulțimea arcelor (edges)

Arc: pereche ordonată de noduri (sursă, destinație) – stabilește relația succesori/predecesor direct.



Descriere graf orientat

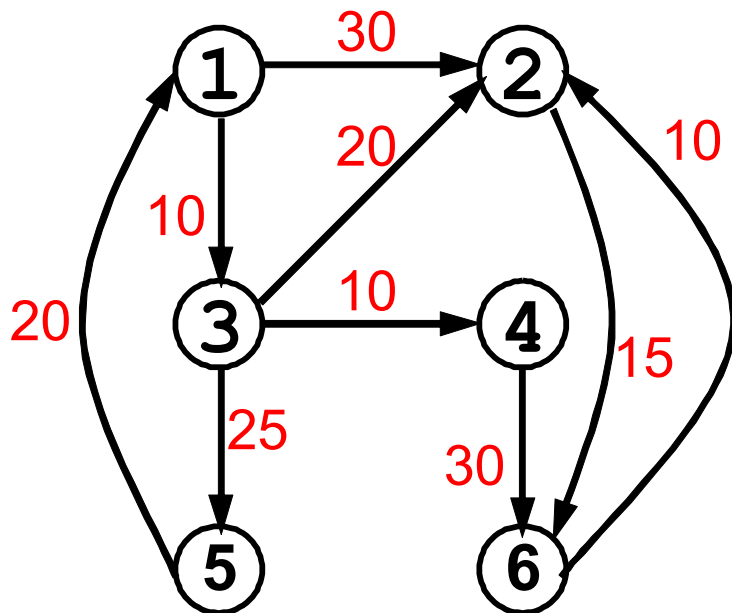
Număr noduri și listă arce



6

1	2	30
1	3	10
2	6	15
3	2	20
3	4	10
3	5	25
4	6	30
5	1	20
6	2	10

Reprezentare graf orientat



a. Matrice de adiacențe (costuri)

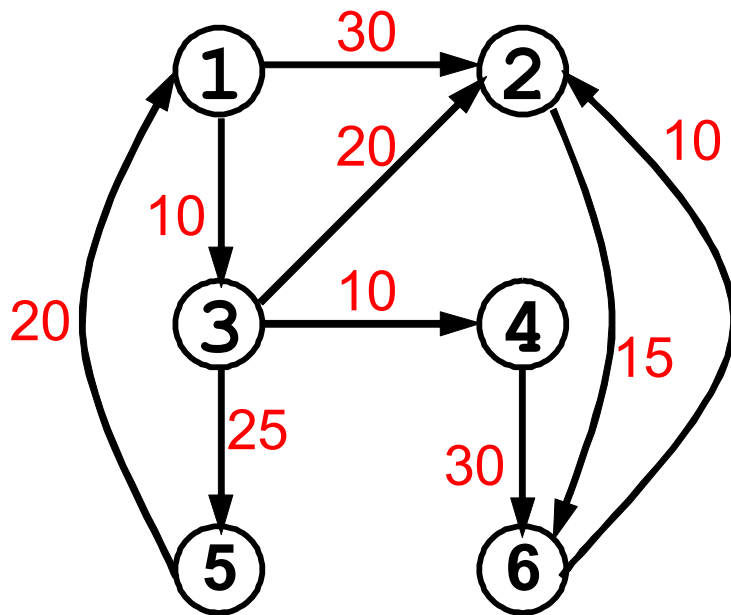
	1	2	3	4	5	6
1	-	30	10	-	-	-
2	-	-	-	-	-	15
3	-	20	-	10	25	-
4	-	-	-	-	-	30
5	20	-	-	-	-	-
6	-	10	-	-	-	-

b. Liste de adiacențe

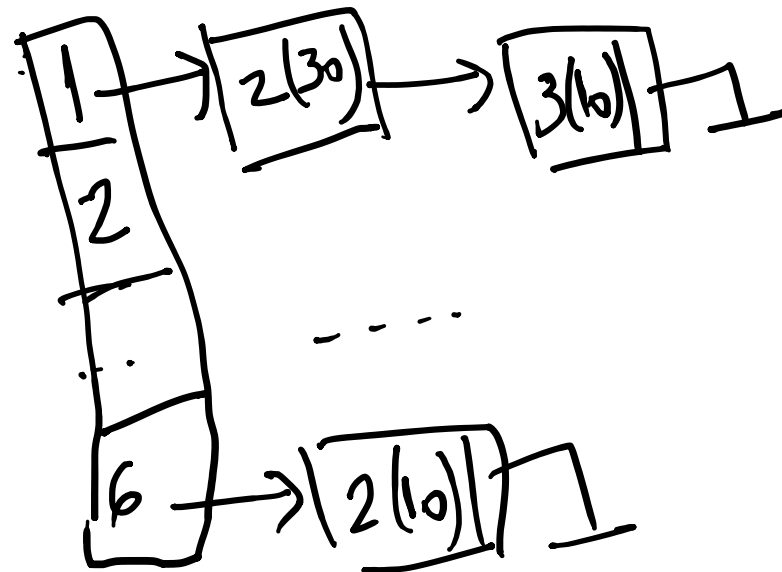
1 : 2 (30) 3 (10)
2 : 6 (15)
3 : 2 (20) 4 (10) 5 (25)
4 : 6 (30)
5 : 1 (20)
6 : 2 (10)

Reprezentare graf orientat

b. Liste de adiacențe

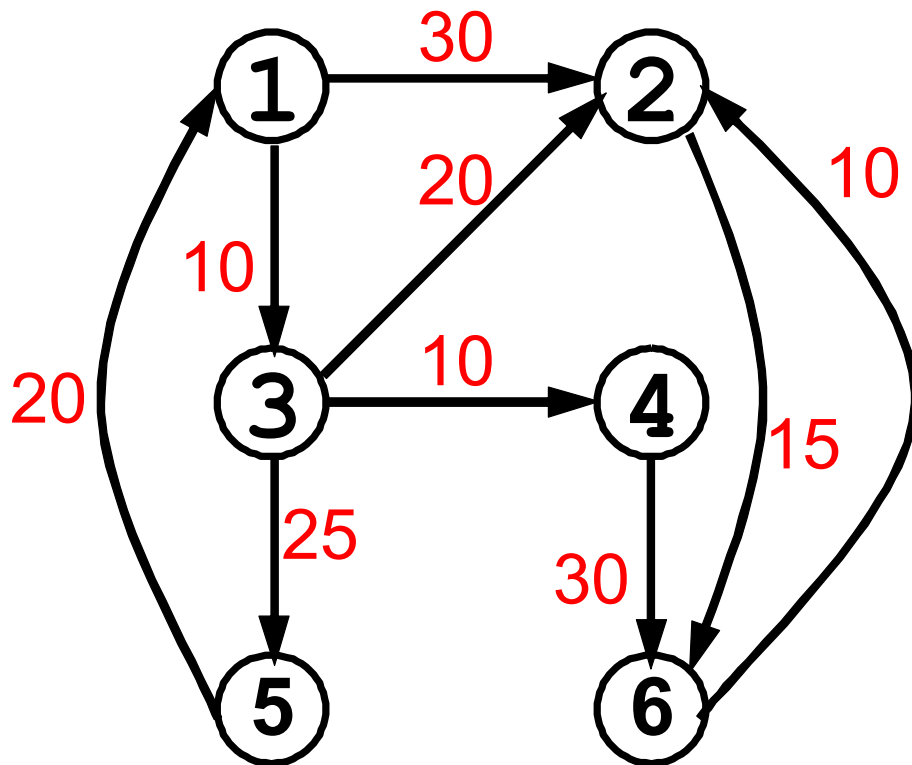


1 :	2 (30)	3 (10)	
2 :	6 (15)		
3 :	2 (20)	4 (10)	5 (25)
4 :	6 (30)		
5 :	1 (20)		
6 :	2 (10)		



Grafuri

- i-grad/e-grad: număr arce care ajung în /pleacă din nod
- i-grad: gradul de intrare al unui nod
- e-grad: gradul de iesire al unui nod

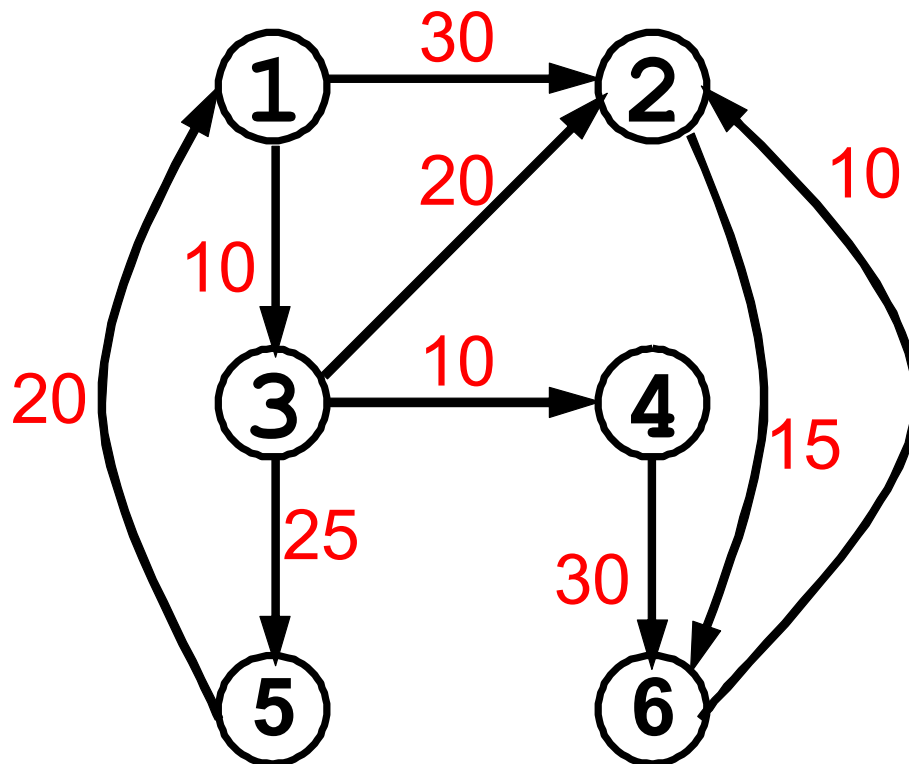


i-grad(2)=(3)
e-grad(2)=1

Noduri
adiacente cu
nodul n:
nodurile
pentru care
existe arce de
la n la v

Grafuri

- **cale**: succesiune de noduri conectate prin arce;
numărul de arce parcurse = lungimea căii
- **cale simplă**: trece numai prin noduri distincte
- **ciclu**: cale pe care primul nod = ultimul nod
- **graf aciclic**: nu conține cicluri.



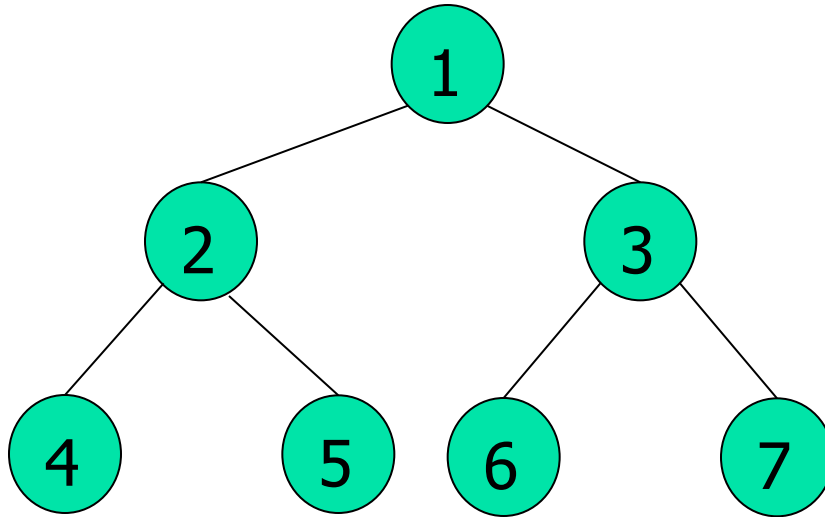
Cale: 1, 2, 6, 2

Cale: 1, 3, 4, 6, 2, 6

Ciclu: 1, 3, 5, 1

Grafuri

Arbori:



Arborii sunt cazuri particulare de grafuri aciclice orientate, cu un singur nod cu $i\text{-grad} = 0$ și restul cu $i\text{-grad} = 1$.

Parcurgere grafuri

- Parcurgere în adâncime (Depth First Search / DFS)
- Parcurgere în lățime (Breadth First Search / BFS)

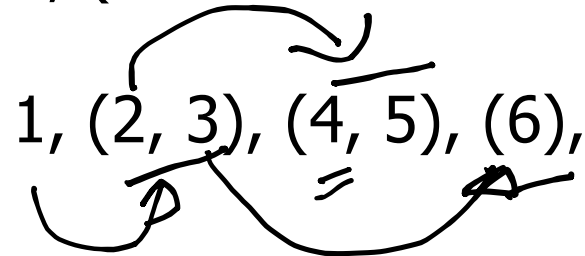
Parcursare în latime

Parcursare începând de la nodul 1

Descriere graf

1	2	30
1	3	10
2	4	15
2	5	20
3	6	10
4	6	30
5	3	20
5	4	15
6	2	10

1, (noduri adiacente cu 1), ...



Coada: inserare noduri adiacente
vizitat: 0/1 (daca nodul i a fost vizitat)
– in momentul in care se introduce
nodul in coada

1, 2, 3, 4, 5, 6

Parcurgere în latime

Parcurgere începând de la nodul 1

Descriere graf

1 2 30

1 3 10

2 4 15

2 5 20

3 6 10

4 6 30

5 3 20

5 4 15

6 2 10

Coadă:1

vizitat: 0, 0, 0, 0, 0, 0

Coadă:

Afisare 1

Vizitat: 1, 0, 0, 0, 0, 0

Coadă: 2, 3

Coadă:3

Afisare 1, 2

Vizitat: 1, 1, 0, 0, 0, 0

Coadă: 3, 4, 5

Parcurgere în latime

Parcurgere începând de la nodul 1

Descriere graf

1 2 30

1 3 10

2 4 15

2 5 20

3 6 10

4 6 30

5 3 20

5 4 15

6 2 10

Coadă:3

Afisare 1, 2

Vizitat: 1, 1, 0, 0, 0, 0

Coadă: 3, 4, 5

Coadă:4, 5

Afisare 1, 2, 3

Vizitat: 1, 1, 1, 0, 0, 0

Coadă: 4, 5, 6

Coadă: 5, 6

Afisare 1, 2, 3, 4

Vizitat: 1, 1, 1, 1, 0, 0

Coadă: 5, 6

Parcurgere în latime

Parcurgere începând de la nodul 1

Descriere graf

1 2 30

1 3 10

2 4 15

2 5 20

3 6 10

4 6 30

5 3 20

5 4 15

6 2 10

Coadă:1

vizitat: 1, 0, 0, 0, 0, 0

Coadă:

Afisare 1

Vizitat: 1, 1, 1, 0, 0, 0

Coadă: 2, 3

Coadă: 3

Afisare 1, 2

Vizitat: 1, 1, 1, 1, 1, 0

Coadă: 3, 4, 5

Parcurgere în latime

Parcurgere începând de la nodul 1

Descriere graf

1 2 30

1 3 10

2 4 15

2 5 20

3 6 10

4 6 30

5 3 20

5 4 15

6 2 10

Coada: 4, 5

Afisare 1, 2, 3

Vizitat: 1, 1, 1, 1, 1, 1

Coada: 4, 5, 6

Coada: 5, 6

Afisare 1, 2, 3, 4

Vizitat: 1, 1, 1, 1, 1, 1

Coada: 5, 6

Coada: -

Afisare: 1, 2, 3, 4, 5, 6

Vizitat: 1, 1, 1, 1, 1, 1

Parcurgere în latime

Parcurgere în adâncime pornind de la nodul n

IntrQ(q, n)

marchează n vizitat

while(q != vida)

{

 n = ExtrQ(q)

 for fiecare a adiacent cu n

 {

 daca a e nevizitat

 {

 marchează a vizitat

 IntrQ(q, a)

 }

 }

}

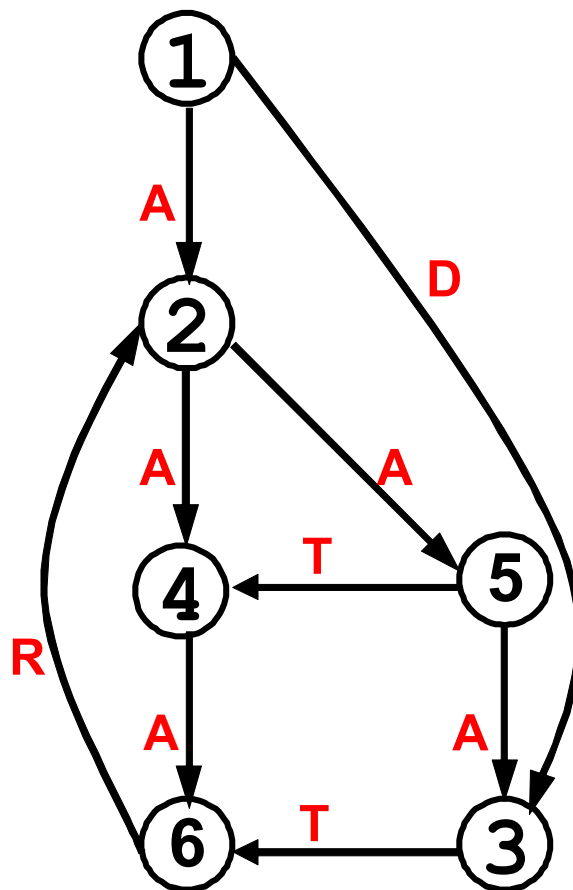
Parcurgere în adâncime

Marcaj arce:

A – arbore; **R** – revenire; **T** – traversare; **D** – direct

Descriere graf Parcurgere începând de la nodul 1

6
1 2 30
1 3 10
2 4 15
2 5 20
3 6 10
4 6 30
5 3 20
5 4 15
6 2 10



1 - 2 (A)
2 - 4 (A)
4 - 6 (A)
6 - 2 (R)
2 - 5 (A)
5 - 3 (A)
3 - 6 (T)
5 - 4 (T)
1 - 3 (D)

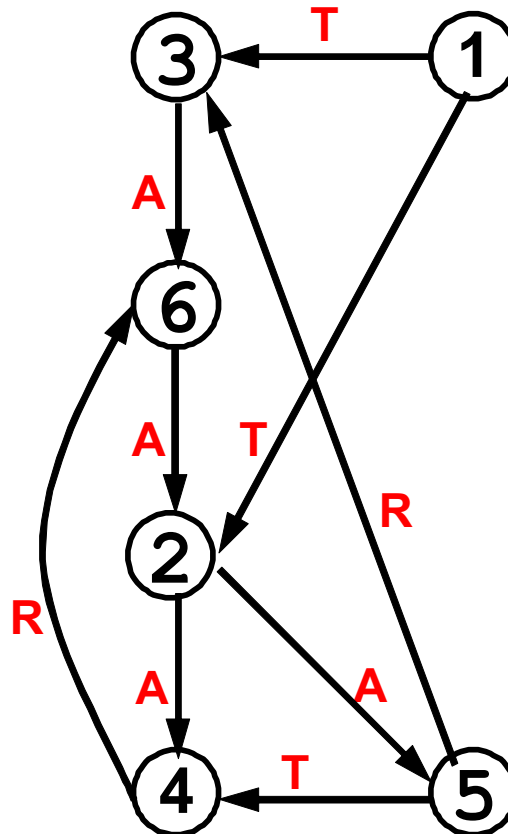
Parcurgere în adâncime

Marcaj arce:

A – arbore; **R** – revenire; **T** – traversare; **D** – direct

Descriere graf Parcurgere începând de la nodul 3

6
1 2 30
1 3 10
2 4 15
2 5 20
3 6 10
4 6 30
5 3 20
5 4 15
5 3 20
6 2 10



3 - 6 (A)
6 - 2 (A)
2 - 4 (A)
4 - 6 (R)
2 - 5 (A)
5 - 3 (R)
5 - 4 (T)
1 - 2 (T)
1 - 3 (T)

Parcurgere în adâncime

Pentru stabilirea tipului arcelor se utilizează **indicele de parcurgere $ip[i]$** , asociat nodului **i** , cu semnificatia

- 0 – netratat (alb)
- > 0 – în curs de tratare (gri)
- < 0 – tratat (negru)

$ip[0]$ = valoare indice curent (**IC**)

Cod_arc(x,a):

- A** - nodul **a** netratat ($ip[a] == 0$)
- R** - nodul **a** in_curs de tratare ($ip[a] > 0$)
- T** - nodul **a** tratat anterior ($ip[a] < 0$ si – $ip[a] < ip[x]$)
- D** - nodul **a** tratat ulterior ($ip[a] < 0$ si – $ip[a] > ip[x]$)

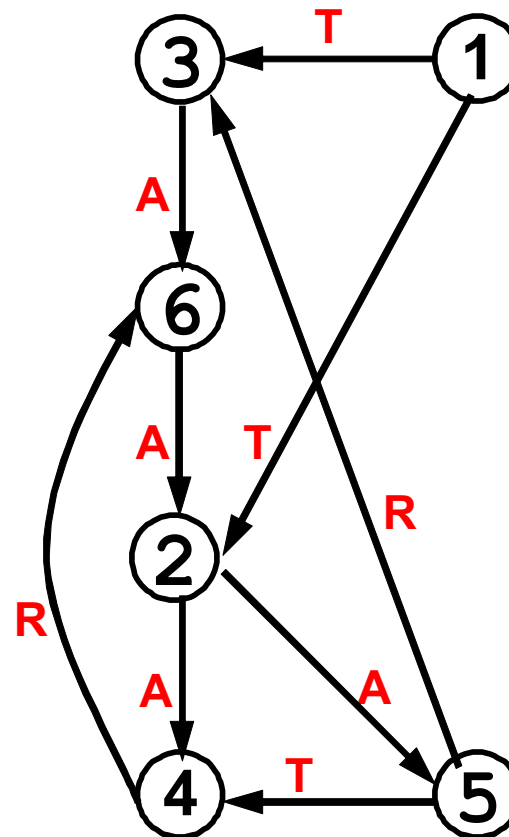
Algoritm parcurgere în adâncime

```
Algoritm ParcAd(info_parcurgere, x)
{ crește IC;
  marchează x "in_curs" (ip[x] = IC);
  pentru fiecare nod adiacent(a)
  { afișează x,a,Cod_arc(x,a)
    dacă (a "netratat") atunci
    { crește nivel;
      ParcAd(info_parcurgere, a);
      reduce nivel;
    }
  }
  marchează x ca "tratată" (schimbă semn ip[x]);
}
```

Algoritm testare graf aciclic

Descriere graf

1	2	30
1	3	10
2	4	15
2	5	20
3	6	10
4	6	30
5	3	20
5	4	15
6	2	10

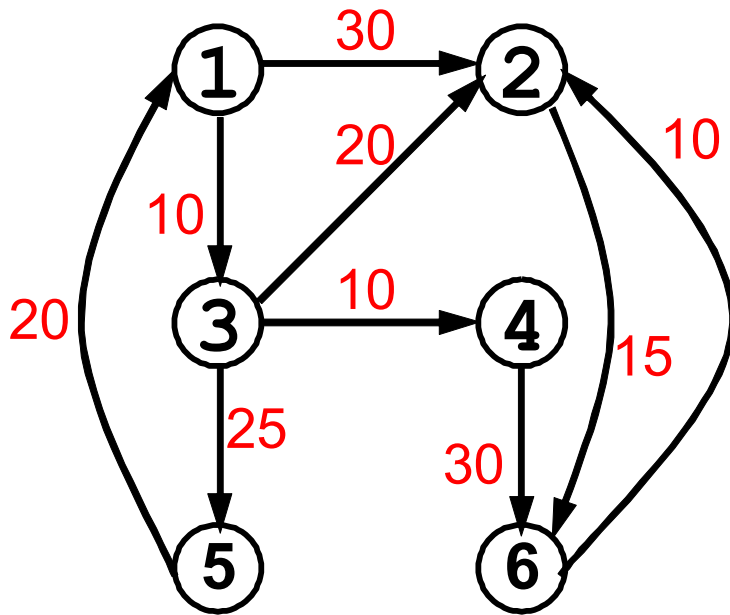


Algoritm testare graf aciclic

```
Algoritm Aciclic(info_parcurgere, x)
{ marchează x "in_curs";
  pentru fiecare nod adiacent(a)
  { dacă (a "in_curs") atunci întoarce 0;
    dacă a "netratat" atunci
      dacă (!Aciclic(info_parcurgere,a))
        atunci întoarce 0;
  }
  marchează x ca "tratată";
  întoarce 1;
}
```

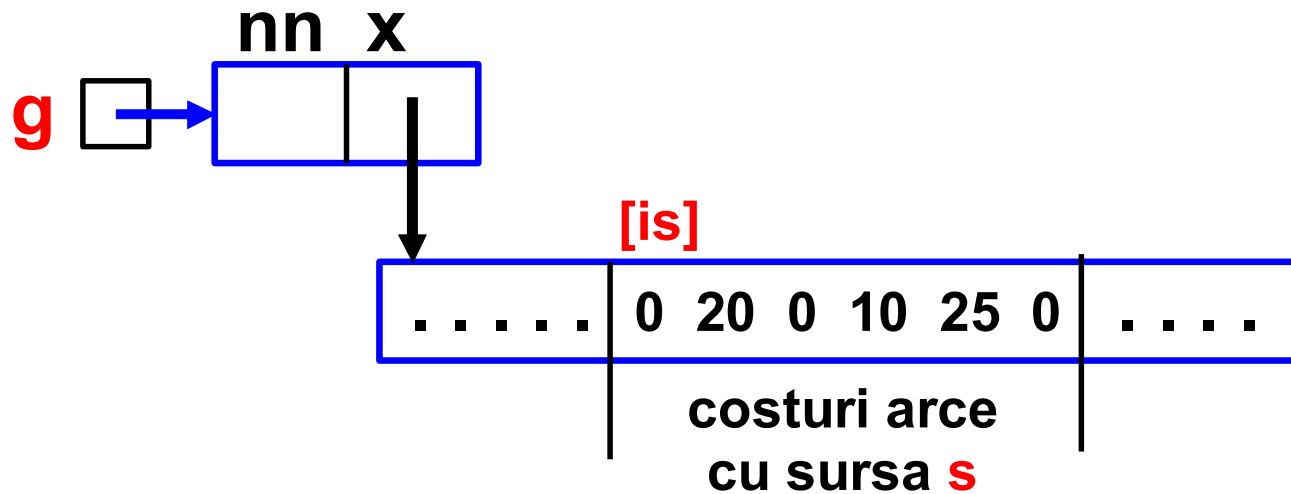
Reprezentare graf

Matrice de adiacențe (costuri)

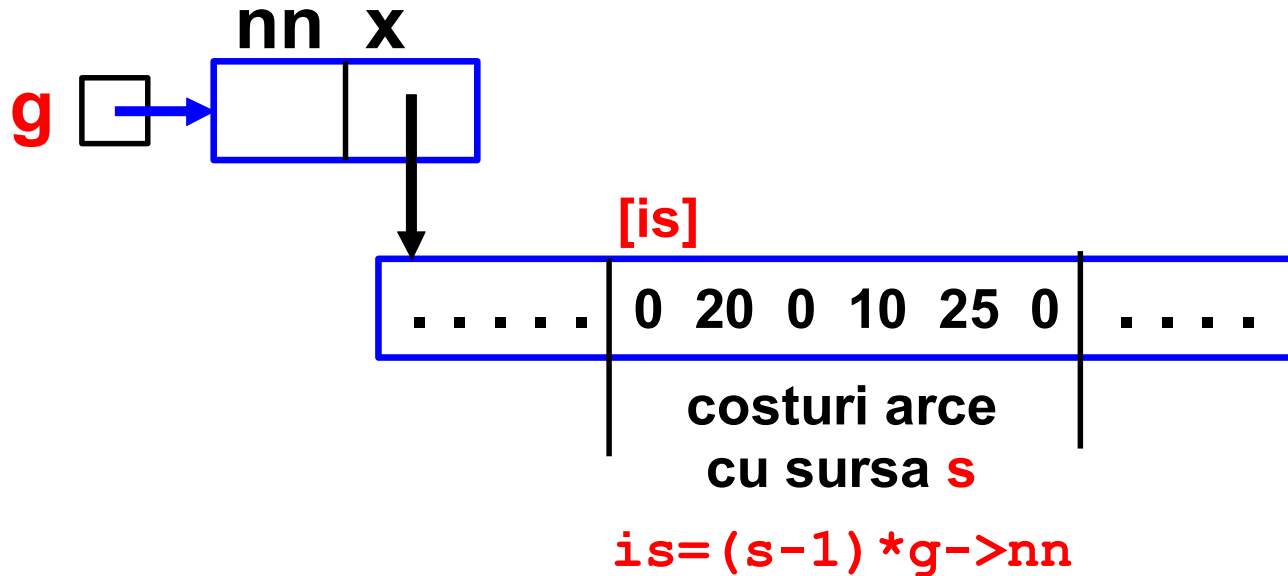


	1	2	3	4	5	6
1	-	30	10	-	-	-
2	-	-	-	-	-	15
3	-	20	-	10	25	-
4	-	-	-	-	-	30
5	20	-	-	-	-	-
6	-	10	-	-	-	-

1. Reprezentare ca matrice liniarizată costuri



Reprezentare ca matrice liniarizată costuri



```
typedef struct
{ int nn;          /* numar noduri */
  int* x;          /* vector costuri =
                    matrice liniarizata */
} TGM, *AG;        /* descriptor */
```

Reprezentare graf prin matrice de adiacență

```
typedef struct
```

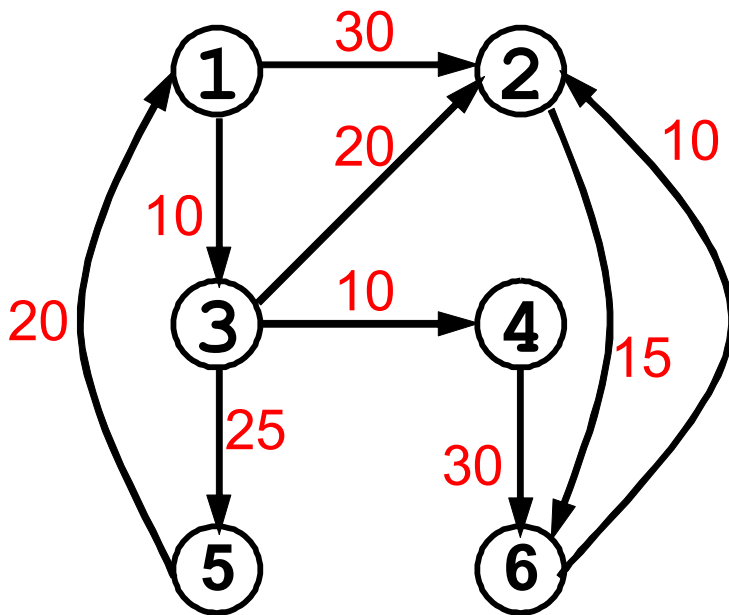
```
{ int n;    /* numar noduri */  
  int* x;   /* costuri = matrice liniarizata */  
} TGM, *AGM;
```

```
void* g; //graf
```

```
Cost arc (i, j) = ?
```

```
((AGM) g) ->x[ (i-1) * ((AGM) g) ->n + j-1] = x;
```

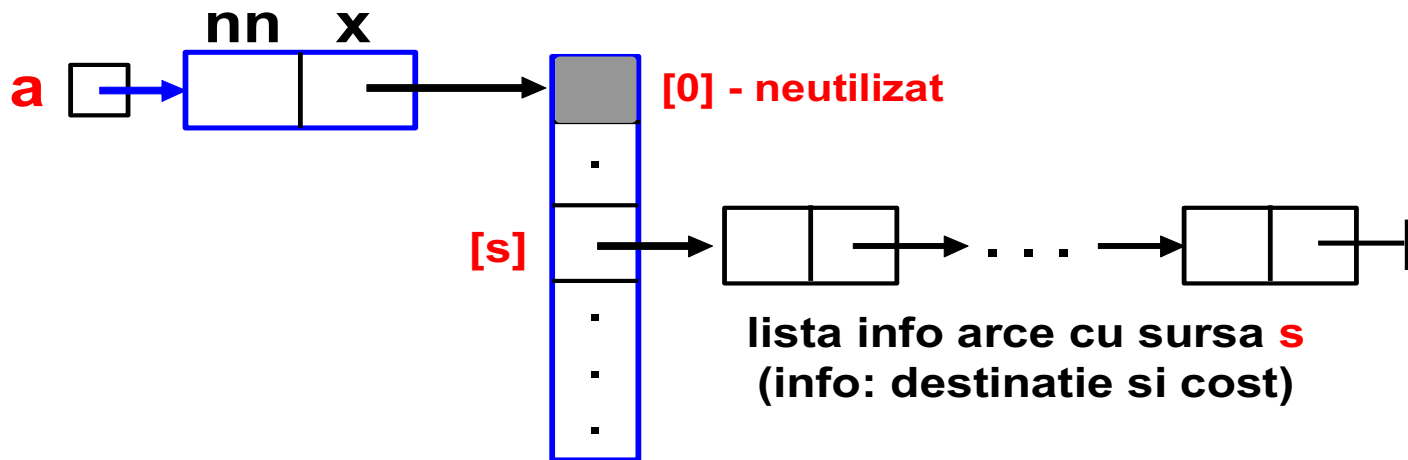
Reprezentare graf prin listă de adiacență



Liste de adiacențe

1 :	2 (30)	3 (10)	
2 :	6 (15)		
3 :	2 (20)	4 (10)	5 (25)
4 :	6 (30)		
5 :	1 (20)		
6 :	2 (10)		

Reprezentare cu liste înlănțuite de info arce



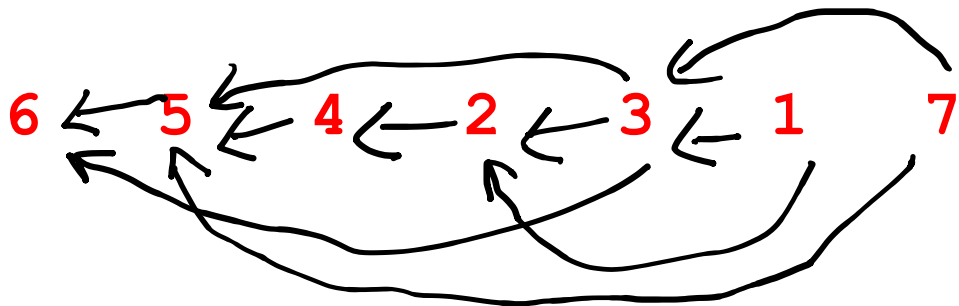
```
typedef struct celarc
{ int d;           /* destinatie arc */
  int c;           /* cost arc */
  struct celarc *urm;
} TCelArc, *AArc;

typedef struct
{ int nn;          /* numar noduri */
  AArc *x;         /* vector adrese liste arce */
} TGL, *AG;
```

Sortare topologică

	1	2	3	4	5	6	7
1	-	40	35	-	-	-	-
2	-	-	-	25	-	-	-
3	-	20	-	-	15	30	-
4	-	-	-	-	20	-	-
5	-	-	-	-	-	15	-
6	-	-	-	-	-	-	-
7	-	-	12	-	15	-	-

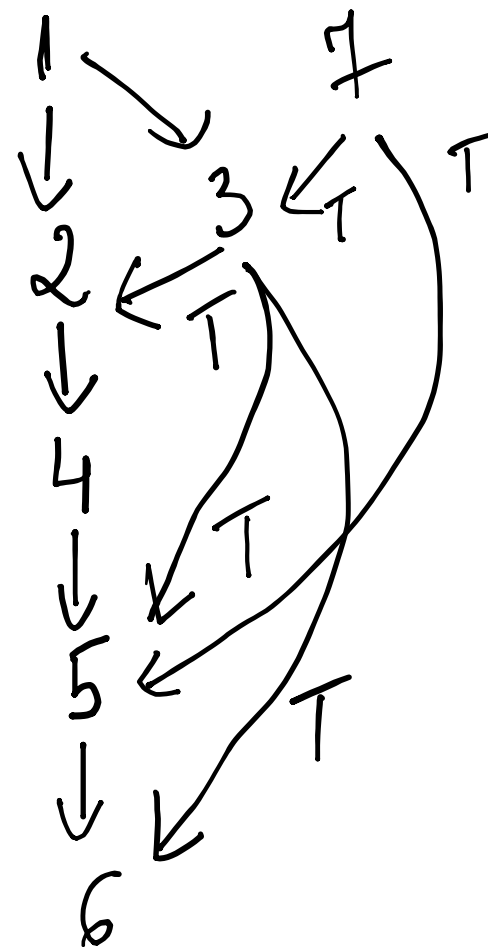
Rezultat sortare topologica:



Sortare topologică

	1	2	3	4	5	6	7
1	-	40	35	-	-	-	-
2	-	-	-	25	-	-	-
3	-	20	-	-	15	30	-
4	-	-	-	-	20	-	-
5	-	-	-	-	-	15	-
6	-	-	-	-	-	-	-
7	-	-	12	-	15	-	-

6, 5, 4, 2, 3, 1, 7



Sortare topologică

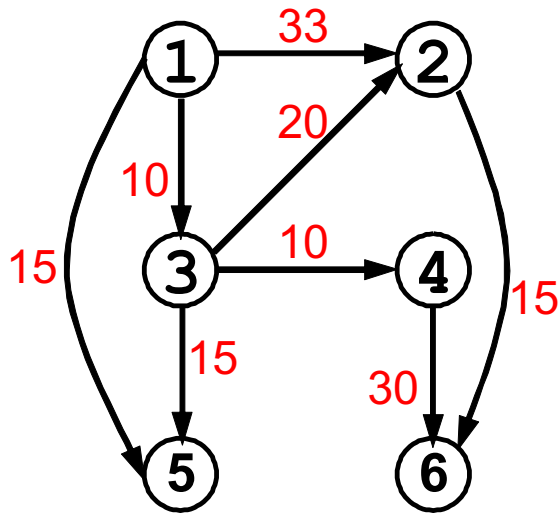
	—	1 —	2 —	3 —	4 —	5 —	6 —
1	—	—	30	10	—	—	—
2	—	—	—	—	—	—	15
3	—	—	20	—	10	25	—
4	—	—	—	—	—	—	30
5	20	—	—	—	—	—	—
6	—	—	10	—	—	—	—

Ciclu pe calea: 1 2 6 2

Algoritm sortare topologică

```
Algoritm SortTopologică(info_parcurgere, x)
{ marchează x "in_curs";
  pentru fiecare nod adiacent(a)
  { dacă (a "in_curs") atunci întoarce 0;
    dacă a "netratat" atunci
      dacă (!SortTopologică(info_parcurgere, a))
        atunci întoarce 0;
  }
  marchează x ca "tratată";
  memorează ca "sortat"
  întoarce 1;
}
```


Algoritmul Dijkstra



Sursa: 1

selectate	Distanțe				
	2	3	4	5	6
1	33	10	INF	15	INF
1, 3	30	10	20	15	INF
1, 3, 5	30	10	20	15	INF
1, 3, 5, 4	30	10	20	15	50
1, 3, 5, 4, 2	30	10	20	15	45

Algoritmul Dijkstra

Dijkstra: Distanța minimă de la sursă la celelalte noduri

```
{ initializeaza structuri de date auxiliare
  (vector distante fata de sursa,
  vector predecesori);
marcheaza sursa "tratat";
initializeaza nr = nr.noduri - 1;
cat timp nr > 0
{ alege x - nodul netratat cel mai apropiat
  de sursa;
  daca nu exista atunci iesire fortata (break);
  marcheaza nodul x ca "tratat";
  pentru fiecare nod netratat (j)
    daca adiacent lui x, inaccesibil din sursa
      sau accesibil pe un drum mai lung
      actualizeaza distanta si predecesor;
  }
}
```

Arbori de selectie (Heap)

Memoreaza elementele conform unei **relatii de ordine** definite pe multimea cheilor si asigura **extragerea rapida** a elementelor in ordinea dictata de aceasta relatie.

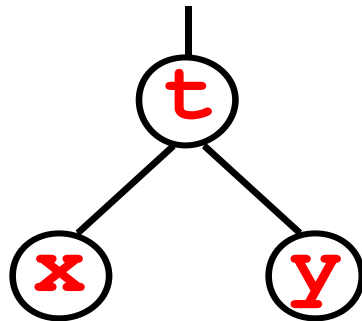
➤ tratarea cererilor de acordare a unei burse in **ordinea descrescatoare a mediei** generale a solicitantilor

In literatura se folosesc si termenii de **Heap** (nu are legatura cu zona de memorie destinata alocarii dinamice), **arbore partial ordonat**, **coada de prioritati**.

Arbori de selectie (Heap)

Caracteristici:

- este arbore binar de **inaltime minima** ($\log n$)
- **inserarea** se realizeaza pe principiul parcurgerii in latime
→ **ultimul nivel este completat de la stanga spre dreapta**
- pentru **orice pereche de noduri tata-fiu** cheile **respecta relatia de ordine prestabilita** (cheia nodului tata este predecesoare sau echivalenta cu cele ale nodurilor fiu)



$$\text{Rel}(t, x) \leq 0$$

$$\text{Rel}(t, y) \leq 0$$

■ qsort

- v, nv, dim elem, fcmp

- fcmp(a,b):

 - 0: a echiv b

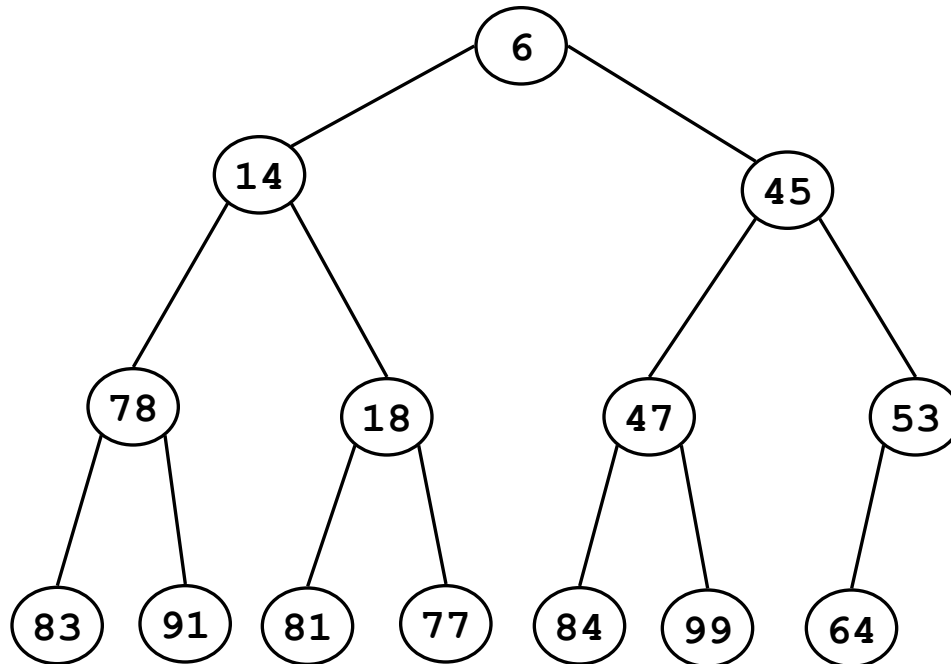
 - < 0 : a, b

 - > 0 : b, a

- Min-heap: $\text{info}(\text{parinte}) < \text{info}(\text{st}), \text{info}(\text{dr})$

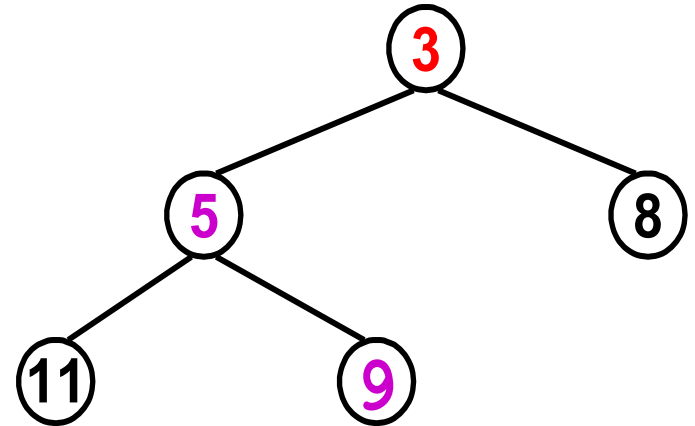
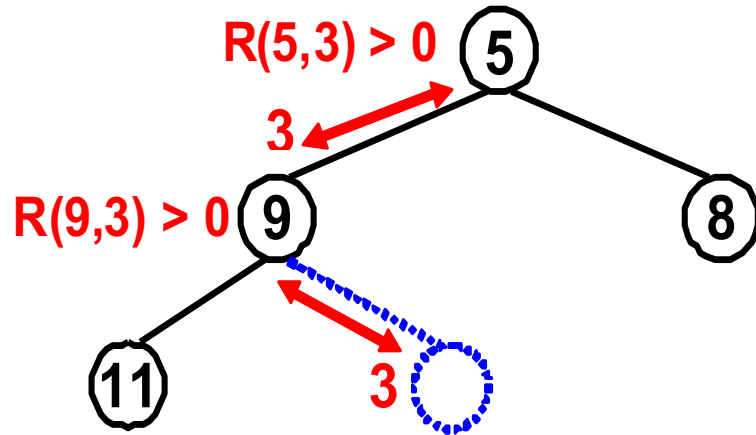
- Max-heap: $\text{info}(\text{parinte}) > \text{info}(\text{st}), \text{info}(\text{dr})$

Exemplu: heap (min-heap)

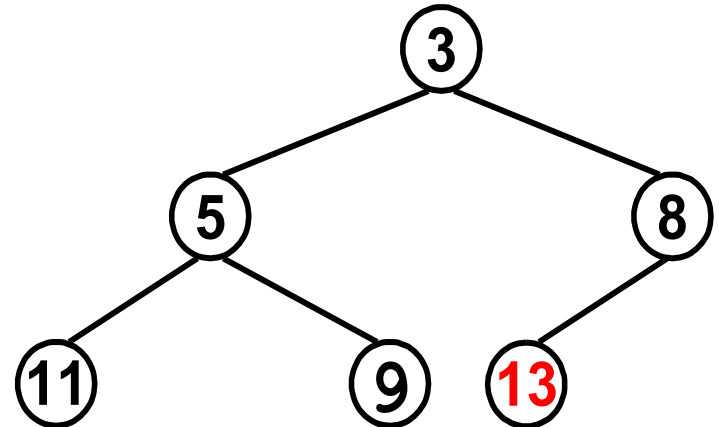
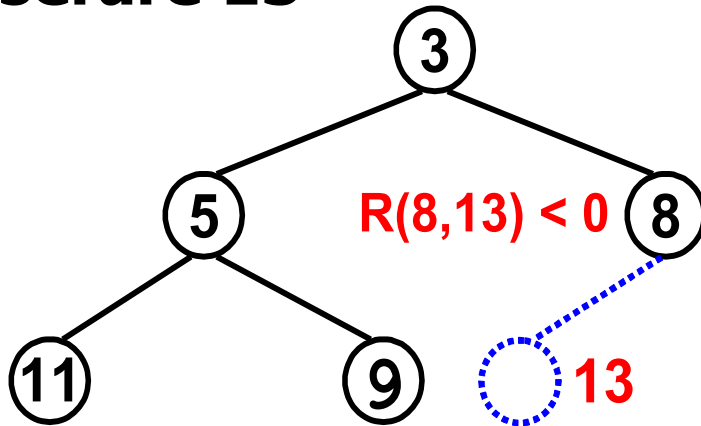


Inserare in heap (min-heap)

Inserare 3

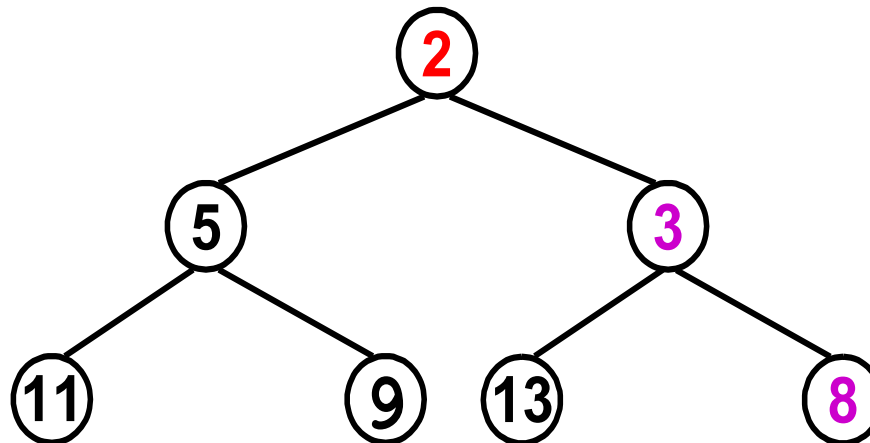
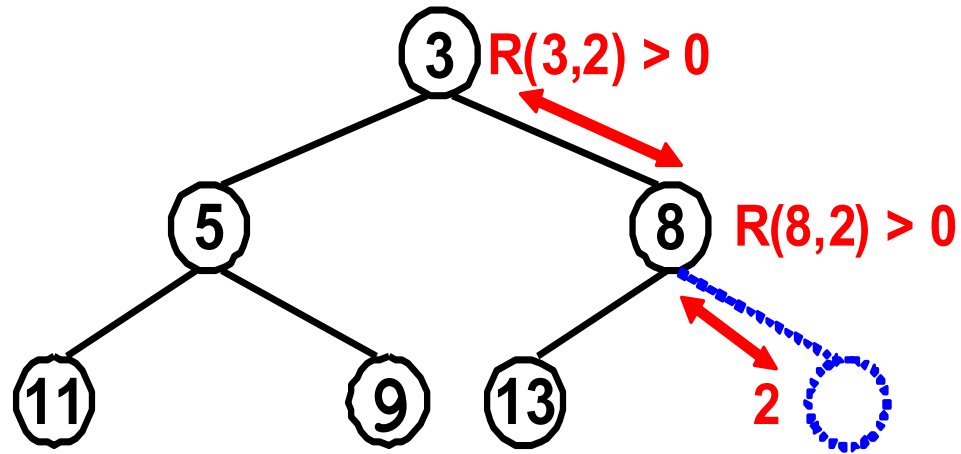


Inserare 13

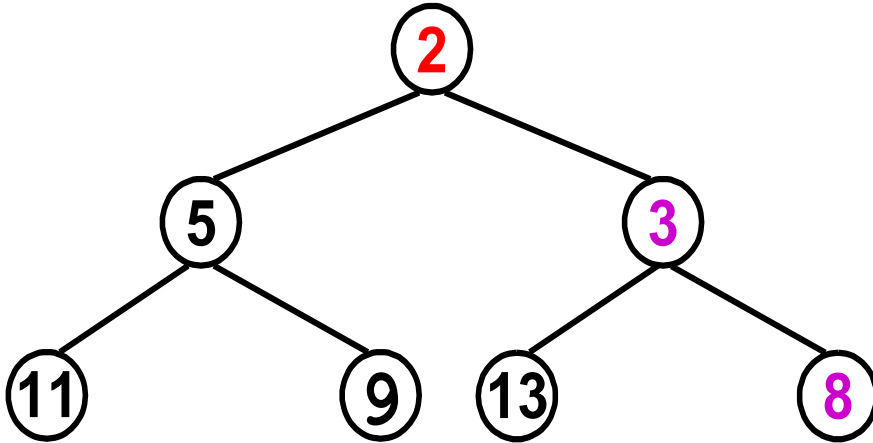


Inserare in heap (min-heap)

Inserare 2



Insertare in heap (min-heap)



Algoritm inserare in heap

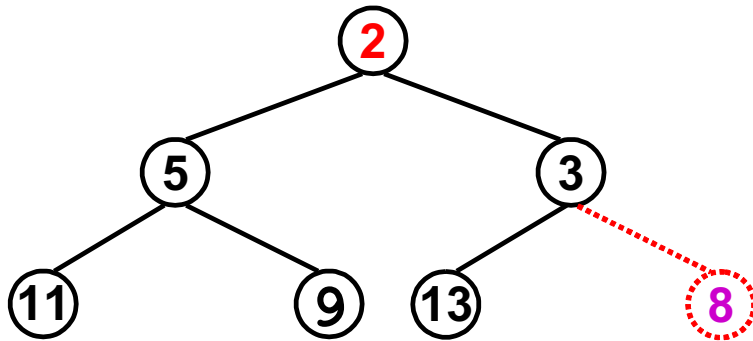
Fie **x** informatia ce trebuie inserata in arborele A

```
{ adauga in A nod Nou, fara informatie;  
  D = Nou;  
  cat timp (D != Radacina(A) si  
            Rel(cheie(Tata(D)), cheie(x)) > 0)  
  { transfer info(Tata(D)) in D;  
    D = Tata(D);  
  }  
  transfera x in nodul D;  
}
```

Extragere din heap

- Se extrage informatia din radacina
- Se elimina nodul din extrema dreapta a ultimului nivel.
- Informatia din nodul eliminat ramane in radacina sau, daca este cazul, se propaga spre nivelurile inferioare, pentru a pastra organizarea de arbore de selectie.

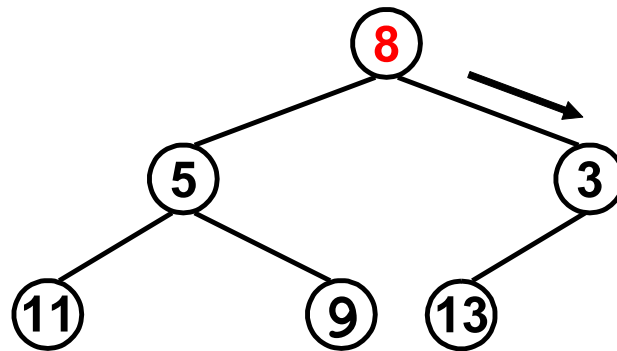
Eliminare din heap (min-heap)



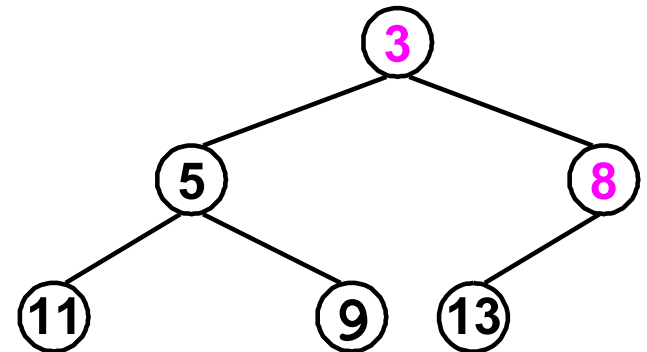
Extras = 2

Temp = 8

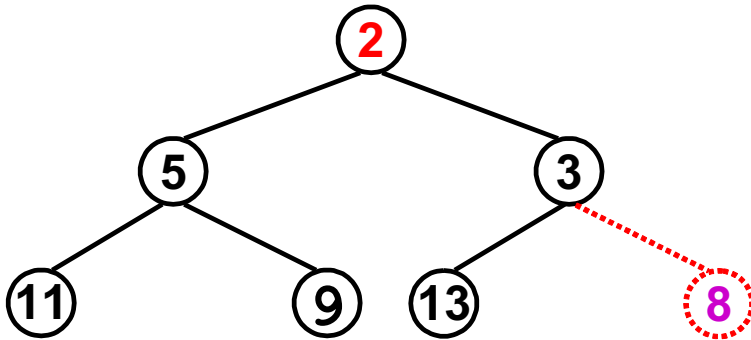
Elim val din rad



$R(3, 8) < 0$



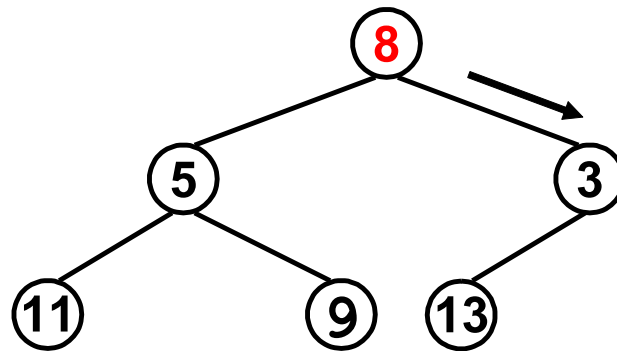
Eliminare din heap (min-heap)



Extras = 2

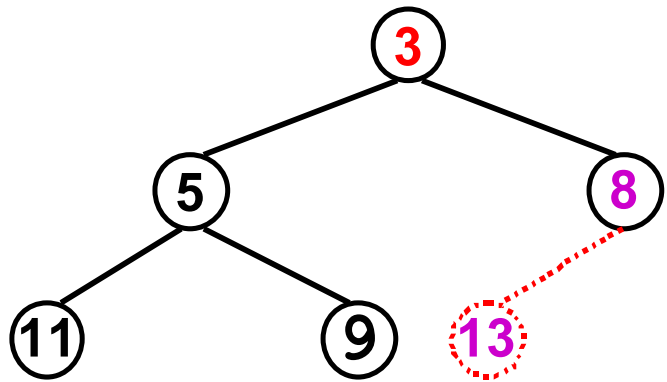
Temp = 8

Elimina
valoarea din
radacina



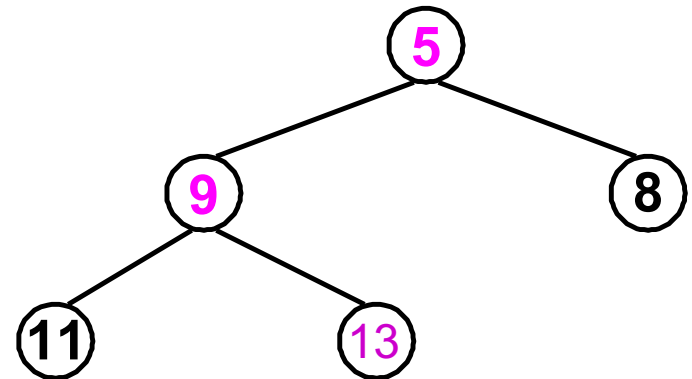
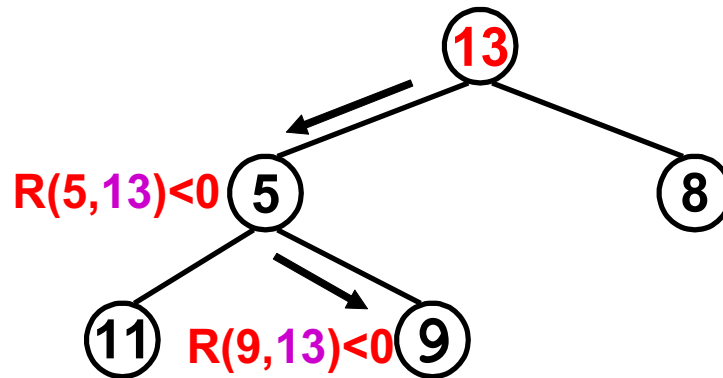
$R(3, 8) < 0$

Eliminare din heap (min-heap)

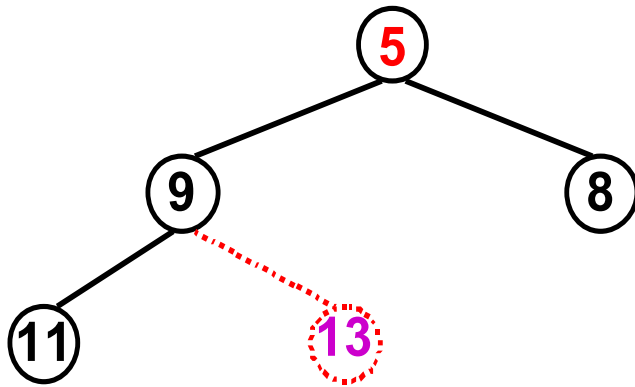


Extras = 3

Temp = 13

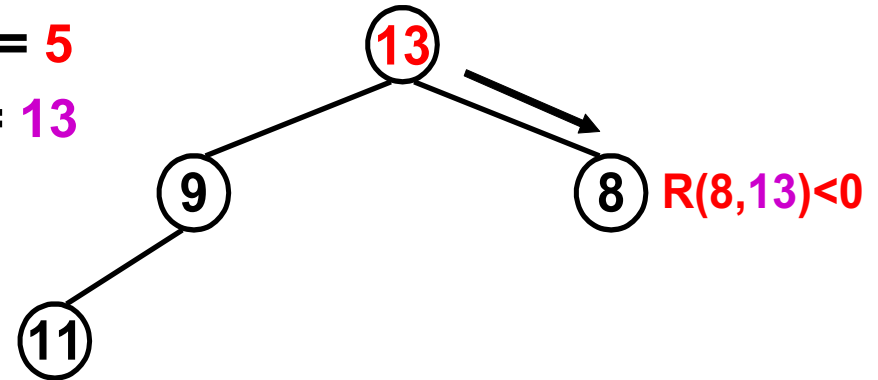


Eliminare din heap (min-heap)

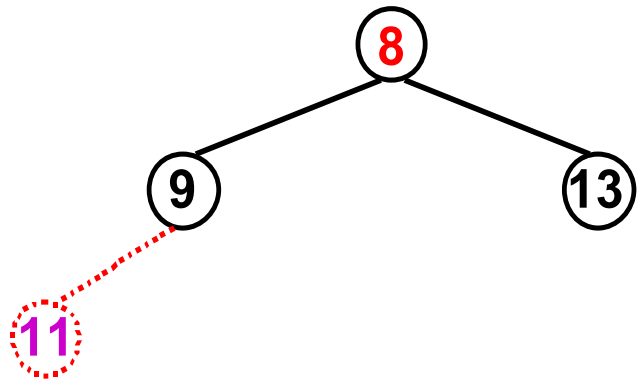


Extras = 5

Temp = 13

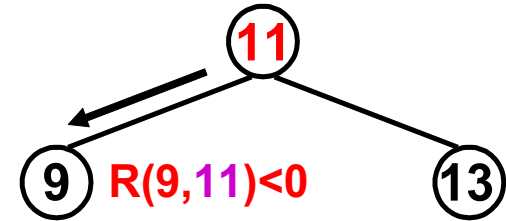


Eliminare din heap (min-heap)

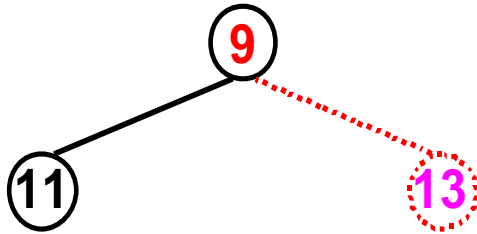


Extras = 8

Temp = 11

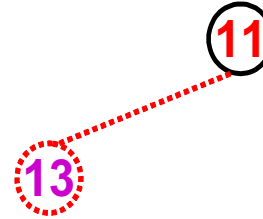


Eliminare din heap (min-heap)



Extras = 9

Temp = 13



Extras = 11

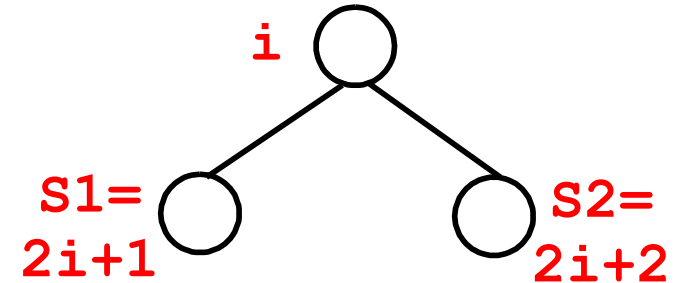
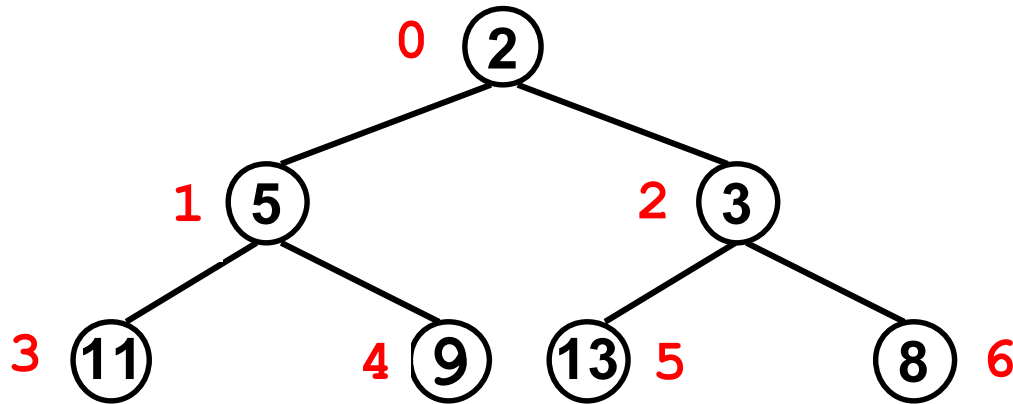
Temp = 13



Algoritm eliminare din heap

```
{ Extras = info(radacina);
  D = radacina;
  T = info(ultimul_nod); info(D) = T;
  elimina ultimul nod;
  cat timp exista fiu_st(D)
  { s = fiu_st;
    daca (exista fiu_dr(D)
          && R(cheie(s), cheie(fiu_dr))) > 0
      atunci s = fiu_dr;
    daca R(cheie(T), cheie(s)) <= 0 atunci iesire
      altfel { interschimba info(D) cu info(s);
              D = s;
            }
  }
  intoarce Extras;
}
```

Reprezentare heap



Indice parinte(i) = $(i-1) / 2$

Indice Subarbore stang(i) = $2*i + 1$

Indice Subarbore stang(i) = $2*i + 1$

Pentru reprezentarea arborelui de selectie se poate utiliza un vector.

In cazul de fata vectorul va contine:

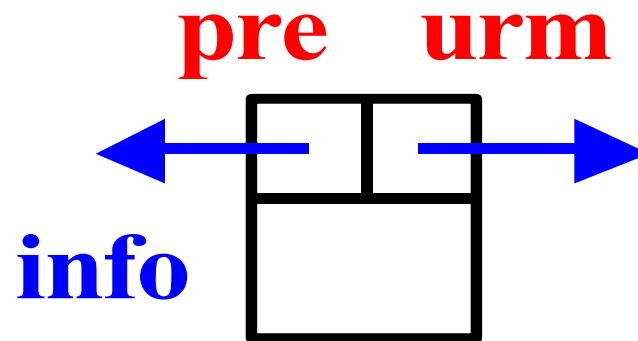
{ 2, 5, 3, 11, 9, 13, 8 }

Liste dublu inlantuite

Celulele din lista au doua campuri legatura:

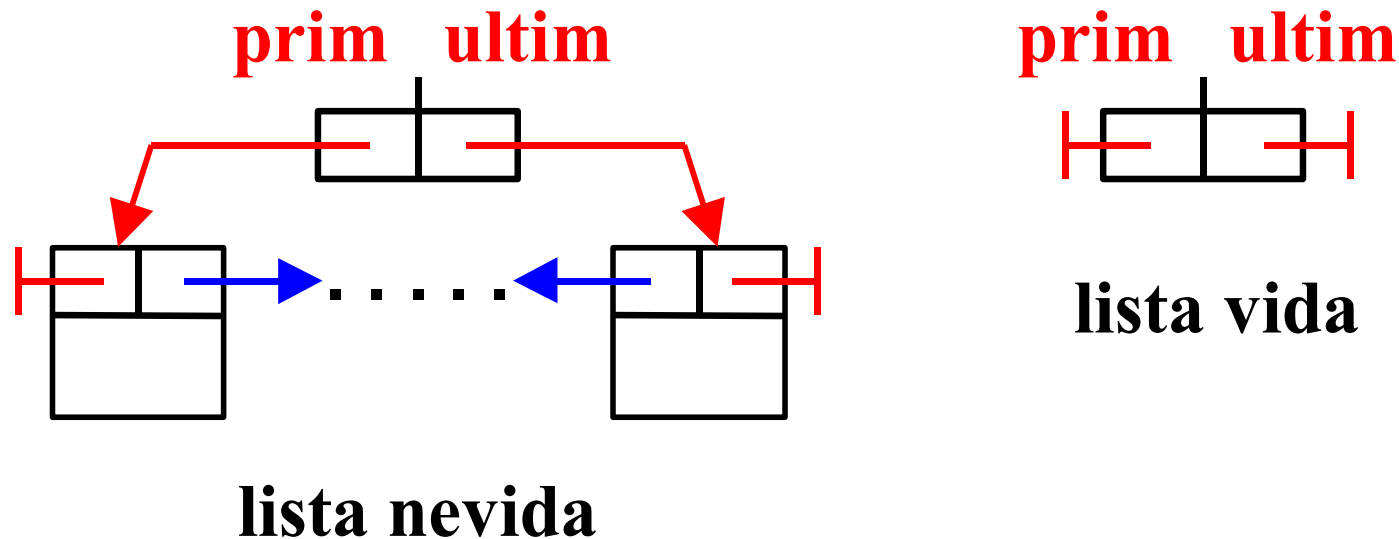
pre – adresa celulei predecesoare

urm – adresa celulei urmatoare



Pentru **parcurgerea eficienta** a listei in oricare din cele doua sensuri (**de la inceput** sau **de la sfarsit**) este necesar accesul la cele doua extremitati.

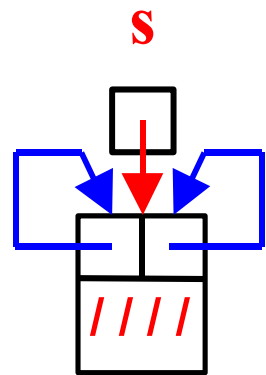
Liste dublu inlantuite deschise



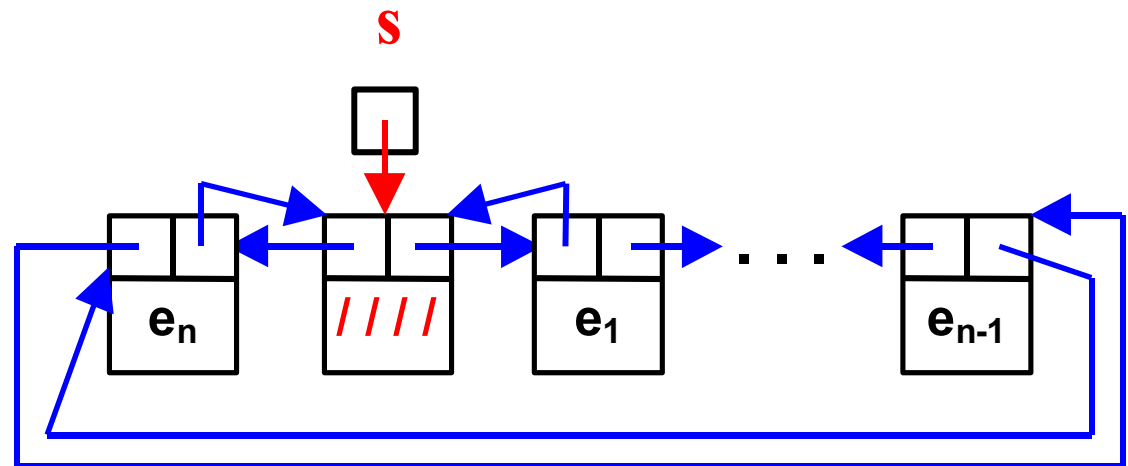
Se observa ca in aceasta abordare exista **diferente de tratare** in cazul modificarilor (inserare / eliminare) **la extremitati**, deoarece acestea efecteaza fie campul **prim**, fie campul **ultim**.

Liste dublu inlantuite circulare

Pentru o **tratare uniforma** se poate adopta solutia de organizare ca **lista circulara cu santinela**.



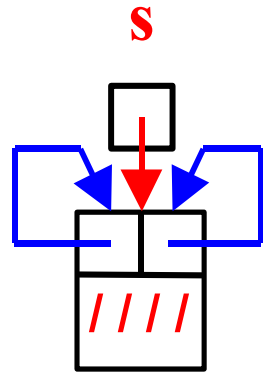
lista vida



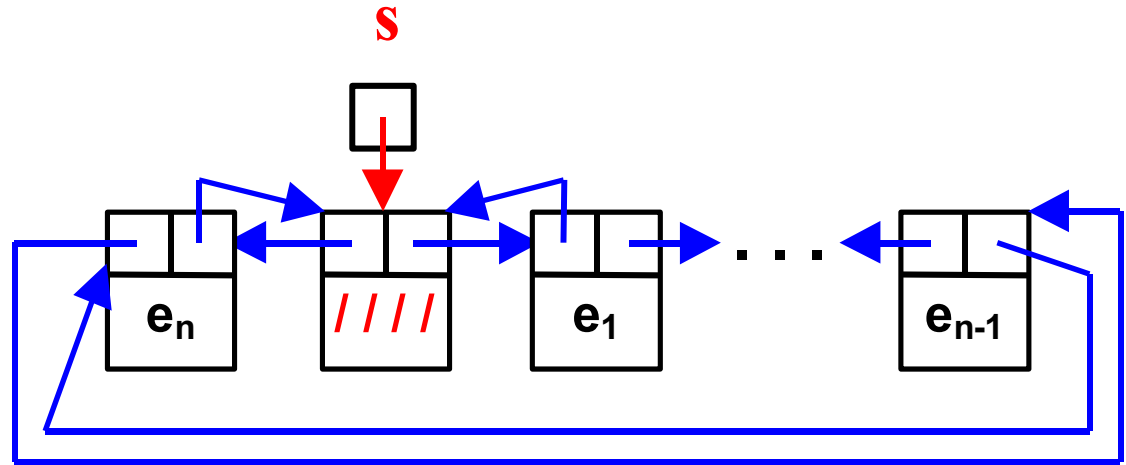
lista nevida

Campul **info** al celulei **santinela** poate fi utilizat pentru pastrarea unor informatii cu caracter general (de ex. referinta la functia de verificare a relatiei de ordine in cazul in care elementele din lista sunt ordonate).

Liste dublu inlantuite circulare



lista vida



lista nevida

```
typedef struct cel2
{ struct cel2 *pre, *urm;
  void* info;
} T Cel2, *TL2;
```

Liste dublu inlantuite circulare

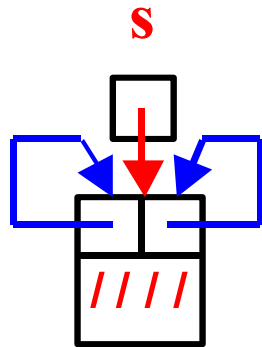
Test lista vida:

$(s \rightarrow \text{urm} == s)$ sau $(s \rightarrow \text{pre} == s)$

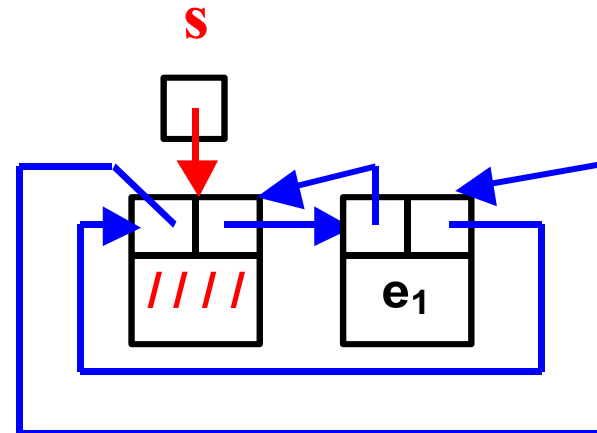
Atentie! Expresia

$(s \rightarrow \text{urm} == s \rightarrow \text{pre})$

are valoarea 1 si in cazul listei cu un singur element.



lista vida

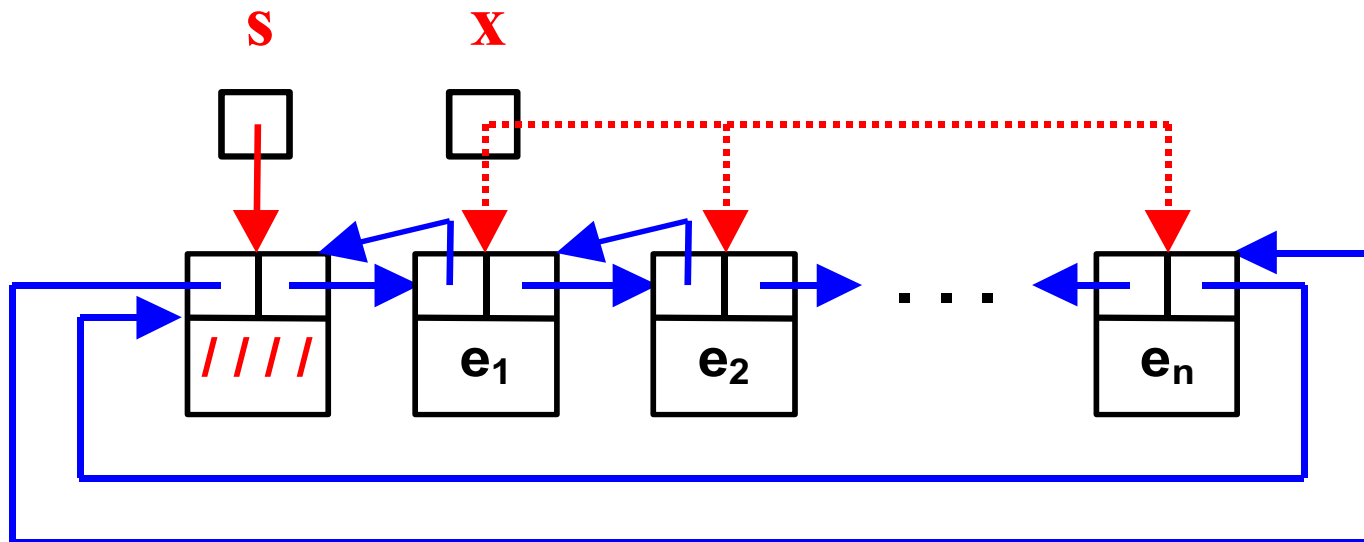


lista cu un element

Parcurgerea listei dublu inlantuite circulare

- de la inceput

```
for (x = s->urm; x != s; x = x->urm)
{ . . . }
```



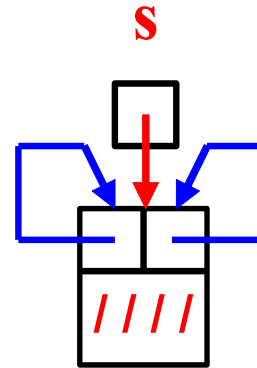
- de la sfarsit

```
for (x = s->pre; x != s; x = x->pre)
{ . . . }
```

Initializare lista dublu inlantuita circulara

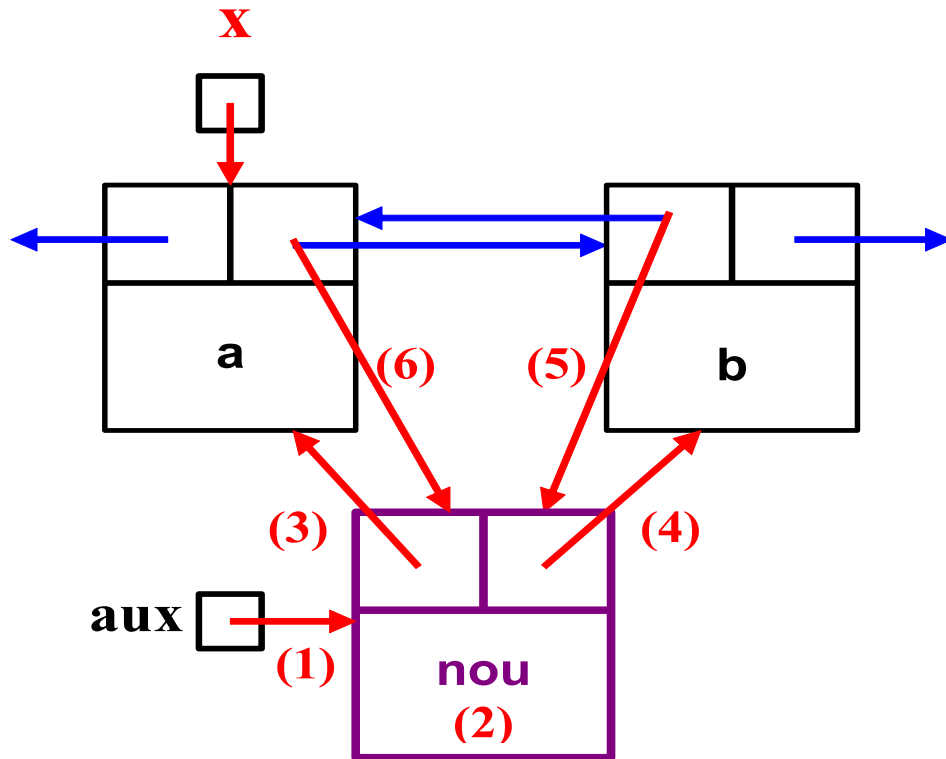
```
TL2 InitL2()
```

```
{  
    TL2 aux = (TL2)malloc(sizeof(TCel2));  
    if(aux)  
    { aux->pre = aux->urm = aux;  
      aux->info = NULL;  
    }  
    return aux;  
}
```



lista vida

Inserare dupa celula cu adresa x



Secventa de operatii:

- **alocare spatiu** pentru o noua celula (**1**)
- **actualizarea campurilor** noii celule (**2,3,4**)
- **actualizare referinte** la noua celula (**5,6**)

Inserare dupa celula cu adresa p

```
int InsDupa(TL2 a, void* x)
```

```
/* inserare dupa celula cu adresa a -> 1/0 */
```

```
{
```

```
    TL2 aux = (TL2)malloc(sizeof(TCelula2));
```

```
    if(!aux) return 0;
```

```
    aux->info = x; /*2*/
```

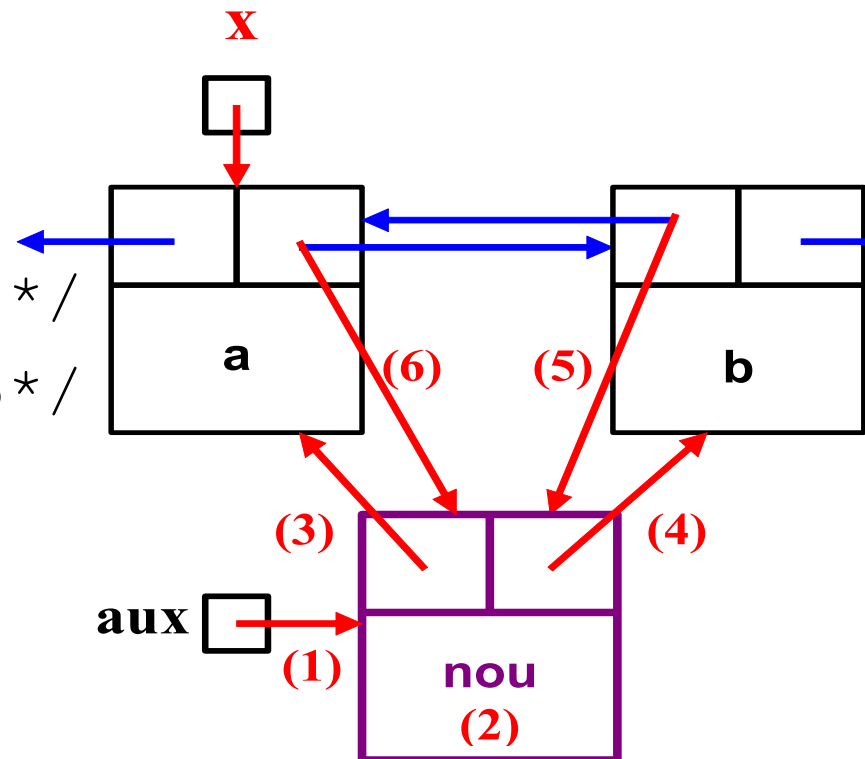
```
    aux->pre = a; /*3*/
```

```
    aux->urm = a->urm; /*4*/
```

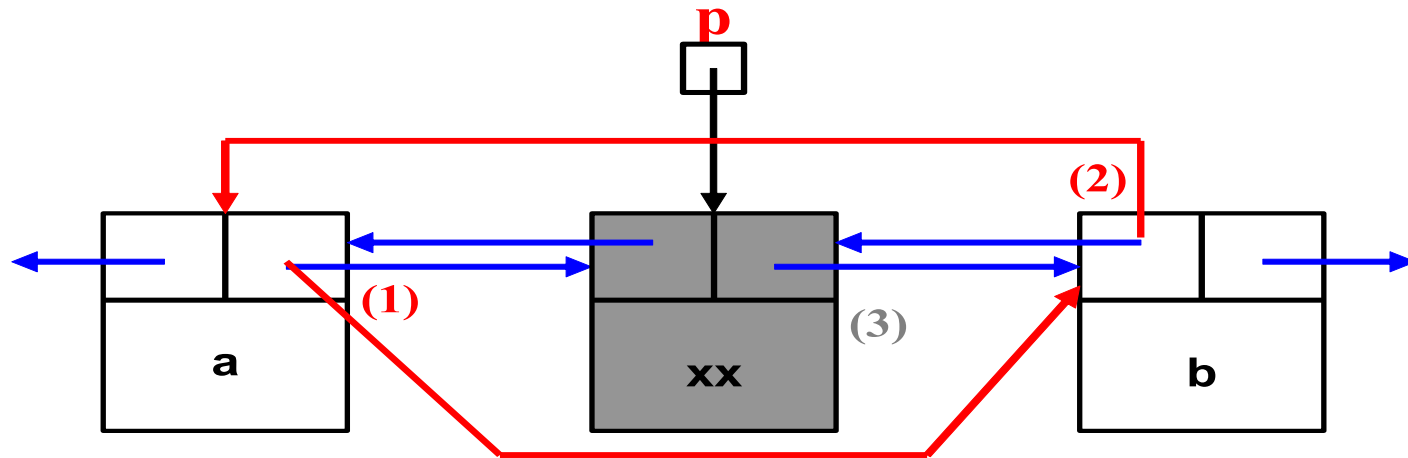
```
    a->urm->pre = aux; /*5*/
```

```
    a->urm = aux; /*6*/
```

```
}
```



Eliminarea celulei de la adresa p ($p \neq s$)



Secventa de operatii:

- actualizare legaturi in celulele vecine (**1,2**)
- eliberarea spatiului ocupat de celula (**3**)

Daca informatia din celula este necesara pentru prelucrari ulterioare, atunci ea trebuie salvata inainte de eliminarea celulei.

Eliminarea celulei de la adresa p ($p \neq s$)

```
void* ExtrL2 (TL2 p)
```

```
/* elimina celula de la adresa p
```

```
    intoarce adresa info */
```

```
{ void* r;
```

```
  if (p == p->urm) return NULL;
```

```
  r = p->info;
```

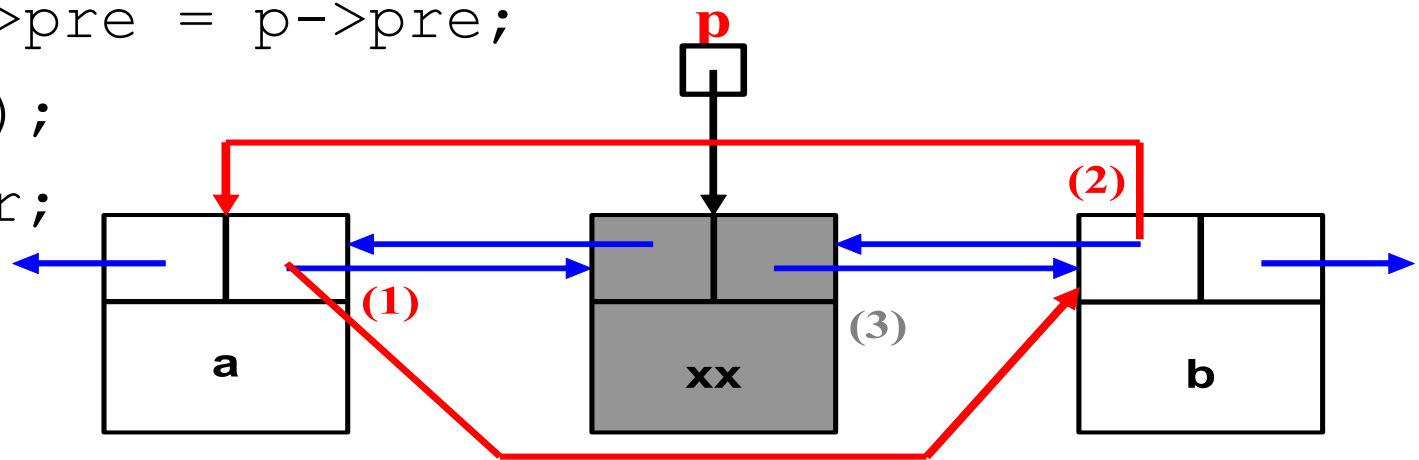
```
  p->pre->urm = p->urm;
```

```
  p->urm->pre = p->pre;
```

```
  free (p);
```

```
  return r;
```

```
}
```



Eliberare spatiul de memorie

/*- functii de eliberare spatiu; parametrul fe -
functie care elibereaza spatiul ocupat de un element */

void ResetL2(TL2 s, void (*fe)(void*))

/* transforma lista in lista vida */

```
{ TL2 p = s->urm, aux;  
  while(p != s)  
  {  
    aux = p;  
    p = p->urm;  
    fe(aux->info); free(aux);  
  }  
  s->urm = s->pre = s;  
}
```

Eliberare tot spatiul de memorie

```
void DistrL2(TL2* as, void (*fe)(void*))
/* elibereaza tot spatiul ocupat de lista,
inclusiv santinela */
{
    ResetL2(*as, fe);
    free(*as);
    *as = NULL;
}
```

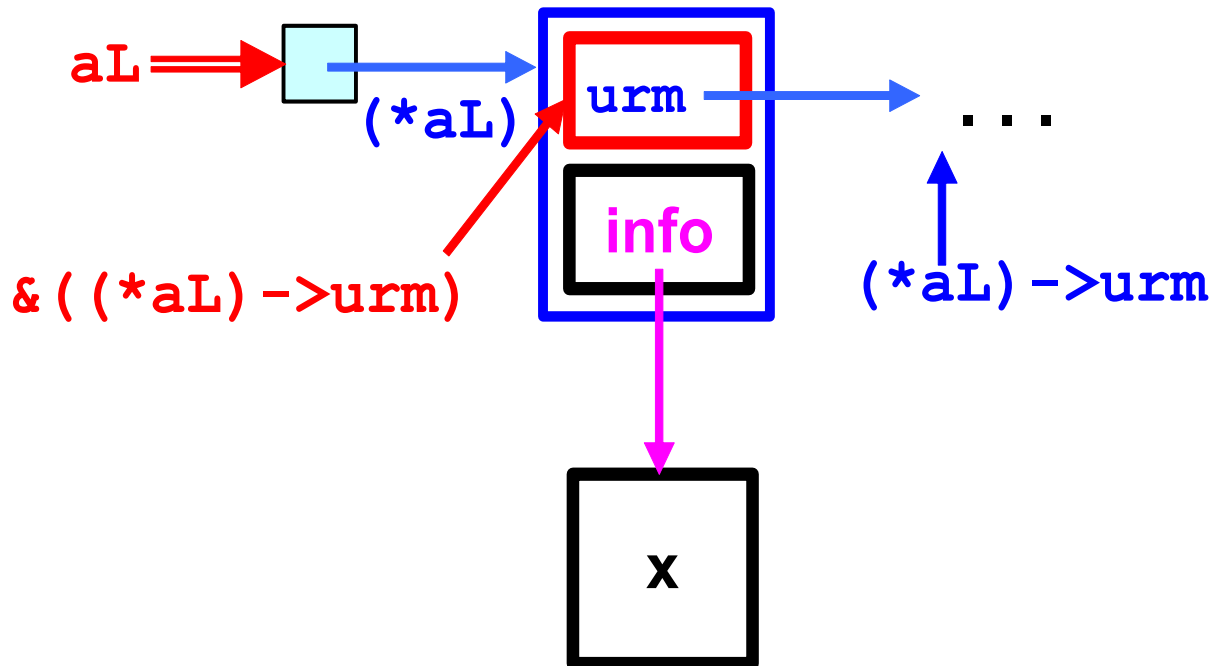

Tipul lista generica

- Pot exista insa situatii in care in aceeaasi aplicatie se lucreaza cu mai multe **liste, cu elemente de tipuri diferite** (de exemplu lista de materii si liste de studenti).
- **operatiile de baza asupra listelor sunt aceleasi**, dar **modul de tratare a partii de informatie difera**, fiind strict legat de tipul elementelor din lista prelucrata la un moment dat.
- Pentru a rezolva aceste situatii vom defini tipul “**lista generica**”, care face abstractie de tipul elementelor concrete.

Cum se memoreaza elementele unei liste generice?

Intr-o zona de memorie gestionata de utilizator

→ la crearea celulei trebuie cunoscuta **adresa** zonei in care se gaseste valoarea elementului



Definirea tipului lista generica

Adresele necesare sunt adresele unor elemente de tip necunoscut in momentul definirii tipului lista generica, deci vor fi declarate **void***, urmand ca la nivelul aplicatiei sa fie convertite la tipul corespunzator.

Campul info este adresa unui element de tip specific aplicatiei, deci de tip **void***.

Definirea tipului lista generica

```
typedef struct celulag
{ void *info;          /* informatie */
  struct celulag *urm; /* adresa urmatoarei celule */
} TCelulaG, *TLG;
/* tipuri: Celula, Lista */
```

Adresarea la nivelul aplicatiei

Sa consideram:

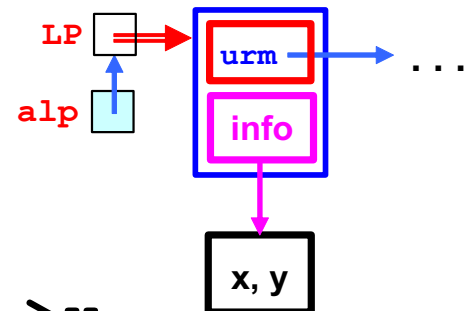
```
typedef struct { int x, y; } TPunct;
                /* punct in plan */
TLG LP = NULL; /* lista puncte */
ALG alp = &LP;
```

Expresia abscisei primului punct din lista este:

`((TPunct*) (LP->info)) ->x`

sau

`((TPunct*) ((*alp) ->info)) ->x`



Inserarea in liste generice

Operatia de inserare are trei etape:

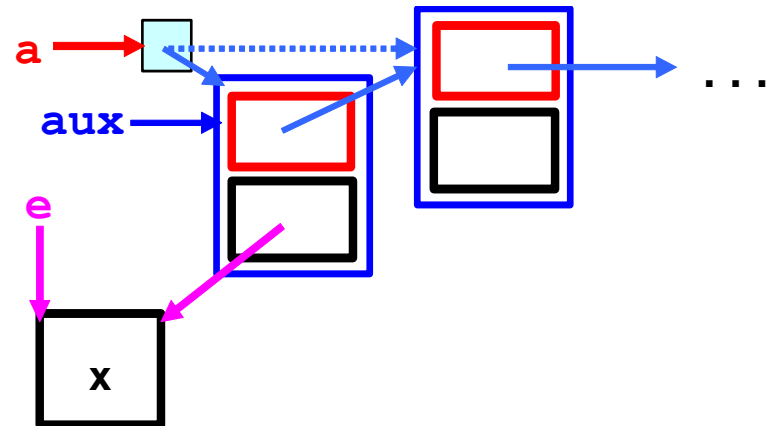
1. **alocarea** de spatiu pentru o noua celula
2. **completarea** campului **info**
3. **conectarea** celulei in lista.

Dimensiunea noii celule este:

`sizeof(TCelulaG)`

Inserarea in liste generice

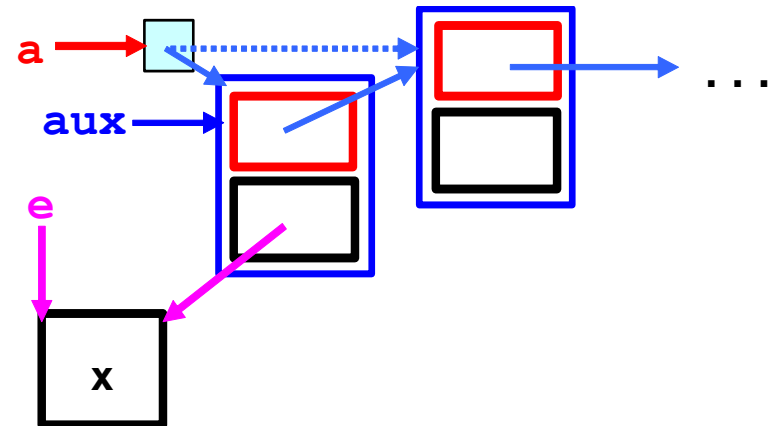
```
int InsLG(ALG aL, void* ae, size_t d)
{
    TLG aux = (TLG)malloc(sizeof(TCelulaG));
    if(!aux) return 0;
    aux->info = malloc(d);
    if (!aux->info) /
    { free(aux); return 0;}
    memcpy(aux->info, ae, d);
    aux->urm = *aL;
    *aL = aux;
    return 1;
}
```



Inserarea in liste generice

O alta varianta:

```
int InsLG(ALG aL, void* ae)
{
    TLG aux = (TLG)malloc(sizeof(TCelulaG)) ;
    if(!aux) return 0;
    aux->info = ae;
    aux->urm = *aL;
    *aL = aux;
    return 1;
}
```



Distruge lista generica

```
void DistrugeLG(ALG aL)  /* eliminare celula si element */
{   TLG aux;
    while(*aL)
    {
        aux = *aL;
        *aL = aux->urm;
        free(aux->info. /* sau utilizare functie
                        de eliminare */
        free(aux) ;
    }
}
```

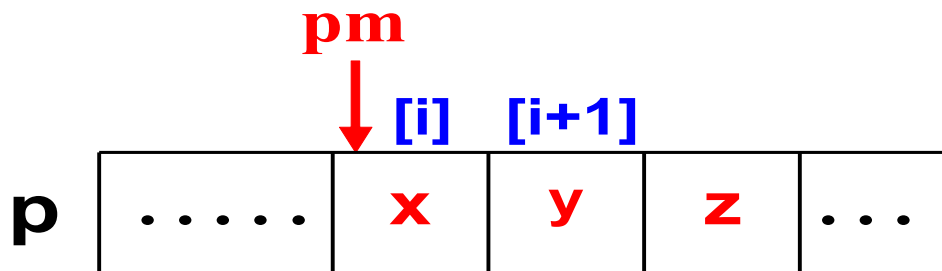
Prelucrare lista de pointeri la puncte din plan

```
void Afisare(ALG aL)
{ TPunct p;
  if(!*aL) { printf("Lista vida\n"); return;}
  printf("Lista: ");
  for(; *aL; aL = &(aL)->urm)
  {
    p = *(TPunct*) ((*aL)->info);
    printf("[%d %d] ", p.x, p.y);
  }
  printf("\n");
}
```

Memorarea elementelor unei multimi

Intr-un **vector**

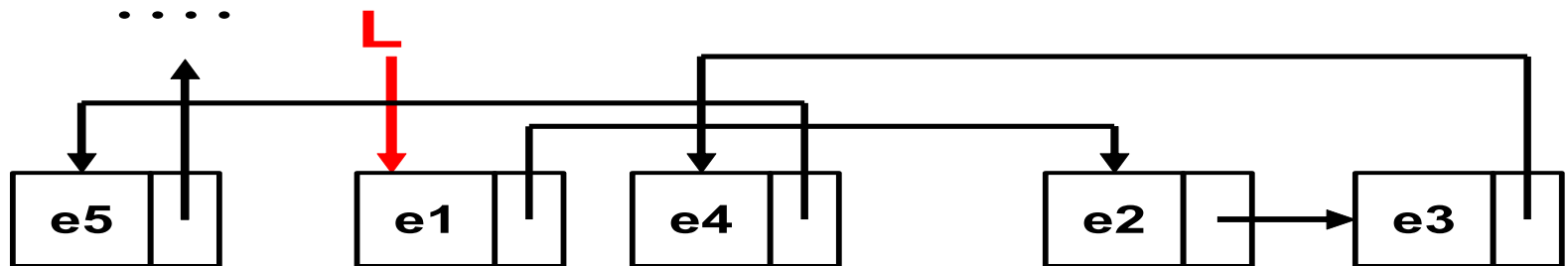
- elementele ocupa **zone de memorie adiacente**
- se alocă, static sau dinamic, **spatiu pentru numarul maxim** de elemente
- este posibila **adresarea indexata**
- **inserarea / eliminarea** unui element in / din interiorul colectiei implica **deplasarea succesorilor la dreapta / stanga**



Memorarea elementelor unei multimii

Intr-o **lista, de exemplu** simplu inlantuita

- elementele ocupa **zone de memorie neadiacente**

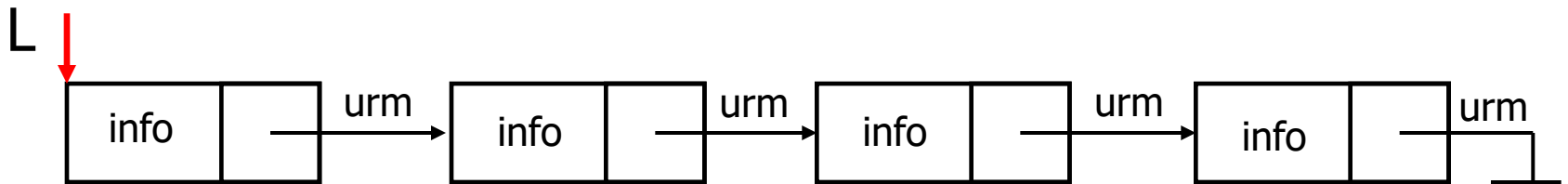


- fiecare element trebuie insotit de cel putin o informatie de legatura - **adresa succesorului**.
- Elementul si informatia de legatura se grupeaza intr-o structura numita **celula**.

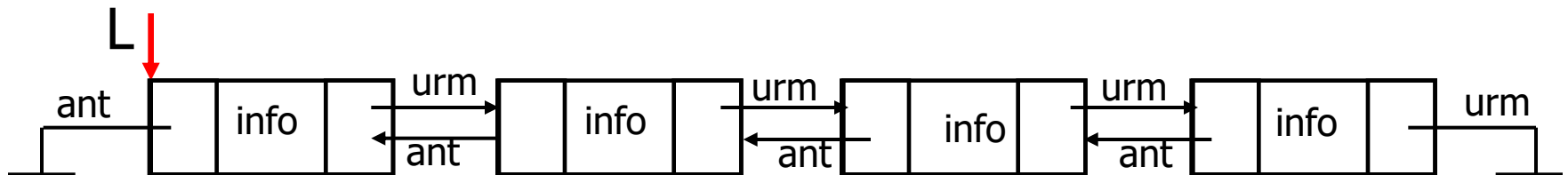
Liste

- O lista inlantuita este o colectie de elemente, alocate dinamic, dispersate in memorie, dar legate intre ele prin pointeri, ca intr-un lant

Lista simplu inlantuita



Lista dublu inlantuita

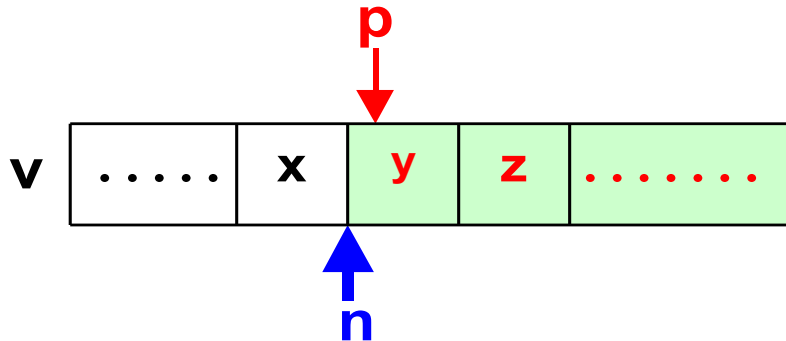


Liste

- o lista inlantuita este o structura dinamica, flexibila, care se poate extinde continuu (singura limita este dimensiunea zonei "heap" din care se alocă memorie)
- alocarea de spatiu se realizeaza la nivel de celula, numai atunci cand este necesar
- pentru localizarea unui element trebuie parcursa lista predecesorilor
- inserarea in lista / eliminarea din lista nu necesita deplasarea altor elemente, ci alocarea, respectiv eliberarea de spatiu si actualizarea unor informatii de legatura.

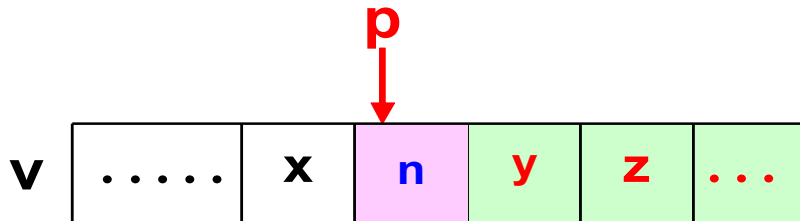
Inserare in lista

Inserarea in lista simplu inlantuita



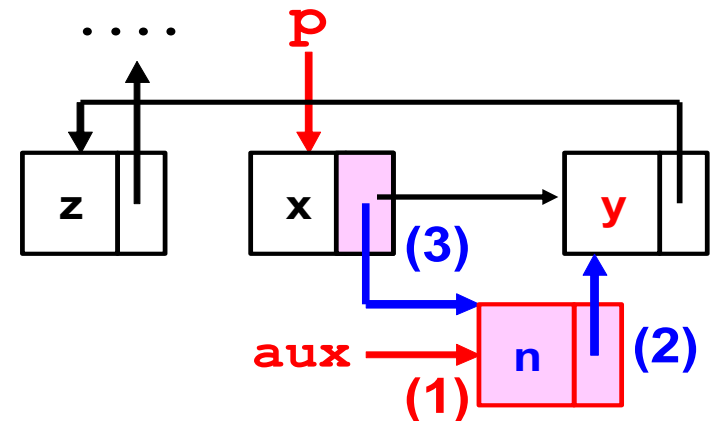
Vector:

- **deplaseaza dreapta** succesorii
- **copiază** noul element



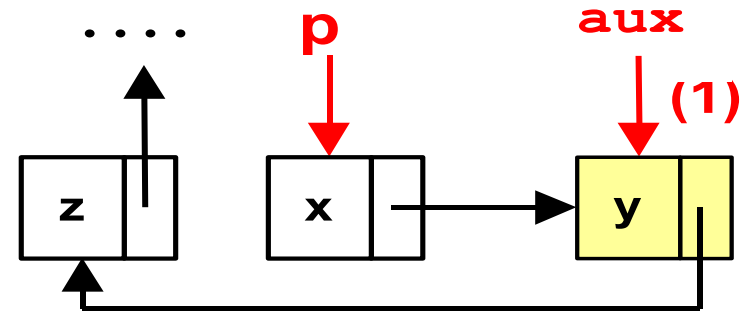
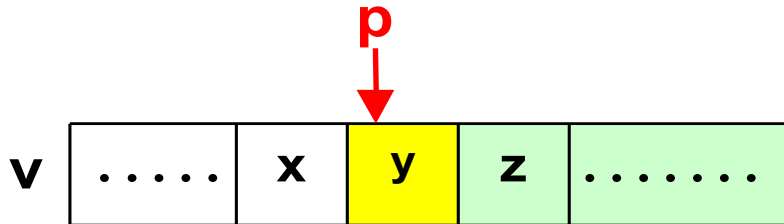
Lista:

- **aloca spatiu** pentru o celula si copiaza noul element (1)
- **actualizeaza legaturi** (2,3)



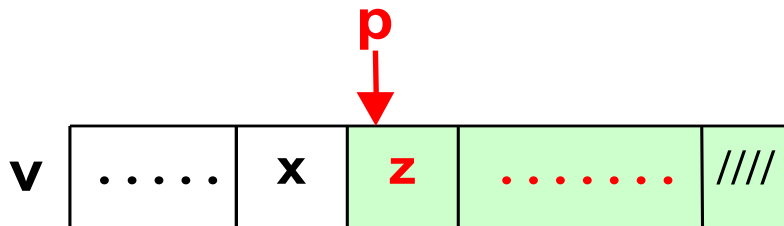
Eliminare din lista

Eliminarea din lista simplu inlantuita



Vector:

- **deplaseaza stanga** succesorii



Lista:

- **memoreaza adresa** celulei eliminate (1)
- **actualizeaza legatura** (2)
- **elibereaza** spatiul ocupat de celula eliminata (3)

Operatii liste

Operatii cu elemente:

- Alocarea unei celule de lista
- Cautarea unui element in lista
- Inserare in lista
 - La inceputul listei
 - La sfarsitul listei
 - In interiorul listei: dupa / inaintea unui element specificat
- Eliminarea unui element din lista
- Modificare element
- Citirea unei liste
- Distrugere lista

Operatii liste

Operatii cu liste

- Iterare asupra listei (afisarea unei liste)
- Sortarea unei liste
- Inversarea unei liste
- Concatenarea a doua liste

Liste simplu inlantuite

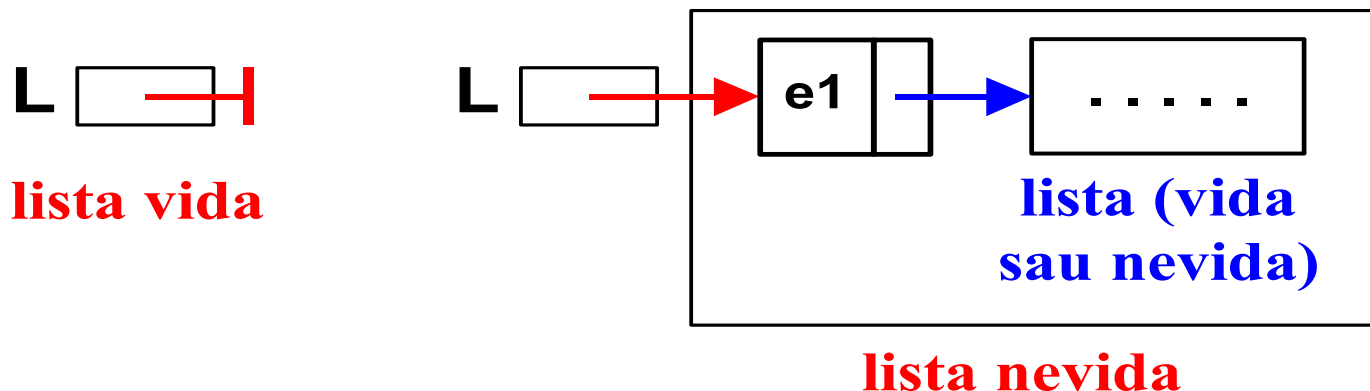
Lista este o structura recursiva, care se defineste:

```
lista: lista_vida | informatie lista
```

Informatia este specifica fiecarui element.

O variabila de tip lista are ca valoare

- **NULL** daca lista este vida
- **adresa primei celule** daca lista este nevida



Liste simplu inlantuite

Se considera **tipul elementelor unei liste: TEL**.

Tipurile **celula**, **lista** si **adresa lista** se definesc astfel:

```
typedef int TEL;
```

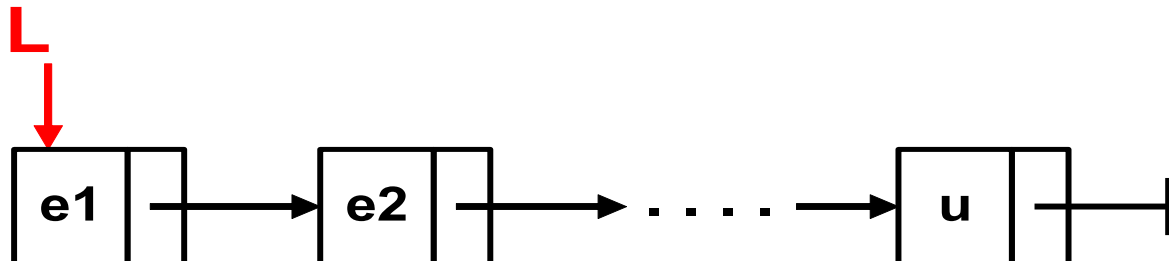
```
typedef struct celula
```

```
{ TEL info;
```

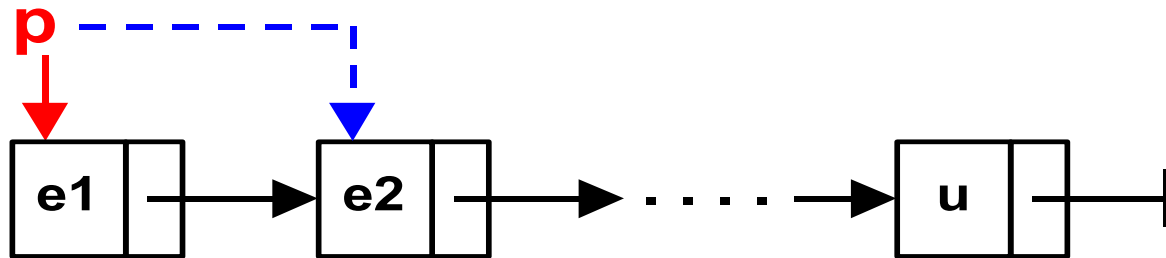
```
    struct celula * urm;
```

```
} TCelula, * TLista, ** ALista;
```

```
TLista L;
```



Liste simplu inlantuite



Lista vida: **TLista p = NULL;**

test lista vida: **(p == NULL)**

valoarea primului element: **p->info**

adresa urmatoarei celule: **p->urm**

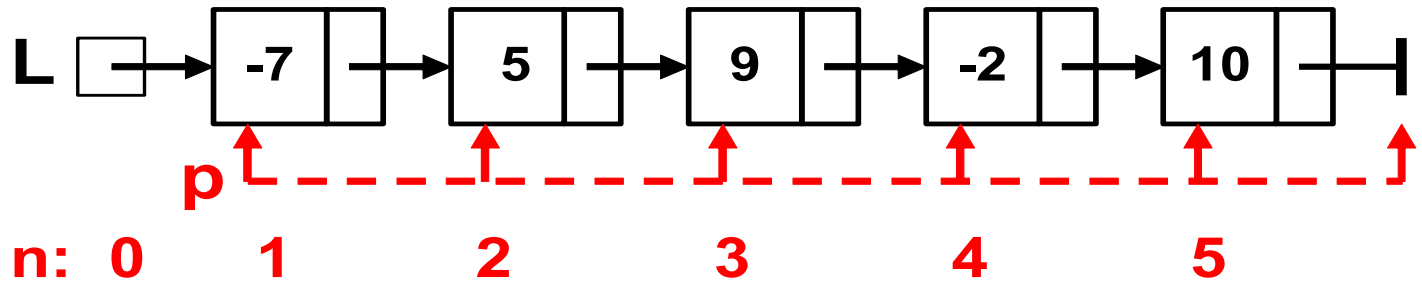
valoarea urmatorului element: **(p->urm->info)**

avans la urmatoarea lista (celula): **p = p->urm**

Parcurgerea unei liste

Parcurgere Lista

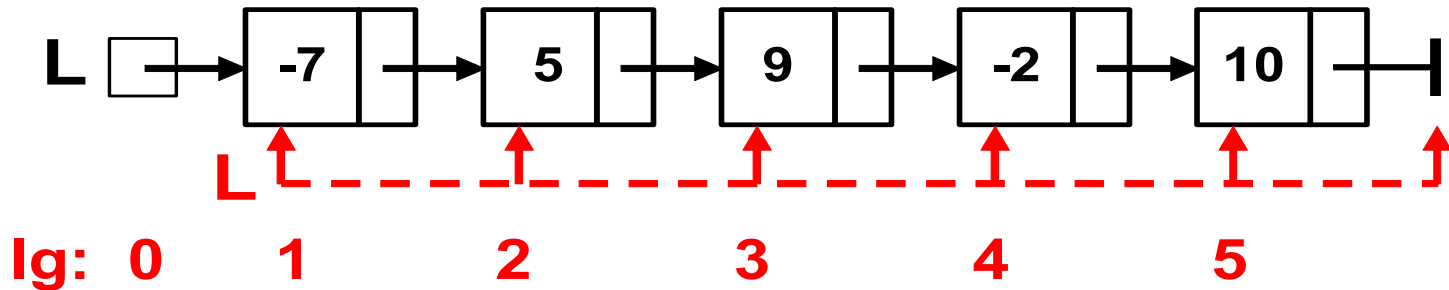
```
{ initializari;  
  pentru fiecare celula din lista  
  { prelucreaza info celula;}  
}
```



Afisare elemente lista

```
void AfisareL(TLista L)
/* afiseaza valorile elementelor din lista */
{
    for (; L != NULL; L = L->urm)
        printf("%d ", L->info);
}
```

Parcuregere iterativa lista

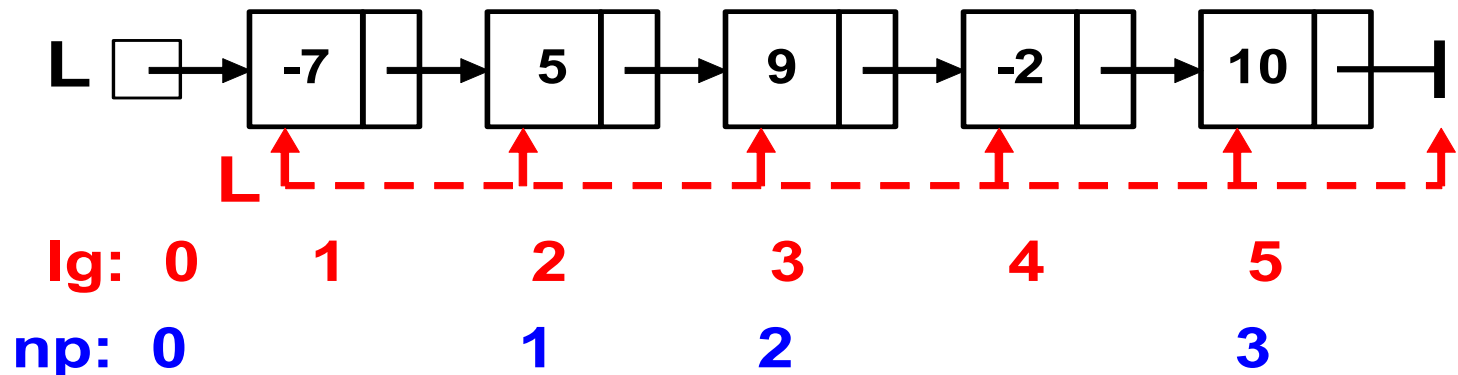


```
size_t LungimeI(TLista L)
/* varianta iterativa */
{ size_t lg = 0;
  for( ; L != NULL; L = L->urm)
    lg++;
  return lg;
}
```

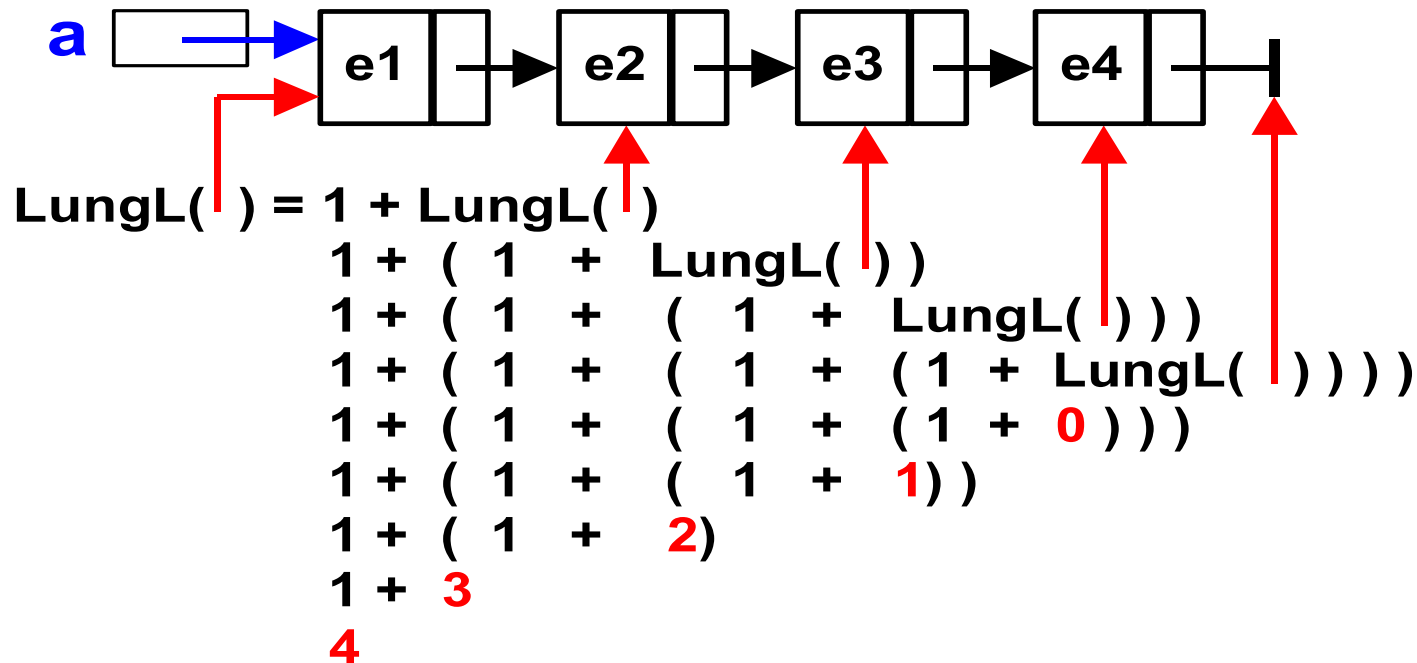

Parcurgerea unei liste

Numar elemente pozitive

```
/* np = numarul de elemente pozitive */  
{ initializari;  
  pentru fiecare celula din lista  
  {  
    daca element > 0 atunci creste np;  
  }  
}
```



Parcurgerea recursiva a unei liste

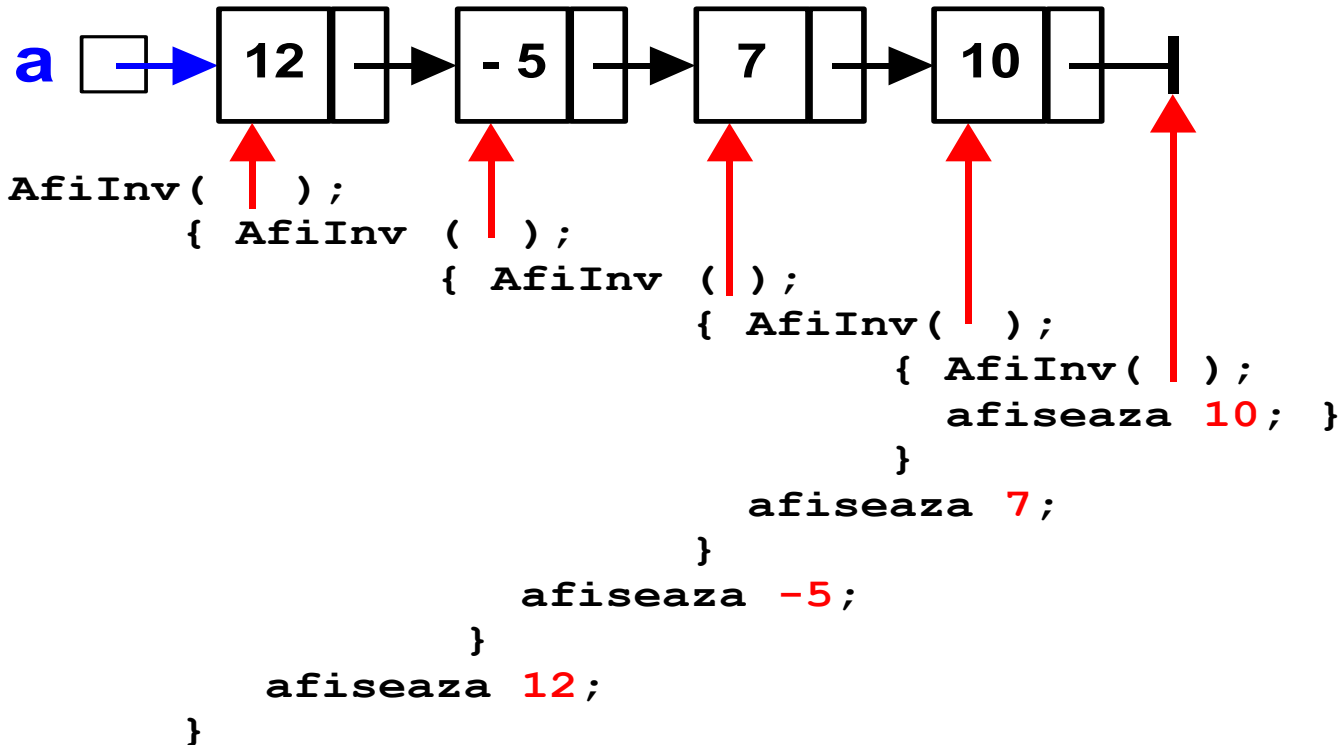


Algoritm LungL(lista)

```

{ daca lista vida
    atunci intoarce 0;
    altfel intoarce 1 + LungL(listaUrmatoare) ;
}
    
```

Afisare continut lista in ordine inversa



```
void AfiInv(TLista x)  
{ if (!x) return;  
  AfiInv (x->urm);  
  printf ("%i  ", x->info);  
}
```

Alocare Celula

```
TLista AlocCelula(TEL e)
```

```
/* adresa celulei create sau NULL */
```

```
{ TLista aux =
```

```
    (TLista)malloc(sizeof(TCelula)); /* (1) */
```

```
if (!aux) return NULL;
```

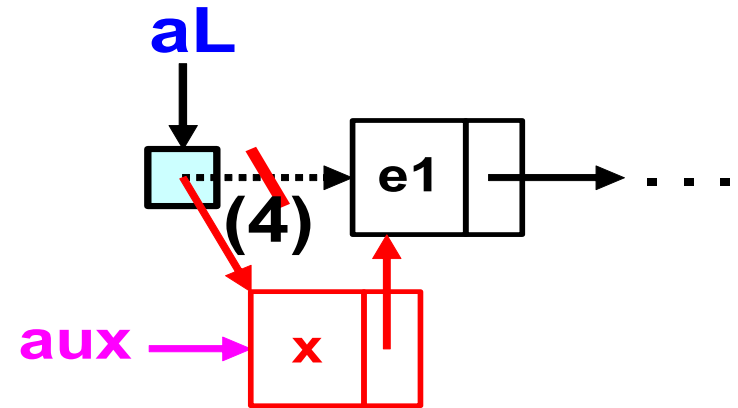
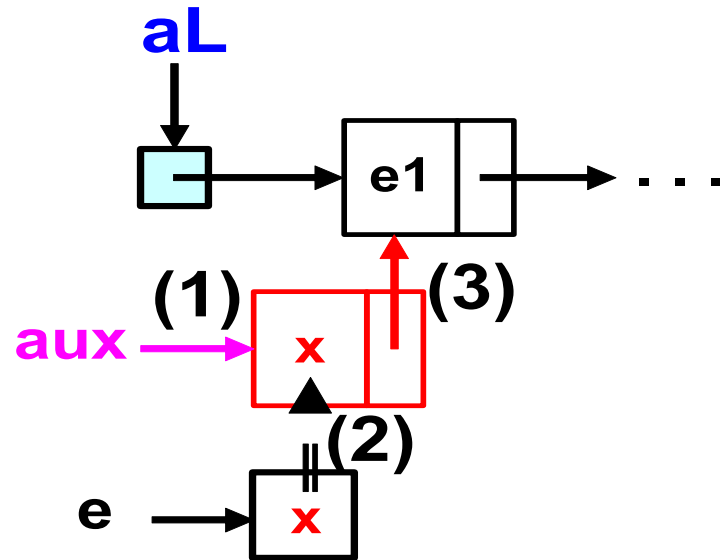
```
aux->info = e; /* (2) */
```

```
aux->urm = NULL; /* (3) */
```

```
return aux;
```

```
}
```

Inserare la inceputul listei



Algoritm InsInc

```
{ incerca alocare celula; (1)
  daca esec atunci intoarce esec;
  completeaza info si urm din noua celula; (2, 3)
  actualizeaza inceput lista; (4)
  intoarce succes;
}
```

Inserare la inceputul listei

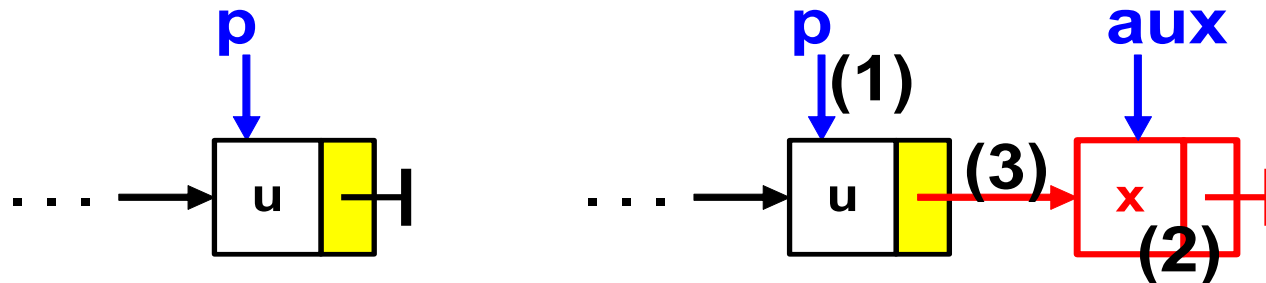
```
int InsInc(ALista aL, TEL e)
/* inserare la inceput reusita sau nu (1/0) */
{
    TLista aux =
        (TLista)malloc(sizeof(TCelula)); /* (1) */
    if (!aux) return 0;
    aux->info = e; /* (2) */
    aux->urm = *aL; /* (3) */
    *aL = aux; /* (4) */
    return 1;
}
```

Inserare la sfarsitul listei

Se realizeaza in doua etape:

A. **Localizarea** ultimei celule (in cazul particular de lista vida $\Rightarrow$ inserare la inceput)

B. **Inserarea** propriu-zisa



Functia va intoarce: **adresa noii celule sau NULL**,
daca nu se poate alocata spatiu pentru aceasta;

Inserare la sfarsitul listei

```
TLista InsSfL(ALista aL, TEL e)
```

```
/* adresa celulei inserate sau NULL */
```

```
{ TLista p = *aL, aux, ultim = NULL;
```

```
    while (p != NULL)
```

```
    {      ultim = p; p = p->urm; } /* (1) */
```

```
    aux= AlocCelula(e); /* (2) */
```

```
    if(!aux) return NULL; /* intoarce NULL */
```

```
    if(ultim) ultim->urm = aux; /* (3) */
```

```
    else *aL = aux;
```

```
    return aux;
```

```
/* intoarce adresa celulei inserate */
```

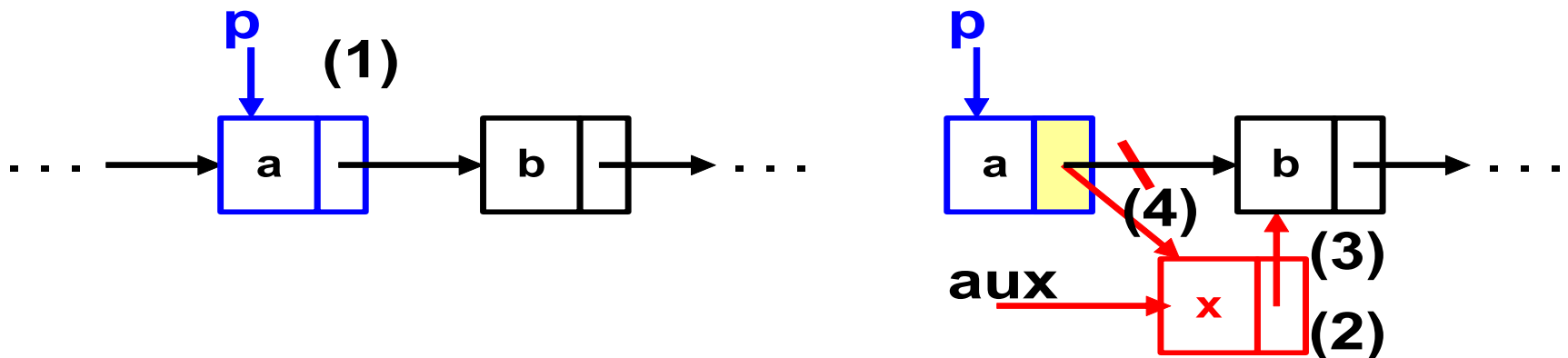
```
}
```


Inserare dupa o anumita celula

Se realizeaza in doua etape:

A. **Localizarea** celulei dupa care se efectueaza inserarea (al carei camp **urm** se modifica) , folosind adresa celula (**p**) sau adresa legatura (**aL**)
Caz particular: celula inexistentă

B. **Inserarea** propriu-zisa.
Caz particular: alocare spatiu imposibila



Inserare dupa o anumita celula

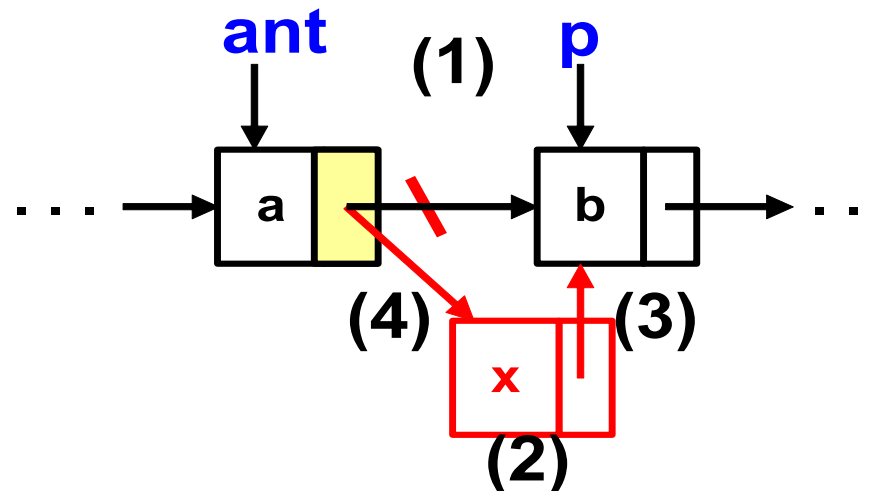
```
TLista InserareDupa(TLista L, TEL e, TEL ref)
/* intoarce adresa celula inserata sau NULL */
{ TLista aux, p;
  for(p = NULL; L != NULL; L = L->urm) /* (1) */
    if(L->info == ref) {p = L; break;}
  if (!p) return NULL;
  aux= AlocCelula(e); /* (2) */
  if(!aux) return NULL;
  aux->urm = p->urm; /* (3) */
  p->urm = aux; /* (4) */
  return aux;
}
```

Inserare in fata unei celule

Se realizeaza in doua etape:

A. **Localizarea** legaturii modificate
(adresa listei sau adresa camp **urm**)
Caz particular: celula inexistenta

B. **Inserarea** propriu-zisa
Caz particular: alocare spatiu imposibila



Inserare in fata unei celule

TLista InserareInainte

(ALista aL, AEL ae, AEL aRef)

/* intoarce adresa celula inserata sau NULL */

{ TLista aux, ant, p, L;

for (ant = NULL, p = NULL, L = *aL ;

L != NULL; ant = L; L = L->urm) /* (1) */

if (L->info == *aRef)

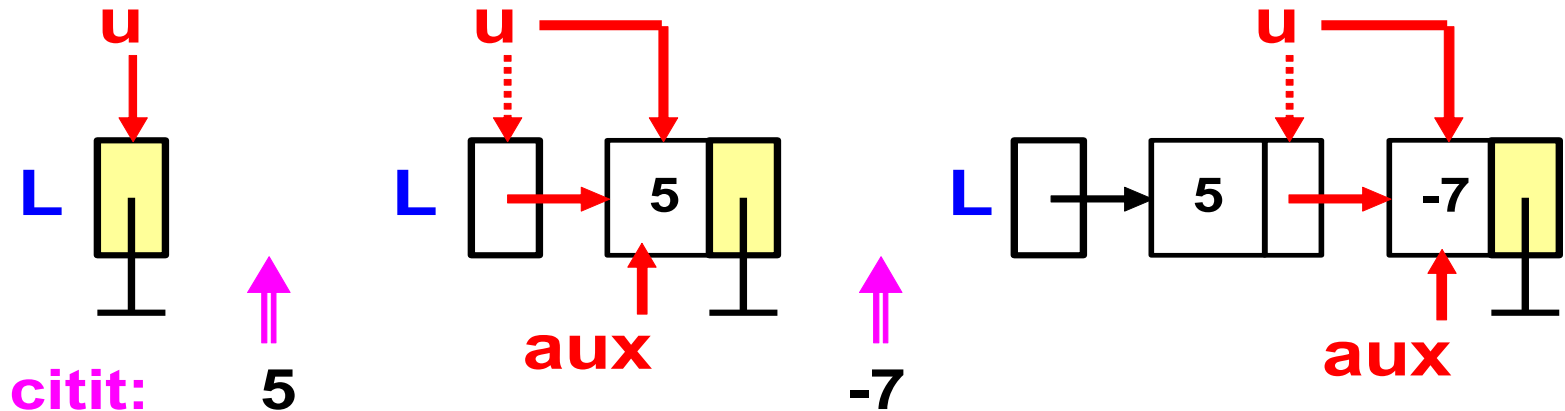
{p = L; break;}

if (!p) return NULL;

Inserare in fata unei celule

```
aux= AlocCel(ae); /* (2) */
if(!aux) return NULL;
if(ant == NULL)
{
    aux->urm = p; /* (3) */
    *aL = aux; /* (4) */
}
else
{
    aux->urm = p; /* (3) */
    ant->urm = aux; /* (4) */
}
return aux;
}
```

Citire lista



```
{ initializeaza lista vida (L) ;  
  cat timp s-a citit o valoare  
  { insereaza urmatoarea celula;  
    daca alocare esuata atunci iesire din bucla  
  }  
  intoarce L;  
}
```

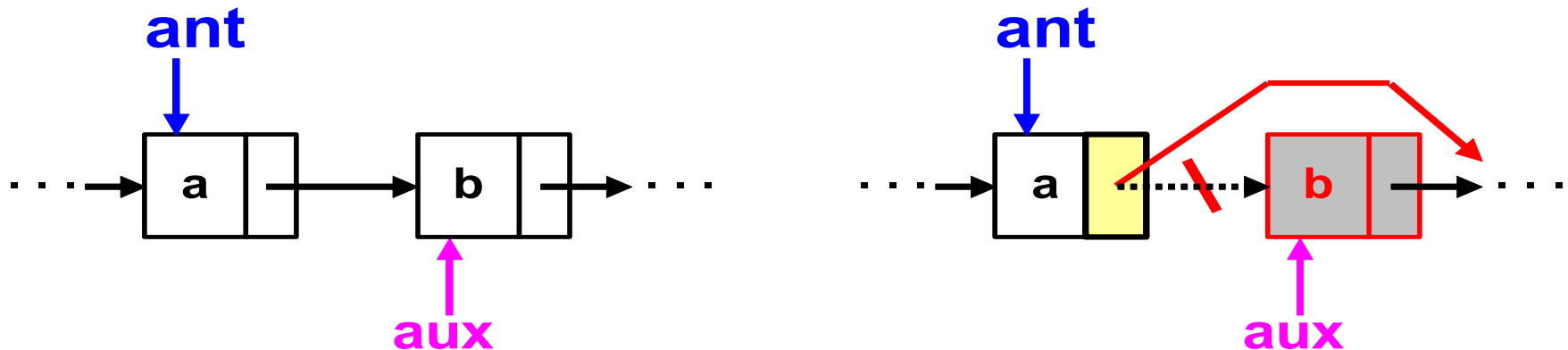
Citire lista (2)

```
TLista CitireL(size_t*lg)
{ TLista L = NULL, u, aux;
  TInfo x;
  for(*lg=0; scanf("%i", &x) ==1;)
  { aux = AlocCel(&x);
    if(!aux) break;
    (*lg)++;
    if(L == NULL) L = aux;
    else u->urm = aux;
    u = aux;
  }
  return L;
}
```

Eliminare element

Se realizeaza in doua etape:

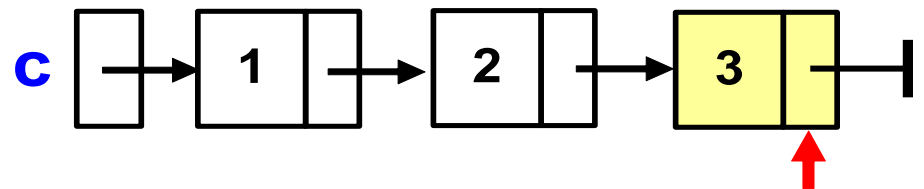
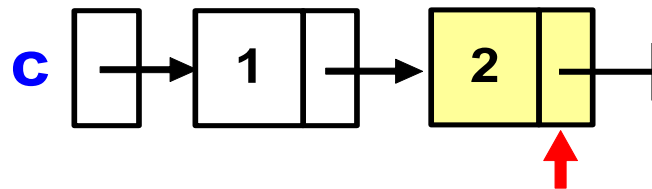
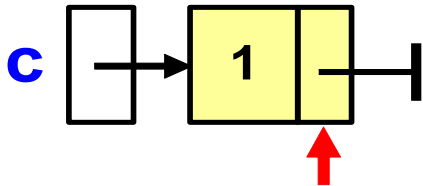
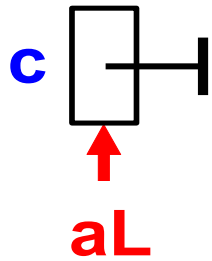
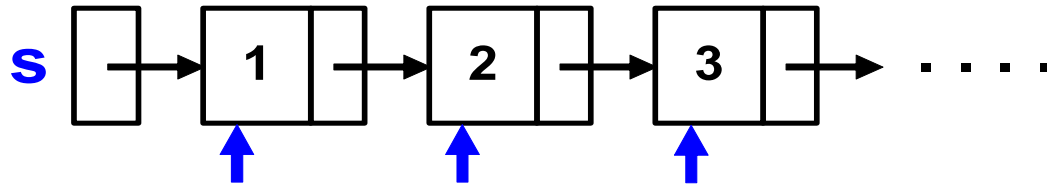
- A. **Localizarea** celulei dupa care se efectueaza eliminarea (al carei camp **urm** se modifica) .
Caz particular: celula inexistenta
- B. **Eliminarea** propriu-zisa; daca este necesar, elementul din celula eliminata este copiat la o adresa furnizata ca parametru



Distrugere lista

```
void Distrugel(ALista aL)
{
    TLista aux;
    while(*aL)
    {
        aux = *aL;
        *aL = aux->urm;
        free(aux) ;
    }
    *aL = NULL;
}
```

Copiare lista

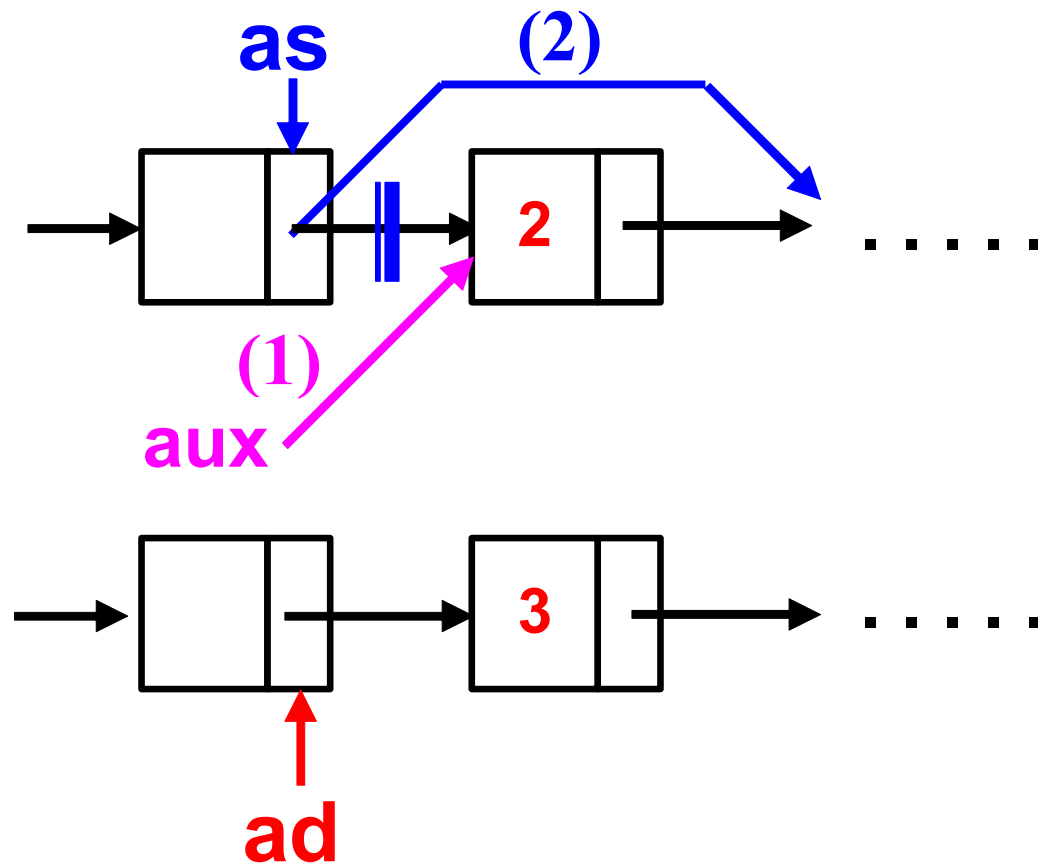


Copiere lista (2)

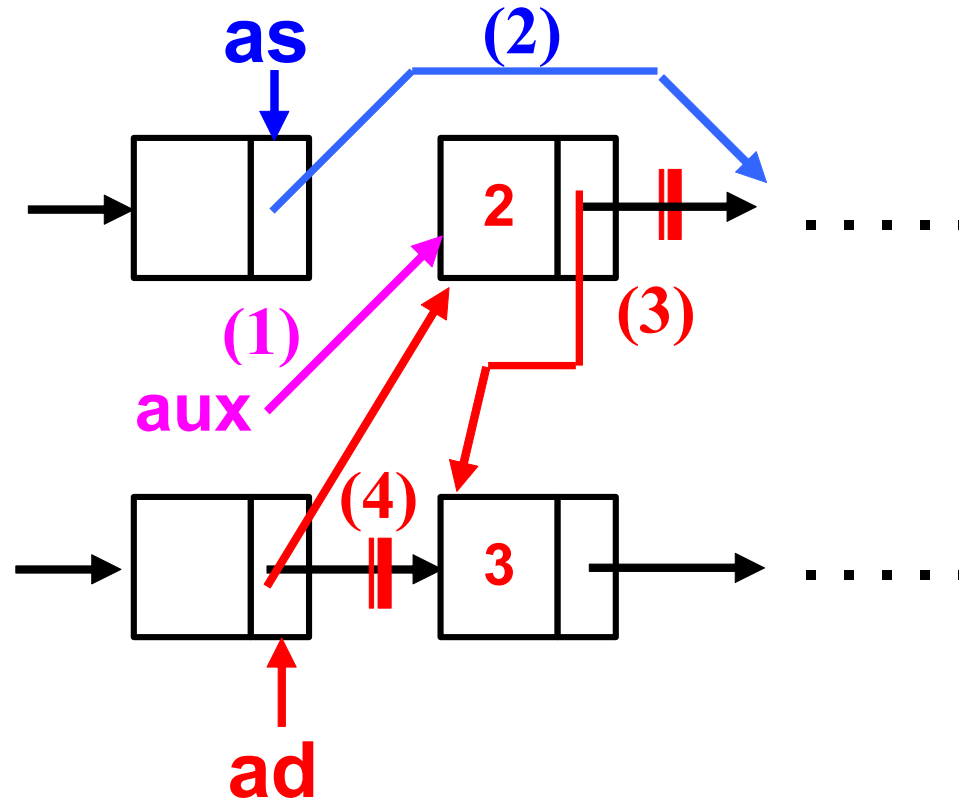
Algoritm Copiere

```
{ initializari;  
  cat timp exista celula de copiat  
  { daca nu reuseste alocare celula copie  
    atunci  
    { distruge copia partiala;  
      intoarce NULL;  
    }  
    altfel  
    { avans in copie partiala;  
      avans in lista sursa;  
    }  
  }  
  intoarce adresa copie;  
}
```

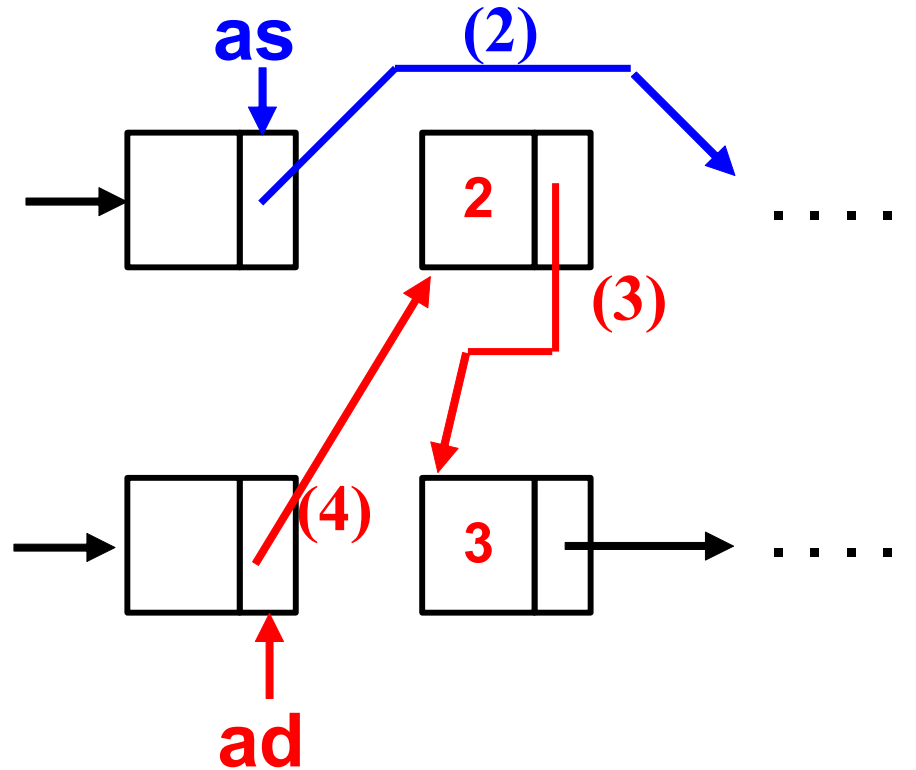
Mutare celula



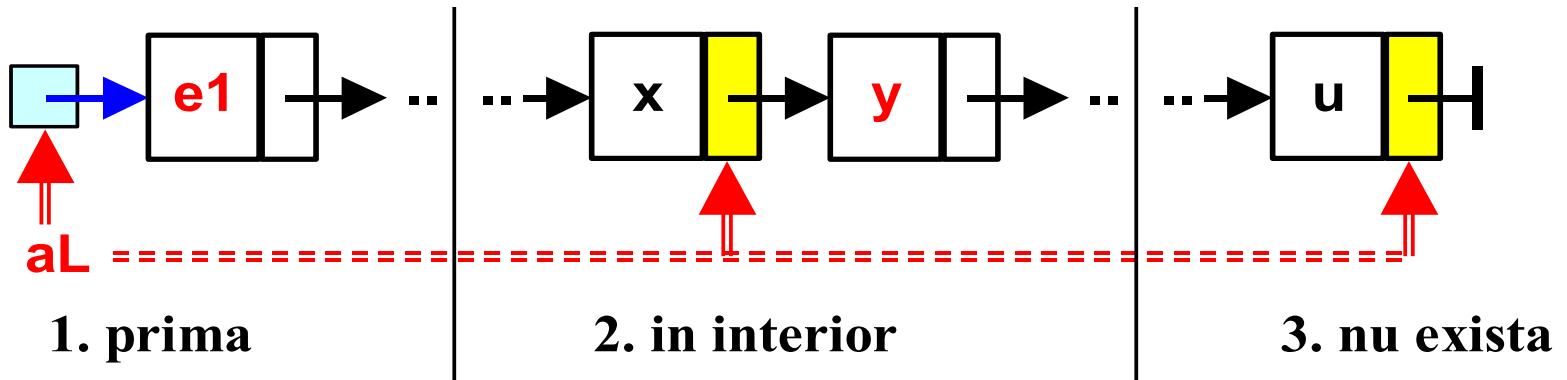
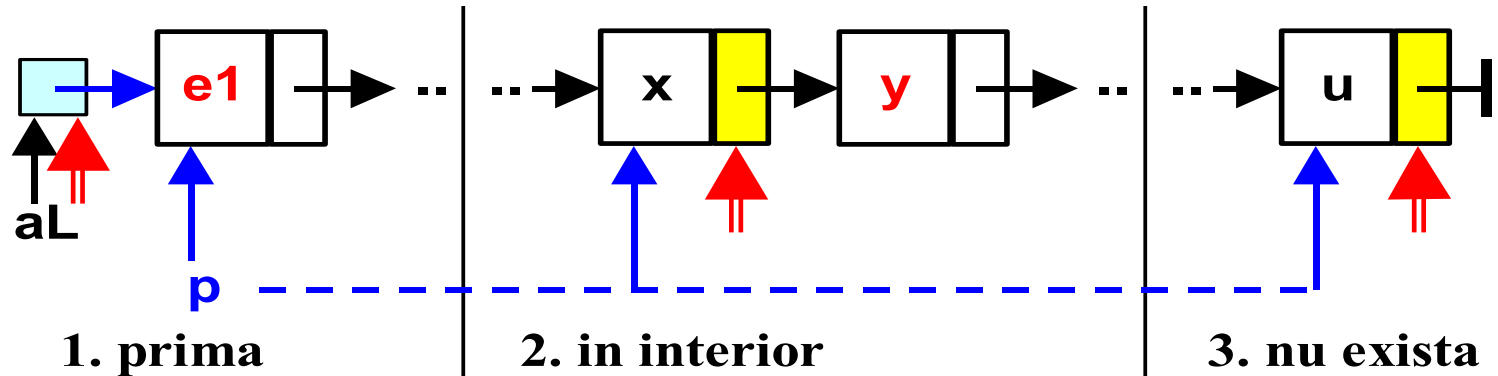
Mutare celula (2)



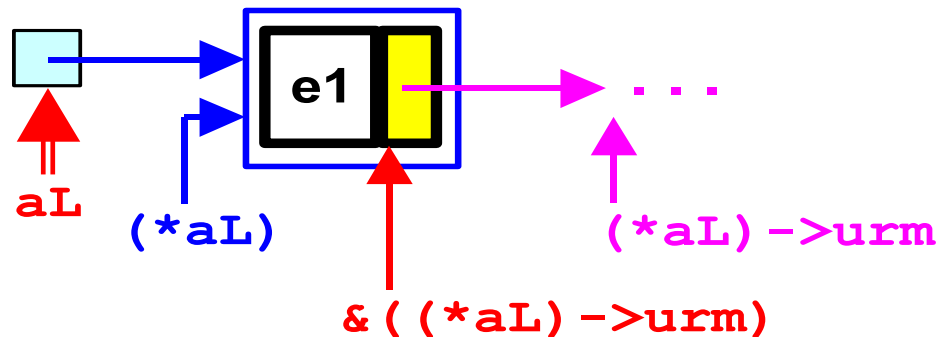
Mutare celula (3)



Cautare legatura spre celula



Cautare legatura spre celula



Daca se lucreaza numai cu adrese de legaturi, atunci avansul in lista se codifica astfel:

$$aL = \& ((*aL) ->urm)$$

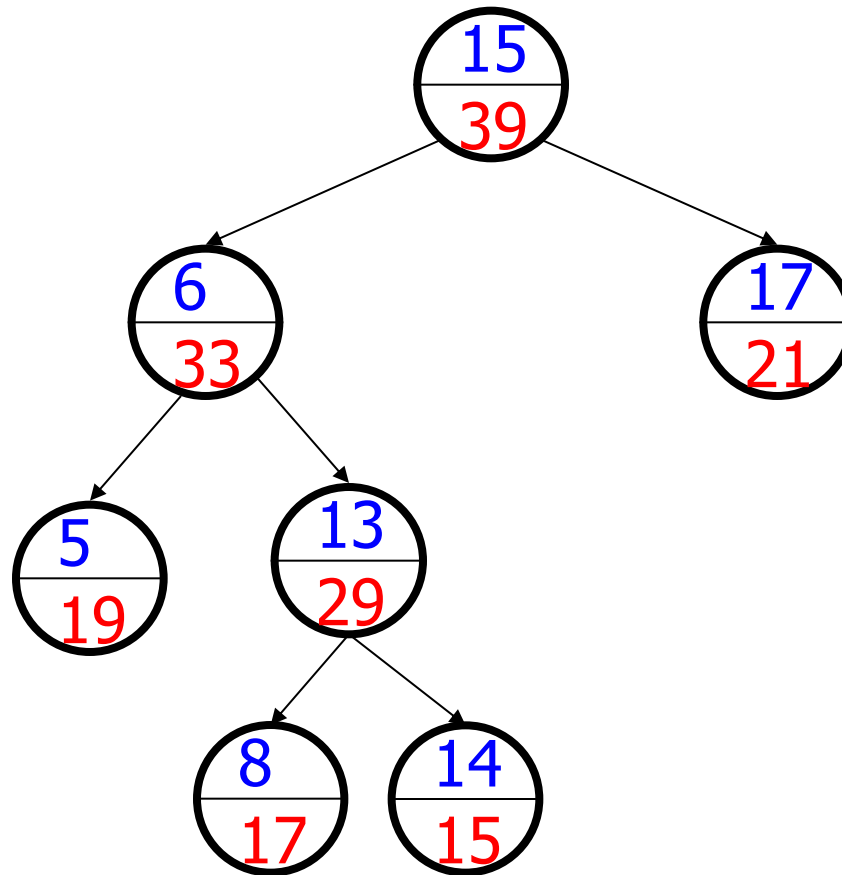
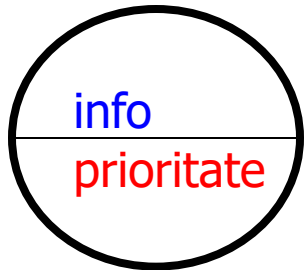
Cautare legatura spre celula

```
Alista Adr_Legatura(Alista aL, Tinfo* ae)
/* adresa legaturii catre prima aparitie
sau NULL */
{
    for(; *aL ; aL = &(*aL)->urm)
        if( (*aL)->info == *ae) reurn aL;
    return NULL;
}
```

Treap tree

- **Tree** + **Heap** $\rightarrow$ **Treap**
- Arbore binar ce contine elemente formate din *(info, prioritate)*
- Treap:
 - Arbore binar de cautare (*info*)
 - Heap (*prioritate*).

Exemplu treap



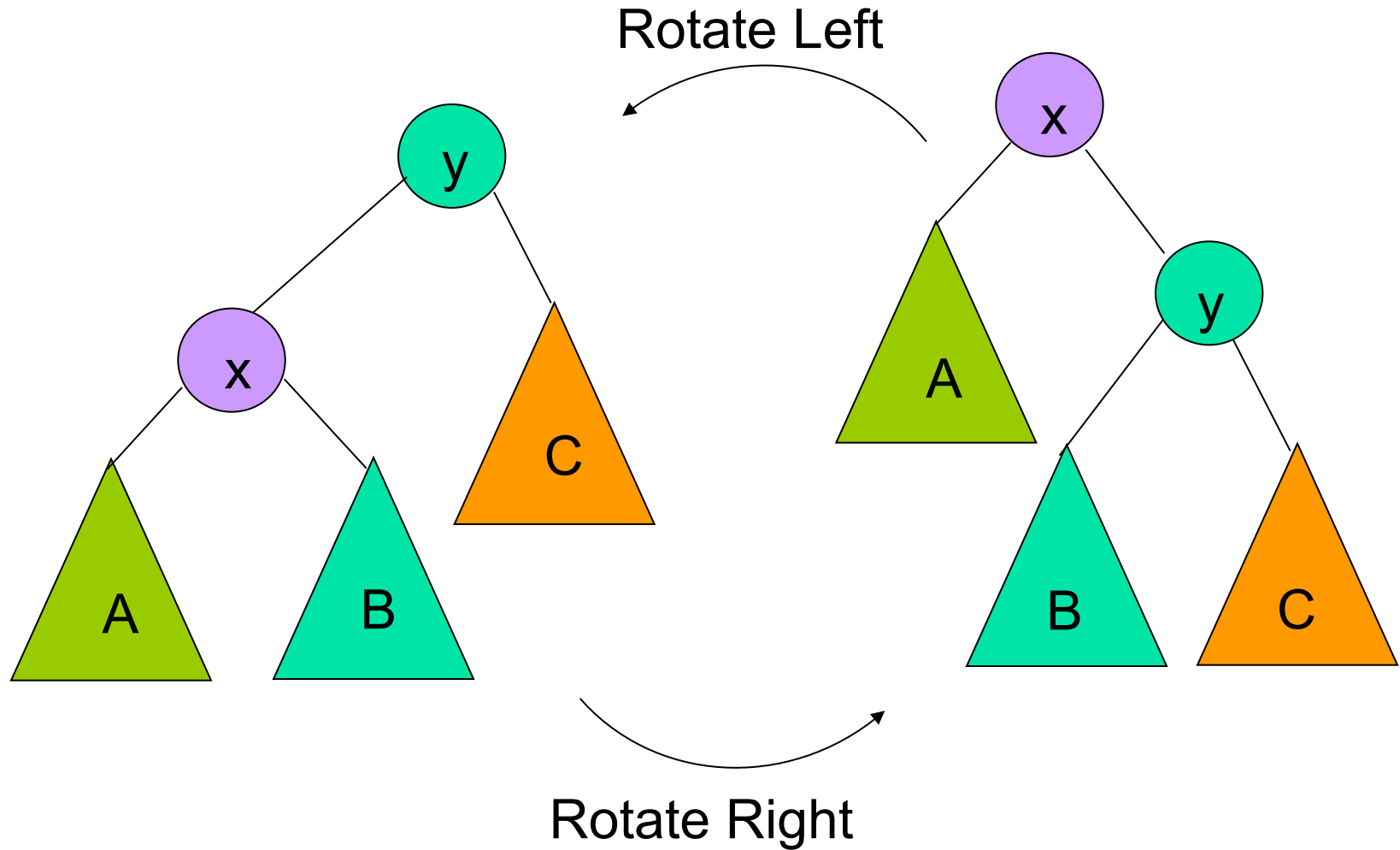
Operatii

- Cautare – analog arbori binari de cautare
- Inserare
- Eliminare

Inserare in treap

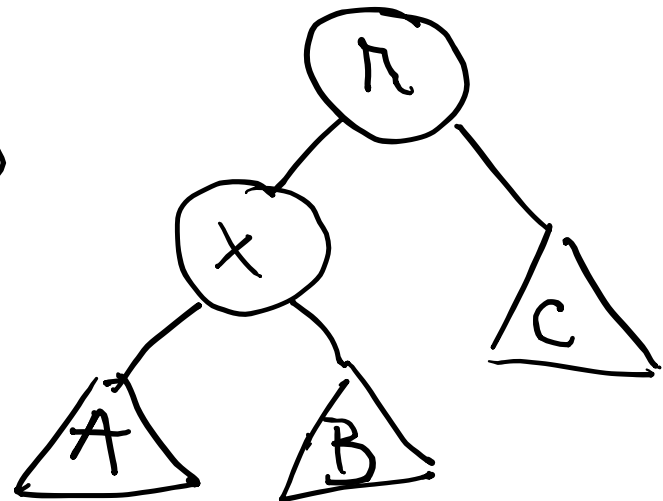
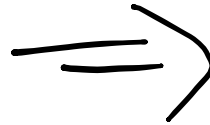
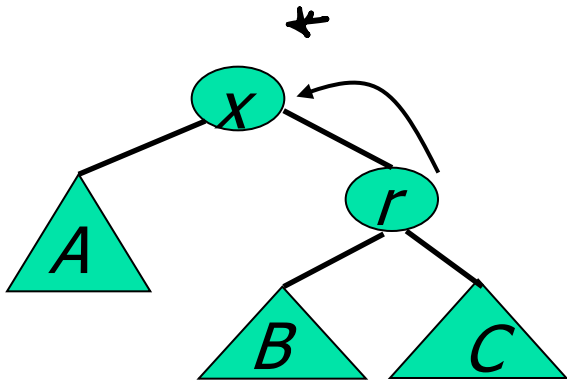
- se insereaza informatia in treap folosind regula de inserare de la arbori binari de cautare
- se genereaza o prioritate aleatoare pentru nodul inserat
- se actualizeaza structura arborelui pentru a se asigura conditia de heap raportata la prioritatile din nodurile arborelui: **rotatii stanga / dreapta**

Rotatii



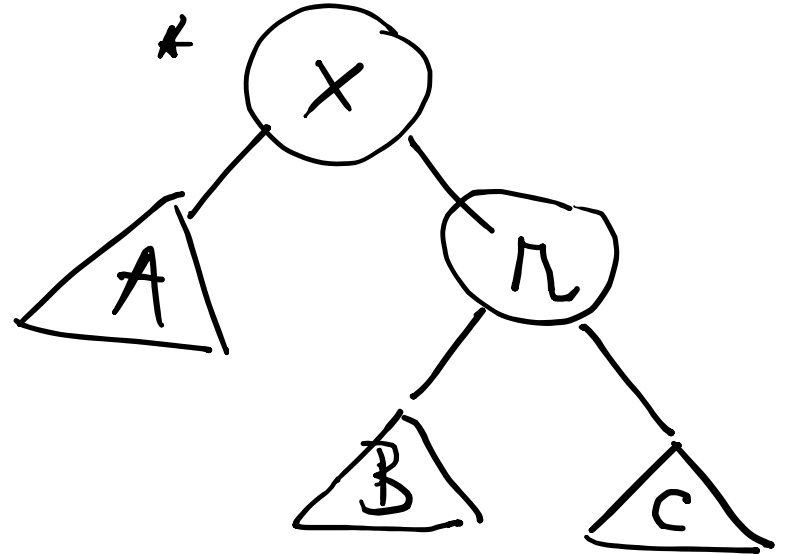
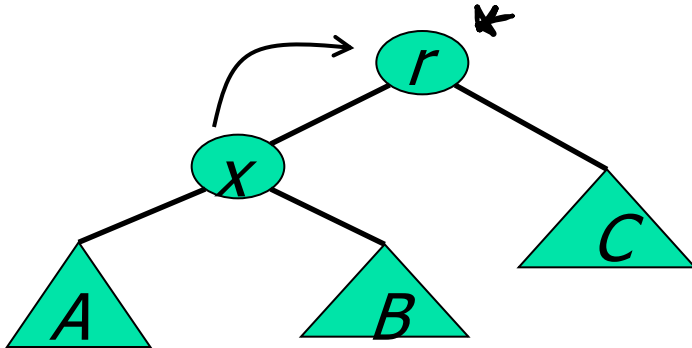
Rotatie stanga / dreapta

Rotatie stanga



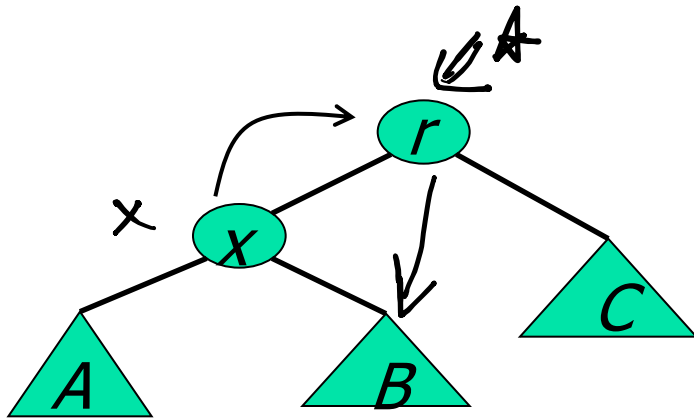
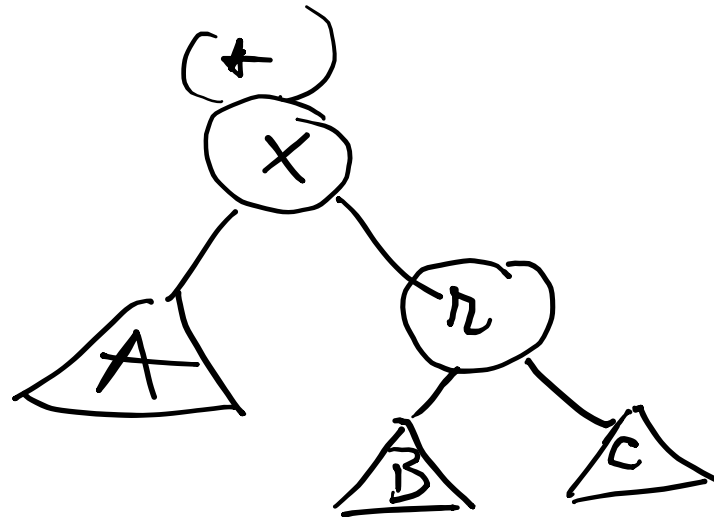
Rotatie stanga / dreapta

Rotatie dreapta



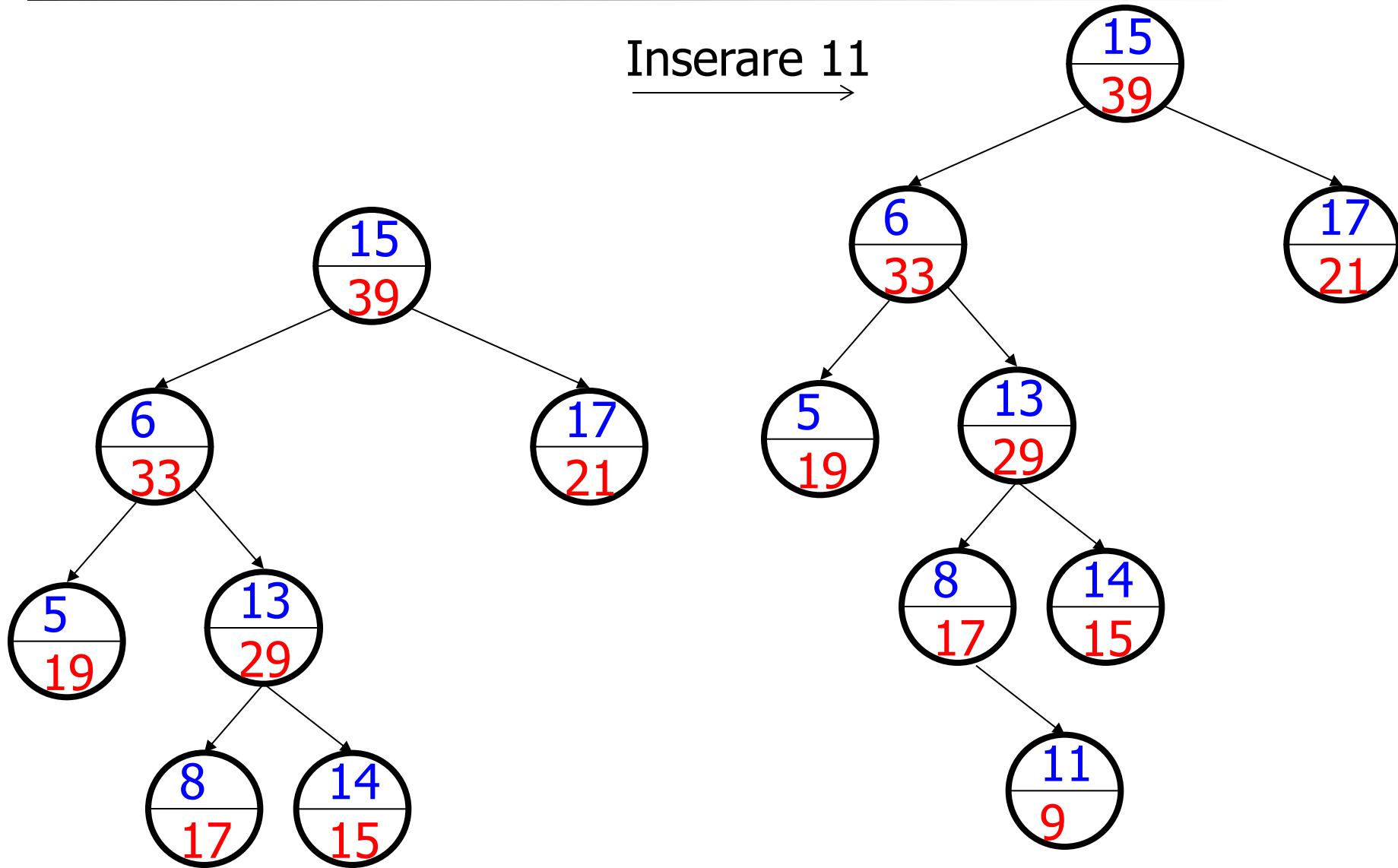
Rotatie stanga / dreapta

Rotatie dreapta


$$\left\{ \begin{array}{l} x = r \rightarrow \text{st}; \\ r \rightarrow \text{st} = x \rightarrow \text{dr}; \\ x \rightarrow \text{dr} = r; \\ r \leftarrow x; \end{array} \right\}$$


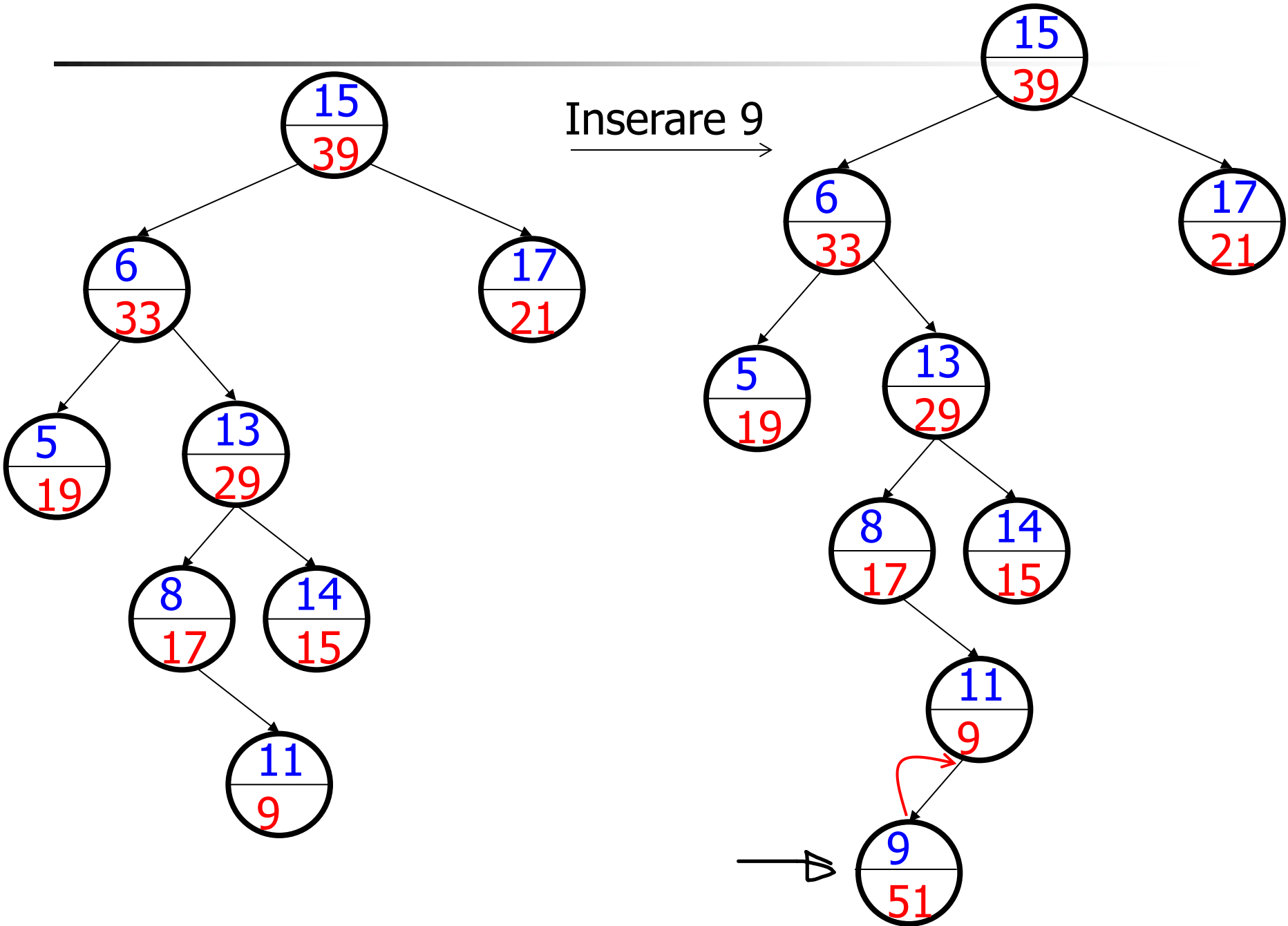
Exemplu inserare

Inserare 11 →

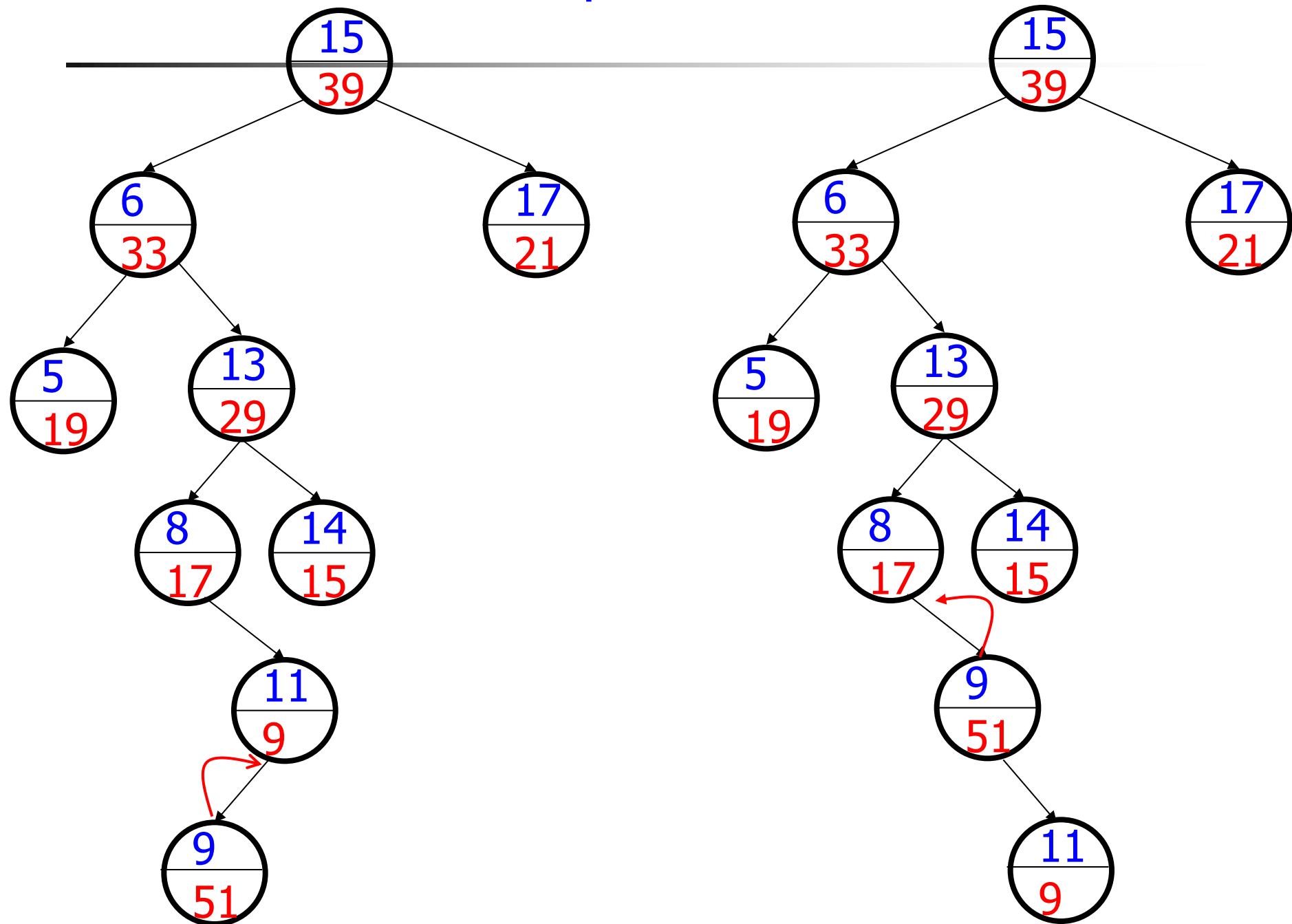


Exemplu inserare

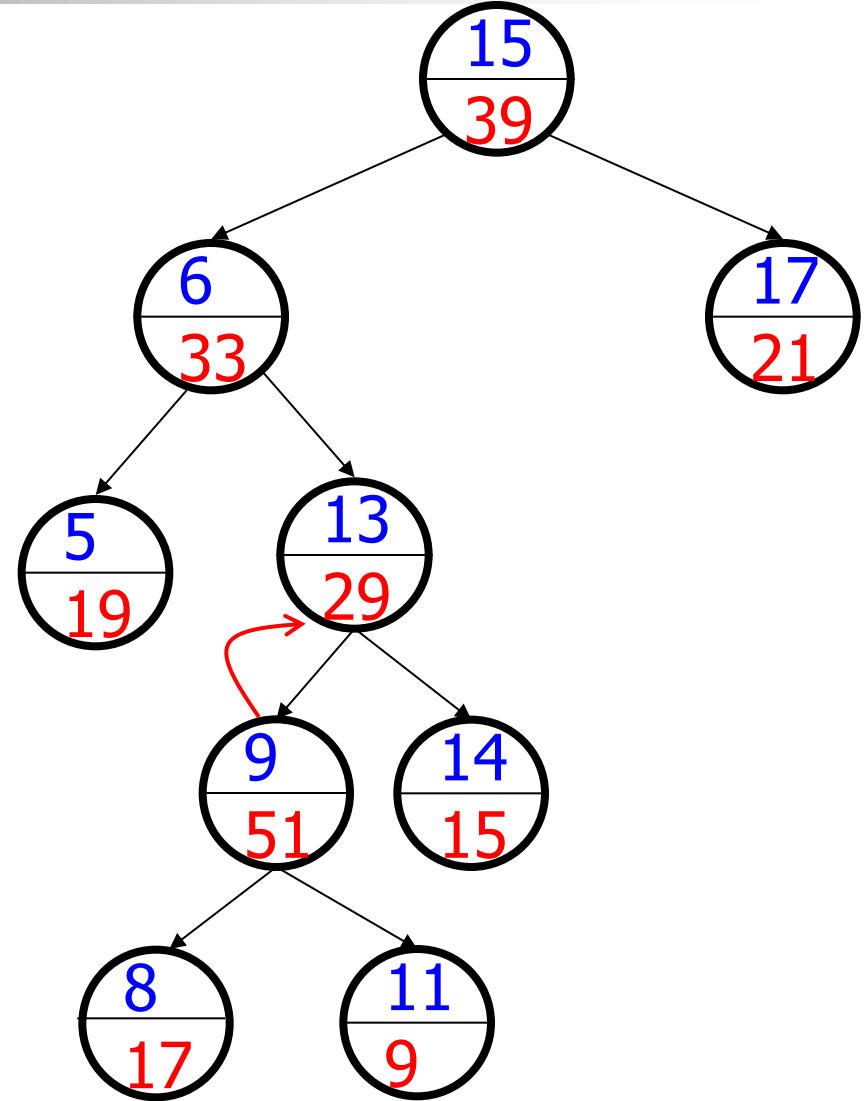
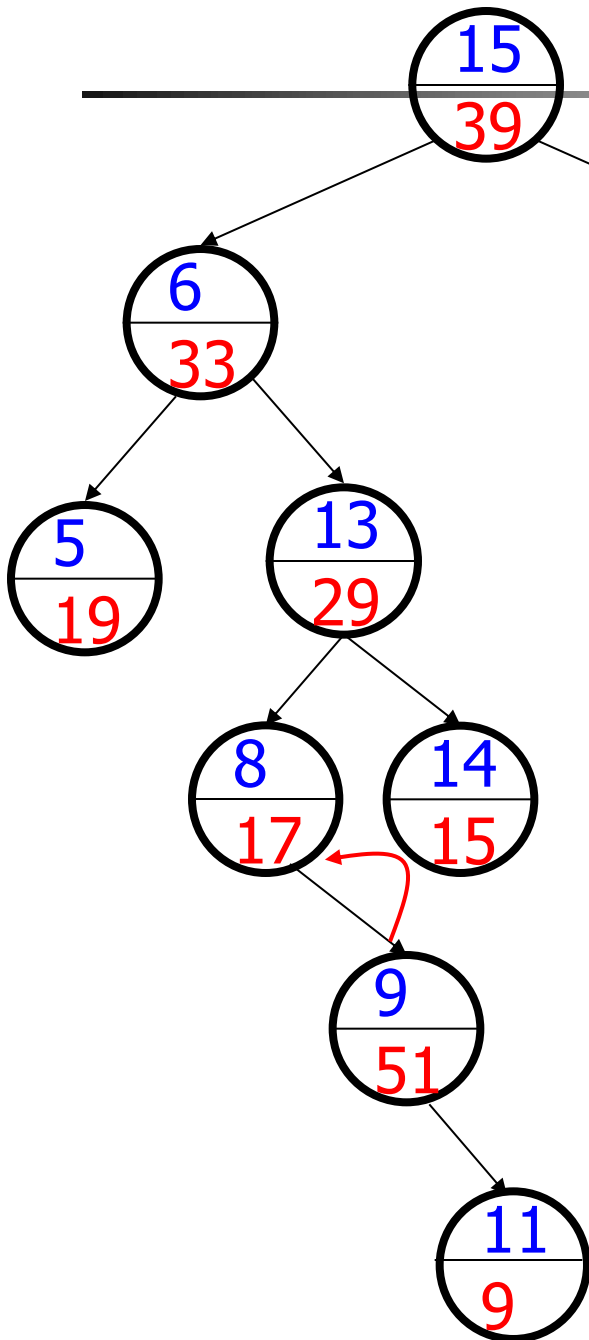
Inserare 9



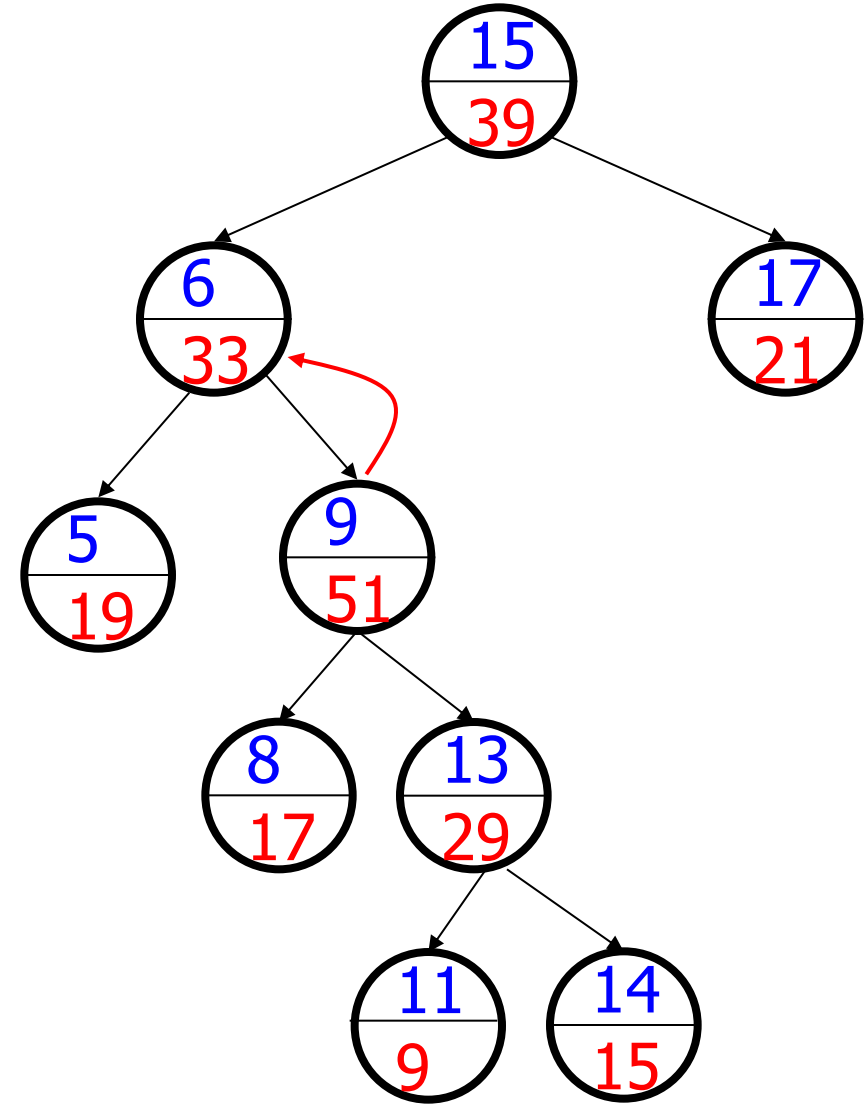
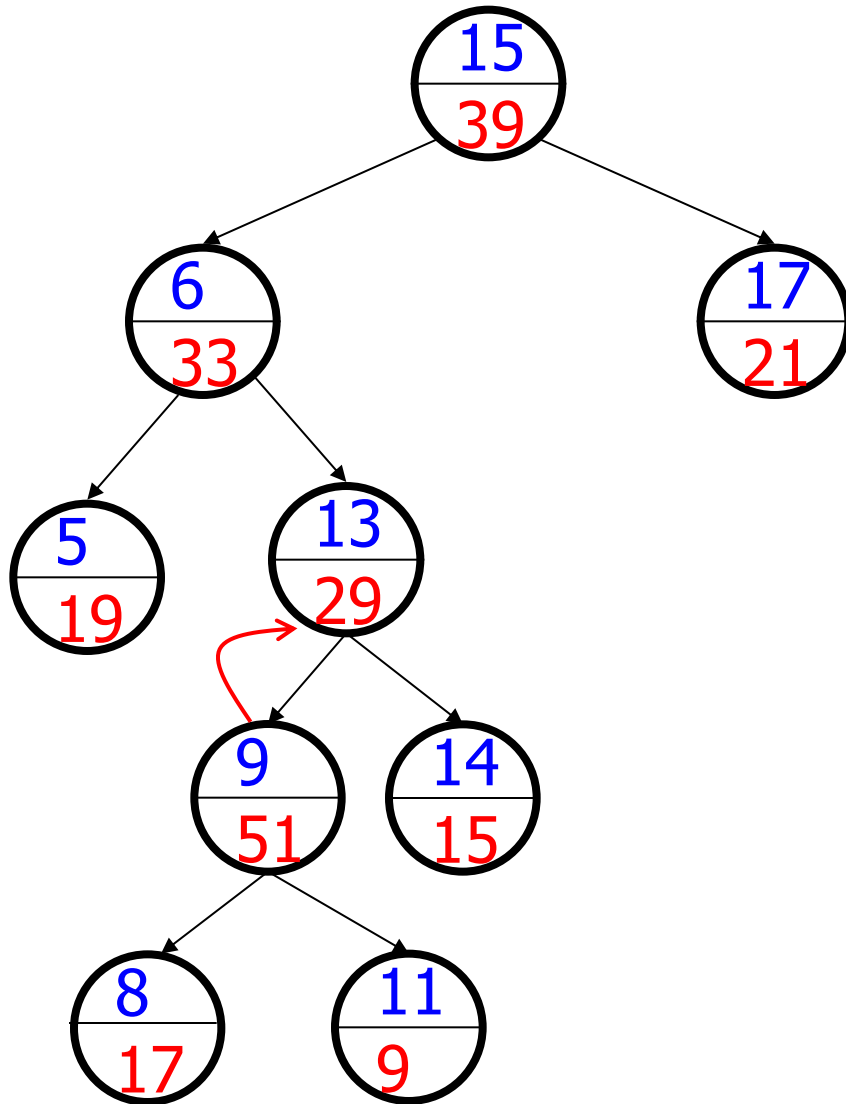
Exemplu inserare



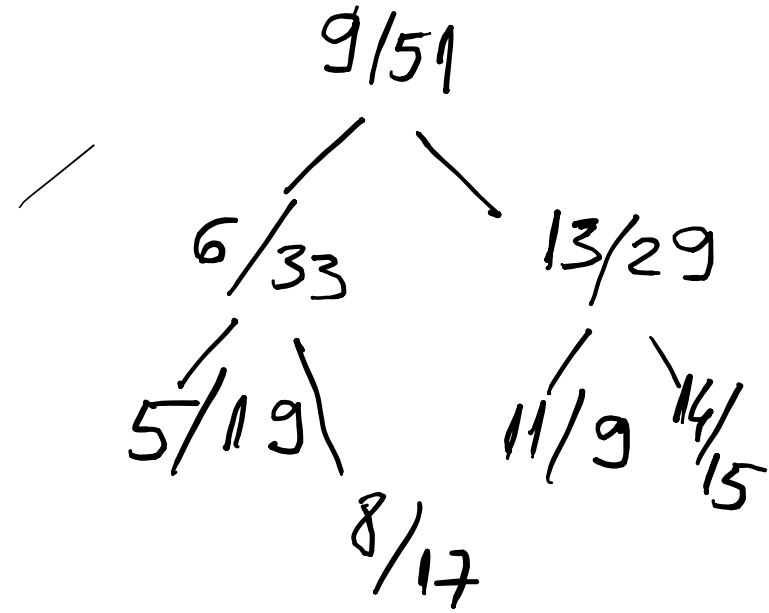
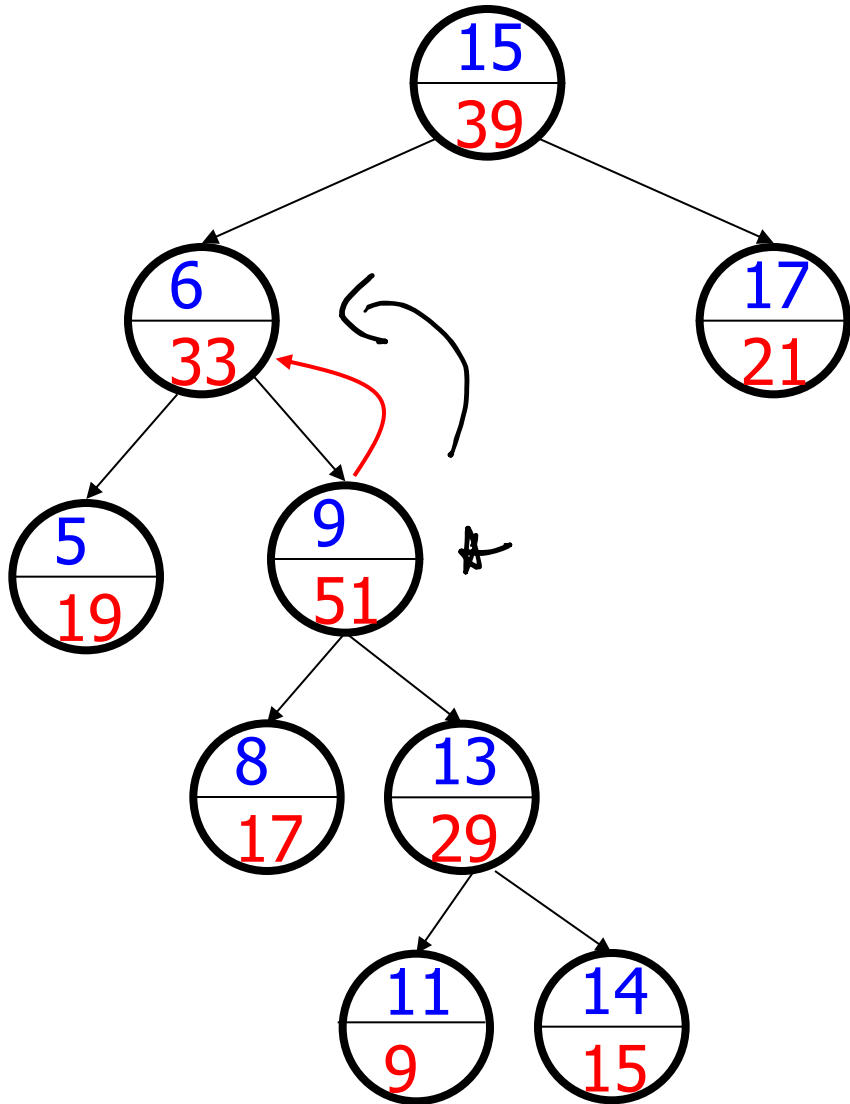
Exemplu inserare



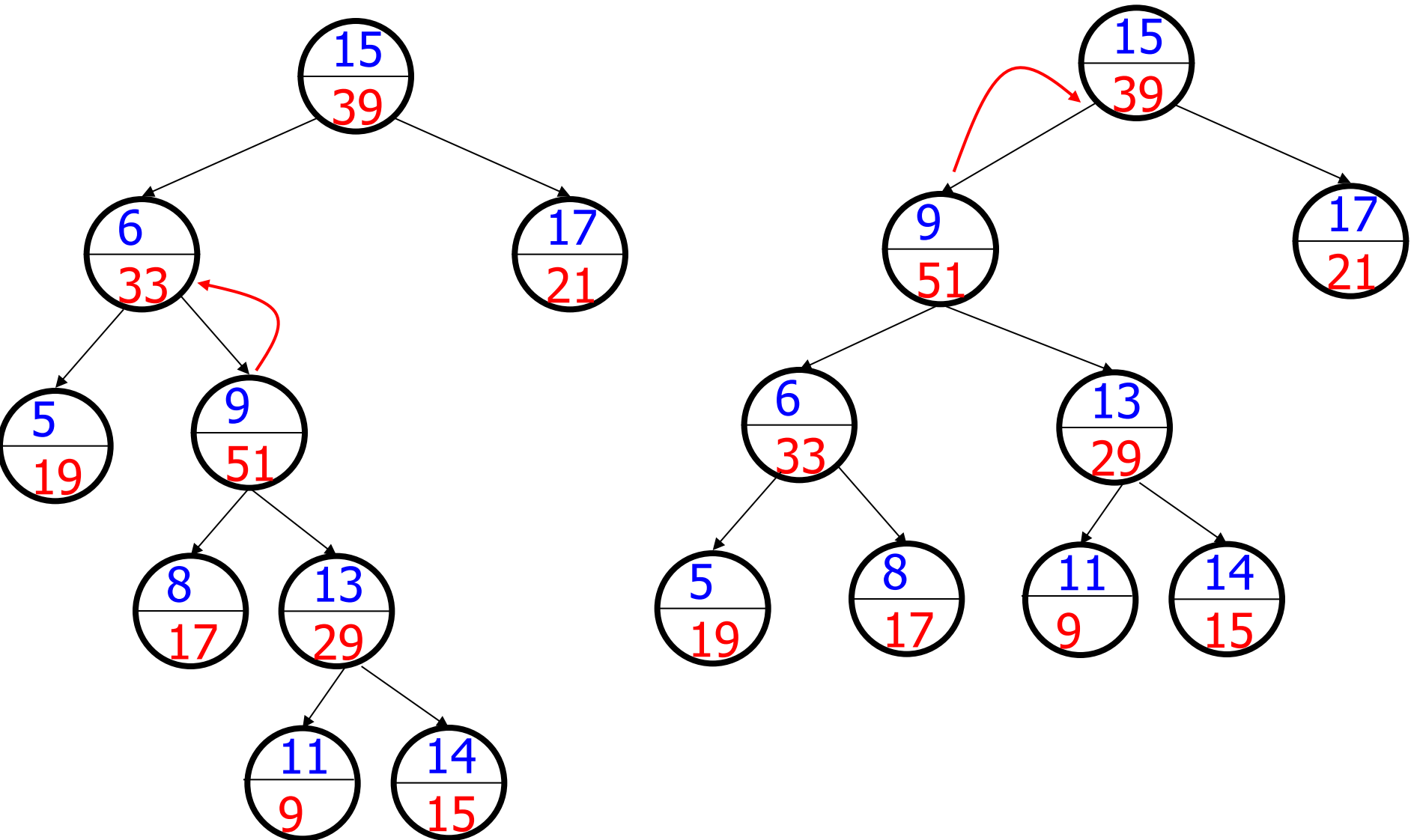
Exemplu inserare



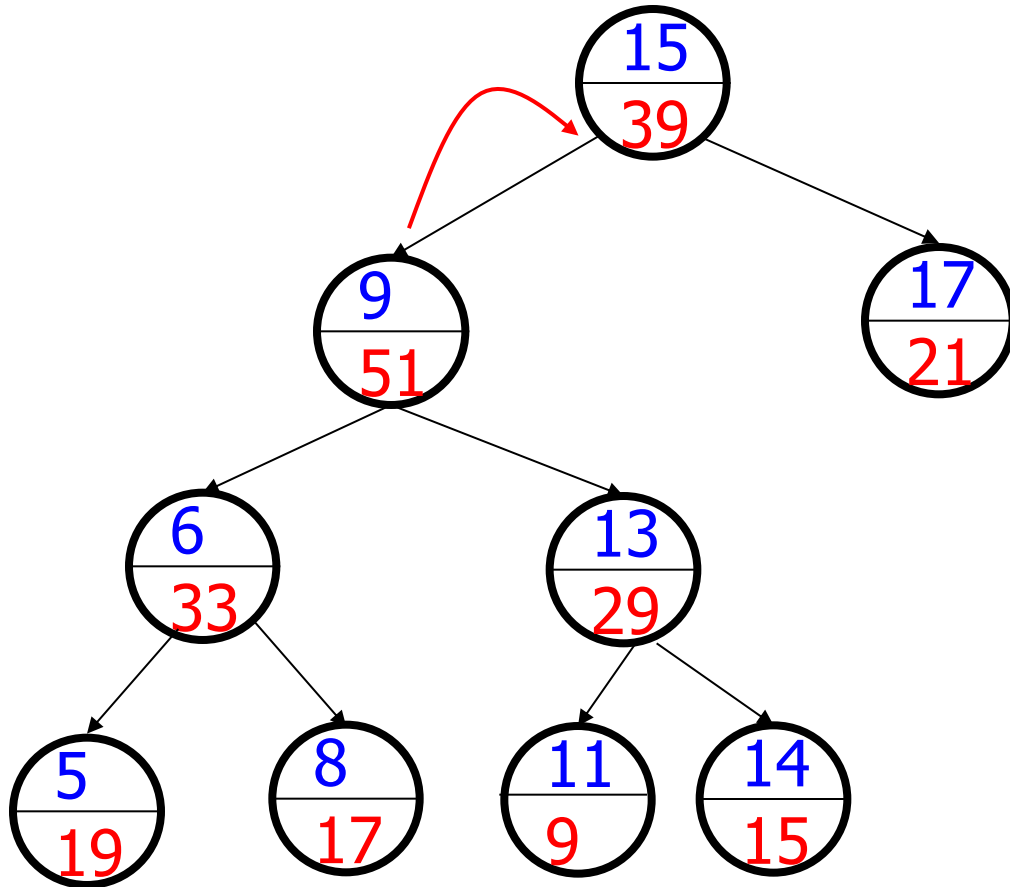
Exemplu inserare



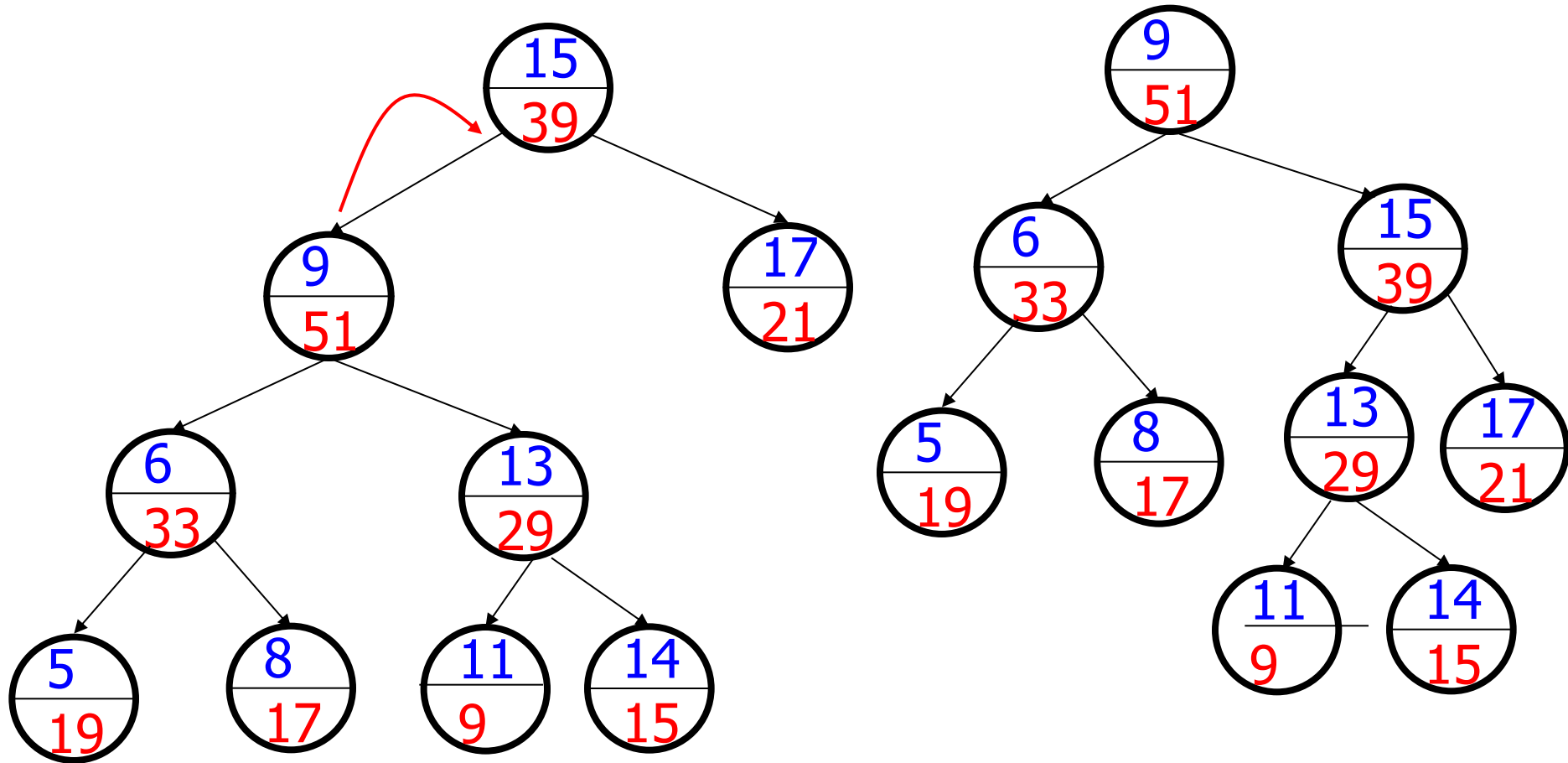
Exemplu inserare



Exemplu inserare

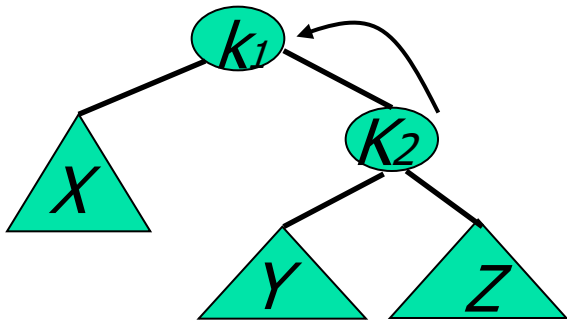


Exemplu inserare

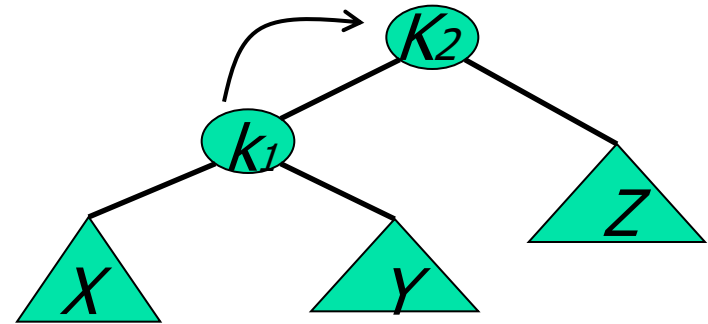


Rotatie stanga / dreapta

Rotatie stanga



Rotatie dreapta



Inserare in treap cheia x

- Genereaza prioritate pentru cheia x
- Insereaza x in treap – inserare frunza analog inserarii in arbore binar de cautare
- Reface proprietatea de heap prin rotatii stanga / dreapta

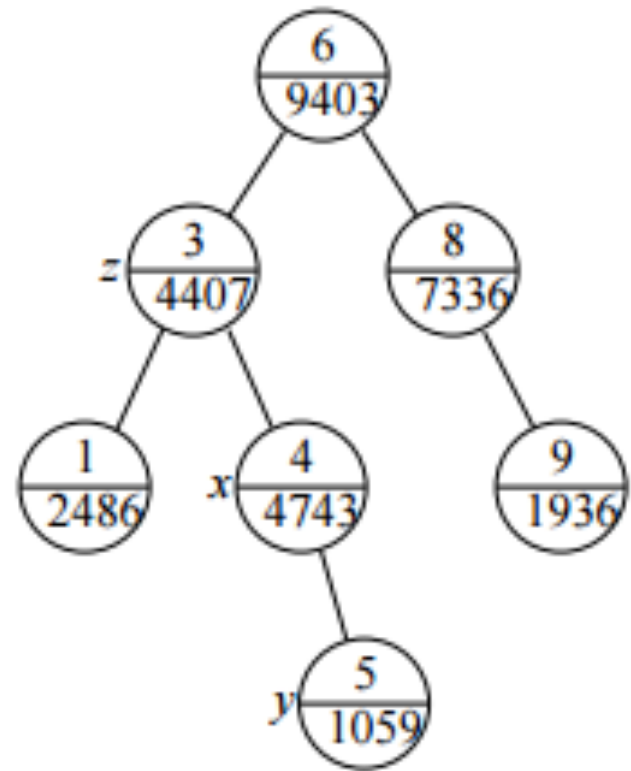
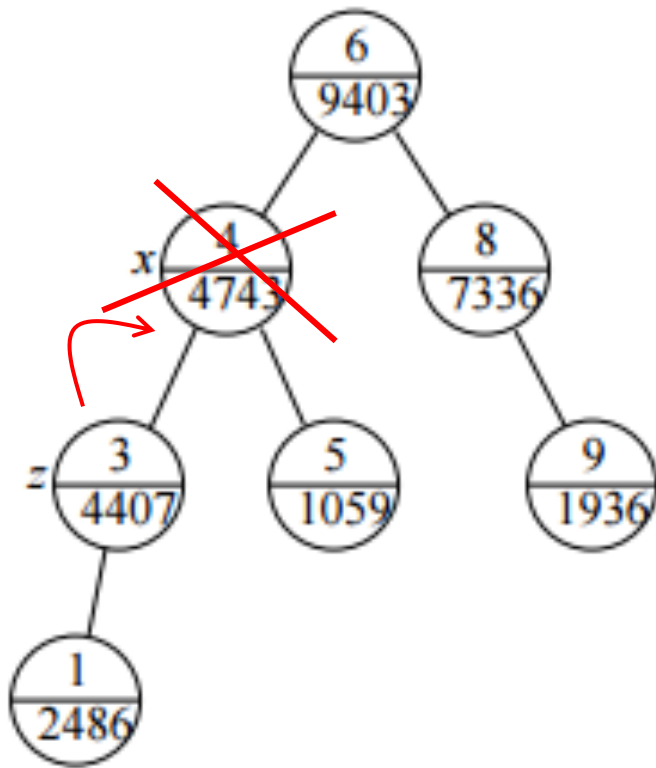
Inserare in treap

```
void insert(nod, cheie, prioritate)
{ if nod == null
    nod = creza nou nod pe baza de cheie si prioritate
  else
    if cheie < nod->info
      insert(nod->st, cheie, prioritate)
    else
      insert(nod->dr, cheie, prioritate)

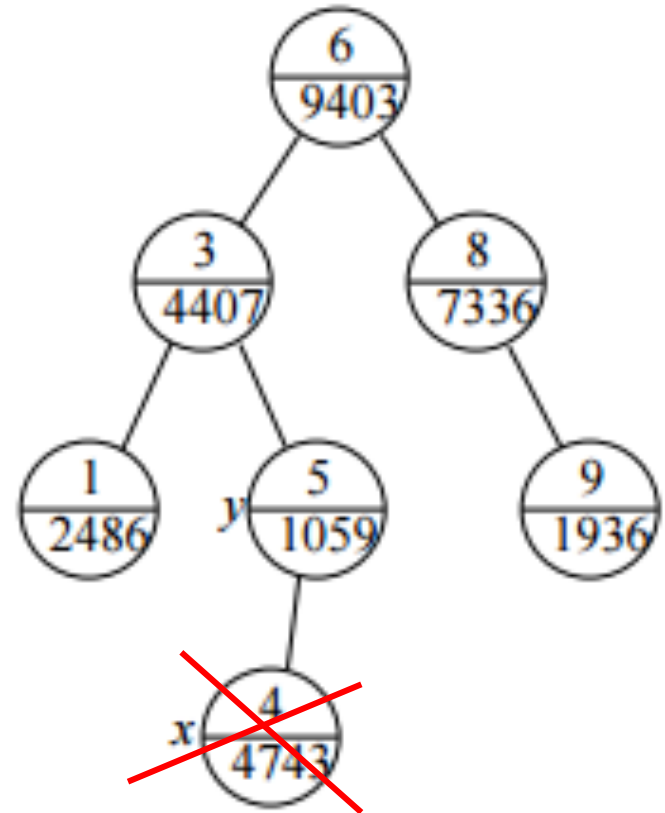
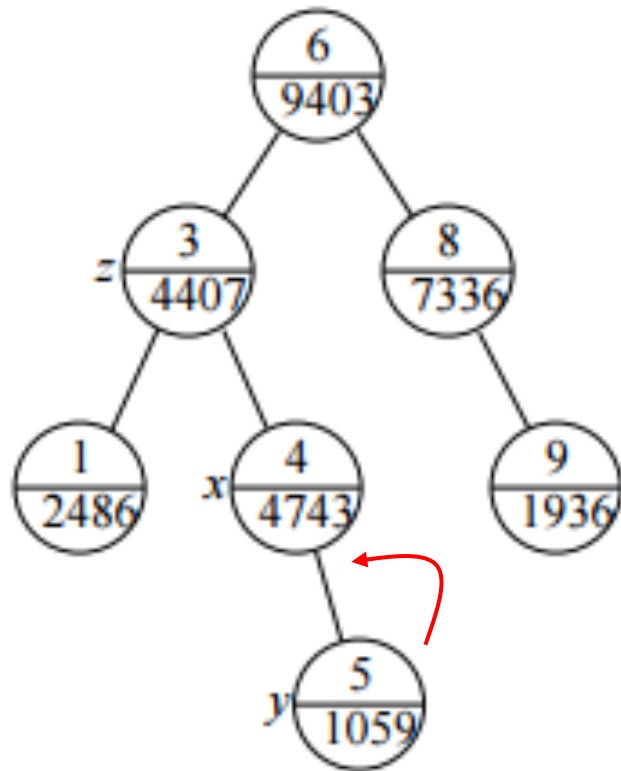
  if nod->st->prioritate > nod->prioritate
    rotireDreapta(nod)
  else
    if nod->dr->prioritate > nod->prioritate
      rotireStanga(nod) }
```

Eliminare

■ Operatia inversa inserarii



Eliminare



Eliminare

- Cauta x in arbore (analog cautare in arbore binar de cautare)
- Daca x este frunza atunci sterge nod
- Altfel propaga nodul cu informatia x pana ajunge o frunza (prin rotatii succesive)
- Sterge frunza ce contine informatia x din arbore

Eliminare

```
void sterge(nod, x)
```

```
{ if nod == null return
```

```
  if x < nod->info
```

```
    sterge(nod->st, x)
```

```
  else if x > nod->info
```

```
    sterge(nod->dr, x)
```

```
  else if frunza(nod)
```

```
    sterge nod
```

```
  else
```

```
    if nod->st->prioritate > nod->dr->prioritate
```

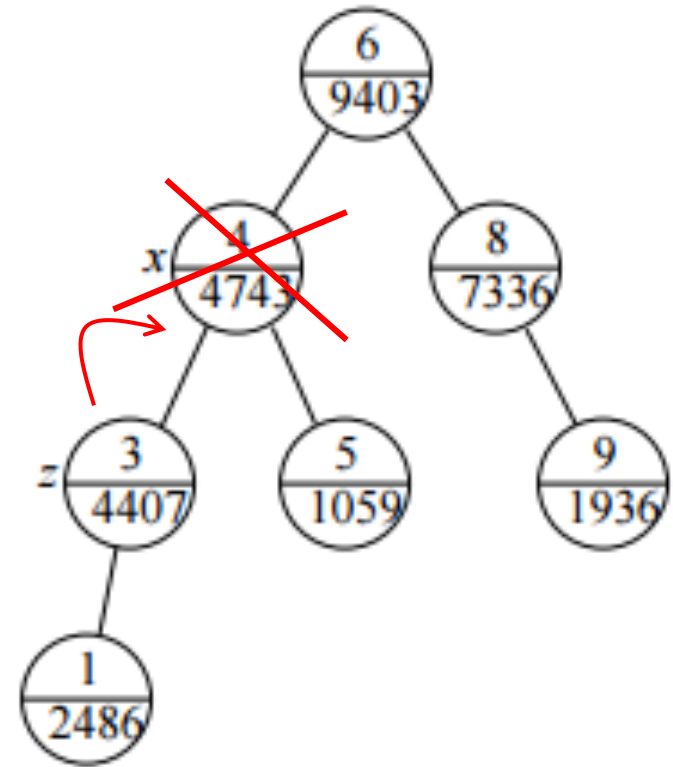
```
      rotireDreapta(nod);
```

```
      sterge(nod, x);
```

```
    else
```

```
      rotireStanga(nod);
```

```
      sterge(nod, x) }
```



Eliminare

```
void sterge(nod, x)
{ if nod == null return
  if x < nod->info
    sterge(nod->st, x)
  else if x > nod->info
    sterge(nod->dr, x)
  else if frunza(nod)
    sterge nod
  else
    if nod->st->prioritate > nod->dr->prioritate
      rotireDreapta(nod);
      sterge(nod, x);
    else
      rotireStanga(nod);
      sterge(nod, x) }
```

